Algoritmos

Estruturas de dados, paradigmas de projeto e algoritmos em grafos

Anizio Silva Correia Júnior

com orientação e material didático do
Prof. Uéverton Souza

IMPA Tech

Produção auxiliada por inteligência artificial.
Este texto foi elaborado por Anizio Silva Correia Júnior com o auxílio de ferramentas de inteligência artificial, a partir da orientação e do material didático proporcionados pelo Prof. Uéverton Souza.

Prefácio

Este livro apresenta os fundamentos de projeto e análise de algoritmos na sequência em que eles são estudados na disciplina de Algoritmos do IMPA Tech: começa pela linguagem da análise de complexidade, passa pelas estruturas de dados clássicas (filas de prioridades, conjuntos disjuntos, árvores de busca), pela ordenação, pela seleção e pela técnica de divisão e conquista, e chega à programação dinâmica, aos algoritmos gulosos e aos algoritmos em grafos.

Cada capítulo trata de um tópico. Ele começa com um quadro que diz o que será estudado, desenvolve o assunto com definições, algoritmos em pseudocódigo, demonstrações e exemplos resolvidos passo a passo, e termina com um resumo e uma lista de exercícios. Os exemplos numéricos foram conferidos por simulação.

Como o livro está organizado

A Parte I trata de análise de algoritmos e de estruturas de dados: complexidade (Capítulo 1), heaps binários, binomiais e de Fibonacci com análise amortizada (Capítulos 2 e 3), conjuntos disjuntos (Capítulo 4), árvores AVL (Capítulo 5), ordenação (Capítulos 6 e 7), seleção (Capítulo 8) e divisão e conquista (Capítulo 9). O Capítulo 10 reúne os critérios para escolher entre essas ferramentas.

A Parte II trata de paradigmas de projeto e de grafos: programação dinâmica (Capítulo 11), o algoritmo de Floyd-Warshall (Capítulo 12), algoritmos gulosos e matroides (Capítulo 13), árvores geradoras mínimas (Capítulo 14), busca em largura (Capítulo 15), o algoritmo de Dijkstra (Capítulo 16) e a busca em profundidade em grafos e digrafos (Capítulos 17 e 18). O Apêndice A trata de tabelas hash e o Apêndice B traz as respostas dos exercícios.

As caixas

Ao longo do texto, caixas coloridas ajudam a situar cada trecho:

> **Neste capítulo**
> O que o capítulo vai apresentar.

Pontos-chave

O que precisa ficar claro ao fim de uma seção ou de um capítulo.

Roteiro

Um método passo a passo para resolver um tipo de problema ou escolher entre alternativas.

Armadilhas comuns

Erros frequentes e confusões que costumam aparecer.

Para saber mais

Aprofundamentos que vão além do essencial.

Os exercícios

Cada capítulo termina com pelo menos vinte exercícios, divididos em dois grupos. Os **exercícios do curso** vêm do material da disciplina; uma etiqueta indica a origem de cada um:

[Slides]	exercícios propostos nos slides da disciplina;
[Lista]	listas de exercícios da disciplina;
[Monitoria]	exercícios das monitorias;
[Avaliação]	questões das avaliações da disciplina;
[Pesquisa]	temas dos trabalhos de pesquisa;
[Material de apoio]	listas de outras disciplinas indicadas como material complementar.

Os **exercícios complementares**, etiquetados [Complementar], foram escolhidos para fixar e aprofundar o conteúdo; os mais difíceis estão marcados com ★. Todos têm resposta, sugestão ou esboço de solução no Apêndice B. O link "resposta: p. X" ao lado de cada enunciado leva até ela, e o número da resposta leva de volta ao enunciado.

Convenções

- Heaps são **de máximo** e guardados em vetor indexado a partir de 1: o pai de i é $\lfloor i/2 \rfloor$ e os filhos são $2i$ e $2i+1$.
- Em árvores AVL, o balanço é $\text{bal}(v) = h_D(v) - h_E(v)$ (altura da direita menos a da esquerda).
- Em grafos, $n = |V|$ e $m = |E|$; $N(v)$ é a vizinhança de v e, num digrafo, $N_{out}(v)$ é a vizinhança de saída.
- log sem base indica $\log_2$; o símbolo * indica custo amortizado.

Nota sobre a produção

Este livro foi produzido por **Anizio Silva Correia Júnior**, com o **auxílio de ferramentas de inteligência artificial**, a partir da orientação e do material didático proporcionados pelo **Prof. Uéverton Souza** na disciplina de Algoritmos do IMPA Tech. As fontes utilizadas estão listadas nas Referências bibliográficas, ao final do volume.

Sumário

Prefácio i

I Análise de algoritmos e estruturas de dados 1

1 Complexidade de algoritmos 2
1.1 Medidas de complexidade . . . 2
1.2 Notações assintóticas . . . 4
1.3 Analisando laços na prática . . . 6
1.4 Complexidade de problemas e de algoritmos . . . 8
1.5 Algoritmos polinomiais e exponenciais . . . 9
1.6 Exercícios . . . 9

2 Listas de prioridades e heaps binários 12
2.1 Listas de prioridades . . . 12
2.2 Definição e representação . . . 13
2.3 As operações subir e descer . . . 14
2.4 Inserção, remoção e alteração de prioridade . . . 15
2.5 Construção em tempo linear . . . 16
2.6 Heapsort . . . 18
2.7 Propriedades de heaps e suas demonstrações . . . 19
2.8 Exercícios . . . 20

3 Heaps binomiais, heaps de Fibonacci e análise amortizada 23
3.1 Árvores binomiais . . . 23
3.2 Heap binomial . . . 24
3.3 Análise amortizada . . . 27
3.4 Heap de Fibonacci . . . 28
3.5 Comparação e escolha da estrutura . . . 32
3.6 Exercícios . . . 33

4 Conjuntos disjuntos: Union-Find 36
4.1 O problema . . . 36
4.2 Implementações . . . 37
4.3 União por rank com compressão: $O(m \log^* n)$ . . . 40
4.4 Qual implementação usar? . . . 41
4.5 Exemplos resolvidos . . . 42
4.6 Exercícios . . . 43

5 Árvores binárias de busca e árvores AVL 46
5.1 Árvores binárias de busca . . . 46
5.2 Árvores AVL . . . 48
5.3 Inserção e atualização dos balanços . . . 49
5.4 As rotações . . . 50
5.5 Pseudocódigo da inserção . . . 52

5.6 Remoção . . . 53
5.7 Exemplos resolvidos . . . 54
5.8 Heap, ABB ou AVL? . . . 57
5.9 Árvores rubro-negras . . . 57
5.10 Exercícios . . . 59

6 Ordenação por comparação e limite inferior 62
6.1 O problema da ordenação . . . 62
6.2 Recorrências . . . 63
6.3 Ordenação por comparação . . . 65
6.4 Limite inferior para ordenação por comparação . . . 68
6.5 Exercícios . . . 69

7 Ordenação em tempo linear 71
7.1 Counting sort . . . 71
7.2 Radix sort . . . 73
7.3 Bucket sort . . . 74
7.4 American Flag sort . . . 75
7.5 Qual algoritmo de ordenação usar? . . . 76
7.6 Exercícios . . . 77

8 Seleção em tempo linear: a mediana das medianas 80
8.1 O problema e o quickselect . . . 80
8.2 O algoritmo da mediana das medianas . . . 82
8.3 Análise . . . 83
8.4 Exercícios . . . 84

9 Divisão e conquista 87
9.1 O paradigma . . . 87
9.2 Contagem de inversões . . . 88
9.3 Subvetor contíguo de soma máxima . . . 90
9.4 Multiplicação de matrizes: o algoritmo de Strassen . . . 91
9.5 Outros exemplos clássicos . . . 92
9.6 Exercícios . . . 95

10 Escolhendo estruturas de dados e algoritmos 97
10.1 Um método de escolha . . . 97
10.2 Quadro comparativo . . . 99
10.3 Resumo das complexidades . . . 100
10.4 Exemplos . . . 101
10.5 Exercícios . . . 102

II Paradigmas de projeto e algoritmos em grafos 104

11 Programação dinâmica 105
11.1 A técnica . . . 105
11.2 Subsequência comum máxima . . . 107
11.3 Multiplicação de cadeia de matrizes . . . 109
11.4 Mochila 0-1 . . . 111
11.5 Exercícios . . . 113

12 Caminhos mínimos entre todos os pares: Floyd-Warshall **116**
12.1 Grafos: notação e representação . . . 116
12.2 Matriz de pesos . . . 117
12.3 O algoritmo de Floyd-Warshall . . . 117
12.4 Exercícios . . . 120

13 Algoritmos gulosos e matroides **123**
13.1 O método guloso . . . 123
13.2 Matroides . . . 124
13.3 Exercícios . . . 128

14 Árvores geradoras mínimas: Kruskal e Prim **131**
14.1 O problema e a propriedade do corte . . . 131
14.2 O algoritmo de Kruskal . . . 132
14.3 O algoritmo de Prim . . . 133
14.4 Exercícios . . . 135

15 Busca em largura **137**
15.1 O algoritmo . . . 137
15.2 Propriedades e aplicações . . . 138
15.3 Exercícios . . . 140

16 Caminhos mínimos de uma origem: Dijkstra **143**
16.1 O algoritmo . . . 143
16.2 Correção . . . 144
16.3 Implementações e complexidade . . . 145
16.4 Exercícios . . . 146

17 Busca em profundidade **149**
17.1 O algoritmo . . . 149
17.2 Propriedades . . . 150
17.3 Aplicações . . . 151
17.4 Articulações, pontes e blocos . . . 151
17.5 Exercícios . . . 153

18 Busca em profundidade em digrafos **155**
18.1 Classificação das arestas . . . 155
18.2 Componentes fortemente conexas . . . 156
18.3 Ordenação topológica . . . 158
18.4 Exercícios . . . 159

A Tabelas hash **162**
A.1 Endereçamento direto e funções hash . . . 162
A.2 Colisões . . . 163
A.3 Complexidade . . . 165
A.4 Exercícios . . . 166

B Respostas e sugestões **169**

Referências bibliográficas **199**

Parte I

Análise de algoritmos e estruturas de dados

1

Complexidade de algoritmos

Neste capítulo

- Medir o custo de um algoritmo em função do tamanho da entrada: pior caso, caso médio e melhor caso.
- As notações assintóticas O, Ω, Θ, o e ω e suas propriedades.
- Analisar laços na prática: como contar as iterações de um trecho de código.
- A diferença entre a complexidade de um *algoritmo* e a de um *problema*: limites superiores, inferiores e algoritmos ótimos.
- Por que a fronteira entre algoritmos polinomiais e exponenciais importa, e o que é um algoritmo pseudopolinomial.

Dois programas que resolvem o mesmo problema podem levar tempos muito diferentes. Considere a tarefa de calcular a soma $1 + 2 + \cdots + N$. Um programa faz um laço que soma os números um a um; outro usa a fórmula de Gauss, $N(N+1)/2$.

Soma com laço

```
s := 0
para i := 1 até N faça
    s := s + i
retorna s
```

Soma com a fórmula

```
retorna N(N + 1)/2
```

Para $N = 10$, ambos são instantâneos. Para $N = 10^{12}$, o primeiro faz um trilhão de somas e o segundo continua fazendo três operações. A diferença não está na máquina nem na linguagem: está no *número de passos* que cada algoritmo executa e em *como esse número cresce* quando a entrada aumenta.

Este capítulo fixa a linguagem que será usada no resto do livro para fazer esse tipo de comparação. As técnicas para resolver recorrências aparecem no Capítulo 6, junto com o primeiro algoritmo recursivo estudado, e a análise amortizada aparece no Capítulo 3.

1.1 Medidas de complexidade

Intuição

Medir um algoritmo pelo tempo de relógio é pouco confiável: o mesmo programa roda em tempos diferentes em computadores diferentes. Em vez disso, contamos **passos elementares** (uma comparação, uma soma, uma atribuição, um acesso a uma posição de vetor), cada um com custo constante, e expressamos o total como uma *função do tamanho da entrada*.

O problema é que entradas do mesmo tamanho podem dar trabalhos diferentes. Fixe um algoritmo A e um tamanho de entrada n. Sejam $E_1, \ldots, E_m$ todas as entradas possíveis de tamanho n e T_i o número de passos que A executa sobre E_i. Há três maneiras naturais de resumir os números $T_1, \ldots, T_m$ num só valor:

- a complexidade de **pior caso**, $\max_i T_i$;
- a complexidade de **caso médio**, $\sum_i p_i T_i$, onde p_i é a probabilidade de a entrada ser E_i;
- a complexidade de **melhor caso**, $\min_i T_i$.

Exemplo 1.1 (Os três casos da ordenação por inserção)**.** A ordenação por inserção percorre o vetor da esquerda para a direita e insere cada elemento na posição correta entre os anteriores, que já estão ordenados. Para isso, compara o elemento com os anteriores, do mais próximo para o mais distante, até achar um menor ou chegar ao início.

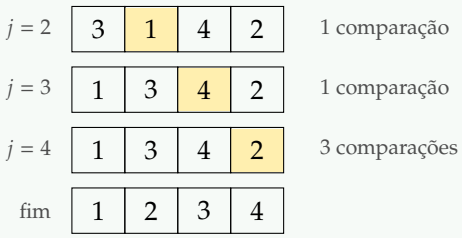


Figura 1.1. Ordenação por inserção sobre $[3, 1, 4, 2]$. Em cada linha, o elemento destacado é inserido entre os anteriores (já ordenados); à direita, as comparações feitas.

Para $n = 4$, as 24 ordens possíveis de quatro números distintos dão:

- **melhor caso**: o vetor já ordenado, $[1, 2, 3, 4]$; cada elemento é comparado só com o vizinho e não se move: 3 comparações, e em geral $n - 1$;
- **pior caso**: o vetor decrescente, $[4, 3, 2, 1]$; cada elemento é comparado com todos os anteriores: $1 + 2 + 3 = 6$ comparações, e em geral $n(n-1)/2$;
- **caso médio**, supondo as 24 ordens igualmente prováveis: a média é $59/12 \approx 4{,}92$ comparações (a entrada $[3, 1, 4, 2]$ da figura faz 5).

O mesmo algoritmo é linear em entradas favoráveis e quadrático nas desfavoráveis.

O pior caso é a medida padrão: ele dá uma *garantia* que vale para toda entrada, e não depende de hipóteses sobre a distribuição das entradas, que nem sempre são realistas. O caso médio é útil quando essa distribuição é conhecida (o quicksort, por exemplo, é rápido em média e lento no pior caso). O melhor caso raramente diz algo útil, mas aparece para mostrar que um algoritmo pode ser rápido em entradas favoráveis, como a ordenação por inserção num vetor já ordenado.

Uma quarta medida, a complexidade **amortizada**, divide o custo de uma *sequência* de operações sobre uma estrutura de dados pelo número de operações, no pior caso da sequência. Ela não envolve probabilidade e será estudada no Capítulo 3.

Tamanho da entrada

O tamanho de uma entrada é o comprimento da sua codificação, em bits ou em símbolos. Para uma lista de n números de tamanho limitado, o tamanho é proporcional a n. Mas quando a entrada é um número N, ela ocupa apenas cerca de $\log_2 N$ bits (ou $\log_{10} N$ dígitos), e um algoritmo cujo custo cresce com o *valor* de N pode ser exponencial no tamanho da entrada.

Exemplo 1.2 (A soma com laço é exponencial no número de dígitos)**.** Volte à soma $1+2+\cdots+N$. O laço faz N iterações, o que parece "linear". Mas a entrada é o próprio número N, escrito com $d \approx \log_{10} N$ dígitos:

dígitos de N	iterações do laço	tempo a 10^9 iterações/s
3	10^3	1 microssegundo
9	10^9	1 segundo
15	10^{15}	$\approx$ 11,6 dias
30	10^{30}	$\approx 3 \cdot 10^{13}$ anos

Cada dígito a mais multiplica o tempo por 10: o custo é $\Theta(10^d)$, exponencial no tamanho d da entrada. A fórmula $N(N+1)/2$, por outro lado, faz uma multiplicação de números com d dígitos, o que custa $O(d^2)$ pelo método escolar: polinomial.

Um algoritmo é **pseudopolinomial** quando é polinomial no *valor* dos números da entrada (isto é, quando eles são escritos em unário), mas não necessariamente no seu tamanho em bits. A soma com laço é um exemplo artificial; exemplos importantes são o *counting sort* do Capítulo 7, de custo $\Theta(n+k)$ para inteiros em $[0, k]$, e a programação dinâmica para a mochila do Capítulo 11, de custo $\Theta(nW)$.

1.2 Notações assintóticas

Intuição

Quando n é grande, só o termo que cresce mais rápido importa. Em $n^2 + 10n + 20$, para $n = 1000$ o termo n^2 vale 10^6, enquanto $10n + 20$ vale 10 020, cerca de 1% do total. E constantes multiplicativas dependem da máquina: um computador duas vezes mais rápido transforma $2n^2$ em n^2. Por isso dizemos simplesmente que esse custo "é da ordem de n^2". As notações assintóticas tornam essa frase precisa.

As notações abaixo valem para funções f, g não negativas a partir de algum ponto. Leia $f = O(g)$ como "f cresce no máximo como g".

Notação	Definição	Via limite (se existir)	Leitura
$f = O(g)$	$\exists c > 0, n_0 : \ f(n) \le c\, g(n)\ \forall n \ge n_0$	$\lim f/g < \infty$	"$\le$"
$f = \Omega(g)$	$\exists c > 0, n_0 : \ f(n) \ge c\, g(n)\ \forall n \ge n_0$	$\lim f/g > 0$	"$\ge$"
$f = \Theta(g)$	$f = O(g)$ e $f = \Omega(g)$	$\lim f/g = c \in (0, \infty)$	"$=$"
$f = o(g)$	$\forall c > 0\ \exists n_0 : \ f(n) < c\, g(n)\ \forall n \ge n_0$	$\lim f/g = 0$	"$<$"
$f = \omega(g)$	$\forall c > 0\ \exists n_0 : \ f(n) > c\, g(n)\ \forall n \ge n_0$	$\lim f/g = \infty$	"$>$"

Formalmente, $O(g)$ é um *conjunto* de funções, e $f = O(g)$ significa $f \in O(g)$; o sinal de igualdade é um abuso de notação consagrado, que deve ser lido sempre da esquerda para a direita ($n = O(n^2)$ está certo; "$O(n^2) = n$" não faz sentido).

As notações O, Ω e Θ numa figura

A definição de O tem duas constantes, e cada uma tem um papel: c absorve as constantes multiplicativas, e n_0 permite ignorar os valores pequenos de n, onde termos de ordem menor ainda pesam. A Figura 1.2 mostra as duas em ação.

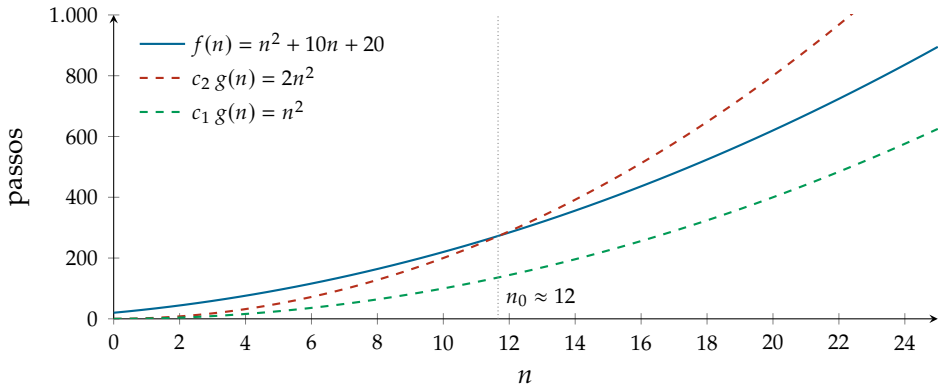


Figura 1.2. A função $f(n) = n^2 + 10n + 20$ fica "ensanduichada" entre $1 \cdot n^2$ e $2 \cdot n^2$ a partir de $n_0 = 12$. A curva de baixo prova $f = \Omega(n^2)$, a de cima prova $f = O(n^2)$ e as duas juntas provam $f = \Theta(n^2)$.

Exemplo 1.3 (Provando O, Ω e Θ pela definição)**.** Seja $f(n) = n^2 + 10n + 20$.

- $f = O(n^2)$: queremos c e n_0 com $n^2 + 10n + 20 \le c\, n^2$. Com $c = 2$, a desigualdade vira $10n + 20 \le n^2$, que vale para $n \ge 12$ (para $n = 12$: $140 \le 144$). Logo $c = 2$, $n_0 = 12$ servem. Também serviria $c = 31$, $n_0 = 1$, pois $10n \le 10n^2$ e $20 \le 20n^2$ para $n \ge 1$: as constantes não são únicas, basta exibir *uma* escolha.
- $f = \Omega(n^2)$: com $c = 1$ e $n_0 = 1$, $f(n) \ge n^2$ é óbvio.
- $f = \Theta(n^2)$: segue dos dois itens. Note que f também é $O(n^3)$ (um limite superior folgado), mas *não* é $O(n)$: se fosse, existiria c com $n^2 \le f(n) \le c\, n$ para n grande, ou seja, $n \le c$, o que é falso para $n > c$.

Exemplo 1.4. Pela mesma técnica, a função $4n^3 + 5n$ é $O(n^4)$ e $O(n^3)$, mas não é $O(n^2)$; é $\Omega(n^2)$ e $\Omega(n^3)$, mas não é $\Omega(n^4)$; portanto é $\Theta(n^3)$. Para provar $4n^3 + 5n = O(n^3)$ pela definição, basta tomar $c = 9$ e $n_0 = 1$: $4n^3 + 5n \le 4n^3 + 5n^3 = 9n^3$ para todo $n \ge 1$.

As versões estritas o e ω

Em $f = O(g)$, a função f pode crescer *tanto quanto* g; em $f = o(g)$, ela cresce *estritamente menos*: para *qualquer* constante c, por menor que seja, f acaba ficando abaixo de $c\, g$. A diferença está no quantificador: em O basta *existir* um c, em o a desigualdade vale *para todo* c.

Exemplo 1.5 (Usando limites)**.** Quando o limite de f/g existe, ele resolve tudo de uma vez:

- $\lim\limits_{n\to\infty} \dfrac{3n^2 + 5n}{n^2} = 3$, um número em $(0, \infty)$: logo $3n^2 + 5n = \Theta(n^2)$.
- $\lim \dfrac{n}{n^2} = \lim \dfrac{1}{n} = 0$: logo $n = o(n^2)$. Já $n^2 \ne o(n^2)$, pois $n^2/n^2 = 1 \not\to 0$.
- $\lim \dfrac{\ln n}{\sqrt{n}} = \lim \dfrac{1/n}{1/(2\sqrt{n})} = \lim \dfrac{2}{\sqrt{n}} = 0$ (regra de L'Hôpital): logo $\log n = o(\sqrt{n})$. Logaritmos crescem mais devagar que qualquer raiz.
- $\lim \dfrac{2^n}{n^3} = \infty$ (aplique L'Hôpital três vezes): logo $2^n = \omega(n^3)$. Exponenciais crescem mais rápido que qualquer polinômio.

Propriedades

As notações se comportam como as desigualdades que representam:

- **transitividade**: $f = O(g)$ e $g = O(h)$ implicam $f = O(h)$ (e o mesmo para as outras notações);
- **simetria transposta**: $f = O(g)$ se e somente se $g = \Omega(f)$;
- **a soma é dominada pela maior parcela**: $f + g = \Theta(\max\{f, g\})$;
- **a base do logaritmo não importa**: $\log_a n = \log_b n / \log_b a$ difere de $\log_b n$ pelo fator constante $1/\log_b a$.

Exemplo 1.6 (Aplicando as propriedades)**.**

- $5n^3 + 100\,n^2 \log n + 7 = \Theta(n^3)$: a soma é dominada pela maior parcela, e $n^2 \log n = o(n^3)$ porque $\log n = o(n)$.
- $\log_{10} n = \log_2 n / \log_2 10 \approx 0{,}301 \log_2 n$; logo $\log_{10} n = \Theta(\log_2 n)$, e podemos escrever simplesmente $\Theta(\log n)$.
- $\log(n^3) = 3 \log n = \Theta(\log n)$, mas $\log(3^n) = n \log 3 = \Theta(n)$: o expoente *dentro* do logaritmo vira constante; o expoente que depende de n, não.

Também valem $\log(n!) = \Theta(n \log n)$, $(\log n)^c = o(n^\varepsilon)$ para quaisquer $c > 0$ e $\varepsilon > 0$, e $n^c = o(a^n)$ para todo $a > 1$. A escala abaixo resume as ordens de crescimento mais frequentes:

$$1 < \log\log n < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < n^{\log n} < 2^n < n! < n^n.$$

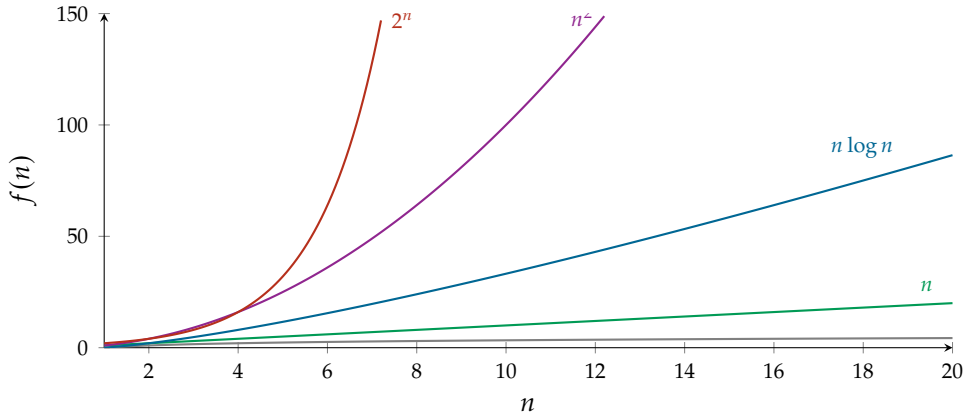


Figura 1.3. As funções mais comuns para n entre 1 e 20. Para n pequeno, todas parecem próximas; logo a exponencial 2^n sai do gráfico, e n^2 se afasta de $n \log n$.

A Tabela 1.1 traduz essas ordens em tempo real, supondo um computador que executa um bilhão de passos por segundo. Ela mostra por que a ordem de crescimento importa mais que qualquer constante.

Tabela 1.1. Tempo para executar $f(n)$ passos a 10^9 passos por segundo.

n	$\log n$	n	$n \log n$	n^2	n^3	2^n
10	3 ns	10 ns	33 ns	100 ns	1 μs	1 μs
100	7 ns	100 ns	664 ns	10 μs	1 ms	$4 \cdot 10^{13}$ anos
1000	10 ns	1 μs	10 μs	1 ms	1 s	—
10^6	20 ns	1 ms	20 ms	17 min	32 anos	—

1.3 Analisando laços na prática

Na maior parte das vezes, analisar um algoritmo iterativo é contar quantas vezes o corpo dos seus laços é executado.

Como analisar um trecho com laços

1. Identifique o **corpo mais interno** e verifique que ele custa $O(1)$.
2. Para cada laço, escreva quantas vezes ele itera *em função das variáveis dos laços externos*.
3. Some: laços aninhados viram somatórios aninhados.
4. Resolva o somatório e simplifique com as notações assintóticas.

Exemplo 1.7 (Três padrões de laço). **(a) Laços aninhados independentes.** Em "**para** $i := 1$ **até** n, **para** $j := 1$ **até** n: corpo", o corpo executa $n \cdot n = n^2$ vezes: $\Theta(n^2)$.

(b) Laço interno que depende do externo. Em "**para** $i := 1$ **até** n, **para** $j := 1$ **até** i: corpo", o laço interno faz i iterações, e o total é

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2).$$

A Figura 1.4 mostra a contagem como uma escada: as execuções ocupam cerca de metade do quadrado $n \times n$, e metade de n^2 ainda é $\Theta(n^2)$.

(c) Laço que divide pela metade. Em "$i := n$; **enquanto** $i > 1$ **faça** $i := \lfloor i/2 \rfloor$", o valor de i passa por $n,\ n/2,\ n/4, \ldots$ até chegar a 1. Depois de k iterações, $i \approx n/2^k$, que chega a 1 quando $k \approx \log_2 n$: $\Theta(\log n)$. Para $n = 16$, i vale $16, 8, 4, 2, 1$: quatro iterações.

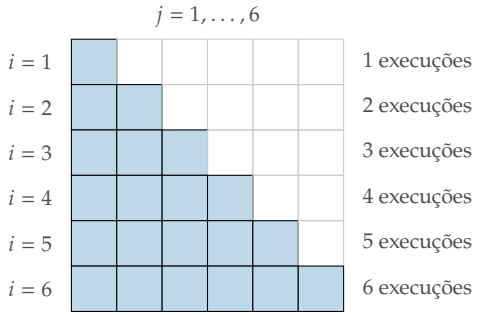


Figura 1.4. O laço do item (b) com $n = 6$: na linha i, o corpo executa i vezes (células preenchidas). O total, $1 + 2 + \cdots + 6 = 21$, é pouco mais que a metade das 36 células.

Um laço cuja variável é multiplicada ou dividida por uma constante a cada passo dá $\Theta(\log n)$ iterações. É isso que torna a **busca binária** tão rápida: a cada comparação, o trecho do vetor ordenado onde o elemento procurado pode estar cai pela metade.

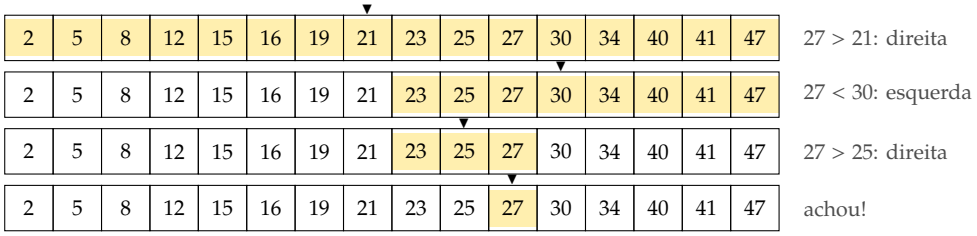


Figura 1.5. Busca binária de $x = 27$ num vetor ordenado de 16 posições. O trecho candidato (em destaque) cai pela metade a cada comparação com o elemento do meio (▼).

Erros comuns na análise de laços

- Multiplicar os limites dos laços quando o interno depende do externo: isso sempre dá um limite *superior*, mas ele pode ser folgado. No item (b) a ordem saiu certa; no Exercício 1.8(a), em que i dobra a cada volta, faça a soma com cuidado antes de concluir.
- Esquecer que Θ descarta constantes: $n(n+1)/2$ e n^2 são da mesma ordem.
- Confundir o número de iterações com o *valor* das variáveis: em (c), i começa em n, mas o laço só dá $\log_2 n$ voltas.

1.4 Complexidade de problemas e de algoritmos

A complexidade de um *algoritmo* é uma função que descreve o seu custo. A complexidade de um *problema* é uma pergunta mais difícil: qual o menor custo possível entre *todos* os algoritmos que o resolvem, inclusive os que ainda não foram inventados?

Definição 1.8.

- Uma função $f(n)$ é um **limite superior** para o problema Π se *existe* algum algoritmo que resolve Π com no máximo $f(n)$ passos no pior caso. A complexidade de pior caso de qualquer algoritmo para Π é um limite superior para Π.
- Uma função $f(n)$ é um **limite inferior** para Π se *todo* algoritmo que resolve Π executa pelo menos $f(n)$ passos no pior caso.
- Um algoritmo é **ótimo** se sua complexidade de pior caso coincide com um limite inferior do problema.

Exemplo 1.9 (Encontrar o máximo é um problema de custo exatamente $n-1$)**.** Considere o problema de achar o maior de n números distintos usando comparações.

- **Limite superior**: o algoritmo que guarda o maior visto até agora e o compara com cada novo elemento faz $n-1$ comparações. Logo $n-1$ é um limite superior para o problema.
- **Limite inferior**: numa comparação "$a < b$?", diremos que o menor dos dois *perdeu*. Um algoritmo só pode ter certeza de que x é o máximo se cada um dos outros $n-1$ elementos já perdeu alguma comparação: se $y \neq x$ nunca perdeu, y poderia ser o máximo sem contradizer nenhuma resposta obtida. Como cada comparação produz exatamente um perdedor, são necessárias pelo menos $n-1$ comparações, *qualquer que seja o algoritmo*.

Os dois limites coincidem, e o algoritmo simples é ótimo.

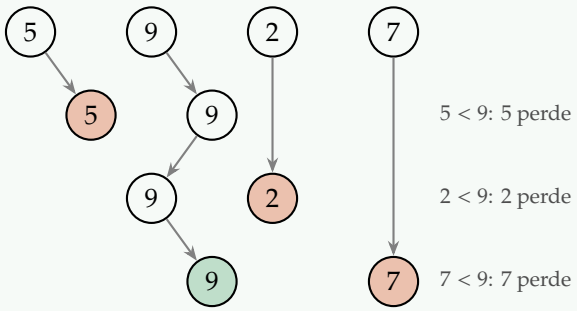


Figura 1.6. O máximo de $[5, 9, 2, 7]$. Cada comparação (seta) elimina um candidato, o perdedor (em vermelho). Com $n = 4$ elementos, três perdedores exigem três comparações.

Limites superiores se provam exibindo um algoritmo; limites inferiores exigem um argumento sobre todos os algoritmos possíveis, como o argumento do exemplo acima ou o da árvore de decisão, que mostra que ordenar por comparações exige $\Omega(n \log n)$ comparações (Capítulo 6). O mergesort e o heapsort, que fazem $O(n \log n)$ comparações, são portanto ótimos.

As notações assintóticas são afirmações sobre funções, e não sobre "limites" no sentido acima: nada impede dizer que o pior caso de um algoritmo é $\Omega(n^3)$, ou que conhecemos para um problema um limite inferior que é $O(\log n)$.

1.5 Algoritmos polinomiais e exponenciais

Um algoritmo é **polinomial** se seu custo de pior caso é $O(n^c)$ para alguma constante c; a classe dos problemas que admitem algoritmos polinomiais é chamada de P. Custos como 2^n, k^n, $n!$, 2^{n^2} e $n^{\log n}$ são **exponenciais** (ou superpolinomiais).

O consenso é que algoritmos polinomiais são eficientes e exponenciais não, por um motivo que vai além do tempo absoluto da Tabela 1.1: algoritmos polinomiais aproveitam o avanço tecnológico. Com uma máquina mais rápida, o maior tamanho de entrada resolvido num tempo fixo é *multiplicado* por uma constante quando o custo é polinomial, mas apenas *acrescido* de uma constante quando é exponencial.

Exemplo 1.10 (O que uma máquina 100 vezes mais rápida compra)**.** Suponha que hoje, em uma hora, resolvemos entradas de tamanho até N. Com a máquina nova, no mesmo tempo, fazemos 100 vezes mais passos:

custo	novo tamanho máximo	por quê
n	$100N$	$n' = 100N$ dá $100N$ passos
n^2	$\sqrt{100}\,N = 10\,N$	$(10N)^2 = 100N^2$
n^3	$\sqrt[3]{100}\,N \approx 4{,}64\,N$	$(4{,}64N)^3 \approx 100N^3$
2^n	$N + 6{,}6$	$2^{N+6{,}6} \approx 100 \cdot 2^N$

Se hoje resolvemos $N = 1000$ com o algoritmo quadrático, passaremos a resolver 10 000. Se resolvemos $N = 40$ com o exponencial, passaremos a resolver só 46: cem vezes mais velocidade compram seis posições a mais.

Pontos-chave

Pior caso é a medida padrão. O é "$\leq$", Ω é "$\geq$", Θ é "$=$" a menos de constantes, para n grande; a base do logaritmo não importa. Laços aninhados viram somatórios; laços que dividem a variável por uma constante dão $\Theta(\log n)$ voltas. Limite superior de um problema: *existe* um algoritmo; limite inferior: *todo* algoritmo; ótimo: pior caso igual ao limite inferior. O tamanho da entrada se mede em bits, e algoritmos cujo custo cresce com o *valor* de um número são pseudopolinomiais.

1.6 Exercícios

Exercícios do curso

1.1 [Slides] resposta: p. 169

Diga, justificando pela definição, se $2n^2 + 1$ é: (a) $O(n)$; (b) $O(n^2)$; (c) $O(n^3)$; (d) $\Omega(n)$; (e) $\Omega(n^3)$; (f) $\Theta(n^2)$.

1.2 [Slides] resposta: p. 169
Uma máquina nova é 10 vezes mais rápida que a antiga. Para algoritmos de custo n, n^2 e 2^n, em quanto aumenta o maior tamanho de entrada que se resolve no mesmo tempo? O que isso diz sobre algoritmos polinomiais e exponenciais?

1.3 [Slides] resposta: p. 169
O *counting sort* ordena n inteiros em $[0, k]$ em tempo $\Theta(n + k)$. Explique por que ele **não** é polinomial no tamanho da entrada quando $k = 2^n$, e por que é linear quando $k = O(n)$.

Exercícios complementares

1.4 [Complementar] resposta: p. 169
Ordene as funções pela taxa de crescimento: $\sqrt{n}$, $\log^2 n$, $2^{\log_2 n}$, $n \log n$, $n^{\log_2 3}$, $1{,}01^n$, n^{100}, $n!$, 2^{2^n}.

1.5 [Complementar] resposta: p. 169
Verdadeiro ou falso? Justifique. (a) $2^{n+1} = O(2^n)$; (b) $2^{2n} = O(2^n)$; (c) $\log(n^2) = \Theta(\log n)$; (d) $(\log n)^{100} = O(n^{0{,}01})$; (e) $n^{\log n} = O(2^n)$; (f) se $f = O(g)$ então $2^f = O(2^g)$.

1.6 [Complementar] resposta: p. 169
Mostre que $\log(n!) = \Theta(n \log n)$ sem usar a fórmula de Stirling.

1.7 [Complementar] resposta: p. 169
Dê funções positivas f e g tais que nem $f = O(g)$ nem $g = O(f)$.

1.8 [Complementar] resposta: p. 169
Dê a complexidade, em Θ, de cada trecho (o corpo mais interno custa $O(1)$):

(a)
```
i := 1
enquanto i ≤ n faça
  para j := 1 até i faça (corpo)
  i := 2i
```

(b)
```
para i := 1 até n faça
  j := 1
  enquanto j ≤ n faça
    (corpo); j := 2j
```

1.9 [Complementar] resposta: p. 169
O algoritmo que testa se N é primo tentando dividir N por $2, 3, \ldots, \lfloor \sqrt{N} \rfloor$ faz $O(\sqrt{N})$ divisões. Ele é polinomial? (Pense no tamanho da entrada.)

1.10 [Complementar] resposta: p. 169
Prove pela definição, exibindo as constantes, que $3n^3 - 100n^2 + 7 = \Theta(n^3)$.

1.11 [Complementar] resposta: p. 169
Prove que $n^2 \neq O(n \log n)$.

1.12 [Complementar] resposta: p. 169
Prove que, para quaisquer constantes $a, b > 1$, $\log_a n = \Theta(\log_b n)$. Por que, por outro lado, a^n e b^n não são da mesma ordem quando $a \neq b$?

1.13 [Complementar] resposta: p. 169
Prove: (a) se $f = O(g)$ e $g = O(h)$, então $f = O(h)$; (b) $f + g = \Theta(\max\{f, g\})$; (c) $f = \Theta(g)$ se e somente se $g = \Theta(f)$.

1.14 [Complementar] resposta: p. 170
Mostre que $\sum_{i=1}^{n} i^k = \Theta(n^{k+1})$ para toda constante inteira $k \geq 1$.

1.15 [Complementar] resposta: p. 170
Mostre que $H_n = \sum_{i=1}^{n} \frac{1}{i} = \Theta(\log n)$, comparando a soma com a integral de $1/x$.

1.16 [Complementar] resposta: p. 170
Na busca sequencial de x num vetor de n elementos distintos, quantas comparações são feitas no pior caso, no melhor caso e no caso médio, supondo que x está no vetor e é igualmente provável em cada posição?

1.17 [Complementar] resposta: p. 170
Quantas vezes o corpo é executado no trecho "**para** $i := 1$ **até** n, **para** $j := i$ **até** n, **para** $k := i$ **até** j: corpo"? Dê a fórmula exata e a ordem em Θ.

1.18 [Complementar] resposta: p. 170
Qual a complexidade do trecho "$j := 2$; **enquanto** $j \leq n$ **faça** $j := j^2$"?

1.19 [Complementar] resposta: p. 170
Mostre que $2^n = o(n!)$ e que $n! = o(n^n)$.

1.20 [Complementar] resposta: p. 170
Quantas comparações a busca binária faz no pior caso num vetor ordenado com 1000 elementos? E com 10^6? Dê a fórmula geral.

1.21 [Complementar] resposta: p. 170
Mostre que todo algoritmo que decide se x está num vetor *não ordenado* de n elementos, usando apenas comparações de x com elementos do vetor, faz pelo menos n comparações no pior caso. Conclua que a busca sequencial é ótima nesse modelo.

1.22 [Complementar] resposta: p. 170
Verdadeiro ou falso? (a) $\max\{f, g\} = \Theta(f + g)$; (b) $f = O(g)$ implica $\log f = O(\log g)$, para $f, g \geq 2$; (c) $f = O(g)$ implica $f^2 = O(g^2)$; (d) $f(n) = O(f(n/2))$ para toda f crescente.

2

Listas de prioridades e heaps binários

Neste capítulo

- O tipo abstrato lista de prioridades e o custo de implementá-lo com listas.
- O heap binário: definição, representação em vetor e as operações *subir* e *descer*.
- Inserção, remoção e alteração de prioridade em $O(\log n)$, e construção em $O(n)$.
- O heapsort e as técnicas para demonstrar propriedades de heaps.

Muitas aplicações precisam, repetidamente, do elemento *mais importante* de uma coleção que muda o tempo todo: o próximo processo a ser executado, o paciente mais grave de uma emergência, o próximo evento de uma simulação, o vértice mais próximo no algoritmo de Dijkstra. O tipo abstrato que atende a essas aplicações é a **lista de prioridades**, e sua implementação clássica é o **heap binário**: um vetor com uma ordem parcial que permite todas as operações em tempo logarítmico e a construção em tempo linear.

2.1 Listas de prioridades

Uma lista de prioridades é uma tabela em que cada elemento tem uma **prioridade**, e que oferece as operações de

- *seleção* do elemento de maior prioridade;
- *inserção* de um novo elemento;
- *remoção* do elemento de maior prioridade;
- *alteração* da prioridade de um elemento;
- *construção* da tabela a partir de um conjunto inicial.

A operação de *fusão* de duas listas será tratada no Capítulo 3.

Exemplo 2.1 (A fila de um pronto-socorro)**.** Num pronto-socorro, cada paciente recebe uma nota de gravidade de 0 a 100 na triagem. Uma sequência típica de eventos é:

evento	operação
chegam Ana (40), Bruno (75) e Carla (20)	construção com $\{40, 75, 20\}$
o médico fica livre	seleção e remoção: atende Bruno (75)
chega Davi (90)	inserção de 90
o estado de Carla piora para 95	alteração: $20 \to 95$
o médico fica livre	seleção e remoção: atende Carla (95)

Note que nunca precisamos da ordem completa dos pacientes, apenas de quem é o *primeiro*. É essa observação que o heap explora.

As implementações mais simples são listas. Numa lista não ordenada, inserir é imediato, mas achar o máximo exige percorrer tudo; numa lista ordenada, o máximo está no início, mas cada inserção precisa encontrar a posição certa e deslocar elementos. Por exemplo, inserir 50 na lista ordenada $[90, 70, 40, 20]$ exige deslocar o 40 e o 20 uma posição para a direita, e no pior caso (inserir um novo máximo) todos os elementos se deslocam. A tabela compara essas opções com o heap.

Operação	Lista não ordenada	Lista ordenada	Heap
seleção	$\Theta(n)$	$\Theta(1)$	$O(1)$
inserção	$\Theta(1)$	$\Theta(n)$	$O(\log n)$
remoção	$\Theta(n)$	$\Theta(1)$	$O(\log n)$
alteração	$\Theta(n)$	$\Theta(n)$	$O(\log n)$
construção	$\Theta(n)$	$\Theta(n \log n)$	$O(n)$

Cada lista tem alguma operação de atualização que custa $\Omega(n)$. O heap abre mão de manter a ordem total e mantém só o necessário para que o máximo esteja sempre num lugar conhecido; com isso, todas as operações ficam logarítmicas.

2.2 Definição e representação

> **Intuição**
>
> Pense num organograma em que cada chefe é mais importante que seus subordinados diretos. Quem está no topo é o mais importante de todos, mas nada se exige entre colegas do mesmo nível, nem entre pessoas de departamentos diferentes. Um heap é exatamente isso: uma hierarquia em que *cada pai domina os seus filhos*, e nada mais.

Definição 2.2 (Heap)**.** Um **heap** (de máximo) é uma sequência $s_1, \ldots, s_n$ tal que

$$s_i \leq s_{\lfloor i/2 \rfloor} \qquad \text{para todo } 1 < i \leq n.$$

Num heap *de mínimo*, a desigualdade é invertida: $s_i \geq s_{\lfloor i/2 \rfloor}$.

A definição fica clara quando se desenha a sequência como uma **árvore binária completa**, numerada por níveis e, em cada nível, da esquerda para a direita: o nó i tem pai $\lfloor i/2 \rfloor$, filho esquerdo $2i$ e filho direito $2i + 1$ (quando esses índices são $\leq n$). A condição diz que *cada nó tem prioridade maior ou igual à de seus filhos*. Não há ponteiros: a árvore é só uma maneira de olhar para o vetor (Figura 2.1).

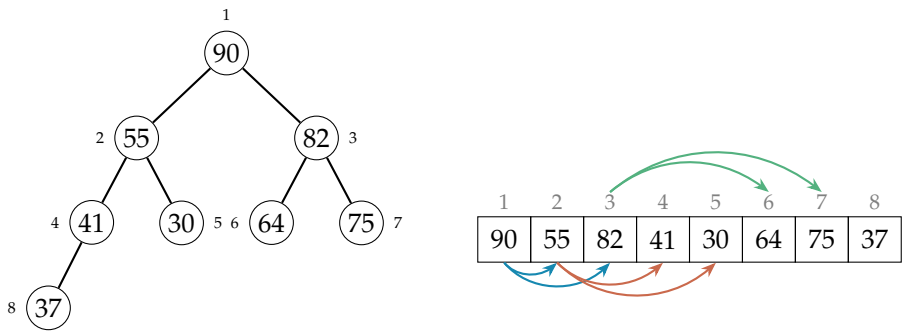


Figura 2.1. O mesmo heap visto como árvore e como vetor. Os números pequenos são os índices; os filhos do nó i estão nas posições $2i$ e $2i + 1$, como indicam as setas desenhadas junto ao vetor.

Exemplo 2.3 (Reconhecendo um heap)**.** Para verificar se uma sequência é um heap, basta comparar cada posição $i \geq 2$ com o seu pai $\lfloor i/2 \rfloor$.

- $[9, 7, 8, 3, 5, 6]$: os pares (filho, pai) são $(7, 9)$, $(8, 9)$, $(3, 7)$, $(5, 7)$ e $(6, 8)$; em todos o filho é menor. É um heap.
- $[9, 5, 8, 6, 4, 7]$: a posição 4 guarda 6 e o seu pai, a posição 2, guarda 5. Como $6 > 5$, **não** é um heap (Figura 2.2). Todos os outros pares estão corretos: basta uma violação.

Observe que no heap $[9, 7, 8, 3, 5, 6]$ os irmãos 7 e 8 aparecem em ordem crescente, e o 6, à direita, é maior que o 3 e o 5, à esquerda: nada disso importa. Só se comparam pais e filhos.

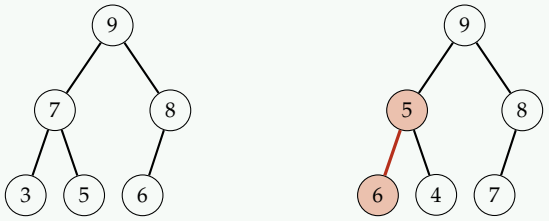


Figura 2.2. À esquerda, um heap. À direita, uma sequência que não é heap: a aresta em vermelho liga o pai 5 a um filho maior, 6.

Alguns fatos sobre os índices aparecem em todas as demonstrações deste capítulo:

- o nó i está no nível $\lfloor \log_2 i \rfloor + 1$ (a raiz está no nível 1), e a árvore tem $\lfloor \log_2 n \rfloor + 1$ níveis;
- o nó i tem filho se e somente se $2i \leq n$; logo as **folhas** são exatamente as posições $\lfloor n/2 \rfloor + 1, \ldots, n$, e há $\lceil n/2 \rceil$ delas;
- neste capítulo, a altura de um nó é medida em níveis: **folhas têm altura 1**.

Exemplo 2.4 (Fazendo contas com índices)**.** Num heap com $n = 10$ elementos: o pai do nó 7 é $\lfloor 7/2 \rfloor = 3$; os filhos do nó 4 são 8 e 9; o nó 5 tem só o filho esquerdo 10, pois $11 > n$; as folhas são as posições $6, \ldots, 10$ (são $\lceil 10/2 \rceil = 5$ folhas); o nó 9 está no nível $\lfloor \log_2 9 \rfloor + 1 = 4$, e a árvore tem $\lfloor \log_2 10 \rfloor + 1 = 4$ níveis.

2.3 As operações subir e descer

Toda operação sobre um heap altera uma posição e depois restaura a propriedade de heap, que pode ter sido violada em apenas um ponto:

- se a prioridade de um nó *aumentou*, ele pode ter ficado maior que o pai, e precisa **subir**;
- se *diminuiu*, pode ter ficado menor que um filho, e precisa **descer**.

subir(i) **– após *aumentar* a prioridade de** i

```
Procedimento SUBIR(i)
    j := ⌊i/2⌋
    se j ≥ 1 então
        se T[i].chave > T[j].chave então
            T[i] ⇔ T[j]
            SUBIR(j)
```

descer(i, n) **– após *diminuir* a prioridade de** i

```
Procedimento DESCER(i, n)
    j := 2i
    se j ≤ n então
        se j < n e T[j + 1].chave > T[j].chave
então
            j := j + 1                    % maior filho
        se T[i].chave < T[j].chave então
            T[i] ⇔ T[j]
            DESCER(j, n)
```

Ambos percorrem um único caminho da árvore, logo custam $O(\log n)$. Um detalhe essencial do *descer* é que ele troca o nó com o **maior** dos filhos: se trocasse com o menor, o maior filho

ficaria abaixo de um valor menor que ele, e a propriedade continuaria violada.

Exemplo 2.5 (Por que trocar com o maior filho)**.** Suponha que a raiz de $[4, 9, 7]$ precise descer. Trocando com o maior filho, 9, obtemos $[9, 4, 7]$, um heap. Trocando com o menor, 7, obteríamos $[7, 9, 4]$, e agora o filho 9 é maior que o pai 7: a violação não foi corrigida, só mudou de lugar.

2.4 Inserção, remoção e alteração de prioridade

Com *subir* e *descer*, as demais operações são imediatas:

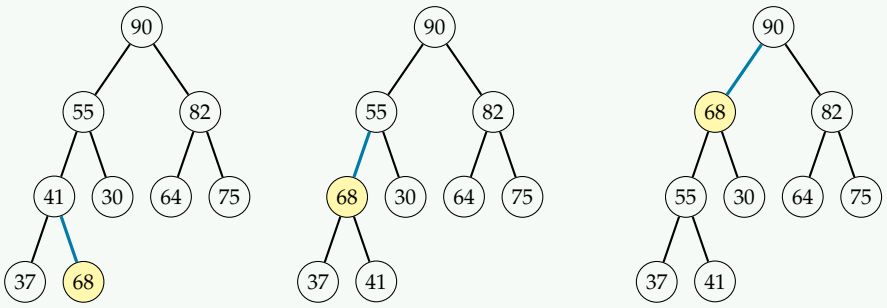
- para **inserir**, o novo elemento ocupa a primeira posição livre do vetor, a $n + 1$, e sobe;
- para **remover o máximo**, o último elemento é copiado para a raiz e desce;
- para **alterar** uma prioridade, o elemento sobe ou desce conforme a prioridade tenha aumentado ou diminuído.

Em todos os casos o *formato* da árvore continua o de uma árvore completa (só a última posição é criada ou apagada), e a ordem é consertada ao longo de um único caminho.

Inserção, remoção do máximo e alteração de prioridade

```
Procedimento INSERIR(novo)                          % M é a capacidade do vetor
    se n < M então
        T[n + 1] := novo; n := n + 1; SUBIR(n)
    senão overflow
Procedimento REMOVER
    se n ≠ 0 então
        processar(T[1]); T[1] := T[n]; n := n − 1; DESCER(1, n)
    senão underflow
Procedimento ALTERAR(i, p)
    antiga := T[i].chave; T[i].chave := p
    se p > antiga então
        SUBIR(i)                                    % aumentou: sobe
    senão
        DESCER(i, n)                                % diminuiu: desce
```

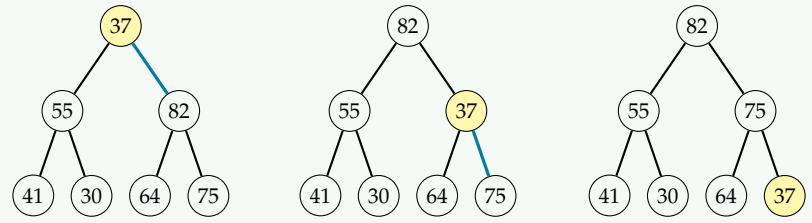
Exemplo 2.6 (Inserção passo a passo)**.** Inserimos 68 no heap $[90, 55, 82, 41, 30, 64, 75, 37]$ da Figura 2.1. Ele entra na posição 9, como filho de 41. Como $68 > 41$, troca com o pai; na posição 4, $68 > 55$, e troca de novo; na posição 2, $68 < 90$, e para. O resultado é $[90, 68, 82, 55, 30, 64, 75, 37, 41]$. Foram duas trocas, ao longo do caminho de uma folha até a raiz.



(a) entra na posição 9 (b) troca com 41 (c) troca com 55; $68 < 90$: fim

Figura 2.3. Inserção de 68: o novo elemento (em destaque) entra na primeira posição livre e sobe enquanto for maior que o pai. A aresta azul é a comparação seguinte.

Exemplo 2.7 (Remoção passo a passo)**.** Removemos o máximo de $H = [90, 55, 82, 41, 30, 64, 75, 37]$. O último elemento, 37, vai para a raiz, formando $[37, 55, 82, 41, 30, 64, 75]$. Agora 37 desce: os filhos são 55 e 82, e ele troca com o maior, 82; na posição 3, os filhos são 64 e 75, e ele troca com 75; na posição 7, é folha, e para. Resultado: $[82, 55, 75, 41, 30, 64, 37]$.



(a) 37 vai para a raiz (b) troca com 82 (c) troca com 75: fim

Figura 2.4. Remoção do máximo: o último elemento ocupa a raiz e desce, sempre trocando com o maior filho.

Exemplo 2.8 (Alteração de prioridade)**.** No mesmo heap $H = [90, 55, 82, 41, 30, 64, 75, 37]$:

- **aumentar** a prioridade do nó 6 de 64 para 95: ele sobe trocando com 82 (posição 3) e com 90 (posição 1), e o heap fica $[95, 55, 90, 41, 30, 82, 75, 37]$;
- **diminuir** a raiz de 90 para 33: ela desce trocando com 82 e com 75: $[82, 55, 75, 41, 30, 64, 33, 37]$.

Subir ou descer?

É comum inverter as duas operações ao alterar uma prioridade, escrevendo algo como "**se** $H[i] > H[\lfloor i/2 \rfloor]$ **então** descer(i) **senão** subir(i)". O correto é o contrário: se o novo valor ficou maior que o pai, a violação é *com o pai*, e o elemento precisa **subir**; se não ficou, a única violação possível é com os filhos, e ele precisa **descer**. Em pseudocódigo: **se** $i > 1$ **e** $H[i] > H[\lfloor i/2 \rfloor]$ **então** subir(i) **senão** descer(i, n).

2.5 Construção em tempo linear

Para transformar um vetor arbitrário num heap, a primeira ideia é inseri-lo elemento a elemento: **para** $i := 2, \ldots, n$ **faça** subir(i). Isso custa $O(n \log n)$, e de fato $\Theta(n \log n)$ no pior caso, por exemplo quando a entrada é crescente e cada elemento sobe até a raiz.

Uma ideia melhor é trabalhar de baixo para cima. As folhas, isto é, as posições $i > \lfloor n/2 \rfloor$, já são heaps de um único elemento. Percorrendo os demais nós do último para o primeiro, cada um encontra as duas subárvores dos filhos já organizadas como heaps, e basta fazê-lo descer.

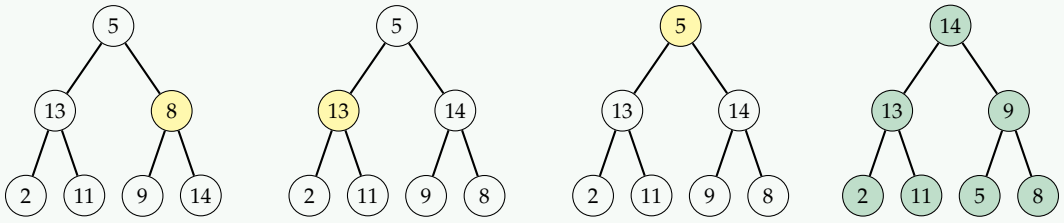
arranjar(n): construção do heap

```
Procedimento ARRANJAR(n)
    para i := ⌊n/2⌋ decrescendo até 1 faça
        DESCER(i, n)
```

A correção segue do invariante "antes de processar i, as subárvores com raízes $i+1, \ldots, n$ são heaps".

Exemplo 2.9 (O *arranjar* passo a passo)**.** Construímos um heap a partir de $[5, 13, 8, 2, 11, 9, 14]$, com $n = 7$. As folhas são as posições $4, \ldots, 7$; processamos $i = 3, 2, 1$ (Figura 2.5).

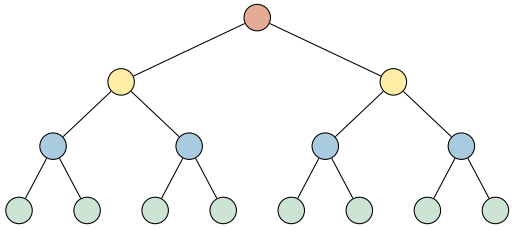
- $i = 3$ (valor 8): os filhos são 9 e 14; troca com 14. Vetor: $[5, 13, 14, 2, 11, 9, 8]$.
- $i = 2$ (valor 13): os filhos são 2 e 11, ambos menores; nada muda.
- $i = 1$ (valor 5): os filhos são 13 e 14; troca com 14, e na posição 3 os filhos são 9 e 8; troca com 9. Vetor final: $[14, 13, 9, 2, 11, 5, 8]$.



(a) $i = 3$ (b) $i = 2$: nada muda (c) $i = 1$ (d) heap pronto

Figura 2.5. O *arranjar* sobre $[5, 13, 8, 2, 11, 9, 14]$. Em cada passo, o nó processado (em destaque) desce enquanto for menor que o maior filho; as subárvores abaixo dele já são heaps.

O ponto surpreendente é o custo: a maioria dos nós está perto das folhas e desce pouco. A Figura 2.6 mostra a conta numa árvore cheia com 15 nós.



altura 4: 1 nó × até 3 trocas

altura 3: 2 nós × até 2 trocas

altura 2: 4 nós × até 1 troca

altura 1: 8 folhas × 0 trocas

Figura 2.6. Numa árvore completa, metade dos nós são folhas e não descem nada; um quarto desce no máximo um nível; e assim por diante. Com 15 nós, o *arranjar* faz no máximo $4 \cdot 1 + 2 \cdot 2 + 1 \cdot 3 = 11$ trocas, menos que n.

Lema 2.10. *Numa árvore binária completa com n nós (folhas com altura 1), no máximo $\lceil n/2^j \rceil$ nós têm altura j.*

Demonstração. Numa árvore completa, o caminho mais longo que desce de i é o da esquerda: $i \to 2i \to 4i \to \cdots$. Logo o nó i tem altura $\geq j$ se e somente se o seu descendente $j-1$ níveis abaixo existe, isto é, $i\,2^{j-1} \leq n$, ou $i \leq \lfloor n/2^{j-1} \rfloor$. Assim, com $x = n/2^{j-1}$ e $m = \lfloor x \rfloor$,

$$\#\{\text{altura} = j\} = \lfloor x \rfloor - \lfloor x/2 \rfloor = m - \lfloor m/2 \rfloor = \lceil m/2 \rceil \leq \lceil x/2 \rceil = \lceil n/2^j \rceil. \qquad \square$$

Teorema 2.11. *O procedimento arranjar(n) roda em tempo $O(n)$.*

Demonstração. Num nó de altura j, descer faz no máximo $j-1$ trocas. Pelo Lema 2.10, o total de trocas é

$$\sum_{j=2}^{h} (j-1)\left\lceil \frac{n}{2^j} \right\rceil \;\leq\; \sum_{j \geq 2} (j-1)\left(\frac{n}{2^j} + 1\right) \;=\; n \sum_{j \geq 2} \frac{j-1}{2^j} + O(\log^2 n) \;=\; n + O(\log^2 n) = O(n),$$

pois $\sum_{j \geq 1} \frac{j-1}{2^j} = \sum_{j \geq 1} \frac{j}{2^j} - \sum_{j \geq 1} \frac{1}{2^j} = 2 - 1 = 1$. $\square$

A intuição é a da Figura 2.6: metade dos nós (as folhas) não custa nada, um quarto custa no máximo uma troca, um oitavo no máximo duas, e assim por diante; só a raiz pode descer $\log n$ níveis. Na construção por inserções sucessivas acontece o contrário: os muitos nós de baixo são justamente os que podem subir mais.

Exemplo 2.12 (Mais duas construções)**.**

- $[12, 30, 21, 8, 27, 40, 16, 35, 19]$, com $n = 9$, processa $i = 4, 3, 2, 1$. Em $i = 4$, troca $8 \leftrightarrow 35$; em $i = 3$, $21 \leftrightarrow 40$; em $i = 2$, $30 \leftrightarrow 35$, e o 30 para na posição 4, pois os seus novos filhos, 8 e 19, são menores; em $i = 1$, $12 \leftrightarrow 40$ e depois $12 \leftrightarrow 21$. Resultado: $[40, 35, 21, 30, 27, 12, 16, 8, 19]$.
- $[6, 9, 14, 3, 12, 10, 2]$ processa $i = 3, 2, 1$: nada muda em $i = 3$, pois 14 já é maior que 10 e 2; em $i = 2$, $9 \leftrightarrow 12$; em $i = 1$, $6 \leftrightarrow 14$ e depois $6 \leftrightarrow 10$. Resultado: $[14, 12, 10, 3, 9, 6, 2]$.

2.6 Heapsort

O heap leva a um algoritmo de ordenação simples: construa o heap e, repetidamente, troque a raiz (o maior elemento restante) com a última posição da parte ainda não ordenada e faça a nova raiz descer. O vetor fica dividido em duas partes: à esquerda, um heap que encolhe; à direita, a parte já ordenada, que cresce.

Heapsort

```
Procedimento HEAPSORT(T, n)
    ARRANJAR(n)                                  % O(n)
    para m := n decrescendo até 2 faça
        T[1] ⇔ T[m]                              % o máximo vai para o fim
        DESCER(1, m − 1)                         % O(log m)
```

Exemplo 2.13 (Heapsort de $[5, 2, 9, 4, 7]$)**.** O *arranjar* processa $i = 2$ (troca $2 \leftrightarrow 7$) e $i = 1$ (troca $5 \leftrightarrow 9$), e o heap é $[9, 7, 5, 4, 2]$. Em seguida, a cada passo o máximo vai para o fim e a nova raiz desce (Figura 2.7).

9	7	5	4	2	heap inicial
7	4	5	2	9	$9 \leftrightarrow 2$; o 2 desce: troca com 7 e com 4
5	4	2	7	9	$7 \leftrightarrow 2$; o 2 desce: troca com 5
4	2	5	7	9	$5 \leftrightarrow 2$; o 2 desce: troca com 4
2	4	5	7	9	$4 \leftrightarrow 2$: ordenado

Figura 2.7. Heapsort de $[5, 2, 9, 4, 7]$. A parte verde já está na posição final; a parte branca é um heap. Em cada linha, a raiz trocou de lugar com a última posição do heap e depois desceu.

O custo é $O(n) + \sum_m O(\log m) = \Theta(n \log n)$ em *todos* os casos, mesmo com a entrada já ordenada. O heapsort é **in-place** (só faz trocas dentro do vetor) e **não é estável**, pois a troca da raiz com o último elemento pode inverter a ordem relativa de elementos iguais. Como faz $O(n \log n)$ comparações, é ótimo entre os algoritmos baseados em comparações (Capítulo 6). Por exemplo, ele transforma $[12, 30, 21, 8, 27, 40, 16, 35, 19]$ em $[8, 12, 16, 19, 21, 27, 30, 35, 40]$.

2.7 Propriedades de heaps e suas demonstrações

Como demonstrar propriedades de heaps

Quase toda demonstração sobre heaps usa só três ferramentas: (i) a definição $s_i \leq s_{\lfloor i/2 \rfloor}$ combinada com a **transitividade ao longo de um caminho**; (ii) os **fatos sobre índices** (pai, filhos, folhas, níveis); (iii) **indução** na altura, no nível ou no número de nós. Para "provar ou dar contraexemplo", teste primeiro exemplos pequenos, com $n \leq 8$.

Proposição 2.14 (O ancestral domina)**.** *Se j é ancestral de i num heap, então $s_i \leq s_j$. Em particular, todo caminho da raiz até uma folha é não crescente, a* ***raiz é o máximo*** *e* ***toda subárvore é um heap****.*

Demonstração. O caminho de i até j é $i, \lfloor i/2 \rfloor, \lfloor i/4 \rfloor, \ldots, j$. Aplicando a definição em cada aresta e a transitividade de $\leq$, obtemos $s_i \leq s_{\lfloor i/2 \rfloor} \leq \cdots \leq s_j$. A raiz é ancestral de todos os nós. A condição de heap numa subárvore é a mesma condição, restrita a ela. □

Por exemplo, no heap da Figura 2.1, o caminho da raiz até a posição 8 é $90 \geq 55 \geq 41 \geq 37$.

Proposição 2.15 (O mínimo está numa folha)**.** *Num heap de máximo com chaves distintas, o menor elemento está numa folha, isto é, numa posição maior que $\lfloor n/2 \rfloor$.*

Demonstração. Se i não é folha, tem o filho $2i$ e $s_{2i} < s_i$, pois as chaves são distintas; logo s_i não é o mínimo. Com repetições, *algum* mínimo está numa folha. □

No heap da Figura 2.1, as folhas são as posições $5, \ldots, 8$, com valores $30, 64, 75, 37$, e o mínimo, 30, está entre elas. Note que o mínimo pode estar em *qualquer* folha, não necessariamente na última posição.

Proposição 2.16 (Onde está o k-ésimo maior)**.** *Com chaves distintas, o k-ésimo maior elemento está num nível $\leq k$, isto é, numa posição $\leq 2^k - 1$. Em particular, o segundo maior é filho da raiz (posição 2 ou 3), e o terceiro maior está entre as posições 2 e 7.*

Demonstração. Pela Proposição 2.14, todos os ancestrais de x são maiores que x. Se x é o k-ésimo maior, existem só $k-1$ elementos maiores que ele, e portanto x tem no máximo $k-1$ ancestrais: está no nível $\leq k$. As posições dos níveis $1, \ldots, k$ são $1, \ldots, 2^k - 1$. □

No heap da Figura 2.1, o segundo maior, 82, está na posição 3, e o terceiro maior, 75, na posição 7, no limite permitido.

Proposição 2.17 (Altura)**.** *Um heap com n elementos tem altura $\lfloor \log_2 n \rfloor + 1$ (em níveis), que é $\Theta(\log n)$. Por isso subir, descer, inserir e remover custam $O(\log n)$.*

Demonstração. O último nó, n, está no nível $\lfloor \log_2 n \rfloor + 1$, e numa árvore completa todos os níveis anteriores estão cheios: $2^{h-1} \leq n \leq 2^h - 1$, logo $h = \lfloor \log_2 n \rfloor + 1$. □

Um heap com um milhão de elementos tem apenas $\lfloor \log_2 10^6 \rfloor + 1 = 20$ níveis: nenhuma operação faz mais que 19 trocas.

Proposição 2.18 (Heaps e ordenação)**.** *Toda sequência em ordem não crescente é um heap; a recíproca é falsa, como mostra $[3, 1, 2]$. Construir um heap é, portanto, mais fácil que ordenar: $O(n)$ contra $\Omega(n \log n)$. E, como o heapsort ordena com uma construção e n remoções, nenhuma implementação de heap baseada em comparações pode ter construção* e *remoção com custo amortizado $o(\log n)$; caso contrário, ordenaríamos em $o(n \log n)$.*

Proposição 2.19 (Troca de dois elementos)**.** *Sejam S um heap e $i < j$. Trocar s_i com s_j não produz necessariamente um heap, nem quando $s_i < s_j$ nem quando $s_i > s_j$.*

Contraexemplos. Para $s_i < s_j$: $S = [10, 9, 1, 8, 7, 0, 0, 6]$ é um heap; trocando $s_3 = 1$ com $s_4 = 8$, obtemos $[10, 9, 8, 1, 7, 0, 0, 6]$, em que a posição 8 (6) tem pai na posição 4 (1). O valor pequeno foi para uma posição cujos filhos podem ser maiores que ele. Para $s_i > s_j$: em $S = [10, 9]$, trocar as duas posições dá $[9, 10]$, com o filho maior que o pai. □

Proposição 2.20 (Número de heaps)**.** *Seja $H(n)$ o número de permutações de $1, \ldots, n$ que são heaps (o mesmo vale para heaps de mínimo). Então $H(0) = H(1) = 1$ e*

$$H(n) = \binom{n-1}{L} H(L)\, H(n-1-L),$$

onde L é o número de nós da subárvore esquerda da raiz: com $h = \lfloor \log_2 n \rfloor$ e $u = n - (2^h - 1)$ nós no último nível, $L = (2^{h-1} - 1) + \min\{u, 2^{h-1}\}$. Os valores para $n = 1, \ldots, 10$ são $1, 1, 2, 3, 8, 20, 80, 210, 896, 3360$.

Demonstração. A raiz é necessariamente n. O formato da árvore é fixo, então L é determinado por n. Escolhem-se quais L dos $n-1$ valores restantes vão para a esquerda, de $\binom{n-1}{L}$ modos; cada lado precisa ser um heap sobre os seus valores, e o número de heaps depende só da quantidade de valores. □

> **Exemplo 2.21** (Os três heaps de $\{1, 2, 3, 4\}$)**.** Com $n = 4$, a raiz é 4, a subárvore esquerda tem as posições 2 e 4 ($L = 2$) e a direita, só a posição 3. Escolhemos quais 2 dos valores $\{1, 2, 3\}$ vão para a esquerda, de $\binom{3}{2} = 3$ modos, e cada escolha dá um único heap: $\{1, 2\}$ dá $[4, 2, 3, 1]$; $\{1, 3\}$ dá $[4, 3, 2, 1]$; $\{2, 3\}$ dá $[4, 3, 1, 2]$.

Proposição 2.22 (Correção de subir e descer)**.** *(a) Se T era um heap e a chave de i aumentou, após subir(i) T volta a ser um heap. (b) Se as subárvores dos filhos de i são heaps, após descer(i, n) a subárvore de i é um heap.*

Esboço. (a) Invariante: a única violação possível é entre o nó corrente e o seu pai. Ao trocar com o pai j, o antigo valor de j desce para a posição corrente e é maior ou igual aos filhos dela, que já eram seus descendentes; o novo valor em j é maior que o antigo, e não viola nada com os filhos de j. Repete-se para cima. (b) Invariante: a única violação é entre o nó corrente e os seus filhos. Trocando com o *maior* filho, este passa a dominar o outro filho e o valor que desceu. A violação desce um nível e termina numa folha ou quando deixa de existir. □

Pontos-chave

Heap = vetor com "pai ≥ filho". Sobe-se após um aumento e desce-se após uma diminuição, sempre trocando com o maior filho. Inserir = colocar no fim e subir; remover = levar o último à raiz e descer. A construção é linear porque há no máximo $\lceil n/2^j \rceil$ nós de altura j. O heapsort é $\Theta(n \log n)$, in-place e não estável.

2.8 Exercícios

Exercícios do curso

2.1 [Lista] resposta: p. 170
Verifique se as sequências correspondem a um heap binário (de máximo):
(a) 33 32 28 31 26 29 25 30 27; (b) 33 32 28 31 29 26 25 30 27.

2.2 [Lista] resposta: p. 170
Prove ou dê contraexemplo: se S é um heap e s_i, s_j são chaves com $i < j$ e $s_i < s_j$, então trocar s_i com s_j produz um heap.

2.3 [Lista] resposta: p. 170
Repita o exercício anterior com $i < j$ e $s_i > s_j$.

2.4 [Lista] resposta: p. 170
Determine o número de permutações de $1, \ldots, n$ que são heaps de mínimo.

2.5 [Lista] resposta: p. 170
Aplique o algoritmo linear de construção à lista 18 25 41 34 14 10 52 50 48.

2.6 [Lista] resposta: p. 170
Prove que no máximo $\lceil n/2^j \rceil$ nós têm altura j numa árvore binária completa com n nós.

2.7 [Lista] resposta: p. 170
Escreva os algoritmos de inserção, remoção, aumento e diminuição de prioridade e construção de heaps binários **sem recursão**.

2.8 [Monitoria] resposta: p. 171
(a) Defina heap e a propriedade que $s_1, \ldots, s_n$ satisfazem. (b) Por que o elemento de maior prioridade é selecionado em $O(1)$? (c) Dê a complexidade de pior caso de seleção, inserção, remoção do máximo, alteração de prioridade e construção.

2.9 [Monitoria] resposta: p. 171
No heap $H = [95, 60, 78, 39, 28, 66, 70, 33]$: (a) a prioridade 66 passa a 98; (b) remove-se a raiz. Construa, pelo procedimento baseado em *descer*, heaps a partir de (c) $[20, 15, 30, 40, 10, 25, 5]$ e (d) $[7, 2, 15, 9, 1, 20, 13]$. Mostre as trocas em cada caso.

2.10 [Monitoria] resposta: p. 171
Um aluno escreveu: "$H[i] := nova$; **se** $H[i] > H[\lfloor i/2 \rfloor]$ **então** Descer(i) **senão** Subir(i)". Qual é o erro? Corrija e explique.

2.11 [Slides] resposta: p. 171
Problema *Telemarketing* (Beecrowd 2330): n vendedores, numerados de 1 a n, atendem L ligações em ordem; cada ligação vai para o vendedor livre de menor número, que fica ocupado pela duração da ligação. Quantas ligações cada vendedor atende? Projete a solução com heap e dê a complexidade.

Exercícios complementares

2.12 [Complementar] resposta: p. 171
Num heap d-ário (cada nó tem até d filhos), em vetor indexado a partir de 1: (a) quais são os índices do pai e dos filhos do nó i? (b) Qual a altura? (c) Quanto custam *subir* e *descer*? (d) Por que um d maior pode ser bom quando há muito mais aumentos de prioridade do que remoções?

2.13 [Complementar] resposta: p. 171
Dadas k listas ordenadas com n elementos no total, intercale-as numa única lista ordenada em $O(n \log k)$.

2.14 [Complementar] resposta: p. 171
Um fluxo de n números chega um a um, e você tem memória $O(k)$. Como obter os k maiores ao final? Qual o tempo?

2.15 [Complementar] resposta: p. 171
Escreva um algoritmo $O(n)$ que decide se um vetor é um heap.

2.16 [Complementar] resposta: p. 171
Mostre, com um exemplo de 3 elementos, que o heapsort não é estável.

2.17 [Complementar] resposta: p. 171
Mostre que construir um heap de máximo inserindo $1, 2, \ldots, n$, nessa ordem, por inserções sucessivas custa $\Theta(n \log n)$, enquanto o *arranjar* sobre o mesmo vetor custa $O(n)$.

2.18 [Complementar] resposta: p. 171
Insira $15, 40, 25, 50, 10, 45$, nessa ordem, num heap de máximo inicialmente vazio. Mostre o vetor após cada inserção.

2.19 [Complementar] resposta: p. 171
Remova o máximo duas vezes do heap $[90, 70, 80, 30, 60, 50, 40, 10, 20]$, mostrando o vetor após cada remoção.

2.20 [Complementar] resposta: p. 171
Execute o heapsort sobre $[4, 10, 3, 5, 1]$, mostrando o vetor após o *arranjar* e após cada passo do laço principal.

2.21 [Complementar] resposta: p. 171
Escreva uma operação que remove o elemento da posição i (não necessariamente a raiz) de um heap em $O(\log n)$. Por que, depois de pôr o último elemento na posição i, às vezes é preciso subir e às vezes descer?

2.22 [Complementar] resposta: p. 172
Mostre como manter a mediana de um conjunto que recebe inserções, com inserção em $O(\log n)$ e consulta à mediana em $O(1)$, usando um heap de máximo e um heap de mínimo.

2.23 [Complementar ★] resposta: p. 172
Mostre que o *arranjar* faz no máximo $2n$ comparações entre chaves (para n grande), contando as comparações feitas em cada passo do *descer*.

3

Heaps binomiais, heaps de Fibonacci e análise amortizada

Neste capítulo

- Árvores binomiais e o heap binomial, cuja fusão imita a soma de números binários.
- A análise amortizada pelos métodos do banqueiro e do potencial.
- O heap de Fibonacci: a ideia de adiar trabalho, as marcas e os cortes em cascata, e por que o grau dos nós é logarítmico.
- Como escolher entre os heaps binário, binomial e de Fibonacci.

O heap binário do capítulo anterior tem um ponto fraco: **unir** duas listas de prioridades exige concatenar os vetores e reconstruir o heap, o que custa $\Theta(n)$. Imagine um sistema operacional com uma fila de processos por processador: quando um processador é desligado, a sua fila precisa ser fundida com a de outro. Em aplicações com muitas fusões, pagar $\Theta(n)$ a cada uma é inviável.

Este capítulo apresenta duas estruturas em que a fusão é barata. O **heap binomial** representa a lista de prioridades como uma coleção de árvores cujas ordens correspondem aos bits de n, e funde duas listas como quem soma dois números binários, em $O(\log n)$ no pior caso. O **heap de Fibonacci** vai além: adia toda a arrumação para o momento da remoção do máximo, e só é eficiente quando se considera o custo de uma *sequência* de operações. Por isso a análise amortizada, que formaliza esse tipo de garantia, aparece aqui.

3.1 Árvores binomiais

Definição 3.1. A árvore binomial B_0 é um único nó. Para $k \geq 1$, B_k é obtida **subordinando** (operação *link*) uma B_{k-1} a outra B_{k-1}: a raiz de uma passa a ser filha da raiz da outra. Num heap, a raiz de *maior* prioridade fica por cima.

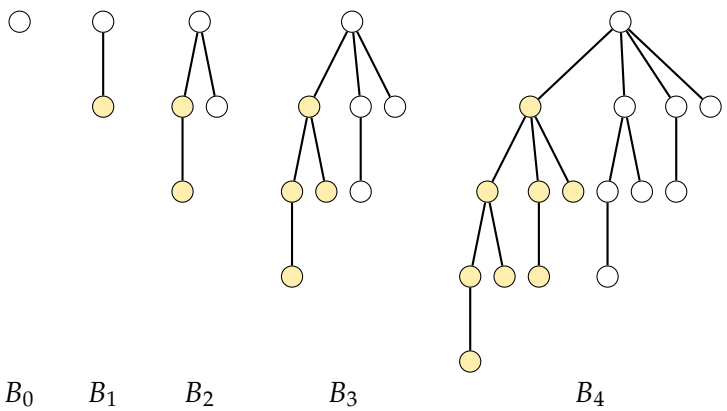


Figura 3.1. As árvores binomiais B_0 a B_4. Em cada B_k, a subárvore em destaque é a cópia de B_{k-1} que foi subordinada à outra.

Proposição 3.2. *Para todo $k \geq 0$:*

1. *B_k tem exatamente 2^k nós;*
2. *o grau da raiz, chamado **ordem** de B_k, é k, e é o maior grau da árvore;*
3. *a altura de B_k, medida em arestas, é k;*
4. *há exatamente $\binom{k}{j}$ nós na profundidade j;*
5. *os filhos da raiz de B_k são raízes de árvores $B_{k-1}, B_{k-2}, \ldots, B_1, B_0$.*

Demonstração. Todas as propriedades seguem por indução em k, olhando para o *link* que forma B_k a partir de duas cópias de B_{k-1}: o número de nós dobra ($2 \cdot 2^{k-1}$); a raiz ganha um filho; a árvore subordinada fica um nível abaixo, e a altura passa a $\max\{k-1,\ 1+(k-1)\} = k$; os nós na profundidade j vêm da cópia de cima (profundidade j) e da de baixo (profundidade $j-1$), e $\binom{k-1}{j} + \binom{k-1}{j-1} = \binom{k}{j}$ — daí o nome "binomial"; e o novo filho da raiz é uma B_{k-1}, que se junta aos filhos $B_{k-2}, \ldots, B_0$ que a raiz já tinha. □

Exemplo 3.3 (Conferindo as propriedades em B_3)**.** Olhe para B_3 na Figura 3.1. Ele tem $8 = 2^3$ nós; a raiz tem 3 filhos e nenhum outro nó tem grau maior; o caminho mais longo da raiz até uma folha tem 3 arestas. Contando os nós por profundidade, obtemos $1, 3, 3, 1$, que são os coeficientes $\binom{3}{0}, \binom{3}{1}, \binom{3}{2}, \binom{3}{3}$ — a linha 3 do triângulo de Pascal. Por fim, os filhos da raiz, da esquerda para a direita, são raízes de uma B_2 (4 nós), uma B_1 (2 nós) e uma B_0 (1 nó).

Duas convenções de altura

A altura de B_k é k quando medida em arestas, e $k+1$ quando medida em níveis (com folhas de altura 1, como nos heaps binários do Capítulo 2). É a mesma árvore. Em exercícios sobre alturas, declare a convenção usada. Neste livro, a ordem de B_k é o grau da raiz, k, e a altura de B_k é k arestas.

3.2 Heap binomial

Definição 3.4. Um **heap binomial** é uma floresta H de árvores binomiais em que (1) cada árvore satisfaz a condição de heap (filho ≤ pai) e (2) as árvores têm **ordens distintas**, isto é, há no máximo uma árvore de cada ordem. As raízes ficam numa lista, em ordem crescente de ordem.

Proposição 3.5. *(a) Para cada n existe um único formato de heap binomial com n nós: B_k pertence a H se e somente se o bit k da representação binária de n é 1. (b) Portanto H tem no máximo $\lfloor \log_2 n \rfloor + 1$ árvores, cada uma de altura $\leq \lfloor \log_2 n \rfloor$.*

Demonstração. A árvore B_k contribui com 2^k nós e as ordens são distintas, logo $n = \sum_{B_k \in H} 2^k$: é a representação binária de n, que é única. Um número n tem $\lfloor \log_2 n \rfloor + 1$ bits. □

Exemplo 3.6 (Um heap binomial com 13 elementos)**.** Como $13 = 1101_2 = 8 + 4 + 1$, um heap binomial com 13 elementos tem exatamente as árvores B_0, B_2 e B_3 (Figura 3.2). Do mesmo modo, $26 = 11010_2$ dá $H = \{B_1, B_3, B_4\}$. O máximo do heap é a maior das raízes, aqui o 50.

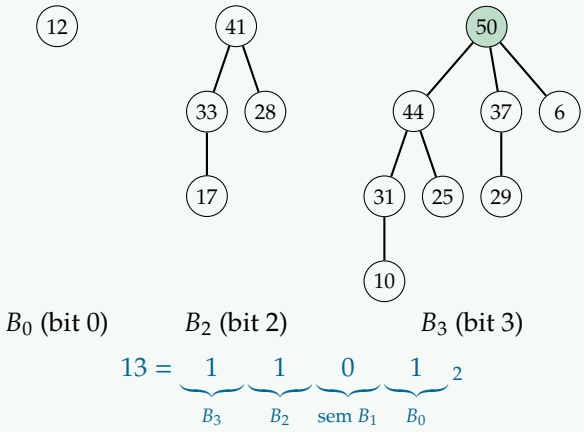


Figura 3.2. Um heap binomial com $13 = 1101_2$ elementos. Cada árvore é um heap (pai ≥ filhos), e há uma árvore para cada bit 1 de n. A lista de raízes está em ordem crescente de ordem.

3.2.1 Operações

A operação central é a **fusão**, e as demais são escritas em termos dela.

Link e fusão de heaps binomiais

Procedimento Link(y, z) % y e z são raízes de árvores B_{k-1}; z fica por cima
  $pai[y] \leftarrow z$; $irmo[y] \leftarrow filho[z]$; $filho[z] \leftarrow y$; $grau[z] \leftarrow grau[z] + 1$
Procedimento Fusão(H', H'')
  $H := H' \cup H''$ % intercala as listas de raízes por ordem crescente: $O(\log n)$
  enquanto H contém duas árvores B, B' de mesma ordem **faça** % a menor ordem repetida primeiro
    se $B.raiz.chave \geq B'.raiz.chave$ **então**
      subordinar B' a B
    senão
      subordinar B a B'

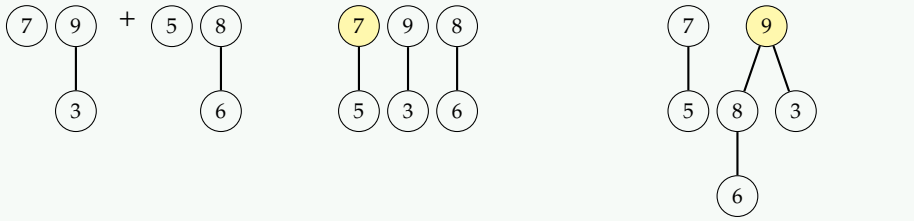
Fundir é somar em binário

Na soma $011_2 + 011_2$, a coluna das unidades tem dois 1, que viram um 0 e um "vai um" para a coluna seguinte. Na fusão, duas árvores B_0 viram uma B_1, que "vai" para a posição das árvores de ordem 1. Ali podem se encontrar até três árvores de mesma ordem (duas originais e a que veio da posição anterior): uma fica, e as outras duas se juntam e vão para a posição seguinte, exatamente como $1 + 1 + 1 = 11_2$.

Há $O(\log n)$ posições e, em cada uma, no máximo três árvores de mesma ordem ao mesmo tempo, de modo que a fusão custa $O(\log n)$ no pior caso. Numa implementação com ponteiros, a lista de raízes é percorrida mantendo o nó anterior, o atual e o próximo, e há quatro casos: ordens diferentes, três árvores seguidas de mesma ordem, e duas árvores de mesma ordem em que vence a atual ou a seguinte.

Exemplo 3.7 (Fusão passo a passo)**.** Sejam $H_1 = \{7,\ 9 \to 3\}$ e $H_2 = \{5,\ 8 \to 6\}$, cada um com $3 = 11_2$ elementos: uma B_0 e uma B_1 (Figura 3.3).

1. Ordem 0: há duas B_0, com raízes 7 e 5. O link põe a maior por cima e forma a B_1 $7 \to 5$ (o "vai um").
2. Ordem 1: agora há três B_1: $7 \to 5$, $9 \to 3$ e $8 \to 6$. A primeira fica; as outras duas se juntam, com 9 por cima, numa B_2.
3. O resultado é $\{B_1, B_2\}$, com $6 = 110_2$ elementos: de fato, $11_2 + 11_2 = 110_2$.



(a) H_1 e H_2 (b) $B_0 + B_0 \to B_1$ (c) duas das três B_1 viram B_2

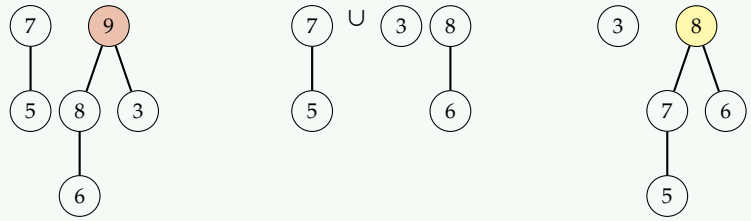
Figura 3.3. A fusão de H_1 com H_2 (Exemplo 3.7) imita a soma $11_2 + 11_2 = 110_2$.

Exemplo 3.8 (Fusão de $H_1 = \{B_1, B_2, B_3\}$ com $H_2 = \{B_0, B_1, B_3\}$)**.** Olhando apenas para os formatos: os heaps têm $14 = 1110_2$ e $11 = 1011_2$ elementos. A B_0 de H_2 fica sozinha na ordem 0. Na ordem 1, as duas B_1 formam uma B_2 (o "vai um"); na ordem 2, essa B_2 se junta à B_2 de H_1 e forma uma B_3; na ordem 3 há, então, três árvores — as B_3 de H_1 e de H_2 e a que acabou de se formar —, e, como na soma $1 + 1 + 1 = 11_2$, uma delas fica e as outras duas formam uma B_4. São três links, e o resultado é $\{B_0, B_3, B_4\}$, isto é, $25 = 11001_2 = 1110_2 + 1011_2$.

As demais operações são as seguintes.

- **Inserção**: fusão com um heap $\{B_0\}$ que contém só o novo elemento. No pior caso custa $O(\log n)$, quando $n = 2^k - 1$, todos os bits são 1 e há k links. Mas inserir equivale a somar 1 a um contador binário, e cada link corresponde a um "vai um". Se os bits de n são equiprováveis, a inserção faz em média um link, pois a posição j do primeiro 0 de n (lido da direita para a esquerda) tem esperança $\sum_{j \geq 1} j/2^j = 2$. E, ao longo de uma sequência de inserções, o custo **amortizado** é $O(1)$, como mostra o exemplo do contador binário da §3.3.
- **Seleção do máximo**: o máximo é uma das no máximo $\lfloor \log n \rfloor + 1$ raízes, e é encontrado em $O(\log n)$, ou em $O(1)$ se um ponteiro para a maior raiz for mantido.
- **Remoção do máximo**: encontre a raiz máxima r, numa árvore B_k, e retire B_k de H, formando $H' = H - B_k$. Removendo r, seus filhos $B_{k-1}, \ldots, B_0$ formam um heap binomial H''. Faça a fusão $H' \cup H''$. Total: $O(\log n)$.
- **Aumento de prioridade**: o nó sobe dentro da própria árvore trocando de lugar com o pai, como no heap binário, em $O(\log n)$, pois a altura é $\leq \log n$.
- **Diminuição de prioridade**: remova o nó (veja abaixo) e reinsira-o com a nova prioridade, em $O(\log n)$. Fazê-lo descer trocando com o maior filho também funciona, mas cada nível tem até $O(\log n)$ filhos a comparar, o que dá $O(\log^2 n)$.
- **Remoção de um nó qualquer**: aumente sua prioridade para $+\infty$, para que ele suba até a raiz, e remova o máximo: $O(\log n)$.
- **Construção**: n inserções sucessivas custam $O(n)$ no total, pelo mesmo argumento do contador binário.

Exemplo 3.9 (Remoção do máximo)**.** No heap resultante do Exemplo 3.7, $\{7 \to 5,\ B_2 \text{ de raiz } 9\}$, o máximo é a raiz 9. Retirando a sua árvore, sobra $H' = \{7 \to 5\}$. Os filhos de 9 formam outro heap binomial, $H'' = \{3,\ 8 \to 6\}$ (uma B_0 e uma B_1). A fusão $H' \cup H''$ encontra duas B_1, $7 \to 5$ e $8 \to 6$, e as junta com 8 por cima. O resultado tem $5 = 101_2$ elementos (Figura 3.4).



(a) o máximo é 9 (b) H' e os filhos de 9 (c) após a fusão

Figura 3.4. Remoção do máximo: os filhos da raiz removida formam um heap binomial, que é fundido com o resto.

3.3 Análise amortizada

Intuição

Pense num cofrinho. Toda vez que você faz algo barato, paga um pouco a mais e guarda o troco no cofrinho. Quando chega uma despesa grande, ela é paga com o dinheiro guardado. Se o cofrinho nunca fica negativo, o total gasto nunca passou do que você "cobrou" de cada operação. A análise amortizada é a contabilidade desse cofrinho.

Em várias estruturas, uma operação individual pode ser cara, mas operações caras são raras, porque cada uma delas desfaz um trabalho acumulado por muitas operações baratas. A **análise amortizada** mede o custo total de uma *sequência* de operações, no pior caso, e o divide pelo número de operações. É uma garantia de pior caso para a sequência e **não envolve probabilidade**.

Método do banqueiro (créditos). Cada operação recebe um número fixo de créditos, que é o seu custo amortizado. Operações baratas deixam créditos guardados na estrutura, e esses créditos pagam as operações caras mais tarde. Se o saldo de créditos nunca fica negativo, o custo real total é no máximo a soma dos custos amortizados.

Método do potencial (físico). Escolhe-se uma função $\Phi \geq 0$ sobre os estados da estrutura, que mede o trabalho "armazenado" (o dinheiro no cofrinho). O custo amortizado da i-ésima operação, de custo real t_i, é $a_i = t_i + \Phi(E_i) - \Phi(E_{i-1})$. Somando, os termos de potencial se cancelam:

$$\sum_{i=1}^{m} t_i = \sum_{i=1}^{m} a_i - \Phi_m + \Phi_0 \ \leq \ \sum_{i=1}^{m} a_i \qquad \text{sempre que } \Phi_m \geq \Phi_0.$$

Exemplo 3.10 (Contador binário)**.** Incrementar um contador de k bits pode trocar k bits (de 0111 para 1000, por exemplo), mas isso é raro. A tabela mostra os oito primeiros incrementos a partir de zero:

incremento	contador	bits trocados (t_i)	Φ (bits 1)	$\Delta\Phi$	$a_i = t_i + \Delta\Phi$	total real
—	**0000**	—	0	—	—	0
1	**0001**	1	1	+1	2	1
2	**0010**	2	1	0	2	3
3	**0011**	1	2	+1	2	4
4	**0100**	3	1	−1	2	7
5	**0101**	1	2	+1	2	8
6	**0110**	2	2	0	2	10
7	**0111**	1	3	+1	2	11
8	**1000**	4	1	−2	2	15

Os custos reais oscilam (1, 2, 1, 3, 1, 2, 1, 4), mas o custo amortizado é sempre 2. Em geral, com Φ = número de bits iguais a 1, um incremento que zera t bits e liga um tem custo amortizado $a = (t + 1) + (1 - t) = 2$. Logo m incrementos trocam no máximo $2m$ bits: na tabela, $15 \leq 16$. É o mesmo argumento que mostra que as inserções num heap binomial custam $O(1)$ amortizado: inserir é somar 1.

Exemplo 3.11 (Pilha com desempilhamento múltiplo)**.** Cada operação desempilha $k \geq 0$ elementos e depois empilha um; uma operação isolada pode custar $\Theta(n)$. Com Φ = número de elementos na pilha, uma operação que encontra p elementos tem custo amortizado $a = (1 + k) + (p - k + 1) - p = 2$. Logo m operações custam no máximo $2m = O(m)$.
Pelo método do banqueiro, a mesma conta fica assim: cada operação paga 2 créditos; um paga o empilhamento e o outro fica "grudado" no elemento empilhado, para pagar o seu futuro desempilhamento. Na sequência com $k = 0, 0, 0, 3, 0$, os custos reais são 1, 1, 1, 4, 1, o total é 8, e os créditos pagos somam $2 \cdot 5 = 10$. Na quarta operação, os três créditos guardados nos elementos da pilha pagam os três desempilhamentos.

Amortizado não é caso médio

Uma garantia amortizada vale para *toda* sequência de operações, sem hipóteses probabilísticas, mas não limita o custo de uma operação *individual*, que ainda pode ser alto (o oitavo incremento da tabela custou 4). Isso é decisivo quando um sistema exige que cada operação termine num tempo previsível.

3.4 Heap de Fibonacci

O heap binomial é "rigoroso": cada inserção ou fusão resolve imediatamente as colisões de ordem. O heap de Fibonacci é **preguiçoso**: adia toda a arrumação até a remoção do máximo, quando ela é feita de uma só vez.

Intuição

É como deixar a louça acumular na pia e lavar tudo de uma vez. Cada refeição (inserção) é rápida; a lavagem (consolidação) é demorada, mas só acontece de vez em quando e é paga pelas refeições que sujaram a louça. A remoção do máximo é o momento da lavagem: como é preciso achar o novo máximo de qualquer jeito, aproveita-se para arrumar a estrutura.

Estrutura. É uma floresta de árvores com a condição de heap. As raízes formam uma **lista circular duplamente encadeada**, e os filhos de cada nó também. Um ponteiro indica a raiz máxima. Cada nó guarda a chave, um ponteiro para o pai, um para um filho, dois para os irmãos

esquerdo e direito, o grau e um bit de **marca**. As listas circulares duplas permitem inserir ou retirar um nó e concatenar duas listas em $O(1)$.

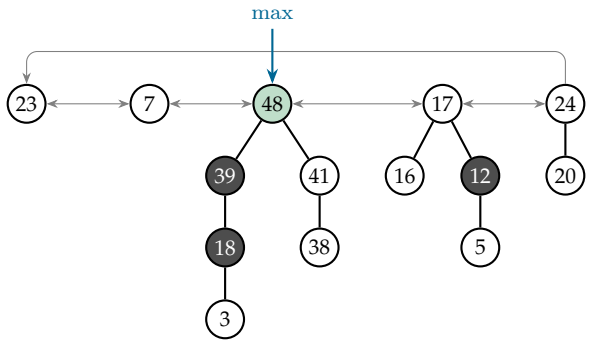


Figura 3.5. Um heap de Fibonacci. As raízes formam uma lista circular dupla (setas duplas; a última volta para a primeira) e o ponteiro max aponta a maior raiz. Os nós escuros estão marcados (já perderam um filho). Os filhos de cada nó também formam uma lista circular dupla, omitida no desenho.

Operações. Descrevemos o heap de máximo; "aumentar a prioridade" corresponde à operação *decrease-key* de um heap de mínimo.

- **Inserção**: cria uma árvore de um nó, coloca-a na lista de raízes e atualiza o ponteiro do máximo. Custo real $O(1)$.
- **Fusão**: concatena as duas listas de raízes; o máximo é o maior dos dois. Custo real $O(1)$.
- **Seleção do máximo**: leitura do ponteiro, $O(1)$.
- **Remoção do máximo**: (1) retira a raiz máxima e promove todos os seus filhos a raízes, desmarcados; (2) **consolida**: percorre as raízes com um vetor $A[0..D]$ indexado pelo grau, onde $D = O(\log n)$ é o grau máximo possível, e, sempre que duas raízes têm o mesmo grau, subordina a de menor prioridade à de maior (o grau aumenta de 1) e repete; (3) reconstrói a lista de raízes a partir de A e encontra o novo máximo. O custo real é $O(D + t)$, onde t é o número de raízes; o amortizado é $O(\log n)$.
- **Aumento de prioridade de** x: aumenta a chave; se ela não ficou maior que a do pai, termina. Senão, **corta** x, com sua subárvore, e o coloca na lista de raízes, desmarcado. Em seguida, olhando o pai y: se y está *desmarcado*, ele é marcado (a menos que seja raiz) e o processo termina; se y já estava *marcado*, isto é, já tinha perdido um filho, y também é cortado, desmarcado e levado à lista de raízes, e o processo continua com o pai de y. São os **cortes em cascata**. Custo amortizado $O(1)$.
- **Remoção de um nó qualquer**: aumenta a prioridade para $+\infty$ e remove o máximo: $O(\log n)$ amortizado.
- **Diminuição de prioridade**: não é a operação barata desta estrutura; faz-se removendo e reinserindo o nó, em $O(\log n)$ amortizado.

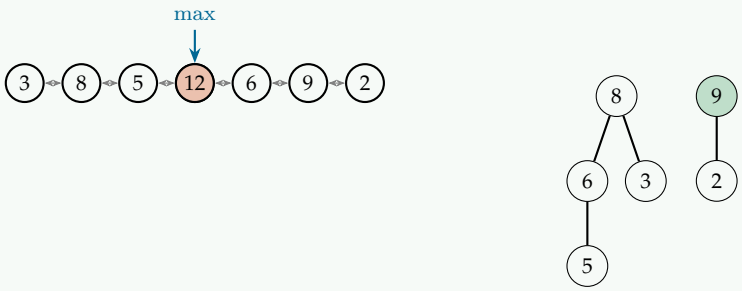
Exemplo 3.12 (Inserções preguiçosas e uma consolidação)**.** Num heap de Fibonacci vazio, inserimos $3, 8, 5, 12, 6, 9, 2$. Cada inserção apenas acrescenta um nó à lista de raízes: são 7 árvores de um nó, e o ponteiro do máximo aponta para 12. Agora removemos o máximo. O 12 sai (não tem filhos) e a consolidação percorre as raízes $3, 8, 5, 6, 9, 2$, usando o vetor A indexado pelo grau:

- 3: $A[0] = 3$. 8: colide com $A[0] = 3$; o link põe 8 por cima, e a árvore $8 \to 3$ tem grau 1: $A[0]$ fica vazio e $A[1] = 8 \to 3$.
- 5: $A[0] = 5$. 6: colide com 5, formando $6 \to 5$, de grau 1, que colide com $A[1] = 8 \to 3$;

o link põe 8 por cima, formando uma árvore de grau 2 em $A[2]$.

- 9: $A[0] = 9$. 2: colide com 9, formando $9 \to 2$ em $A[1]$.

Restam duas raízes, 8 (grau 2) e 9 (grau 1), e o novo máximo é 9 (Figura 3.6). A remoção custou bem mais que uma inserção, mas pagou de uma vez a "preguiça" das sete inserções.



(a) após 7 inserções (b) após remover o máximo

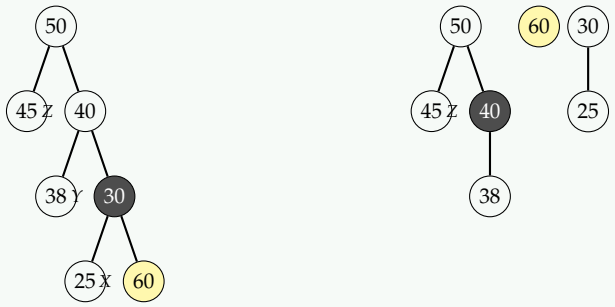
Figura 3.6. A consolidação do Exemplo 3.12: antes, sete raízes; depois de remover o 12 e consolidar, duas árvores de graus distintos.

Regras das marcas. Todos os nós começam desmarcados. Um nó é marcado quando *perde o primeiro filho* por um corte, e, ao perder o segundo, ele próprio é cortado. **Raízes nunca estão marcadas** e podem perder quantos filhos for preciso; quando um nó é subordinado a outro, fica desmarcado.

Exemplo 3.13 (Cortes em cascata)**.** O nó interno X tem pai Y, marcado, e avô Z, desmarcado, que não é raiz. A prioridade de X aumenta e passa a superar a de Y.

1. X é cortado e vai para a lista de raízes, desmarcado.
2. Y acaba de perder o segundo filho: é cortado, desmarcado e levado à lista de raízes.
3. Z, que não é raiz e estava desmarcado, perdeu o seu primeiro filho: passa a ser marcado, e a cascata termina.

Ao final, X e Y estão na lista de raízes, desmarcados, e Z continua no seu lugar, agora marcado: se ele perder mais um filho no futuro, será cortado também. A Figura 3.7 mostra o exemplo com chaves concretas: a de X sobe de 10 para 60.



(a) antes dos cortes (b) depois: três raízes, e Z marcado

Figura 3.7. Cortes em cascata. (a) $Y = 30$ está marcado (escuro) e $Z = 40$, não; a chave de X passa de 10 a 60. (b) X e Y são cortados, um após o outro, e vão para a lista de raízes desmarcados; Z perde um filho e passa a ser marcado. O novo máximo é 60.

3.4.1 Por que funciona: o grau máximo é $O(\log n)$

Lema 3.14. *Seja x um nó e sejam $y_1, \ldots, y_k$ os seus filhos, na ordem em que foram ligados a x. Então* $\text{grau}(y_i) \geq i - 2$.

Demonstração. Quando y_i foi ligado a x, este já tinha os filhos $y_1, \ldots, y_{i-1}$, e portanto grau $\geq i-1$. Só se ligam nós de mesmo grau, logo y_i tinha grau $\geq i-1$. Desde então, y_i perdeu no máximo um filho, pois ao perder o segundo seria cortado. Logo grau$(y_i) \geq i-2$. □

Teorema 3.15. *Um nó de grau k tem uma subárvore com pelo menos $F_{k+2} \geq \varphi^k$ nós, onde F é a sequência de Fibonacci e $\varphi = \frac{1+\sqrt{5}}{2}$. Logo o grau máximo é $D(n) \leq \log_\varphi n = O(\log n)$, e depois de uma consolidação o heap tem $O(\log n)$ árvores.*

Esboço. Seja s_k o menor tamanho possível de uma subárvore cuja raiz tem grau k. Pelo lema, $s_k \geq 2 + \sum_{i=2}^{k} s_{i-2}$, e por indução $s_k \geq F_{k+2}$, pois $F_{k+2} = 1 + \sum_{i=0}^{k} F_i$. É daí que vem o nome da estrutura: as menores árvores de posto $0, 1, 2, 3, 4, 5$ têm $1, 2, 3, 5, 8, 13$ nós. □

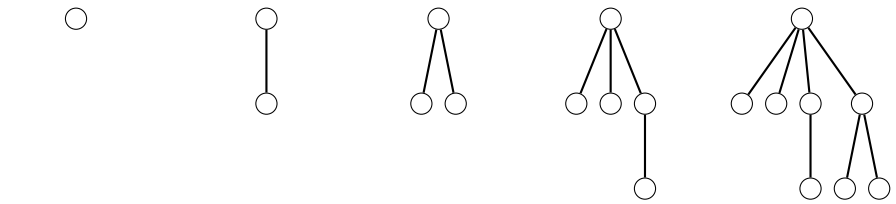


Figura 3.8. As menores árvores possíveis de grau 0 a 4 num heap de Fibonacci. Cada uma é uma raiz cujos filhos são as menores árvores de graus $0, 0, 1, 2, \ldots, k-2$ (o lema). Os tamanhos $1, 2, 3, 5, 8$ são números de Fibonacci.

Sem as marcas e os cortes em cascata, seria possível cortar todos os netos de uma raiz e obter uma árvore de posto k com apenas $k+1$ nós. O posto deixaria de ser $O(\log n)$ e a consolidação ficaria cara. Compare: uma B_4 do heap binomial tem 16 nós; a menor árvore de grau 4 de um heap de Fibonacci tem 8; sem marcas, teria só 5 (uma raiz e quatro folhas).

3.4.2 Análise amortizada

Use o potencial $\Phi(H) = t(H) + 2\,m(H)$, onde t é o número de raízes e m o número de nós marcados.

- **Inserção**: custo real $O(1)$ e $\Delta\Phi = +1$, logo $O(1)$ amortizado. **Fusão**: $\Delta\Phi = 0$, logo $O(1)$.
- **Remoção do máximo**: custo real $O(D + t)$; depois dela há no máximo $D+1$ raízes, então $\Delta\Phi \leq (D+1) - t$, e o custo amortizado é $O(D) = O(\log n)$. O trabalho de consolidar as t raízes acumuladas foi pago pelas inserções e fusões preguiçosas que as criaram.
- **Aumento de prioridade** com c cortes em cascata: custo real $O(c)$; t aumenta de c e m diminui de pelo menos $c-2$, então $\Delta\Phi \leq c + 2(2-c) = 4-c$, e o custo amortizado é $O(1)$. Na linguagem do banqueiro, o corte de um nó marcado é pago pelos créditos deixados pela operação que o marcou.

Exemplo 3.16 (O potencial nos exemplos anteriores)**.** No Exemplo 3.12, depois das 7 inserções há $t = 7$ raízes e nenhuma marca: $\Phi = 7$. A remoção do máximo examinou 6 raízes e fez 4 links, mas terminou com $t = 2$, e Φ caiu de 7 para 2: a queda de 5 unidades "pagou" quase todo o trabalho. No Exemplo 3.13, houve $c = 2$ cortes; t aumentou de 2, e o número de marcas não mudou (a de Y sumiu e a de Z apareceu), logo $\Delta\Phi = 2 + 2 \cdot 0 = 2$, e o custo amortizado é $2 + 2 = 4$, constante. Cada corte a mais na cascata desmarcaria mais um nó, e o potencial cairia 2 para pagá-lo.

Teorema 3.17. *A partir de um heap de Fibonacci vazio, qualquer sequência de a_1 inserções, a_2 remoções do máximo, a_3 aumentos de prioridade e a_4 fusões custa $O(a_1 + a_3 + a_4 + a_2 \log n)$.*

Armadilhas comuns

- Os custos $O(1)$ do heap de Fibonacci são **amortizados**: *uma* remoção do máximo depois de muitas inserções e fusões pode custar $\Theta(n)$, ao consolidar tudo de uma vez. Se cada operação precisa ter tempo previsível, essa estrutura é inadequada.
- As constantes são grandes e cada nó ocupa bastante memória (quatro ponteiros, grau e marca). Na prática, a estrutura só compensa com um número muito grande de aumentos de prioridade, como nos algoritmos de Dijkstra e Prim em grafos densos.
- Uma árvore de um heap de Fibonacci pode ter altura grande, até $\Theta(n)$ em sequências adversárias. O que é logarítmico é o **grau**, não a altura.

3.5 Comparação e escolha da estrutura

	Heap binário	Heap binomial	Heap de Fibonacci
Seleção	$O(1)$	$O(\log n)$ ou $O(1)$ com ponteiro	$O(1)$
Inserção	$O(\log n)$	$O(\log n)$ pior; $O(1)$ amortizado	$O(1)^*$
Remoção do máximo	$O(\log n)$	$O(\log n)$	$O(\log n)^*$
Aumentar prioridade	$O(\log n)$	$O(\log n)$	$O(1)^*$
Fusão	$O(n)$	$O(\log n)$ **pior caso**	$O(1)^*$
Construção	$O(n)$	$O(n)$	$O(n)$
Memória	vetor, sem ponteiros	3 ponteiros + grau por nó	4 ponteiros + grau + marca por nó
Garantia	pior caso	pior caso	amortizada
Simplicidade	muito simples	média	complexa

Como escolher a lista de prioridades

1. Não há fusões? Use o **heap binário**: é o mais simples, usa menos memória e tem boa localidade de acesso.
2. Há fusões frequentes e **cada operação precisa ter tempo previsível** (limite de tempo por operação, sistemas de tempo real)? Use o **heap binomial**, com fusão $O(\log n)$ no pior caso.
3. Há uma quantidade muito grande de **aumentos de prioridade** (ou de fusões) e o que importa é o **custo total** da execução? Use o **heap de Fibonacci**.
4. Há uma restrição forte de **memória**? Use o heap binário; as outras estruturas pagam vários ponteiros por elemento.

Exemplo 3.18 (Aplicando o roteiro)**.**

- Uma fila de impressão que só recebe trabalhos e despacha o mais urgente: não há fusões, e o heap binário basta.
- Um escalonador de tempo real em que filas de processadores são fundidas quando um processador falha, e cada operação tem prazo: heap binomial.
- O algoritmo de Dijkstra num grafo denso, com n remoções e até $m \approx n^2$ aumentos de prioridade: o heap de Fibonacci dá $O(m + n \log n)$, contra $O(m \log n)$ do binário (Capítulo 16).

Pontos-chave

Heap binomial = árvores B_k de ordens distintas = bits de n; fusão = soma binária, $O(\log n)$ no pior caso; inserção $O(1)$ amortizado. Heap de Fibonacci = preguiça + marcas; tudo $O(1)$ amortizado, exceto remoções, $O(\log n)$ amortizado; grau $\leq \log_\varphi n$. Amortizado é garantia para a sequência, não para cada operação, e não é caso médio.

3.6 Exercícios

Exercícios do curso

3.1 [Lista] resposta: p. 172
Desenvolva um algoritmo $O(n)$ para construir um heap binomial com n elementos.

3.2 [Lista] resposta: p. 172
Um heap binomial tem k árvores. Que alturas elas devem ter para que a inserção seja o pior caso, isto é, faça o número máximo de passos?

3.3 [Lista] resposta: p. 172
H é um heap binomial com k árvores, e a árvore cuja raiz tem a maior prioridade tem altura h. Determine as alturas das árvores de H e o valor de h para que a remoção do máximo faça o número máximo de passos.

3.4 [Lista] resposta: p. 172
Prove ou dê contraexemplo: se H_1 e H_2 têm k_1 e k_2 árvores e sua fusão H tem k árvores, então $k \geq \min\{k_1, k_2\}$.

3.5 [Lista] resposta: p. 172
Prove ou dê contraexemplo: nas mesmas condições, $k \leq k_1 + k_2$.

3.6 [Lista] resposta: p. 172
Determine a altura mínima e a máxima de uma árvore de um heap de Fibonacci em função do posto k.

3.7 [Lista] resposta: p. 172
Escreva os algoritmos de seleção, inserção, remoção, aumento e diminuição de prioridade, construção e fusão para heaps binomiais e para heaps de Fibonacci.

3.8 [Monitoria] resposta: p. 172
(a) O que caracteriza uma árvore binomial B_k? Quantos nós tem? Qual a relação entre o grau da raiz e k? (b) Como um heap binomial representa uma coleção de árvores binomiais? (c) Duas árvores B_2 de um heap binomial de mínimo têm raízes de chaves 10 e 15: qual é o resultado do *link*, sua ordem, seu número de nós e o grau da raiz? (d) Como achar o elemento de maior prioridade e qual o custo? (e) Qual o custo da união e por que ela não percorre os n elementos?

3.9 [Monitoria] resposta: p. 172
(a) Qual a principal diferença estrutural entre heap de Fibonacci e heap binomial? (b) Quais operações são $O(1)$ amortizado? Quanto custa remover o mínimo (*Extract-Min*)? (c) Na *Decrease-Key*(x, k) de um heap de mínimo, por que alterar a chave pode violar a propriedade de heap e o que se faz? (d) Para que servem os cortes em cascata, e por que a operação é $O(1)$ amortizado mesmo fazendo vários cortes?

3.10 [Monitoria] resposta: p. 173
Faça a fusão de $H_1 = \{B_0, B_2, B_4\}$ com $H_2 = \{B_0, B_1\}$ como uma soma binária: quais subordinações acontecem, em que ordem, e qual o resultado em binário?

3.11 [Monitoria] resposta: p. 173
Num heap de Fibonacci de máximo, o nó interno X tem pai Y marcado e avô Z marcado. A prioridade de X aumenta e viola a ordem. Descreva os cortes em cascata, os nós movidos para a lista de raízes e as mudanças nas marcas.

3.12 [Monitoria] resposta: p. 173
Por que não podemos simplesmente cortar um nó do pai sem marcar nem cortar em cascata? O que aconteceria com a relação entre a ordem de uma árvore e o seu número de nós?

3.13 [Avaliação] resposta: p. 173
Um aeroporto mantém uma fila de prioridade por setor de controle de tráfego. Fusões de filas (transferências entre setores, unificação de setores) são muito frequentes, com filas de centenas de milhares de aeronaves; alterações de prioridade são raras; e o tempo de *cada* fusão precisa ser previsível, pois as janelas de processamento são rígidas. Qual heap você escolheria? Justifique e explique por que os outros dois são menos adequados.

3.14 [Avaliação] resposta: p. 173
Uma central de atendimento médico mantém cerca de $2 \cdot 10^6$ chamadas numa fila de prioridade, com $2 \cdot 10^6$ inserções, $2 \cdot 10^6$ remoções da chamada mais urgente e poucas alterações de prioridade. Cada chamada ocupa 16 bytes, endereços ocupam 8 bytes e a estrutura pode usar no máximo 40 MB = 41 943 040 bytes. Qual estrutura você usaria? Considere a complexidade das operações, a necessidade de fusão, a memória por elemento e a simplicidade.

3.15 [Avaliação] resposta: p. 173
Um supercomputador tem cerca de $5 \cdot 10^6$ tarefas numa fila de prioridade. Num período típico há $5 \cdot 10^6$ inserções, $5 \cdot 10^6$ remoções da tarefa mais prioritária, $8 \cdot 10^7$ aumentos de prioridade (feitos sem retirar e reinserir a tarefa) e poucas uniões de filas. O custo total da execução é o que importa. Qual heap é mais adequado? Por que os outros dois são menos vantajosos?

Exercícios complementares

3.16 [Complementar] resposta: p. 173
Um vetor dinâmico dobra de capacidade quando fica cheio, copiando todos os elementos. Mostre, com o método do potencial e $\Phi = 2 \cdot (\text{número de elementos}) - \text{capacidade}$, que cada inserção custa $O(1)$ amortizado.

3.17 [Complementar] resposta: p. 173
Um contador binário de k bits passa a aceitar *incremento* e *decremento*. Mostre uma sequência de m operações que custa $\Theta(mk)$. Por que a análise amortizada do contador só com incrementos não se aplica?

3.18 [Complementar] resposta: p. 173
Implemente uma fila com duas pilhas (uma de entrada e outra de saída) e mostre que cada operação custa $O(1)$ amortizado.

3.19 [Complementar] resposta: p. 173
Quais árvores binomiais compõem um heap binomial com 100 elementos? E depois de mais uma inserção? E depois de mais quatro inserções a partir de 100? Quantos *links* cada uma dessas inserções faz?

3.20 [Complementar] resposta: p. 173
Mostre que, se um heap de Fibonacci sofre apenas inserções, fusões e remoções do máximo (sem alterações de prioridade), toda árvore dele é uma árvore binomial (não ordenada) logo após cada consolidação.

3.21 [Complementar] resposta: p. 173
Dê uma sequência de operações, a partir de um heap de Fibonacci vazio, em que *uma única* remoção do máximo tem custo real $\Theta(n)$. Por que isso não contradiz o custo amortizado $O(\log n)$?

4

Conjuntos disjuntos: Union-Find

Neste capítulo

- O problema de manter uma partição sob as operações Make-Set, Find e Union.
- Implementações por vetor de representantes, por listas encadeadas e por florestas.
- As heurísticas de união por rank e de compressão de caminhos, e a análise $O(m \log^* n)$.
- Como escolher a implementação de acordo com as operações exigidas.

Quando objetos vão sendo agrupados ao longo do tempo — usuários que formam comunidades, computadores ligados por cabos, vértices ligados pelas arestas escolhidas pelo algoritmo de Kruskal — e é preciso responder rapidamente se dois deles já estão no mesmo grupo, a estrutura adequada é a de **conjuntos disjuntos**, também chamada *Union-Find*. Este capítulo mostra que, com duas heurísticas simples, cada operação passa a custar, na prática, tempo constante.

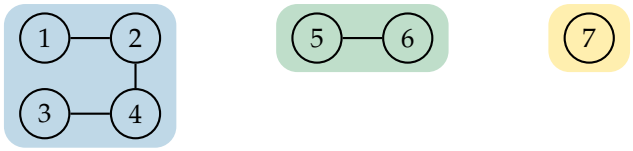


Figura 4.1. Uma rede de computadores que recebe cabos aos poucos. Depois dos cabos 1–2, 3–4, 2–4 e 5–6, há três grupos (componentes): $\{1, 2, 3, 4\}$, $\{5, 6\}$ e $\{7\}$. A pergunta "1 e 3 se comunicam?" tem resposta sim; "1 e 6?", não.

4.1 O problema

Queremos manter uma partição de um universo de n elementos em conjuntos disjuntos, cada um identificado por um elemento chamado **representante**, com três operações:

- Make-Set(x) cria o conjunto unitário $\{x\}$;
- Find(x) devolve o representante do conjunto que contém x; a pergunta "x e y estão juntos?" equivale a testar Find(x) = Find(y);
- Union(x, y) une o conjunto de x ao de y.

Conjuntos **nunca se separam**: não há uma operação de divisão. Numa sequência típica, há n operações Make-Set seguidas de m operações Find e Union intercaladas; como cada união efetiva diminui o número de conjuntos em um, há no máximo $n - 1$ delas.

Exemplo 4.1 (A rede da Figura 4.1 em operações)**.** Cada computador começa sozinho, e cada cabo é uma união:

operação	partição depois dela
Make-Set(1), ..., Make-Set(7)	$\{1\},\{2\},\{3\},\{4\},\{5\},\{6\},\{7\}$
Union(1, 2)	$\{1,2\},\{3\},\{4\},\{5\},\{6\},\{7\}$
Union(3, 4)	$\{1,2\},\{3,4\},\{5\},\{6\},\{7\}$
Union(2, 4)	$\{1,2,3,4\},\{5\},\{6\},\{7\}$
Union(5, 6)	$\{1,2,3,4\},\{5,6\},\{7\}$

Agora Find(1) e Find(3) devolvem o mesmo representante (qual deles depende da implementação), e Find(1) $\neq$ Find(6). Um novo cabo 1–3 não muda nada: Union(1, 3) encontra os dois já no mesmo conjunto.

As aplicações incluem o cálculo de componentes conexas de um grafo que recebe arestas, o algoritmo de Kruskal para árvores geradoras mínimas (Capítulo 14), a equivalência de nomes em compiladores, grupos em redes sociais e problemas de percolação.

4.2 Implementações

Vetor de representantes

Um vetor $rep[x]$ guarda o representante de cada elemento. Find custa $O(1)$, mas Union precisa trocar o representante de todos os elementos de um dos conjuntos, o que custa $O(n)$ por união.

Exemplo 4.2 (Vetor de representantes)**.** Depois das uniões do Exemplo 4.1, uma possibilidade é $rep = [1,1,1,1,5,5,7]$: os elementos $1,\ldots,4$ têm representante 1, os elementos 5 e 6 têm representante 5, e o 7 é o seu próprio representante. Executar agora Union(1, 6) exige percorrer o vetor e trocar por 1 o representante de todos os elementos cujo representante é 5 (as posições 5 e 6, em destaque); em geral, até $n-1$ posições podem mudar numa única união.



Listas encadeadas com ponteiro para o representante

Cada conjunto é uma lista encadeada, e um objeto que representa o conjunto guarda a cabeça, a cauda e o tamanho da lista; todo nó tem um ponteiro direto para esse objeto. Make-Set e Find custam $O(1)$, pois Find segue um único ponteiro. Union concatena as listas em $O(1)$, ligando a cauda de uma à cabeça da outra, mas precisa atualizar o ponteiro de todos os nós da lista anexada (Figura 4.2).

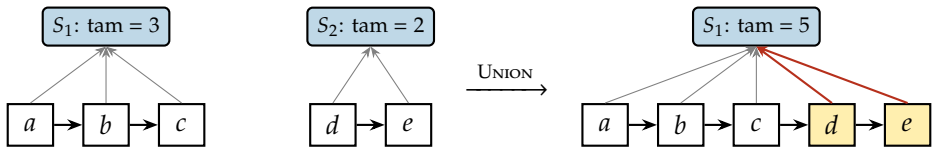


Figura 4.2. Conjuntos como listas encadeadas. Cada nó aponta para o próximo (setas pretas) e para o objeto do seu conjunto (setas cinzas), que guarda cabeça, cauda e tamanho. Na união, a lista menor, $\{d,e\}$, é anexada ao fim da maior, e só os seus dois nós mudam de objeto.

Sem cuidado, isso custa $\Theta(n)$ por união e $\Theta(n^2)$ numa sequência ruim, como anexar sempre a lista grande a um elemento unitário.

Exemplo 4.3 (A sequência ruim)**.** Com $n = 5$ elementos $x_1, \ldots, x_5$, faça Union(x_2, x_1), Union(x_3, x_1), Union(x_4, x_1), Union(x_5, x_1), anexando sempre a lista de x_1 (a grande) à do novo elemento. As uniões atualizam $1, 2, 3, 4$ ponteiros: 10 no total, e, em geral, $1 + 2 + \cdots + (n-1) = \Theta(n^2)$. Anexando a menor à maior, cada uma dessas uniões atualizaria um único ponteiro, o do elemento novo: 4 no total.

A solução é a **união ponderada**: anexa-se sempre a lista *menor* à maior. Cada vez que o ponteiro de um elemento x é atualizado, o conjunto que contém x pelo menos dobra de tamanho; como nenhum conjunto tem mais de n elementos, o ponteiro de x é atualizado no máximo $\log_2 n$ vezes. Assim, n operações Make-Set e m outras operações custam $O(m + n \log n)$: Union custa $O(\log n)$ **amortizado**. Esta é a única implementação que permite **listar os membros** de um conjunto em tempo $O(k)$, proporcional ao tamanho k do conjunto, simplesmente percorrendo a lista.

Por que "menor na maior" dá $\log n$

Acompanhe um único elemento x. Toda vez que o seu ponteiro muda, ele estava na lista menor, e a lista resultante tem pelo menos o dobro do tamanho da que ele estava: $1 \to\geq 2 \to\geq 4 \to\geq 8 \to \cdots$. Depois de k mudanças, x está num conjunto com pelo menos 2^k elementos, e como $2^k \leq n$, são no máximo $\log_2 n$ mudanças. Com $n = 1000$, cada elemento muda de ponteiro no máximo 9 vezes.

Florestas

Cada conjunto é uma árvore em que cada nó aponta para o pai, $pai[x]$, e a raiz é o representante ($pai[r] = r$). Make-Set faz $pai[x] := x$; Find sobe até a raiz, em tempo proporcional à altura; Union encontra as duas raízes e faz uma apontar para a outra, em $O(1)$ depois de encontradas.

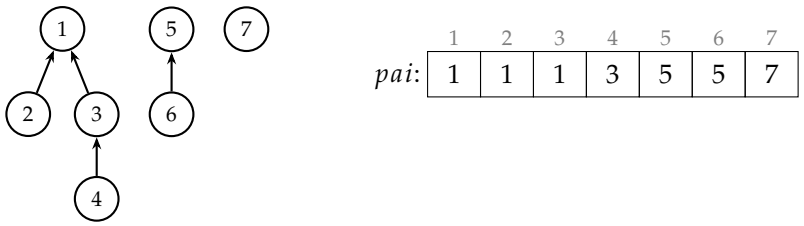


Figura 4.3. Uma floresta e o seu vetor pai. As setas apontam do filho para o pai; as raízes (1 e 5) apontam para si mesmas e são os representantes. Find(4) percorre $4 \to 3 \to 1$.

O problema é que, sem nenhum critério, uma sequência de uniões pode formar uma **cadeia de altura** $n - 1$, e então cada Find custa $\Theta(n)$. Por exemplo, Union$(1, 2)$, Union$(1, 3)$, Union$(1, 4)$, $\ldots$, sempre pondo a raiz atual sob o novo elemento, produz a cadeia $1 \to 2 \to 3 \to \cdots \to n$, e Find$(1)$ percorre todos os nós.

União por tamanho ou por rank

A primeira heurística liga a raiz da árvore *menor* à raiz da maior, medindo o tamanho em número de nós ou por um valor chamado **rank**. As regras do rank são:

- todo nó começa com rank 0;
- só o rank de raízes é atualizado: na união de duas raízes de mesmo rank r, a nova raiz passa a ter rank $r + 1$; com ranks diferentes, a de menor rank vai para baixo e nada muda;
- o rank nunca diminui, e, quando um nó deixa de ser raiz, seu rank nunca mais muda;
- sem compressão de caminhos, o rank é igual à altura; com compressão, é apenas uma cota superior para ela.

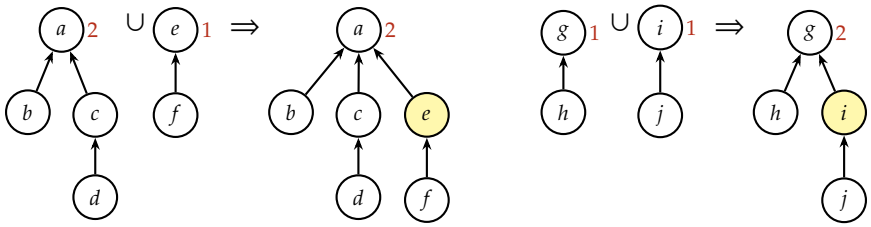


Figura 4.4. União por rank. À esquerda, ranks diferentes: a árvore de rank 1 vai para baixo da de rank 2, e a altura não muda. À direita, ranks iguais: a nova raiz ganha um nível, e o seu rank passa de 1 a 2. Os números em vermelho são os ranks.

Teorema 4.4 (Altura logarítmica)**.** *Com união por rank (ou por tamanho), toda árvore de rank h tem pelo menos 2^h nós. Logo $h \le \lfloor \log_2 n \rfloor$, e* Find *custa $O(\log n)$.*

Demonstração. Indução no número de operações. Uma árvore de um nó tem rank 0 e $2^0 = 1$ nó. O rank de uma árvore só aumenta quando se unem duas árvores de mesmo rank h; pela hipótese de indução, cada uma tem pelo menos 2^h nós, e a nova árvore tem pelo menos $2 \cdot 2^h = 2^{h+1}$ nós e rank $h+1$. Quando a menor é ligada sob a maior, o rank da maior não muda. Como $2^h \le n$, temos $h \le \log_2 n$. □

Por exemplo, com $n = 1000$ elementos nenhuma árvore passa de altura $\lfloor \log_2 1000 \rfloor = 9$, por pior que seja a sequência de uniões; sem a heurística, a altura poderia chegar a 999.

Corolário 4.5. *(i) O número de nós de rank exatamente k é no máximo $n/2^k$: cada um deles é, ou foi, raiz de uma subárvore com pelo menos 2^k nós, e essas subárvores são disjuntas. (ii) Todo rank é no máximo $\lfloor \log_2 n \rfloor$. (iii)* $\text{rank}(x) < \text{rank}(pai[x])$ *para todo x que não é raiz.*

Compressão de caminhos

A segunda heurística aproveita cada Find: depois de encontrar a raiz, faz-se *todo nó do caminho* apontar diretamente para ela. As buscas futuras por esses nós passam a custar $O(1)$. Há variantes que fazem o trabalho numa só passada: na *separação de caminhos*, cada nó do caminho passa a apontar para o avô; na *divisão de caminhos*, isso é feito em nós alternados. Todas têm a mesma complexidade assintótica.

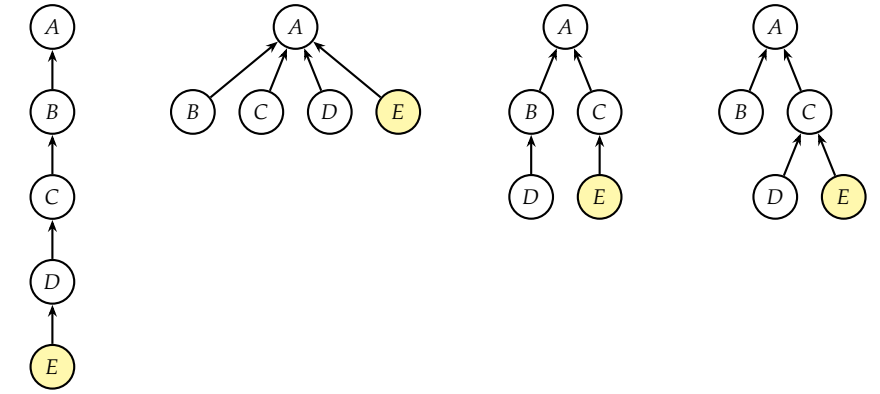


Figura 4.5. Find(E) na cadeia $E \to D \to C \to B \to A$ com as três variantes. Todas encurtam o caminho; a compressão completa deixa todos os nós a um passo da raiz.

Floresta com união por rank e compressão de caminhos

Procedimento Make-Set(x)
$pai[x] := x$; $rank[x] := 0$

```
Função Find(x)
    se pai[x] ≠ x então
        pai[x] := Find(pai[x])                    % compressão
    retorna pai[x]
Procedimento Union(x, y)
    r := Find(x); s := Find(y)
    se r = s então retorna
    se rank[r] < rank[s] então
        pai[r] := s
    senão se rank[r] > rank[s] então
        pai[s] := r
    senão
        pai[s] := r; rank[r] := rank[r] + 1
```

O Find recursivo acima comprime o caminho na volta da recursão: a chamada sobre E chama a de D, que chama a de C, e assim por diante até a raiz A; na volta, cada chamada faz o seu nó apontar para o valor devolvido, que é A. Uma versão iterativa faz duas passadas: a primeira sobe até encontrar a raiz y; a segunda percorre o mesmo caminho fazendo $pai[\cdot] := y$.

4.3 União por rank com compressão: $O(m \log^* n)$

> **Intuição**
>
> O logaritmo iterado $\log^* n$ responde à pergunta: "quantas vezes preciso apertar a tecla $\log_2$ da calculadora, começando em n, até o visor mostrar um número ≤ 1?". Para $n = 1000$: $1000 \to 9{,}97 \to 3{,}32 \to 1{,}73 \to 0{,}79$, quatro vezes. Essa função cresce tão devagar que, para efeitos práticos, é uma constante.

Formalmente, seja $\log^* n$ o número de vezes que é preciso aplicar $\log_2$ a n até chegar a um valor ≤ 1. Definindo $F(0) = 1$ e $F(k) = 2^{F(k-1)}$, temos $F = 1, 2, 4, 16, 65536, 2^{65536}, \ldots$, e $\log^* n$ é o menor k com $F(k) \geq n$:

n	1	2	3, 4	5, …, 16	17, …, 65536	65537, …, 2^{65536}
$\log^* n$	0	1	2	3	4	5

Como 2^{65536} é muito maior que o número de átomos do universo observável (cerca de $10^{80} \approx 2^{266}$), para qualquer n que apareça na prática, $\log^* n \leq 5$.

Dividimos os ranks em **grupos**: o grupo $B \geq 1$ contém os ranks no intervalo $(F(B-1), F(B)]$, e o grupo 0 contém os ranks $\leq F(0) = 1$. Assim, o grupo 0 é $\{0, 1\}$, o grupo 1 é $\{2\}$, o grupo 2 é $\{3, 4\}$, o grupo 3 é $\{5, \ldots, 16\}$ e o grupo 4 é $\{17, \ldots, 65536\}$.

- **Propriedade A**: como os ranks são no máximo $\log n$, há no máximo $\log^* n$ grupos.
- **Propriedade B**: para $B \geq 1$, o número de nós no grupo B é no máximo $\sum_{r=F(B-1)+1}^{F(B)} \frac{n}{2^r} \leq \frac{n}{2^{F(B-1)}} = \frac{n}{F(B)}$; o grupo 0 tem trivialmente no máximo $n = n/F(0)$ nós.

Num Find, distribua o custo de cada nó x do caminho. Se x é a raiz ou filho da raiz, ou se $pai[x]$ está num *grupo diferente* do de x, o custo é cobrado do próprio Find; como os ranks crescem ao longo do caminho, há $O(\log^* n)$ mudanças de grupo por Find, e o total cobrado dessa forma é $O(m \log^* n)$. Caso contrário, quando $pai[x]$ está no mesmo grupo de x, o custo é cobrado do nó x. A compressão dá a x um novo pai de rank *estritamente maior*, e um nó do grupo B só pode ser cobrado assim no máximo $F(B)$ vezes antes que seu pai passe a outro grupo, depois do que nunca mais é cobrado. O total cobrado dos nós de um grupo é, pela Propriedade B, no máximo

$\frac{n}{F(B)} \cdot F(B) = n$, e, somando sobre os $\log^* n$ grupos, $O(n \log^* n)$. O custo total é $O(m \log^* n)$, para $m \geq n$.

Exemplo 4.6 (Onde cada passo de um Find é cobrado)**.** Suponha um caminho de Find com ranks $0, 1, 3, 4, 7, 12, 20$ (da folha para a raiz; os ranks crescem ao longo do caminho). Os grupos são $0, 0, 2, 2, 3, 3, 4$. Os passos que saem de um nó para um pai de *outro grupo* ($1 \to 3$, $4 \to 7$ e $12 \to 20$), mais os dois últimos nós, são cobrados do Find: no máximo $\log^* n + 2$ por consulta. Os demais ($0 \to 1$, $3 \to 4$, $7 \to 12$) são cobrados dos próprios nós; depois da compressão, todos passam a apontar para a raiz, de rank 20, e esses nós ficam mais perto de "mudar de grupo".

Para saber mais

A análise acima não é a melhor possível. O limite justo, devido a Tarjan, é $O(m\,\alpha(m, n))$, onde α é a inversa da função de Ackermann, que cresce ainda mais devagar que $\log^*$ e não passa de 4 para qualquer entrada concebível. Os dois limites mostram que a estrutura é "quase linear".

4.4 Qual implementação usar?

Tabela 4.1. Implementações de Union-Find e quando usar cada uma.

Implementação	Make-Set	Find	Union	Quando é a melhor escolha
Vetor de representantes	$O(1)$	$O(1)$	$O(n)$	Quase só consultas e pouquíssimas uniões, com n pequeno.
Listas encadeadas com ponteiro ao representante e união ponderada	$O(1)$	$O(1)$ **pior caso**	$O(\log n)^*$	As consultas dominam e cada uma precisa custar $O(1)$ *garantido*; é preciso **listar os membros** de um grupo em $O(k)$; há memória para cabeça, cauda, tamanho e ponteiros.
Floresta sem critério	$O(1)$	$O(n)$	$O(1)$ + Finds	Nunca; serve apenas para motivar as heurísticas.
Floresta com união por rank ou tamanho, sem compressão	$O(1)$	$O(\log n)$	$O(\log n)$	Quando é preciso **desfazer uniões**: sem compressão, cada união muda só um ponteiro (e talvez um rank), que pode ser empilhado e restaurado em $O(1)$.
Floresta com rank e compressão de caminhos	$O(1)$	$O(\log^* n)^*$ (ou α)		Caso geral, com muitas operações misturadas e só Find e Union: praticamente $O(1)$ por operação e memória mínima (vetores pai e $rank$).

Cenários típicos

- **Listar os membros do grupo de u.** A floresta só tem ponteiros do filho para o pai; listar exigiria testar Find(i) = Find(u) para todos os n elementos, em $O(n)$. Com **listas encadeadas**, percorre-se a lista em $O(k)$; "mesmo grupo?" custa $O(1)$ e a união custa $O(\log n)$ amortizado, anexando a menor à maior.
- **Conectividade enquanto arestas são destruídas.** Union-Find não desfaz uniões, e a compressão destrói a forma das árvores. A solução é *offline*: leia todas as destruições,

construa o estado final (sem as arestas que serão destruídas) e processe as destruições **de trás para frente**, cada uma como uma Union; guarde as respostas e apresente-as na ordem original.

- **Operações online com desfazer** (*snapshots*). Use floresta **sem compressão**, só com união por rank ou tamanho (altura $O(\log n)$ e Find em $O(\log n)$). Cada Union empilha o ponteiro e o rank que alterou; desfazer é desempilhar e restaurar, em $O(1)$.
- **Algoritmo de Kruskal.** Para cada aresta uv, em ordem crescente de peso, testa-se $\textsc{Find}(u) \neq \textsc{Find}(v)$ e, em caso afirmativo, faz-se $\textsc{Union}(u, v)$. A floresta com rank e compressão é a escolha natural.

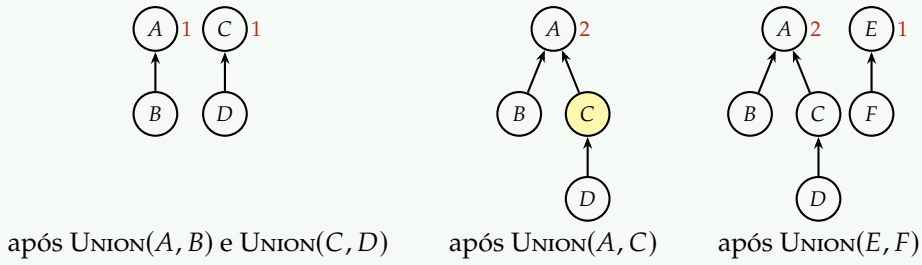
Exemplo 4.7 (A solução offline, de trás para frente)**.** Uma rede tem os cabos 1–2, 2–3 e 3–4, que serão destruídos nessa ordem; queremos o número de componentes após cada destruição. Em vez de destruir, começamos do fim: sem nenhum cabo, há 4 componentes. Reinserimos os cabos na ordem inversa: $\textsc{Union}(3, 4)$ recupera o estado logo após a segunda destruição (3 componentes); $\textsc{Union}(2, 3)$, o estado após a primeira (2 componentes); $\textsc{Union}(1, 2)$, o estado inicial (1 componente). Assim, as respostas, na ordem original das destruições, são 2, 3 e 4 componentes, e cada passo custou só uma Union.

4.5 Exemplos resolvidos

Exemplo 4.8 (Uniões por rank)**.** Partindo de $\{A\}, \ldots, \{F\}$, executam-se $\textsc{Union}(A, B)$, $\textsc{Union}(C, D)$, $\textsc{Union}(A, C)$ e $\textsc{Union}(E, F)$, com união por rank e, em caso de empate, a raiz do primeiro argumento como nova raiz.

- $\textsc{Union}(A, B)$: ranks $0 = 0$, a raiz é A e $\text{rank}(A) = 1$.
- $\textsc{Union}(C, D)$: a raiz é C e $\text{rank}(C) = 1$.
- $\textsc{Union}(A, C)$: ranks $1 = 1$, a raiz é A e $\text{rank}(A) = 2$; a árvore tem A com os filhos B e C, e C com o filho D.
- $\textsc{Union}(E, F)$: a raiz é E e $\text{rank}(E) = 1$.

Ao final há **dois conjuntos**, com representantes A (de $\{A, B, C, D\}$) e E (de $\{E, F\}$).



após $\textsc{Union}(A, B)$ e $\textsc{Union}(C, D)$ — após $\textsc{Union}(A, C)$ — após $\textsc{Union}(E, F)$

Figura 4.6. As quatro uniões do Exemplo 4.8, com os ranks das raízes em vermelho.

Exemplo 4.9 (Compressão numa cadeia)**.** Na cadeia $pai[B] = A$, $pai[C] = B$, $pai[D] = C$, $pai[E] = D$, o $\textsc{Find}(E)$ com compressão devolve A e deixa B, C, D, E apontando diretamente para A (Figura 4.5(b)): a árvore passa a ter altura 1, e as próximas consultas a esses nós custam $O(1)$. Se depois, com $\text{rank}(A) = \text{rank}(D) = 1$, for executada $\textsc{Union}(A, D)$, há empate: a nova raiz é A e $\text{rank}(A) = 2$. O rank só aumenta no empate porque, ligando a árvore menor sob a maior, a altura da maior não muda; só com alturas iguais a nova árvore fica um nível mais alta.

Exemplo 4.10 (Por que usar as duas heurísticas juntas)**.** Só com união por rank, Find custa $\Theta(\log n)$ no pior caso, pois há sequências que atingem essa altura. Só com compressão, uma árvore pode ficar alta (uma cadeia) antes de ser comprimida: o custo total de m operações é $O(m \log n)$, e um Find individual pode custar $\Theta(n)$. Juntas, as duas dão $O(m \log^* n)$ no total, ou $O(m\,\alpha)$, praticamente constante por operação.

Pontos-chave

Listas encadeadas dão Find $O(1)$ no pior caso, Union $O(\log n)$ amortizado com "menor na maior" e listagem de membros em $O(k)$. **Florestas** com **rank e compressão** dão $O(\log^* n)$ (ou α) amortizado por operação, com memória mínima; **sem compressão**, quando é preciso desfazer uniões. Sempre o menor sob o maior; sem isso, listas e árvores degeneram.

4.6 Exercícios

Exercícios do curso

4.1 [Monitoria] resposta: p. 174
(a) Qual o objetivo da estrutura Union-Find? (b) Explique Make-Set, Find e Union. (c) Que informação cada elemento armazena para representar os conjuntos? (d) Como a união por rank evita árvores muito altas?

4.2 [Monitoria] resposta: p. 174
Refaça, sem consultar o texto, os exemplos da §4.5: (a) as uniões Union(A, B), Union(C, D), Union(A, C), Union(E, F) com união por rank; (b) Find(E) com compressão na cadeia $A \leftarrow B \leftarrow C \leftarrow D \leftarrow E$; (c) Union$(A, D)$ com $\text{rank}(A) = \text{rank}(D) = 1$; (d) por que usar rank e compressão juntos.

4.3 [Lista] resposta: p. 174
Considere união por rank sem compressão de caminhos, sempre ligando a árvore de menor rank à de maior. Prove que uma árvore com n elementos tem altura no máximo $\lfloor \log n \rfloor$ e mostre que a cota é justa, com uma sequência de operações que produz altura $\Theta(\log n)$.

4.4 [Lista] resposta: p. 174
Considere apenas compressão de caminhos, sem união por rank ou tamanho. Construa uma sequência de operações que produza uma árvore alta antes da compressão e discuta o custo $O(m \log n)$ de m operações.

4.5 [Avaliação] resposta: p. 174
Uma plataforma tem 10^7 usuários, cada um inicialmente num grupo só seu. Num dia há 10^5 uniões de grupos e 10^8 consultas "u e v estão no mesmo grupo?"; grupos nunca se separam e o histórico das uniões não importa. (a) Que estrutura usar, e como representar cada grupo: como uma árvore em que cada elemento aponta para outro até o representante, ou como uma lista encadeada em que cada elemento aponta diretamente para o representante? (b) Ao unir dois grupos, que estratégia decide quem é anexado a quem, e por que ela importa?

4.6 [Lista] resposta: p. 174
Explique, para cada problema, por que o Union-Find tradicional (floresta com compressão) falha e qual adaptação resolve: (a) estradas são destruídas uma a uma e, após cada destruição, pergunta-se se o país continua conexo; (b) uma rede recebe operações online de adicionar cabo, consultar conectividade, salvar um *snapshot* e desfazer tudo até o último *snapshot*; (c) uma rede social precisa de *Juntar*, *MesmoGrupo* e *ListarMembros*(u).

4.7 [Lista] resposta: p. 174
Problema *Asya and Kittens* (Codeforces 1131F): n gatinhos ocupavam n células enfileiradas, um por célula. Em $n-1$ dias, uma divisória entre duas células vizinhas foi removida por dia, juntando os gatinhos x_i e y_i, que estavam em células vizinhas de blocos diferentes. Dados os pares, reconstrua uma ordem inicial possível dos gatinhos.

Exercícios complementares

4.8 [Complementar] resposta: p. 174
Com floresta, união por rank (no empate, a raiz do primeiro argumento vence) e compressão de caminhos, execute em $\{1,\dots,8\}$: Union$(1,2)$, Union$(3,4)$, Union$(5,6)$, Union$(7,8)$, Union$(1,3)$, Union$(5,7)$, Union$(1,5)$, Find(8). Dê o vetor pai e os ranks antes e depois do Find.

4.9 [Complementar] resposta: p. 174
Use Union-Find para decidir se um grafo não direcionado tem ciclo. Qual a complexidade?

4.10 [Complementar] resposta: p. 174
Prove que, com união por rank e compressão de caminhos, $\text{rank}(x) < \text{rank}(pai[x])$ para todo x que não é raiz.

4.11 [Complementar] resposta: p. 175
Estenda a estrutura para responder, em tempo $O(\alpha(n))$ amortizado, "qual o maior elemento do conjunto de x?" e "quantos elementos tem o conjunto de x?".

4.12 [Complementar] resposta: p. 175
Mostre que a *união por tamanho*, que pendura a árvore com menos nós sob a outra, também garante altura no máximo $\lfloor \log_2 n \rfloor$.

4.13 [Complementar] resposta: p. 175
Na floresta sem critério, em que Union(x,y) faz a raiz do conjunto de x apontar para a raiz do conjunto de y, dê uma sequência de $n-1$ uniões que produz uma árvore de altura $n-1$, e uma sequência de n operações Find que custa $\Theta(n^2)$ no total.

4.14 [Complementar] resposta: p. 175
Com listas encadeadas e união ponderada, una os elementos $1,\dots,8$ em rodadas: primeiro aos pares $(\{1,2\},\{3,4\},\dots)$, depois os pares aos pares, e por fim os dois grupos de 4. Quantas atualizações de ponteiro ao representante são feitas no total? Generalize para $n = 2^k$.

4.15 [Complementar] resposta: p. 175
Prove que, com listas encadeadas e união ponderada, qualquer sequência de $n-1$ uniões faz no máximo $n \log_2 n$ atualizações de ponteiro.

4.16 [Complementar] resposta: p. 175
Mostre que, com listas encadeadas *sem* união ponderada, existe uma sequência de $n-1$ uniões que faz $\Theta(n^2)$ atualizações de ponteiro.

4.17 [Complementar] resposta: p. 175
Calcule $\log^* n$ para $n = 2$, 16, 65536, 2^{65536} e 10^{80} (uma estimativa do número de átomos do universo observável).

4.18 [Complementar] resposta: p. 175
Um grafo tem os vértices $1,\dots,10$ e as arestas 12, 34, 23, 56, 78, 89, 65, 97, processadas nessa ordem por Union-Find. Quais arestas provocam uniões efetivas e quais fecham ciclos? Quantas componentes conexas há ao final?

4.19 [Complementar] resposta: p. 175
Num tabuleiro $n \times n$, casas vão sendo abertas uma a uma. Queremos saber, após cada abertura, se existe um caminho de casas abertas (vizinhas na horizontal ou na vertical) ligando a primeira linha à última. Como resolver com Union-Find usando dois vértices virtuais? Qual o custo total?

4.20 [Complementar] resposta: p. 175
Um compilador recebe declarações de equivalência entre variáveis, como $a \equiv b$, $c \equiv d$, $b \equiv c$, e depois consultas "x e y ocupam a mesma posição de memória?". Mostre como resolver com Union-Find e diga quais variáveis ficam equivalentes no exemplo, se também houver $e \equiv f$.

4.21 [Complementar] resposta: p. 175
Quantas operações Find e quantas Union o algoritmo de Kruskal faz num grafo conexo com n vértices e m arestas, no pior caso? Qual o custo dessas operações com rank e compressão, comparado ao da ordenação das arestas?

4.22 [Complementar] resposta: p. 175
Por que a compressão de caminhos não atualiza os ranks? O que o rank passa a representar quando há compressão, e por que ele ainda serve para decidir as uniões?

5

Árvores binárias de busca e árvores AVL

Neste capítulo

- Árvores binárias de busca: busca, inserção e remoção em tempo proporcional à altura.
- Árvores AVL: a condição de balanceamento e a prova de que a altura é logarítmica.
- As rotações simples e duplas, a inserção com no máximo uma rotação e a remoção.
- Uma visão geral das árvores rubro-negras e critérios para escolher entre heap, ABB e AVL.

Heaps são ótimos para encontrar o máximo, mas não sabem procurar uma chave qualquer. Para um **dicionário ordenado** — buscar, inserir e remover por chave, listar as chaves em ordem, encontrar sucessor e predecessor — usamos árvores binárias de busca. Seu custo é proporcional à altura, que pode degenerar até n. As árvores AVL impedem a degeneração com uma condição local de balanceamento, restaurada após cada atualização por meio de rotações que custam $O(1)$.

5.1 Árvores binárias de busca

Uma lista sequencial ordenada permite busca binária em $\Theta(\log n)$, mas inserções e remoções custam $\Omega(n)$, pois exigem deslocar elementos; uma lista encadeada insere rapidamente, mas a busca custa $\Omega(n)$. A árvore binária de busca combina busca eficiente com uma estrutura dinâmica.

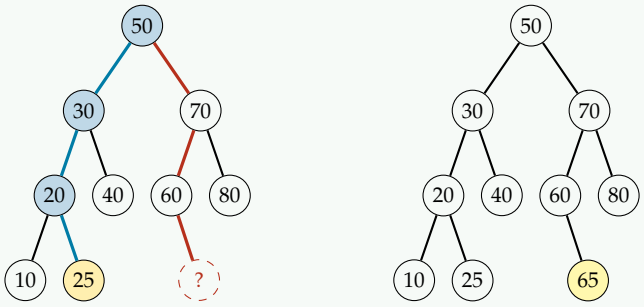
Definição 5.1. Uma **árvore binária de busca** (ABB) é uma árvore binária em que, para todo nó p, a chave de p é maior ou igual às chaves da subárvore esquerda de p e menor ou igual às da subárvore direita. Equivalentemente, o percurso *em ordem* lista as chaves em ordem crescente.

Intuição

Uma ABB é uma busca binária "congelada" numa árvore. A raiz faz o papel do elemento do meio: tudo o que é menor está à esquerda, tudo o que é maior, à direita, e o mesmo vale recursivamente em cada subárvore. Procurar uma chave é responder a uma sequência de perguntas "é menor ou maior?", descendo um nível a cada resposta.

- **Busca**: desce a partir da raiz, indo para a esquerda ou para a direita conforme a comparação; custa $\Theta(h)$, onde h é a altura.
- **Inserção**: busca a posição onde a chave deveria estar e a insere como uma nova **folha**; custa $\Theta(h)$.
- **Remoção** de um nó p: se p é folha, basta retirá-lo; se tem um único filho, esse filho ocupa o lugar de p; se tem dois filhos, o **maior elemento da subárvore esquerda** (o predecessor de p, encontrado descendo uma vez à esquerda e depois sempre à direita) ocupa o lugar de p e é removido da sua posição original, que tem no máximo um filho. Simetricamente, pode-se usar o menor da subárvore direita. Custa $\Theta(h)$.

Exemplo 5.2 (Busca e inserção)**.** Na ABB da Figura 5.1(a), a busca por 25 compara com 50 (menor: esquerda), com 30 (menor: esquerda), com 20 (maior: direita) e encontra 25: quatro comparações. A busca por 65 compara com 50 (direita), com 70 (esquerda) e com 60 (direita), e encontra um ponteiro nulo: 65 não está na árvore. Inserir 65 é exatamente pendurar uma nova folha nesse ponteiro nulo, como filho direito de 60 (Figura 5.1(b)). Percorrendo a árvore em ordem (subárvore esquerda, raiz, subárvore direita), obtemos $10, 20, 25, 30, 40, 50, 60, 65, 70, 80$, em ordem crescente.

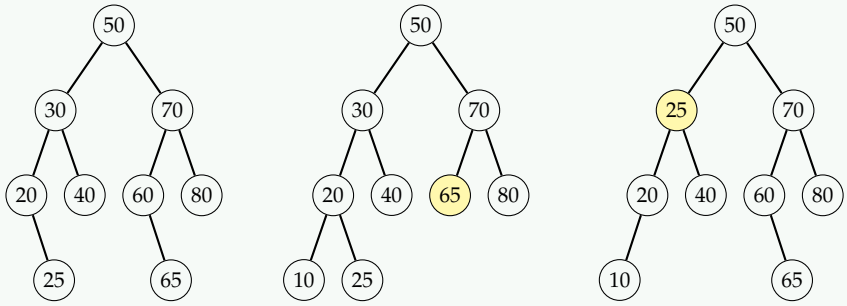


(a) buscas por 25 e por 65 (b) depois de inserir 65

Figura 5.1. (a) As buscas por 25 (caminho azul, sucesso) e por 65 (caminho vermelho, fracasso). (b) A inserção de 65 pendura uma folha onde a busca fracassou.

Exemplo 5.3 (Os três casos da remoção)**.** Partimos da árvore da Figura 5.1(b).

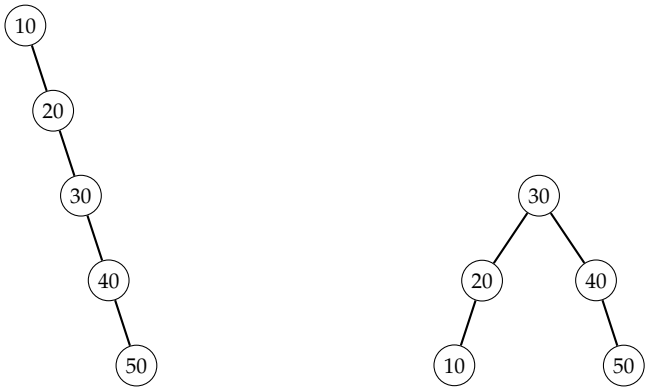
- **Remover 10** (folha): basta retirá-la.
- **Remover 60** (um filho): o filho 65 sobe e ocupa o lugar de 60.
- **Remover 30** (dois filhos): o predecessor de 30 é o maior da subárvore esquerda, encontrado descendo uma vez à esquerda (20) e depois sempre à direita (25). A chave 25 sobe para o lugar de 30, e o nó onde ela estava, uma folha, é retirado. A ordem continua correta: 25 é maior que tudo o que ficou à esquerda e menor que tudo à direita.



(a) sem 10 (folha) (b) sem 60 (um filho) (c) sem 30 (dois filhos)

Figura 5.2. Os três casos da remoção numa ABB, cada um aplicado à árvore da Figura 5.1(b).

A altura de uma ABB com n nós está entre $\lfloor \log_2 n \rfloor + 1$ e n (medida em níveis). Inserir as chaves em ordem crescente ou decrescente produz uma "lista" de altura n, e todas as operações passam a custar $\Theta(n)$ (Figura 5.3).



inserção de 10, 20, 30, 40, 50 inserção de 30, 20, 40, 10, 50

Figura 5.3. As mesmas cinco chaves inseridas em duas ordens. Em ordem crescente, a ABB degenera numa lista de altura 5; na ordem 30, 20, 40, 10, 50, fica com altura 3.

Árvores completas minimizam a altura, mas mantê-las completas depois de cada inserção exige reorganizações globais de custo $\Omega(n)$. A saída é exigir algo mais fraco: que a árvore seja **balanceada**, isto é, que tenha altura $O(\log n)$ e que o mesmo valha para cada subárvore, e que o balanceamento possa ser restaurado em tempo $O(\log n)$.

5.2 Árvores AVL

Definição 5.4. Uma árvore binária é **AVL** se, em todo nó v, as alturas das subárvores esquerda e direita diferem de no máximo 1. O **balanço** de v é

$$\mathrm{bal}(v) = h_D(v) - h_E(v),$$

a altura da subárvore direita menos a da esquerda; um balanço -1 indica que v "pesa" à esquerda. O nó v é *regulado* se $\mathrm{bal}(v) \in \{-1, 0, 1\}$ e *desregulado* caso contrário. Folhas têm balanço 0, e toda árvore completa é AVL (a recíproca é falsa).

Exemplo 5.5 (Calculando balanços)**.** Na árvore da esquerda da Figura 5.4, o nó 10 não tem filho esquerdo (altura 0) e tem o filho direito 12 (altura 1): $\mathrm{bal}(10) = 1 - 0 = +1$. Na raiz, as duas subárvores têm altura 2, e $\mathrm{bal}(8) = 0$. Todos os balanços estão em $\{-1, 0, 1\}$: é uma AVL, embora não seja completa. Na árvore da direita, a subárvore direita de 10 tem altura 2 (o caminho $12 \to 14$) e a esquerda é vazia: $\mathrm{bal}(10) = +2$, e a árvore **não** é AVL, ainda que todos os outros nós, inclusive a raiz, estejam regulados.

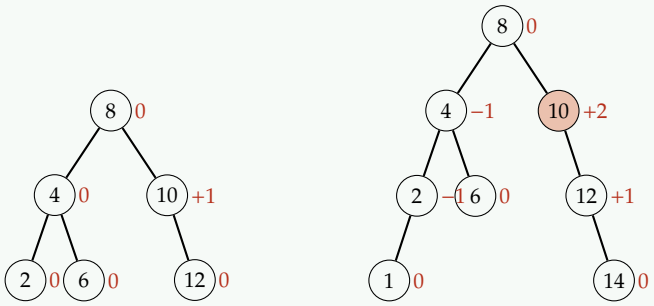


Figura 5.4. Balanços em vermelho. À esquerda, uma AVL; à direita, uma ABB que não é AVL por causa do nó 10.

Teorema 5.6. *Toda árvore AVL com n nós tem altura $O(\log n)$; mais precisamente, $h < 1{,}44 \log_2(n+2)$.*

Demonstração. Seja $N(h)$ o número mínimo de nós de uma AVL de altura h, medida em níveis. Então $N(1) = 1$, $N(2) = 2$ e, para $h \geq 3$, uma AVL mínima de altura h tem a raiz, uma subárvore de altura $h-1$ e a outra com a menor altura permitida, $h-2$, ambas mínimas:

$$N(h) = 1 + N(h-1) + N(h-2).$$

Essas árvores mínimas são chamadas **árvores de Fibonacci**, e $N(h) = F_{h+2} - 1$. Como N é crescente, $N(h) > 2N(h-2)$, de onde $N(h) \geq 2^{\lfloor h/2 \rfloor}$, ou seja, $h \leq 2\log_2 N(h) + 1$. Como qualquer AVL de altura h com n nós tem $n \geq N(h)$, segue que $h = O(\log n)$; o cálculo exato com a fórmula de F_k dá a constante 1,44. □

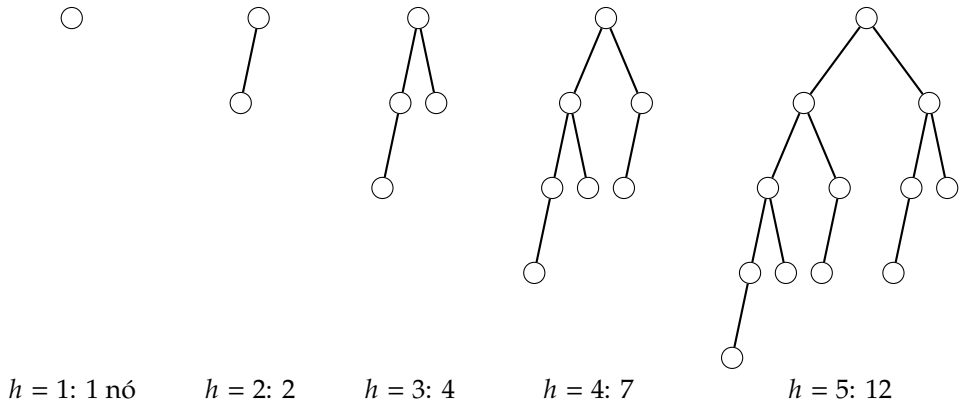


Figura 5.5. As árvores de Fibonacci: as AVL com o menor número de nós para cada altura. Cada uma é uma raiz com as duas anteriores penduradas.

Exemplo 5.7 (Quão alta pode ser uma AVL?)**.** A recorrência dá $N(1), \ldots, N(5) = 1, 2, 4, 7, 12$, os tamanhos das árvores da Figura 5.5. Como uma AVL de altura 5 tem pelo menos 12 nós, uma AVL com 10 nós tem altura no máximo 4, e essa altura é possível, pois $N(4) = 7 \leq 10$; uma árvore completa com 10 nós também tem altura 4. A diferença entre a melhor e a pior AVL é pequena, e continua pequena para n grande: com um milhão de nós, a altura de uma AVL é no máximo 28, contra 20 de uma árvore completa e até 10^6 de uma ABB degenerada.

Assim, a busca numa AVL custa $O(\log n)$. Resta mostrar que a inserção e a remoção, incluindo o rebalanceamento, também custam $O(\log n)$.

5.3 Inserção e atualização dos balanços

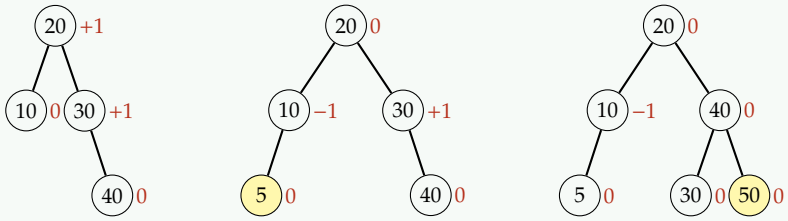
A inserção começa como numa ABB: a nova chave vira uma folha (se a chave já existe, nada é feito). Em seguida, percorre-se o caminho de volta, do pai do novo nó em direção à raiz, atualizando os balanços. Suponha que a inserção ocorreu na subárvore **esquerda** de um nó p; o caso direito é simétrico, com os sinais trocados.

bal(p) antes	depois	altura de $T(p)$	ação
+1	0	não mudou	**para**: os ancestrais não mudam
0	−1	aumentou 1	continua com o pai de p (se p é a raiz, para)
−1	−2	—	p está desregulado: **rotação**, e depois **para**

Uma inserção aumenta alturas em no máximo 1, então o nó desregulado tem balanço ±2. O nó p a rotacionar é o **primeiro nó desregulado encontrado ao subir da folha para a raiz**, isto é, o mais profundo. Como a rotação devolve à subárvore a altura que ela tinha antes da inserção, nenhum ancestral é afetado: **uma inserção faz no máximo uma rotação**, simples ou dupla.

Exemplo 5.8 (As três linhas da tabela)**.** Na árvore $20^{+1}(10,\ 30^{+1}(\cdot, 40))$ (Figura 5.6(a)):

- **Inserir 5**, à esquerda de 10. Em 10, o balanço passa de 0 a −1: a subárvore de 10 ficou mais alta, e subimos. Em 20, a inserção veio da esquerda e o balanço passa de +1 a 0: a altura de $T(20)$ não mudou, e paramos (Figura 5.6(b)).
- **Inserir 50**, à direita de 40. Em 40, $0 \longrightarrow +1$, e subimos; em 30, $+1 \longrightarrow +2$: 30 é o primeiro desregulado. O filho do lado pesado, 40, tem balanço +1 (mesmo sinal), e uma rotação simples à esquerda em 30 produz 40(30, 50), com a altura de antes. Paramos sem tocar em 20 (Figura 5.6(c)).



(a) inicial (b) após inserir 5 (c) após inserir 50

Figura 5.6. Atualização de balanços na inserção (Exemplo 5.8). O nó novo está em destaque.

5.4 As rotações

Seja p o nó desregulado mais profundo. Há quatro casos, determinados pelo sinal do balanço de p e pelo do filho do lado mais pesado.

Caso	bal(p)	filho do lado pesado	sinais	rotação
esquerda-esquerda	−2	u (esq.), bal(u) = −1	iguais	**simples à direita** em p
esquerda-direita	−2	u (esq.), bal(u) = +1	opostos	**dupla à direita**: esquerda em u, direita em p
direita-direita	+2	z (dir.), bal(z) = +1	iguais	**simples à esquerda** em p
direita-esquerda	+2	z (dir.), bal(z) = −1	opostos	**dupla à esquerda**: direita em z, esquerda em p

Uma regra fácil de lembrar: **balanço positivo pede rotações à esquerda e negativo, à direita; sinais iguais pedem rotação simples e sinais opostos, dupla**. Um filho com balanço 0 só ocorre na remoção, e então se usa a rotação simples.

Exemplo 5.9 (Os quatro casos com três chaves)**.** A menor situação de cada caso aparece ao inserir 10, 20 e 30 numa árvore vazia, em ordens diferentes (Figura 5.7). Nos quatro casos o resultado é o mesmo, 20(10, 30): a rotação sempre leva para a raiz a chave do meio.

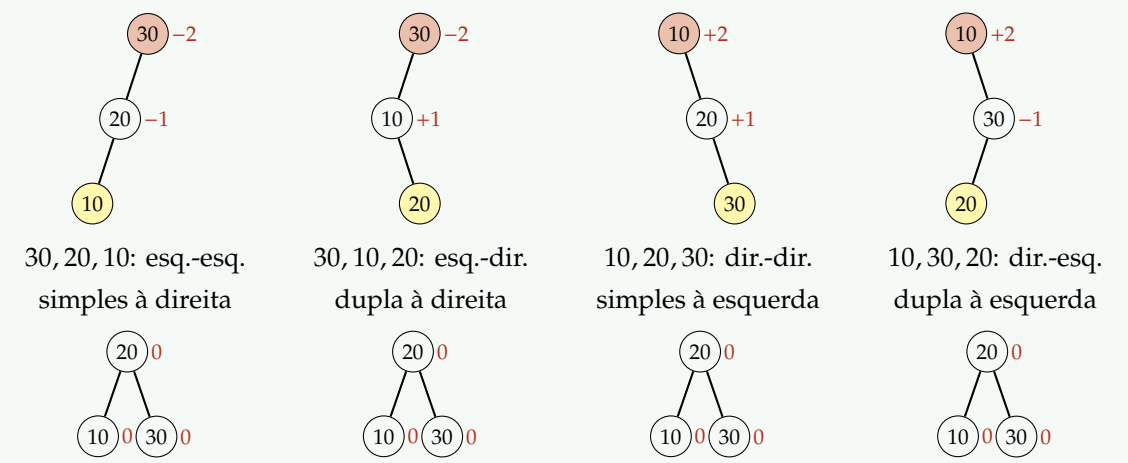


Figura 5.7. Os quatro casos de rotação. Na linha de cima, a árvore logo após a terceira inserção, com o nó desregulado em vermelho; na de baixo, depois da rotação.

Em geral, as subárvores penduradas nos nós envolvidos precisam ser redistribuídas. As figuras abaixo mostram as rotações à direita com subárvores genéricas $T_1, \ldots, T_4$.

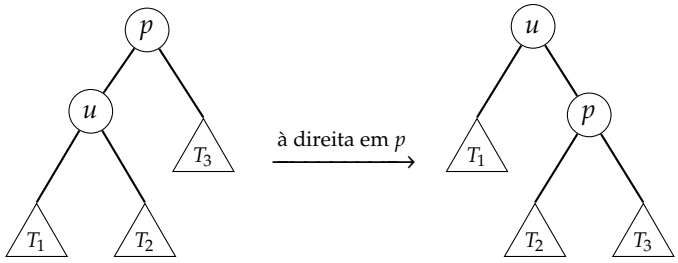


Figura 5.8. Rotação simples à direita (caso esquerda-esquerda), com $h(T_1) = h + 1$ e $h(T_2) = h(T_3) = h$. Depois da rotação, $\text{bal}(u) = \text{bal}(p) = 0$ e a altura volta a $h + 2$.

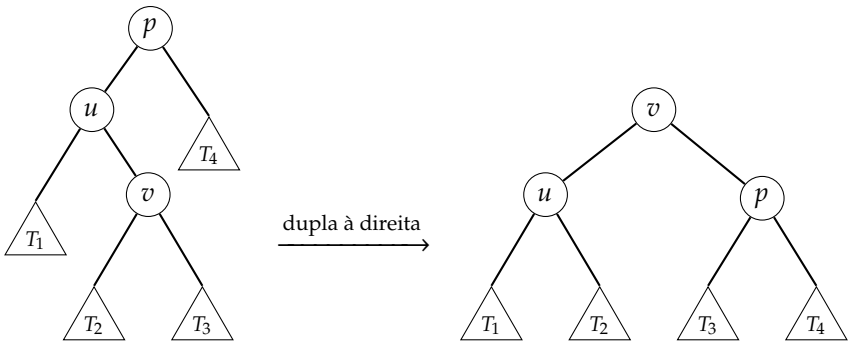


Figura 5.9. Rotação dupla à direita (caso esquerda-direita): v sobe para a raiz e fica com $\text{bal}(v) = 0$. Se $\text{bal}(v)$ era -1, ficam $\text{bal}(u) = 0$ e $\text{bal}(p) = +1$; se era $+1$, $\text{bal}(u) = -1$ e $\text{bal}(p) = 0$; se era 0 (v é o nó inserido), ambos ficam 0.

Os casos direita-direita e direita-esquerda são os espelhos desses: na rotação simples à esquerda, o filho direito z sobe, p passa a ser seu filho esquerdo e o antigo filho esquerdo de z passa a ser filho direito de p. **Rotações preservam a ordem da ABB**, pois o percurso em ordem T_1, u, T_2, p, T_3 é o mesmo antes e depois, e custam $O(1)$, porque só alteram alguns ponteiros.

Por que a rotação dupla?

No caso esquerda-direita, uma rotação simples à direita em p não resolve: a subárvore pesada, a de v, mudaria de lado junto com u e continuaria pendurada no lugar errado, só que agora do lado direito. A primeira rotação (à esquerda em u) transforma o "zigue-

zague" $p \to u \to v$ numa "linha reta" $p \to v \to u$, e a segunda (à direita em p) é o caso esquerda-esquerda comum. Teste com 30, 10, 20: rotacionar só em 30 daria $10(\cdot, 30(20, \cdot))$, ainda desregulado.

5.5 Pseudocódigo da inserção

Cada nó tem os campos `esq`, `dir`, `chave` e `bal`; λ é o ponteiro nulo e $pt\uparrow.campo$ acessa um campo do nó apontado por pt. A chamada externa é Ins-AVL$(x, ptraiz)$. A função devolve um valor lógico h que indica se **a altura da subárvore aumentou**; só nesse caso o pai precisa atualizar o seu balanço.

Inserção em árvore AVL

```
Procedimento Iniciar-nó(pt, x)
    alocar(pt); pt↑.esq := λ; pt↑.dir := λ; pt↑.chave := x; pt↑.bal := 0
Função Ins-AVL(x, ref pt)                    % verdadeiro se a altura de T(pt) aumentou
    se pt = λ então
        Iniciar-nó(pt, x); retorna verdadeiro
    se x = pt↑.chave então
        retorna falso                        % chave repetida
    se x < pt↑.chave então
        h := Ins-AVL(x, pt↑.esq)
        se h então
            (pt, h) := AtualizaBalanço(pt, "esquerda")
    senão
        h := Ins-AVL(x, pt↑.dir)
        se h então
            (pt, h) := AtualizaBalanço(pt, "direita")
    retorna h
Função AtualizaBalanço(pt, a)
    h := falso
    se a = "esquerda" então
        se pt↑.bal = 1 então
            pt↑.bal := 0
        senão se pt↑.bal = 0 então
            pt↑.bal := −1; h := verdadeiro   % a altura aumentou
        senão
            pt := RotaçõesDireita(pt)        % era −1 e ficaria −2
    senão
        se pt↑.bal = −1 então
            pt↑.bal := 0
        senão se pt↑.bal = 0 então
            pt↑.bal := 1; h := verdadeiro
        senão
            pt := RotaçõesEsquerda(pt)
    retorna (pt, h)          % após uma rotação, pt aponta para a nova raiz da subárvore
```

Rotações

```
Função RotaçõesDireita(pt)
    ptu := pt↑.esq
    se ptu↑.bal = −1 então                   % simples à direita
        pt↑.esq := ptu↑.dir; ptu↑.dir := pt; pt↑.bal := 0; ptu↑.bal := 0; pt := ptu
    senão                                    % dupla à direita
```

```
        ptv := ptu↑.dir
        ptu↑.dir := ptv↑.esq; ptv↑.esq := ptu; pt↑.esq := ptv↑.dir; ptv↑.dir := pt
        se ptv↑.bal = −1 então pt↑.bal := 1
        senão pt↑.bal := 0
        se ptv↑.bal = 1 então ptu↑.bal := −1
        senão ptu↑.bal := 0
        ptv↑.bal := 0; pt := ptv
    retorna pt
Função RotaçõesEsquerda(pt)
    ptz := pt↑.dir
    se ptz↑.bal = 1 então                                   % simples à esquerda
        pt↑.dir := ptz↑.esq; ptz↑.esq := pt; pt↑.bal := 0; ptz↑.bal := 0; pt := ptz
    senão                                                   % dupla à esquerda
        pty := ptz↑.esq
        ptz↑.esq := pty↑.dir; pty↑.dir := ptz; pt↑.dir := pty↑.esq; pty↑.esq := pt
        se pty↑.bal = 1 então pt↑.bal := −1
        senão pt↑.bal := 0
        se pty↑.bal = −1 então ptz↑.bal := 1
        senão ptz↑.bal := 0
        pty↑.bal := 0; pt := pty
    retorna pt
```

Detalhes do pseudocódigo

- **Para que serve h?** Ele indica se a altura da subárvore cresceu. Se não cresceu, nenhum ancestral muda de balanço; por isso AtualizaBalanço só é chamada quando h é verdadeiro.
- **Por que atualizar pt depois da rotação?** A raiz da subárvore muda (u, v, z ou y sobe). Como pt é passado por referência, a atribuição religa a nova raiz ao pai, ou atualiza a raiz da árvore.
- Os três testes de AtualizaBalanço precisam ser **mutuamente exclusivos** ("senão se"). Escritos como "se" consecutivos, um nó com balanço 1 viraria 0 e, em seguida, o teste "balanço = 0" dispararia de novo, deixando o nó com −1 e h verdadeiro.
- Depois de uma rotação na inserção, h é falso: a altura voltou ao valor antigo.

5.6 Remoção

A remoção começa como numa ABB (se o nó tem dois filhos, o maior elemento da subárvore esquerda toma o seu lugar) e sobe a partir do nó *fisicamente removido*, atualizando os balanços. Agora a pergunta é se a altura **diminuiu**. Para uma remoção na subárvore **direita** de p (o caso esquerdo é simétrico):

bal(p) antes	depois	altura de $T(p)$	ação
+1	0	**diminuiu**	continua com o pai
0	−1	não mudou	**para**
−1	−2	—	rotação à direita (ver abaixo)

Na rotação, olha-se o filho esquerdo u de p:

- se bal(u) = −1, faz-se a rotação simples à direita; ambos ficam com balanço 0 e a altura **diminuiu**, de modo que o processo continua subindo;

- se bal(u) = 0, caso que só ocorre na remoção, faz-se a rotação simples à direita; ficam bal(p) = −1 e bal(u) = +1, a altura **não muda** e o processo para;
- se bal(u) = +1, faz-se a rotação dupla à direita; a altura **diminuiu** e o processo continua subindo.

Por isso uma remoção pode exigir uma rotação em **cada** ancestral: são $O(\log n)$ rotações de custo $O(1)$, e o total continua $O(\log n)$.

Por que a remoção pode propagar e a inserção não

Na inserção, o nó desregulado ficou desregulado porque a sua subárvore *cresceu*; a rotação a encolhe de volta à altura original, e os ancestrais nem percebem. Na remoção, a subárvore desregulada *encolheu* de um lado; a rotação conserta o nó, mas em geral a subárvore resultante fica um nível mais baixa do que era antes da remoção, e esse encolhimento pode desregular o ancestral seguinte.

Reaproveitar as rotações da inserção

As funções RotaçõesDireita e RotaçõesEsquerda acima testam apenas "bal(u) = −1" (ou "= 1") para decidir entre a rotação simples e a dupla. Na remoção, o caso bal(u) = 0 cairia na rotação *dupla*, o que está errado: com bal(u) = 0 deve-se fazer a rotação **simples** (o teste passa a ser "≤ 0"), ajustar bal(p) = −1 e bal(u) = +1 e informar que a altura não diminuiu.

Exemplo 5.10 (Remoções pequenas)**.**

- Em $20^{+1}(10,\ 40^{0}(30, 50))$, remover 10 deixa bal(20) = +2, com o filho 40 de balanço 0: rotação simples à esquerda em 20, resultando em $40^{-1}(20^{+1}(\cdot, 30),\ 50)$, com a altura inalterada.
- Em $20^{-1}(10^{+1}(\cdot, 15),\ 30)$, remover 30 deixa bal(20) = −2, com o filho 10 de balanço +1: rotação dupla à direita, resultando em 15(10, 20).

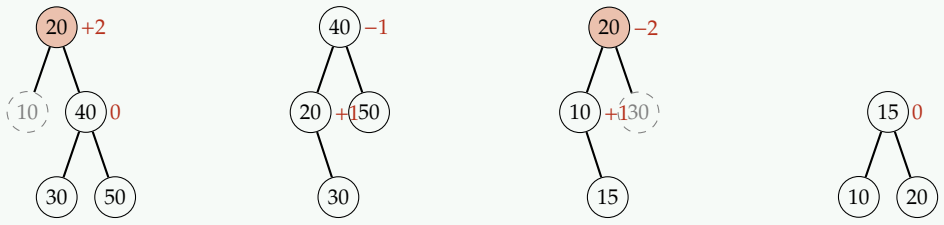
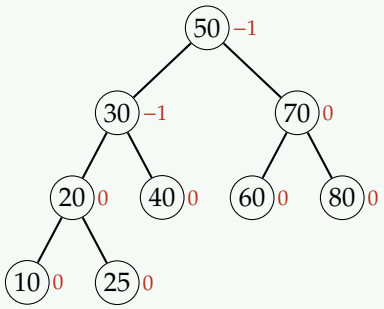


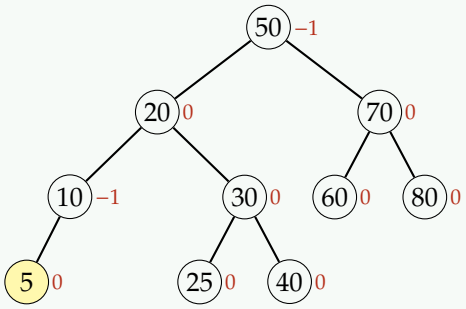
Figura 5.10. As duas remoções do Exemplo 5.10. O nó removido está tracejado; o desregulado, em vermelho.

5.7 Exemplos resolvidos

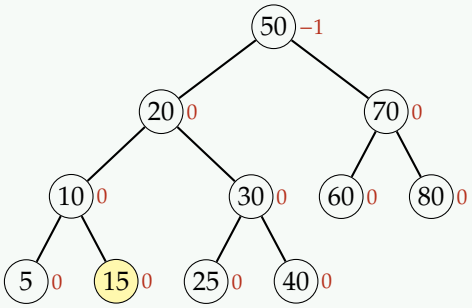
Exemplo 5.11 (Inserções com rotação simples e dupla)**.** Partimos da árvore abaixo (balanços em vermelho) e inserimos 5, 15, 35 e 45.



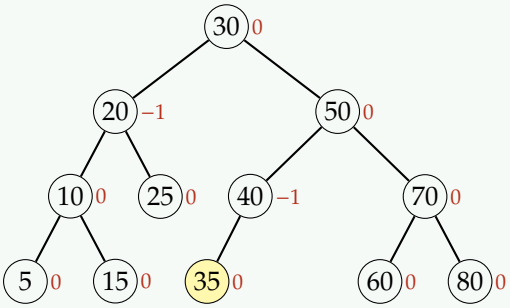
Inserir 5 (filho esquerdo de 10). Subindo: bal(10) = −1, bal(20) = −1, bal(30) = −2. O primeiro nó desregulado é **30**, e o filho esquerdo 20 tem balanço −1 (sinais iguais): caso esquerda-esquerda, **rotação simples à direita em 30**. A altura da subárvore volta a 3, e 50 continua com −1.



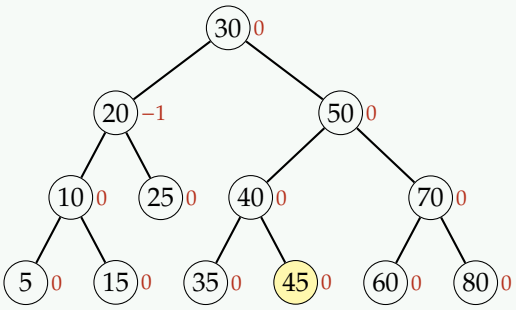
Inserir 15 (filho direito de 10): bal(10) passa de −1 a 0 e a altura de $T(10)$ não muda; o processo para, sem rotação.



Inserir 35 (filho esquerdo de 40). Subindo: bal(40) = −1, bal(30) = +1, bal(20) = +1, bal(50) = −2. O primeiro nó desregulado é **50**, e o filho esquerdo 20 tem balanço +1 (sinais opostos): caso esquerda-direita, **rotação dupla à direita** — à esquerda em 20 e depois à direita em 50. O nó $v = 30$ sobe para a raiz; como bal(30) era +1, ficam bal(20) = −1 e bal(50) = 0.

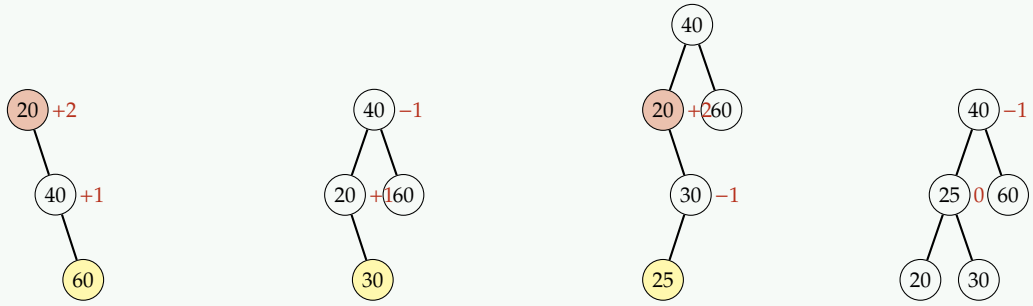


Inserir 45 (filho direito de 40): bal(40) passa de −1 a 0 e a altura de $T(40)$ não muda. A árvore final é:



Numa ABB comum, inserções em ordem desfavorável (crescente, decrescente ou em ziguezague) produziriam altura $\Theta(n)$ e operações de custo $\Theta(n)$. A AVL garante altura no máximo $1{,}44 \log_2 n$ (Teorema 5.6), qualquer que seja a ordem de chegada, pagando só $O(1)$ rotações por inserção.

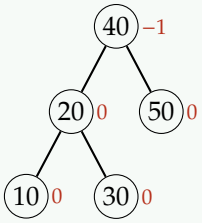
Exemplo 5.12 (Inserindo 20, 40, 60, 30, 25 numa árvore vazia)**.** Depois de 20 e 40, temos $20^{+1}(\cdot, 40)$. Inserir 60 deixa bal(20) = +2 com o filho 40 de balanço +1: rotação simples à esquerda em 20, resultando em $40^{0}(20, 60)$. Inserir 30 dá $40^{-1}(20^{+1}(\cdot, 30),\ 60)$. Inserir 25 (à esquerda de 30) deixa bal(30) = −1 e bal(20) = +2: o primeiro desregulado é **20**, com filho direito 30 de balanço −1, e a rotação é **dupla à esquerda** (à direita em 30, à esquerda em 20). A árvore final é $40^{-1}(25^{0}(20^{0}, 30^{0}),\ 60^{0})$.



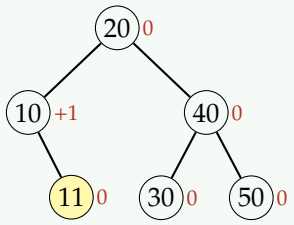
(a) 60 desregula 20 (b) após a rotação e o 30 (c) 25 desregula 20 (d) após a dupla

Figura 5.11. As inserções do Exemplo 5.12: uma rotação simples e uma dupla, ambas à esquerda.

Exemplo 5.13 (Inserindo 50, 40, 30, 20, 10 e depois 11)**.** Inserir 30 deixa bal(50) = −2 com o filho 40 de balanço −1: rotação simples à direita em 50, resultando em 40(30, 50). Inserir 20 dá $40^{-1}(30^{-1}(20, \cdot),\ 50)$. Inserir 10: subindo, bal(20) = −1 e bal(30) = −2, de modo que o **primeiro desregulado é 30**, e não 40; faz-se a rotação simples à direita em 30, e a árvore fica $40^{-1}(20^{0}(10^{0}, 30^{0}),\ 50^{0})$.



Inserir 11 (à direita de 10) deixa bal(10) = +1, bal(20) = −1 e bal(40) = −2: o desregulado é 40, com filho esquerdo 20 de balanço −1, e a rotação é **simples à direita em 40**, resultando em $20^{0}(10^{+1}(\cdot, 11), 40^{0}(30, 50))$.



Compare com a Figura 5.3: numa ABB comum, as chaves de 10 a 50 em ordem decrescente formariam uma lista, como em ordem crescente; aqui formam uma árvore de altura 3.

Rotacionar no nó certo

No exemplo anterior, ao inserir 10, é tentador rotacionar em 40, a raiz. Mas a rotação deve ser feita no desregulado *mais profundo*, 30; depois dela, o nó 40 fica com balanço -1 e nem chega a se desregular. Rotacionar em 40 produziria a árvore $30(20(10,\cdot),40(\cdot,50))$, e a inserção seguinte de 11 levaria, erradamente, a uma rotação dupla.

5.8 Heap, ABB ou AVL?

Como escolher

- **Fila de prioridade pura** (sempre atender o elemento de maior prioridade): **heap**. O máximo sai em $O(1)$, inserir e remover custam $O(\log n)$, e a estrutura é um vetor simples. A AVL também faria tudo em $O(\log n)$, mas não dá o máximo em $O(1)$ sem um ponteiro extra e é mais pesada.
- **Busca por chave arbitrária, remoção de um elemento específico, percurso em ordem, sucessor e predecessor, consultas por intervalo**: **AVL**. O heap não tem ordem entre irmãos, e buscar uma chave nele custa $O(n)$.
- **ABB simples ou AVL**: com inserções e remoções frequentes e ordem de chegada imprevisível, **AVL**, pois a ABB pode degenerar para altura $\Theta(n)$.

Exemplo 5.14 (Aplicando o roteiro)**.** Uma agenda de compromissos precisa listar os eventos entre duas datas e remover um evento cancelado qualquer: consultas por intervalo e remoções arbitrárias pedem uma **AVL** indexada pela data. Já o atendimento de chamados por urgência, em que só interessa o mais urgente, pede um **heap**. Um dicionário que é carregado uma vez a partir de um arquivo ordenado e depois só consultado degeneraria numa ABB simples (as chaves chegam em ordem crescente), mas pode ser um vetor ordenado com busca binária, ou uma AVL.

5.9 Árvores rubro-negras

As árvores rubro-negras são outra forma clássica de manter uma ABB balanceada, e são a escolha mais comum em bibliotecas de linguagens de programação.

Definição 5.15. Uma **árvore rubro-negra** é uma ABB em que cada nó é *vermelho* ou *preto* (um bit extra por nó) e: (1) a raiz é preta; (2) as folhas nulas (λ) são pretas; (3) um nó vermelho tem os dois filhos pretos, isto é, não há dois vermelhos seguidos; (4) todo caminho de um nó até as folhas nulas abaixo dele tem o mesmo número de nós pretos, chamado *altura negra* do nó, bh.

Exemplo 5.16 (Conferindo as propriedades)**.** Na árvore da Figura 5.12, a raiz 13 é preta; os vermelhos 8, 17, 6, 22 e 27 têm apenas filhos pretos (inclusive as folhas nulas, não desenhadas). Contando os pretos de 13 até as folhas nulas, sem contar o próprio 13 e contando a folha nula, todo caminho encontra 2: por exemplo, $13 \to 8 \to 1 \to 6 \to \lambda$ passa pelos pretos 1 e λ, e $13 \to 17 \to 25 \to 22 \to \lambda$ passa por 25 e λ. Logo $bh(13) = 2$.

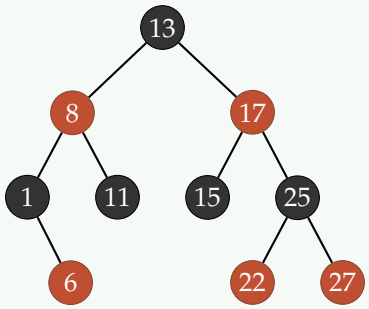


Figura 5.12. Uma árvore rubro-negra com altura negra 2. As folhas nulas, todas pretas, foram omitidas.

Por que é balanceada. Pela propriedade (4), a subárvore de um nó x tem pelo menos $2^{bh(x)} - 1$ nós internos (por indução). Pela propriedade (3), pelo menos metade dos nós de qualquer caminho da raiz até uma folha é preta, então $bh(\text{raiz}) \geq h/2$. Logo $n \geq 2^{h/2} - 1$, isto é, $h \leq 2\log_2(n+1)$. Dito de outro modo: como todos os caminhos têm o mesmo número de pretos e não há dois vermelhos seguidos, o caminho mais longo da raiz até uma folha nula tem no máximo cerca do dobro de nós do mais curto. Na Figura 5.12, os caminhos vão de 3 nós ($13 \to 8 \to 11$) a 4 nós ($13 \to 17 \to 25 \to 22$).

Inserção. Insere-se como numa ABB e pinta-se o novo nó z de *vermelho*, o que não altera alturas negras; se o pai de z também é vermelho, a propriedade (3) é violada. A correção olha o *tio* y de z: (caso 1) se o tio é vermelho, pintam-se o pai e o tio de preto e o avô de vermelho, e o processo continua a partir do avô; (caso 2) se o tio é preto e z está em zigue-zague com o pai e o avô, uma rotação no pai o transforma no caso 3; (caso 3) se o tio é preto e z está "em linha", faz-se uma rotação no avô e trocam-se as cores do pai e do avô. Ao final, a raiz é pintada de preto. São no máximo **duas rotações** por inserção, e $O(\log n)$ recolorações.

Exemplo 5.17 (Inserindo 10, 20, 30 numa rubro-negra vazia)**.** O 10 entra como raiz e é pintado de preto. O 20 entra vermelho, à direita de 10: o pai é preto, e nada é violado. O 30 entra vermelho, à direita de 20: pai e filho vermelhos violam a regra (3). O tio de 30 (filho esquerdo de 10) é uma folha nula, preta, e $10 \to 20 \to 30$ estão em linha: é o caso 3. A rotação à esquerda no avô 10 leva 20 para a raiz, e a troca de cores deixa 20 preto e 10 vermelho (Figura 5.13). É a mesma rotação que a AVL faria (Figura 5.7), seguida de uma recoloração.

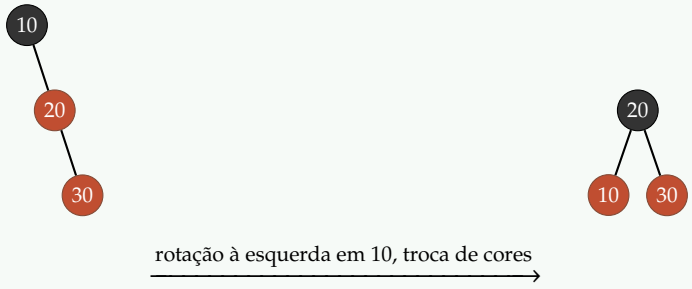


Figura 5.13. Inserção de 30 na rubro-negra $10(\cdot, 20)$: o caso 3 resolve-se com uma rotação e uma troca de cores.

Remoção. Remove-se como numa ABB; se o nó retirado era preto, surge um "preto duplo", resolvido olhando o irmão, em quatro casos, com no máximo **três rotações**.

AVL ou rubro-negra?

A AVL é mais rigidamente balanceada ($h \leq 1{,}44 \log n$, contra $2 \log n$), o que torna as buscas um pouco mais rápidas. A rubro-negra faz menos rotações nas atualizações — $O(1)$ rotações na remoção, contra $O(\log n)$ na AVL —, o que a torna preferida em bibliotecas e sistemas com muitas atualizações.

Pontos-chave

AVL: bal $= h_D - h_E \in \{-1, 0, 1\}$ e altura no máximo $1{,}44 \log n$ (árvores de Fibonacci). Inserção: sobe atualizando os balanços; $\pm 1 \to 0$ para, $0 \to \pm 1$ continua, $\pm 1 \to \pm 2$ rotaciona no desregulado **mais profundo** e para (no máximo uma rotação). Sinais iguais: simples; opostos: dupla. Remoção: pode rotacionar em todos os ancestrais; filho com balanço 0 pede rotação simples, e a altura não muda.

5.10 Exercícios

Exercícios do curso

5.1 [Monitoria] resposta: p. 175
(a) Defina árvore binária de busca balanceada. (b) Defina árvore AVL. (c) Qual a relação entre árvores AVL e árvores balanceadas? (d) Defina o fator de balanço de um nó.

5.2 [Monitoria] resposta: p. 176
Um nó é inserido na subárvore esquerda de p. (a) Para bal$(p) = 1, 0$ e -1 antes da inserção, diga o novo balanço, se a altura de $T(p)$ mudou e se é preciso continuar subindo. (b) Se bal$(p) = -2$ e o filho esquerdo u tem bal$(u) = -1$, qual rotação deve ser feita? E se bal$(u) = +1$? Qual a diferença entre os dois casos?

5.3 [Monitoria] resposta: p. 176
No pseudocódigo da inserção: (a) qual a finalidade da variável lógica h? (b) Por que AtualizaBalanço só é chamada quando h é verdadeiro? (c) Por que o ponteiro pt precisa ser atualizado após uma rotação? (d) Um aluno escreveu os três testes de AtualizaBalanço como "se" consecutivos: o que dá errado?

5.4 [Monitoria] resposta: p. 176
Insira (a) $40, 20, 10, 30, 35$ e (b) $10, 20, 30, 40, 50$ numa AVL vazia, indicando os balanços, as rotações (simples ou dupla, e a direção) e a árvore final. (c) Em seguida, insira 49 na árvore de (b).

5.5 [Avaliação] resposta: p. 176
Na AVL $50(30(20(10, 25), 40), 70(60, 80))$, que armazena códigos de produtos, insira $5, 15, 35, 45$, nessa ordem. (a) Após cada inserção, verifique os balanços e desenhe a árvore após cada rotação. (b) Quando a árvore deixar de ser AVL, identifique o primeiro nó desbalanceado encontrado ao subir da folha para a raiz, o caso e a rotação necessária. (c) Por que uma ABB comum poderia ter desempenho muito pior que a AVL se os códigos chegassem numa ordem desfavorável?

5.6 [Lista] resposta: p. 176
Construa uma AVL em que remover um único nó exija rotações em *todos* os ancestrais; o nó removido deve ter pelo menos três ancestrais.

5.7 [Lista] resposta: p. 176
Descreva em alto nível uma estratégia para a fusão de duas árvores AVL.

5.8 [Lista] resposta: p. 176
É possível usar árvores AVL como estrutura de conjuntos disjuntos? Qual seria a complexidade de Find e Union?

5.9 [Monitoria] resposta: p. 176
(a) Uma base de registros tem inserções e remoções frequentes e consultas rápidas por chave: ABB simples ou AVL? (b) Uma fila de impressão sempre processa a tarefa de maior prioridade: heap ou AVL? Justifique.

5.10 [Slides] resposta: p. 176
Escreva o pseudocódigo das rotações, da inserção e da remoção em árvores AVL.

5.11 [Slides] resposta: p. 176
Resolva os problemas 1195 (*Árvore Binária de Busca*), 1200 e 1201 (*Operações em ABP I e II*) e 1191 (*Recuperação da Árvore*) do Beecrowd, e o problema SID do SPOJ. No 1191, dados os percursos em pré-ordem e em ordem de uma árvore binária, deve-se imprimir o percurso em pós-ordem.

5.12 [Pesquisa] resposta: p. 176
Apresente a definição de árvore rubro-negra e suas propriedades; justifique por que elas garantem altura $O(\log n)$; explique os algoritmos de inserção e remoção, com rotações e recolorações.

Exercícios complementares

5.13 [Complementar] resposta: p. 177
Insira $1, 2, 3, 4, 5, 6, 7$ numa AVL vazia. Em quais inserções ocorrem rotações, e de que tipo? Qual a árvore final?

5.14 [Complementar] resposta: p. 177
Insira $10, 20, 30, 25, 28, 27$ numa AVL vazia, indicando cada rotação e a árvore final com os balanços.

5.15 [Complementar] resposta: p. 177
Na árvore final do Exercício 5.13, remova 1, depois 3, depois 2 (na remoção de nó com dois filhos, use o maior da subárvore esquerda). Indique balanços e rotações.

5.16 [Complementar] resposta: p. 177
Qual a altura máxima (em níveis) de uma AVL com 20 nós? E com 32 nós? Use $N(h) = 1 + N(h-1) + N(h-2)$.

5.17 [Complementar] resposta: p. 177
Prove que as rotações simples e duplas preservam o percurso em ordem e, portanto, a propriedade de ABB.

5.18 [Complementar] resposta: p. 177
Quantas ABBs distintas armazenam as chaves $\{1, 2, 3\}$? E $\{1, 2, 3, 4\}$? Dê a recorrência.

5.19 [Complementar] resposta: p. 177
Na árvore final do Exemplo 5.11, remova a raiz 30 (substituindo-a pelo maior elemento da subárvore esquerda). Indique os balanços e a rotação necessária.

5.20 [Complementar] resposta: p. 177
Insira $3, 2, 1, 4, 5, 6, 7, 16, 15, 14$, nessa ordem, numa AVL vazia. Em quais inserções há rotação, de que tipo e em que nó? Qual a árvore final?

5.21 [Complementar] resposta: p. 177
Mostre que toda árvore binária com n nós tem altura (em níveis) pelo menos $\lceil \log_2(n+1) \rceil$. Conclua que a altura de uma AVL é $\Theta(\log n)$.

5.22 [Complementar] resposta: p. 177
Escreva um algoritmo $O(n)$ que decide se uma árvore binária é uma árvore AVL (isto é, uma ABB que satisfaz a condição de balanceamento).

5.23 [Complementar] resposta: p. 177
Escreva um algoritmo que encontra o sucessor de um nó numa ABB em tempo $O(h)$, supondo que cada nó tem um ponteiro para o pai.

6

Ordenação por comparação e limite inferior

Neste capítulo

- Ferramentas para resolver recorrências: árvore de recursão, substituição e Teorema Mestre.
- Os algoritmos clássicos de ordenação por comparação: mergesort, quicksort e heapsort.
- Estabilidade e ordenação in-place.
- A prova de que toda ordenação por comparação faz $\Omega(n \log n)$ comparações.

Ordenar é o problema algorítmico mais estudado: aparece sozinho e como passo de quase tudo, da busca binária à eliminação de repetidos e ao algoritmo de Kruskal. Este capítulo apresenta os algoritmos que ordenam *comparando* pares de elementos e prova que nenhum algoritmo desse tipo pode fazer menos de $\Omega(n \log n)$ comparações no pior caso. Como o mergesort e o quicksort são recursivos, começamos pelas ferramentas para analisar algoritmos recursivos.

6.1 O problema da ordenação

Dada uma sequência S, queremos uma permutação $s_1, \ldots, s_n$ de seus elementos com $s_i \leq s_{i+1}$ para todo i. Duas propriedades de um algoritmo de ordenação importam além do tempo:

- ele é **estável** se elementos de chaves iguais aparecem na saída na mesma ordem relativa da entrada. A estabilidade é essencial para ordenar por várias chaves (primeiro pela secundária, depois pela principal) e para o radix sort do Capítulo 7;
- ele é **in-place** se usa memória adicional que não depende de n, ou no máximo $O(\log n)$ para a pilha de recursão. O quicksort e o heapsort são in-place; o mergesort não é.

Exemplo 6.1 (Estabilidade e ordenação por duas chaves)**.** Uma turma está em ordem alfabética: (Ana, 8), (Bia, 7), (Caio, 8), (Davi, 7). Ordenando por nota com um algoritmo **estável**, os empates mantêm a ordem de entrada, e o resultado é (Bia, 7), (Davi, 7), (Ana, 8), (Caio, 8): dentro de cada nota, os nomes continuam em ordem alfabética. Um algoritmo não estável poderia devolver (Davi, 7), (Bia, 7), (Caio, 8), (Ana, 8), que também está ordenado por nota, mas perdeu a ordem dos nomes.
É assim que se ordena por duas chaves: para obter a lista por nota e, nos empates, por nome, ordena-se *primeiro pelo nome* (a chave secundária) e *depois, de forma estável, pela nota* (a principal).

6.2 Recorrências

O custo de um algoritmo recursivo é descrito por uma **recorrência**. Um algoritmo que divide uma entrada de tamanho n em a subproblemas de tamanho n/b e gasta $f(n)$ para dividir e combinar tem custo

$$T(n) = a\,T(n/b) + f(n).$$

Por exemplo, o mergesort (§6.3) resolve $a = 2$ subproblemas de tamanho $n/2$ e gasta $f(n) = \Theta(n)$ para intercalar: $T(n) = 2T(n/2) + \Theta(n)$. A busca binária resolve $a = 1$ subproblema de tamanho $n/2$ e gasta $O(1)$: $T(n) = T(n/2) + O(1)$.

Árvore de recursão. Desenha-se a árvore das chamadas e soma-se o custo nível a nível. Para $T(n) = 2T(n/2) + cn$, há $\log_2 n$ níveis, e cada um custa cn no total, pois os subproblemas de um nível somam n; logo $T(n) = \Theta(n \log n)$ (Figura 6.1). É o custo do mergesort e de vários algoritmos de divisão e conquista do Capítulo 9.

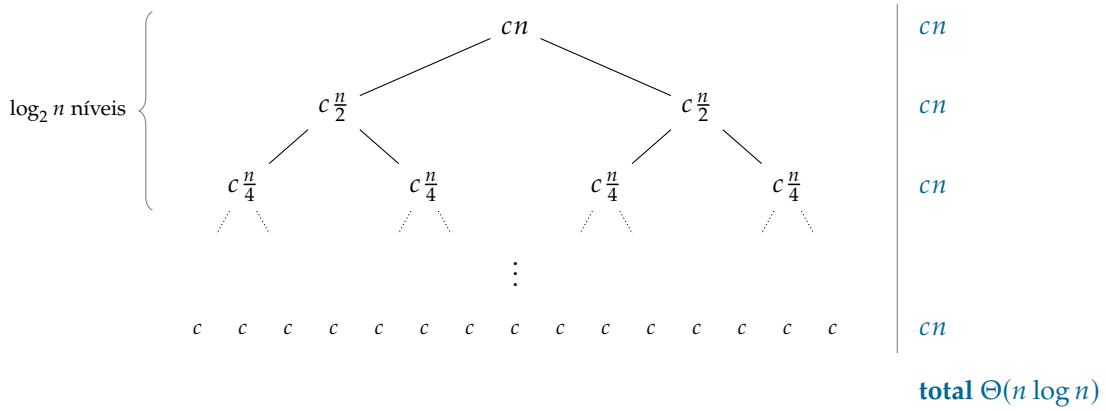


Figura 6.1. Árvore de recursão de $T(n) = 2T(n/2) + cn$. Cada nó mostra o custo da chamada fora das chamadas recursivas; à direita, a soma de cada nível. Há $\log_2 n$ níveis de custo cn e mais n folhas de custo c.

Substituição. Chuta-se a forma da solução, por exemplo $T(n) \le c\,g(n)$, e prova-se o chute por indução forte, ajustando a constante. É assim que se mostra, no Capítulo 8, que a seleção pela mediana das medianas é linear.

Exemplo 6.2 (Substituição: $T(n) = T(n/2) + T(n/4) + n$ é linear)**.** Chutamos $T(n) \le c\,n$ e supomos, por indução, que isso vale para todos os valores menores que n. Então

$$T(n) = T(n/2) + T(n/4) + n \;\le\; c\,\frac{n}{2} + c\,\frac{n}{4} + n = \left(\frac{3c}{4} + 1\right)n.$$

Para fechar a indução, precisamos de $\frac{3c}{4} + 1 \le c$, isto é, $c \ge 4$. Com $c = 4$ (e escolhendo a base de modo que $T(n) \le 4n$ para n pequeno), temos $T(n) \le 4n = O(n)$. A ideia por trás: os dois subproblemas somam $\frac{3}{4}$ da entrada, menos que o todo, e os custos dos níveis da árvore de recursão formam uma progressão geométrica decrescente, $n + \frac{3}{4}n + \frac{9}{16}n + \cdots \le 4n$.

Teorema 6.3 (Teorema Mestre)**.** *Seja $T(n) = a\,T(n/b) + f(n)$, com $a \ge 1$ e $b > 1$, e seja $E = \log_b a$ o* expoente crítico.

1. *Se $f(n) = O(n^{E-\varepsilon})$ para algum $\varepsilon > 0$, então $T(n) = \Theta(n^E)$: as folhas dominam.*
2. *Se $f(n) = \Theta(n^E)$, então $T(n) = \Theta(n^E \log n)$: todos os níveis pesam o mesmo.*
3. *Se $f(n) = \Omega(n^{E+\varepsilon})$ para algum $\varepsilon > 0$ e a $f(n/b) \le c\,f(n)$ para algum $c < 1$, então $T(n) = \Theta(f(n))$: a raiz domina.*

A intuição vem da árvore de recursão: o nível j tem a^j subproblemas de tamanho n/b^j, e há n^E folhas. O teorema compara o custo da raiz, $f(n)$, com o custo total das folhas, n^E (Figura 6.2).

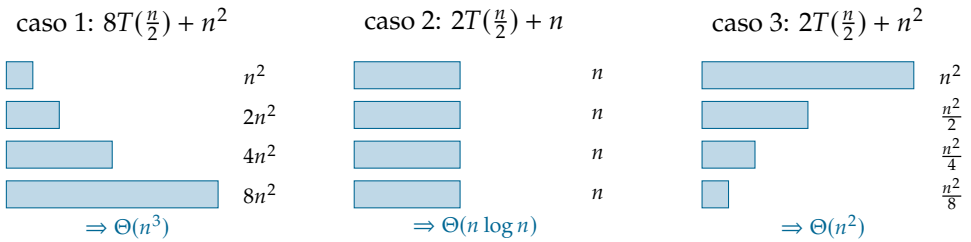


Figura 6.2. Os três casos do Teorema Mestre vistos pelo custo de cada nível da árvore de recursão (barras, de cima para baixo). No caso 1 os custos crescem e o último nível domina; no caso 2 são iguais e o total é o custo de um nível vezes o número de níveis; no caso 3 decrescem e a raiz domina.

Como aplicar o Teorema Mestre

1. Identifique a (número de subproblemas), b (fator de redução do tamanho) e $f(n)$ (custo fora das chamadas).
2. Calcule o expoente crítico $E = \log_b a$.
3. Compare $f(n)$ com n^E: se f é *polinomialmente* menor, caso 1; se é da mesma ordem, caso 2; se é polinomialmente maior (e satisfaz a condição de regularidade), caso 3.
4. Se f e n^E diferem só por um fator logarítmico, o teorema, nesta forma, não se aplica: use a árvore de recursão.

Exemplo 6.4 (Aplicando o roteiro)**.**

- $T(n) = 3T(n/2) + n$ (o algoritmo de Karatsuba para multiplicar inteiros): $a = 3$, $b = 2$, $E = \log_2 3 \approx 1{,}585$. Como $f(n) = n = O(n^{1{,}585-\varepsilon})$ com $\varepsilon = 0{,}5$, é o caso 1: $T(n) = \Theta(n^{\log_2 3})$.
- $T(n) = T(2n/3) + 1$: $a = 1$, $b = 3/2$, $E = \log_{3/2} 1 = 0$, e $f(n) = 1 = \Theta(n^0)$: caso 2, $T(n) = \Theta(\log n)$.
- $T(n) = 3T(n/4) + n \log n$: $E = \log_4 3 \approx 0{,}79$, e $f(n) = n \log n = \Omega(n^{0{,}79+\varepsilon})$ com $\varepsilon = 0{,}2$. A regularidade vale: $3 \cdot \frac{n}{4} \log \frac{n}{4} \le \frac{3}{4} n \log n$, com $c = \frac{3}{4} < 1$. Caso 3: $T(n) = \Theta(n \log n)$.

Recorrência	Exemplo	Solução
$T(n) = T(n/2) + O(1)$	busca binária	$\Theta(\log n)$
$T(n) = 2T(n/2) + O(1)$	máximo por divisão e conquista	$\Theta(n)$
$T(n) = 2T(n/2) + \Theta(n)$	mergesort, contagem de inversões	$\Theta(n \log n)$
$T(n) = 8T(n/2) + \Theta(n^2)$	produto de matrizes por blocos	$\Theta(n^3)$
$T(n) = 7T(n/2) + \Theta(n^2)$	algoritmo de Strassen	$\Theta(n^{\log_2 7}) \approx \Theta(n^{2{,}807})$
$T(n) = T(n/5) + T(3n/4) + \Theta(n)$	mediana das medianas	$\Theta(n)$, pois $\frac{1}{5} + \frac{3}{4} < 1$
$T(n) = T(n-1) + \Theta(n)$	quicksort no pior caso	$\Theta(n^2)$
$T(n) = T(9n/10) + T(n/10) + \Theta(n)$	quicksort com divisão desbalanceada fixa	$\Theta(n \log n)$

Teorema 6.5 (Divisão e conquista balanceada)**.** *Se um algoritmo divide a instância em k partes de tamanho no máximo n/c, com $c > 1$ e tamanhos que diferem por uma constante, e gasta tempo linear para dividir e para combinar, então roda em $O(n \log n)$.*

Ideia. Na árvore de recursão, os subproblemas de um mesmo nível são partes disjuntas da entrada, então cada nível custa $O(n)$; como o tamanho cai por um fator c a cada nível, há $O(\log_c n)$ níveis. □

Somatórios úteis. $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$; $\sum_{k\geq 0} x^k = \frac{1}{1-x}$ para $|x| < 1$; $\sum_{k\geq 0} \frac{k}{2^k} = 2$; $\sum_{k=0}^{\log n} \frac{n}{2^k} \leq 2n$; $\sum_{i=1}^{n} \frac{1}{i} = \Theta(\log n)$; $\log(n!) = \sum_{i=1}^{n} \log i = \Theta(n \log n)$.

6.3 Ordenação por comparação

Mergesort

O mergesort divide o vetor ao meio, ordena recursivamente cada metade e *intercala* as duas metades ordenadas, percorrendo-as com dois índices e copiando sempre o menor dos elementos apontados.

> **Intuição**
>
> Intercalar é como juntar dois montes de provas, cada um já em ordem alfabética, num monte só: olhe a prova do topo de cada monte, pegue a que vem primeiro e repita. Cada comparação põe uma prova no lugar definitivo, então juntar montes com m provas no total custa no máximo $m - 1$ comparações.

MergeSort e Intercala

```
Procedimento MergeSort(V, ini, fim)
    se ini < fim então
        meio := ⌊(ini + fim)/2⌋
        MergeSort(V, ini, meio); MergeSort(V, meio + 1, fim)
        Intercala(V, ini, meio, fim)
Procedimento Intercala(V, ini, meio, fim)
    i := ini; j := meio + 1; k := 1                    % W é um vetor auxiliar
    enquanto i ≤ meio e j ≤ fim faça
        se V[i] ≤ V[j] então
            W[k] := V[i]; i := i + 1                   % "≤" mantém a estabilidade
        senão
            W[k] := V[j]; j := j + 1
        k := k + 1
    copie o que sobrou de V[i..meio] ou de V[j..fim] para o fim de W
    copie W[1..fim − ini + 1] de volta para V[ini..fim]
```

Exemplo 6.6 (Mergesort de $(5, 2, 4, 7, 1, 3, 2, 6)$)**.** A Figura 6.3 mostra a execução: a metade de cima divide o vetor até sobrarem elementos isolados (que já estão ordenados), e a de baixo intercala os pedaços, dois a dois. Na última intercalação, de $(2, 4, 5, 7)$ com $(1, 2, 3, 6)$, as comparações são: $2 > 1$ (sai 1); $2 \leq 2$ (sai o 2 da esquerda, o que preserva a estabilidade); $4 > 2$ (sai 2); $4 > 3$ (sai 3); $4 \leq 6$ (sai 4); $5 \leq 6$ (sai 5); $7 > 6$ (sai 6); e a metade direita acabou, de modo que o 7 é copiado sem comparação. Foram 7 comparações para 8 elementos.

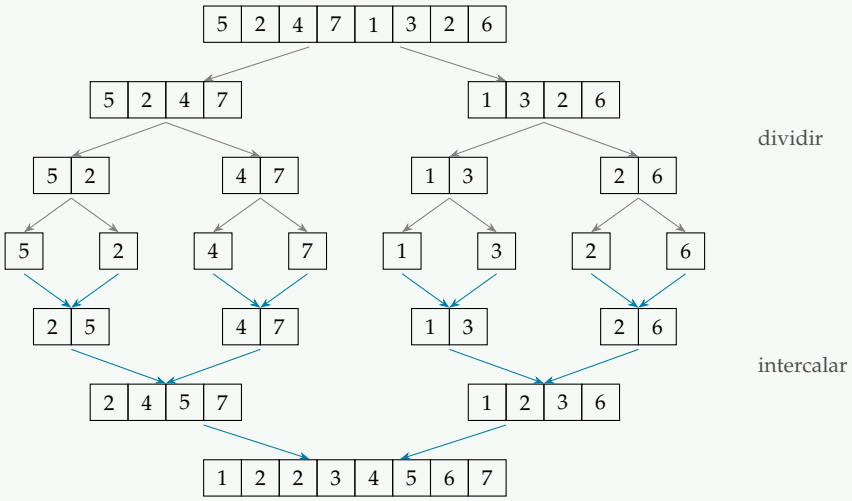


Figura 6.3. Mergesort de $(5, 2, 4, 7, 1, 3, 2, 6)$. As setas cinzas dividem; as azuis intercalam.

A intercalação de duas metades que somam m elementos custa $\Theta(m)$, e portanto $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$ em todos os casos. O mergesort é **estável**, desde que, no empate, o elemento da metade esquerda seja copiado primeiro, e **não é in-place**, pois o vetor auxiliar ocupa $\Theta(n)$. Existe também uma versão iterativa, de baixo para cima, que intercala blocos de tamanho $1, 2, 4, \ldots$ ao longo do vetor.

Quicksort

O quicksort faz o trabalho na divisão, e não na combinação: escolhe um **pivô** x, particiona os elementos em $S_1 = \{w < x\}$ e $S_2 = \{w \geq x\}$, ordena cada parte recursivamente e concatena. Depois da partição, o pivô já está na sua posição definitiva: todos à sua esquerda são menores e todos à sua direita, maiores ou iguais.

Quicksort com a partição de Lomuto

```
Procedimento QuickSort(V, ini, fim)
    se ini < fim então
        q := Particiona(V, ini, fim)
        QuickSort(V, ini, q − 1); QuickSort(V, q + 1, fim)
Função Particiona(V, ini, fim)
    x := V[fim]; i := ini − 1                % o pivô fica no fim; troque-o para lá se escolher outro
    para j := ini até fim − 1 faça
        se V[j] < x então
            i := i + 1; V[i] ⇔ V[j]
    V[i + 1] ⇔ V[fim]; retorna i + 1
```

Exemplo 6.7 (A partição de Lomuto passo a passo)**.** Particionamos $(6, 3, 9, 1, 8, 2, 5)$ com pivô $x = 5$, o último elemento. O índice j percorre o vetor; as posições $ini..i$ guardam os elementos já vistos que são menores que o pivô (em verde), e as posições $i + 1..j$, os já vistos maiores ou iguais (em amarelo). Quando $V[j] < x$, o índice i avança e $V[j]$ é trocado para dentro da região verde (Figura 6.4). No fim, o pivô é trocado com o primeiro elemento da região amarela, e fica na posição 4, que é devolvida.

6	3	9	1	8	2	5	$j = 1$: $6 \geq 5$, nada muda
3	6	9	1	8	2	5	$j = 2$: $3 < 5$: $i = 1$, troca $V[1] \leftrightarrow V[2]$
3	6	9	1	8	2	5	$j = 3$: $9 \geq 5$
3	1	9	6	8	2	5	$j = 4$: $1 < 5$: $i = 2$, troca $V[2] \leftrightarrow V[4]$
3	1	9	6	8	2	5	$j = 5$: $8 \geq 5$
3	1	2	6	8	9	5	$j = 6$: $2 < 5$: $i = 3$, troca $V[3] \leftrightarrow V[6]$
3	1	2	5	8	9	6	fim: troca o pivô com $V[4]$; devolve $q = 4$

Figura 6.4. A partição de Lomuto sobre $(6, 3, 9, 1, 8, 2, 5)$. Verde: menores que o pivô; amarelo: maiores ou iguais; azul: o pivô.

O resultado, $(3, 1, 2, 5, 8, 9, 6)$, tem o pivô no lugar certo; resta ordenar $(3, 1, 2)$ e $(8, 9, 6)$ recursivamente.

- **Pior caso**, $\Theta(n^2)$: quando as partições têm sempre tamanhos 0 e $n - 1$, como ao usar o primeiro elemento como pivô num vetor já ordenado ou invertido. A recorrência é $T(n) = T(n-1) + \Theta(n)$.
- **Melhor caso e caso médio**, $\Theta(n \log n)$. Com um pivô aleatório, em metade das vezes a parte maior tem tamanho no máximo $3n/4$. Se isso acontecesse sempre, no nível t da recursão os subproblemas teriam tamanho no máximo $(3/4)^t n$, e haveria $O(\log n)$ níveis de custo $O(n)$. Como acontece com probabilidade $1/2$, em média o tamanho cai por $3/4$ a cada dois níveis, e o custo esperado continua $O(n \log n)$.
- **Escolha do pivô**: o primeiro elemento é ruim para entradas já ordenadas; boas opções são um elemento aleatório ou a **mediana de três** (primeiro, meio e último). Usar a **mediana exata**, calculada em tempo linear (Capítulo 8), garante $O(n \log n)$ no *pior* caso, mas a constante é alta e na prática não compensa.
- O quicksort é **in-place**: a partição é feita por trocas, e a pilha de recursão tem $O(\log n)$ níveis se a recursão for feita primeiro na parte menor. **Não é estável**, porque as trocas de longa distância embaralham elementos iguais; pode ser tornado estável com uma partição estável, ao custo de $O(n)$ de memória extra.

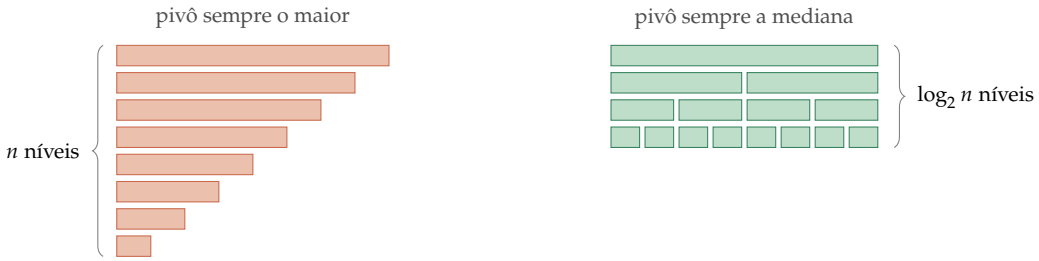


Figura 6.5. O trabalho do quicksort, nível a nível. Cada barra é um subproblema; a largura é o seu tamanho, e a área total é proporcional ao custo. À esquerda, o pivô é sempre o maior: n níveis e área $\approx n^2/2$. À direita, o pivô divide ao meio: $\log_2 n$ níveis de largura n e área $\approx n \log n$.

Exemplo 6.8 (O quicksort não é estável)**.** Três alunos com a mesma nota, Ana, Bruno e Carla, nessa ordem. Na primeira partição, o pivô é Carla (o último); ninguém é estritamente menor, e a troca final leva Carla para a primeira posição: (Carla, Bruno, Ana). A partição seguinte, sobre (Bruno, Ana), troca os dois. A saída é (Carla, Ana, Bruno), embora a entrada estivesse na ordem (Ana, Bruno, Carla): elementos iguais mudaram de ordem relativa.

Heapsort

O heapsort (Capítulo 2) constrói um heap em $O(n)$ e remove o máximo n vezes, em $\Theta(n \log n)$ em todos os casos. É in-place e não é estável.

Algoritmos quadráticos

A **ordenação por inserção** insere $V[i]$ na parte já ordenada $V[1..i-1]$, deslocando os elementos maiores (veja o Exemplo 1.1). Ela custa $\Theta(n^2)$ no pior caso e no caso médio, mas $\Theta(n)$ se o vetor já está ordenado; é estável e in-place, e é a melhor escolha para vetores pequenos ou quase ordenados. A **ordenação por bolha** também é $\Theta(n^2)$ e estável.

algoritmo	melhor caso	caso médio	pior caso	estável?	in-place?
inserção	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	sim	sim
mergesort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	sim	não
quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	não	sim
heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	não	sim

6.4 Limite inferior para ordenação por comparação

Intuição

Ordenar por comparações é como jogar "adivinhe a permutação" fazendo só perguntas de sim ou não do tipo "$a_i \le a_j$?". Cada resposta, no máximo, divide ao meio o conjunto de permutações ainda possíveis. Começando com $n!$ possibilidades, são necessárias pelo menos $\log_2(n!)$ perguntas para chegar a uma só.

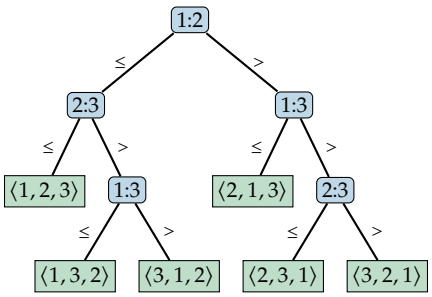


Figura 6.6. Árvore de decisão da ordenação por inserção para $n = 3$. Cada nó interno compara dois elementos ("i:j" pergunta se $a_i \le a_j$); cada folha é a ordem encontrada. Há $3! = 6$ folhas e altura 3: no pior caso, três comparações.

Teorema 6.9. *Todo algoritmo de ordenação baseado em comparações faz $\Omega(n \log n)$ comparações no pior caso.*

Demonstração. Para cada n, o algoritmo pode ser descrito por uma **árvore de decisão**: cada nó interno é uma comparação "$a_i \le a_j$?", com dois filhos, um para cada resposta; cada folha é um estado final, em que o algoritmo já sabe qual permutação ordena a entrada (Figura 6.6). O número de comparações no pior caso é a altura h da árvore. Como qualquer uma das $n!$ permutações pode ser a resposta, há pelo menos $n!$ folhas, e uma árvore binária de altura h tem no máximo 2^h folhas. Logo

$$2^h \ge n! \ge n(n-1)\cdots\left(\tfrac{n}{2}\right) \ge \left(\tfrac{n}{2}\right)^{n/2} \;\Rightarrow\; h \ge \tfrac{n}{2}\log_2 \tfrac{n}{2} = \tfrac{n}{2}\log n - \tfrac{n}{2} = \Omega(n \log n). \qquad \square$$

Exemplo 6.10 (O limite para $n = 3$ e $n = 4$)**.** Com $n = 3$, há $3! = 6$ ordens possíveis; como $2^2 = 4 < 6$, duas comparações não bastam, e todo algoritmo precisa de pelo menos $\lceil \log_2 6 \rceil = 3$ comparações no pior caso. A árvore da Figura 6.6 atinge esse limite. Com $n = 4$, há 24 ordens, e $2^4 = 16 < 24 \leq 32 = 2^5$: são necessárias pelo menos 5 comparações. O mergesort sobre 4 elementos faz no máximo $1 + 1 + 3 = 5$: é ótimo para $n = 4$.

Corolário 6.11. *O heapsort e o mergesort são* ***ótimos*** *entre os algoritmos baseados em comparações, supondo que cada comparação custa $O(1)$. Para ordenar em $o(n \log n)$ é preciso usar a estrutura das chaves — inteiros pequenos, dígitos, distribuição conhecida —, e não apenas compará-las. É o assunto do próximo capítulo.*

Pontos-chave

Recorrências se resolvem por árvore de recursão, substituição ou Teorema Mestre ($E = \log_b a$). Qualquer ordenação por comparação faz $\Omega(n \log n)$ comparações (árvore de decisão, $2^h \geq n!$). Mergesort: $\Theta(n \log n)$ sempre, estável, $\Theta(n)$ de memória extra. Quicksort: in-place, $\Theta(n \log n)$ em média e $\Theta(n^2)$ no pior caso, não estável. Heapsort: in-place, $\Theta(n \log n)$ sempre, não estável. Mergesort e heapsort são ótimos.

6.5 Exercícios

Exercícios do curso

6.1 [Slides] resposta: p. 177
Escreva o pseudocódigo do procedimento de intercalação (*Merge*) do mergesort.

6.2 [Slides] resposta: p. 177
Escreva uma versão iterativa, não recursiva, do mergesort.

6.3 [Slides] resposta: p. 178
Desenvolva uma estratégia para ordenar os elementos de um vetor que não seja baseada em comparações. Quais hipóteses sobre os elementos ela exige?

6.4 [Slides] resposta: p. 178
(a) É possível implementar o mergesort de forma estável? (b) E o quicksort? Justifique.

Exercícios complementares

6.5 [Complementar] resposta: p. 178
Resolva pelo Teorema Mestre: (a) $T(n) = 4T(n/2) + n$; (b) $T(n) = 4T(n/2) + n^2$; (c) $T(n) = 4T(n/2) + n^3$; (d) $T(n) = 2T(n/4) + \sqrt{n}$; (e) $T(n) = 9T(n/3) + n$; (f) $T(n) = T(n/2) + n$.

6.6 [Complementar] resposta: p. 178
Use a árvore de recursão para resolver $T(n) = T(n/3) + T(2n/3) + n$.

6.7 [Complementar] resposta: p. 178
Quantas comparações o mergesort faz no pior caso para $n = 8$? Dê a recorrência para potências de 2 e a sua solução exata.

6.8 [Complementar] resposta: p. 178
Mostre que todo algoritmo que encontra o máximo de n elementos por comparações faz pelo menos $n - 1$ comparações. Por que o argumento da árvore de decisão dá um limite pior aqui?

6.9 [Complementar] resposta: p. 178
Qual o custo do quicksort com a partição de Lomuto (teste "$V[j] < x$") quando todos os elementos são iguais? Como corrigir?

6.10 [Complementar] resposta: p. 178
Mostre que o número de deslocamentos feitos pela ordenação por inserção é exatamente o número de inversões do vetor (pares $i < j$ com $V[i] > V[j]$). Conclua que ela é $\Theta(n^2)$ no pior caso e no caso médio.

6.11 [Complementar ★] resposta: p. 178
Mostre que intercalar duas listas ordenadas de n elementos cada exige $2n - 1$ comparações no pior caso.

6.12 [Complementar] resposta: p. 178
Execute o mergesort sobre $[38, 27, 43, 3, 9, 82, 10]$, com $meio = \lfloor (ini + fim)/2 \rfloor$, e mostre o resultado de cada intercalação, na ordem em que acontecem.

6.13 [Complementar] resposta: p. 178
Execute a partição de Lomuto sobre $[2, 8, 7, 1, 3, 5, 6, 4]$ (pivô 4) e dê o vetor resultante e a posição devolvida.

6.14 [Complementar] resposta: p. 178
Quantas comparações exatamente o quicksort com a partição de Lomuto faz para ordenar um vetor já ordenado de n elementos distintos?

6.15 [Complementar] resposta: p. 178
Prove por indução que a solução de $T(1) = 0$, $T(n) = 2T(n/2) + n$ para n potência de 2 é $T(n) = n \log_2 n$.

6.16 [Complementar] resposta: p. 179
A ordenação por seleção acha o menor elemento de $V[i..n]$ e o troca com $V[i]$, para $i = 1, \ldots, n-1$. Quantas comparações e quantas trocas ela faz? Ela é estável? E a ordenação por inserção?

6.17 [Complementar] resposta: p. 179
Qual o número mínimo de comparações que um algoritmo baseado em comparações precisa fazer, no pior caso, para ordenar 5 elementos, segundo o argumento da árvore de decisão? E para 12 elementos?

6.18 [Complementar ★] resposta: p. 179
Um vetor de n elementos tem apenas k valores distintos. Mostre que $\Omega(n \log k)$ comparações são necessárias para ordená-lo e dê um algoritmo com esse custo.

6.19 [Complementar] resposta: p. 179
Resolva $T(n) = 2T(n/2) + n \log n$ pela árvore de recursão. Por que o Teorema Mestre, na forma apresentada, não se aplica?

6.20 [Complementar] resposta: p. 179
Resolva: (a) $T(1) = 1$, $T(n) = T(n-1) + n$; (b) $T(1) = 1$, $T(n) = 2T(n-1) + 1$ (a recorrência das Torres de Hanói); (c) $T(1) = 1$, $T(n) = T(n-1) + \log n$.

7

Ordenação em tempo linear

Neste capítulo

- Como escapar do limite $\Omega(n \log n)$ usando a estrutura das chaves.
- Counting sort, radix sort, bucket sort e American Flag sort.
- O papel da estabilidade no radix sort.
- Um guia para escolher o algoritmo de ordenação adequado a cada situação.

O limite $\Omega(n \log n)$ do capítulo anterior vale para algoritmos que só *comparam* elementos. Qualquer algoritmo de ordenação em tempo linear precisa, portanto, fazer no máximo $O(n)$ comparações e tomar suas decisões de outra forma. Se soubermos algo sobre as chaves — que são inteiros num intervalo pequeno, números com poucos dígitos ou reais uniformemente distribuídos —, podemos usá-las diretamente como *índices* e ordenar em tempo linear. O preço é memória extra e hipóteses sobre os dados.

Intuição

Um carteiro não ordena as cartas comparando endereços dois a dois. Ele olha o número da casa e põe a carta direto no escaninho daquele número; no fim, basta recolher os escaninhos em ordem. Cada carta é manuseada um número constante de vezes, e nenhuma pergunta do tipo "$a_i \le a_j$?" é feita. Por isso o argumento da árvore de decisão (Capítulo 6) não se aplica: ele só limita algoritmos que obtêm informação *exclusivamente* por comparações. Todos os algoritmos deste capítulo são variações dessa ideia de "distribuir em escaninhos".

7.1 Counting sort

O counting sort ordena inteiros no intervalo $[0, k]$. Ele trabalha em três etapas:

1. **contar**: $C[i]$ recebe o número de elementos iguais a i;
2. **acumular**: $C[i]$ passa a ser o número de elementos *menores ou iguais* a i; esse número é exatamente a última posição que o valor i ocupa na saída;
3. **distribuir**: cada elemento é copiado diretamente para a sua posição final no vetor de saída B, e o contador do seu valor é decrementado, para que a próxima cópia do mesmo valor vá uma posição antes.

Counting-Sort(A, n, k)

```
para i := 0 até k faça
    C[i] := 0
para j := 1 até n faça
    C[A[j]] := C[A[j]] + 1
                                   % C[i] = #{j : A[j] = i}
para i := 1 até k faça
```

```
    C[i] := C[i] + C[i − 1]                          % C[i] = #{j : A[j] ≤ i}
para j := n decrescendo até 1 faça                   % de trás para frente: estável
    B[C[A[j]]] := A[j]; C[A[j]] := C[A[j]] − 1
retorna B
```

Exemplo 7.1 (Counting sort passo a passo)**.** Ordenamos $A = (4, 1, 3, 1, 0, 4, 1)$, com $k = 4$. Para acompanhar a estabilidade, os elementos repetidos levam uma etiqueta com a ordem em que aparecem: $A = (4_a, 1_a, 3, 1_b, 0, 4_b, 1_c)$.

- Contagem: o valor 0 aparece uma vez, o 1 três vezes, o 2 nenhuma, o 3 uma e o 4 duas: $C = (1, 3, 0, 1, 2)$, com índices de 0 a 4.
- Acumulação: $C = (1, 4, 4, 5, 7)$. Leia assim: há 4 elementos ≤ 1, logo os 1 ocupam as posições 2 a 4 da saída (a posição 1 é do 0); há 7 elementos ≤ 4, logo os 4 terminam na posição 7.
- Distribuição: percorrendo A de trás para frente, cada elemento vai para a posição indicada pelo contador do seu valor (Figura 7.1).

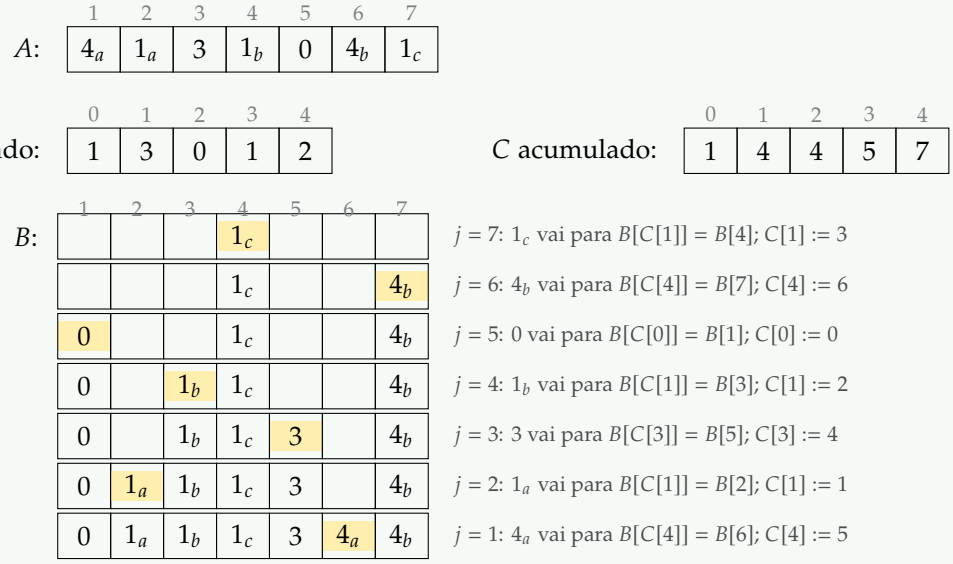


Figura 7.1. Counting sort sobre $A = (4_a, 1_a, 3, 1_b, 0, 4_b, 1_c)$. Acima, o vetor de contadores depois da contagem e depois da acumulação (indexado a partir de 0). Abaixo, a distribuição: em cada linha, o elemento $A[j]$ (em destaque) vai para a posição $C[A[j]]$ de B, e esse contador diminui de 1.

O resultado é $B = (0, 1_a, 1_b, 1_c, 3, 4_a, 4_b)$: os três 1 e os dois 4 saíram na mesma ordem relativa da entrada. Nenhum par de elementos foi comparado; todo o trabalho foi feito com os valores servindo de índices.

O tempo e a memória são $\Theta(n + k)$: os laços sobre i custam $\Theta(k)$ e os laços sobre j, $\Theta(n)$. O algoritmo é, portanto, **linear quando** $k = O(n)$. Ele é **estável**, porque, percorrendo a entrada de trás para frente, a última ocorrência de cada valor vai para a última posição do seu bloco, a penúltima para a penúltima, e assim por diante; não é in-place, pois usa os vetores B e C; e não faz nenhuma comparação entre elementos.

Quando k é grande. Se k é muito maior que n (por exemplo, $k = \Theta(2^n)$), o termo $\Theta(k)$ domina e o algoritmo fica exponencial no tamanho da entrada: é **pseudopolinomial**, porque o custo depende do *valor* das chaves, e não apenas da quantidade delas.

Exemplo 7.2 (Poucos elementos, chaves enormes)**.** Ordenar $n = 1000$ números de identificação, cada um entre 0 e $k = 10^9$, com o counting sort exigiria um vetor de $10^9 + 1$ contadores (vários gigabytes) e 10^9 passos só para zerá-los e acumulá-los, embora a entrada tenha só mil números. Um mergesort faria cerca de $1000 \log_2 1000 \approx 10^4$ comparações. O counting sort só compensa quando o intervalo das chaves é comparável à quantidade de elementos; para chaves grandes com poucos dígitos, a solução é o radix sort, a seguir.

7.2 Radix sort

O radix sort ordena inteiros com d dígitos na base b **dígito a dígito, do menos significativo para o mais significativo**, usando em cada passada um método de ordenação **estável**. Cada passada é um counting sort com $k = b - 1$, que é pequeno mesmo quando os números são enormes. A versão abaixo usa uma fila para cada valor de dígito, o que equivale a um counting sort.

Radix sort com filas

para $i := 1$ **até** d **faça**
  para $j := 1$ **até** n **faça**
    $k :=$ i-ésimo dígito menos significativo de $L[j]$ na base b; insira $L[j]$ no fim da fila $F[k]$
  $j := 1$
  para $k := 0$ **até** $b - 1$ **faça**
    enquanto $F[k]$ não está vazia **faça**
      retire o início de $F[k]$ e coloque-o em $L[j]$; $j := j + 1$

Exemplo 7.3 (Radix sort na base 10)**.** Ordenemos 102, 313, 690, 131, 222, 670, 690, 130, 079, 102. A Figura 7.2 mostra a lista depois de cada passada, com o dígito usado em destaque.
Pelas unidades: 690 670 690 130 131 102 222 102 313 079;
pelas dezenas: 102 102 313 222 130 131 670 079 690 690;
pelas centenas: 079 102 102 130 131 222 313 670 690 690.
Observe a segunda passada: 130 e 131 têm o mesmo dígito das dezenas (3), e continuam na ordem 130, 131 obtida pela passada das unidades. Na terceira, 102, 102, 130 e 131 têm todos centena 1 e mantêm a ordem que já tinham pelos dois últimos dígitos.

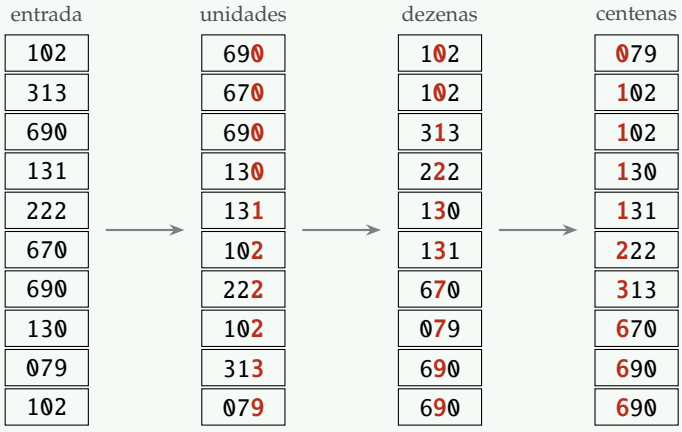


Figura 7.2. Radix sort na base 10. Cada coluna é a lista depois de uma passada; o dígito em vermelho é o que decidiu aquela passada. Dentro de um mesmo valor do dígito, a ordem da coluna anterior é preservada.

Por que a estabilidade é necessária? Depois da passada pelo dígito i, elementos com o mesmo dígito i devem manter entre si a ordem obtida pelos dígitos $1, \ldots, i - 1$. É isso que garante, por indução, que após a passada i a lista está ordenada pelos i últimos dígitos. As filas

preservam a ordem de chegada naturalmente; outra opção é usar o counting sort em cada dígito.

Exemplo 7.4 (O que dá errado sem estabilidade)**.** Ordene $23, 21, 13$. A passada pelas unidades dá $21, 23, 13$ (os dígitos $1, 3, 3$). Na passada pelas dezenas, 21 e 23 empatam (dezena 2). Uma passada estável mantém 21 antes de 23 e produz $13, 21, 23$, correto. Uma passada que não fosse estável poderia devolver $13, 23, 21$: ordenado pelas dezenas, mas com a informação das unidades perdida. É a mesma situação do Exemplo 6.1, em que se ordena primeiro pela chave secundária e depois, de forma estável, pela principal: no radix sort, os dígitos menos significativos fazem o papel das chaves secundárias.

O tempo é $\Theta(d(n+b))$: são d passadas, e cada uma percorre os n elementos e as b filas (ou contadores). Com b e d constantes, $\Theta(n)$. Há uma troca entre d e b: uma base maior deixa os números com menos dígitos (menos passadas), mas cada passada paga $\Theta(b)$ pelos contadores. Uma escolha útil é usar a própria base $b = n$: inteiros menores que n^c têm c dígitos nessa base, e o radix sort os ordena em $\Theta(cn)$.

Exemplo 7.5 (Escolhendo a base)**.** Para ordenar $n = 10^6$ inteiros menores que 10^6, podemos escrevê-los em várias bases:

base b	dígitos d	custo de cada passada	custo total aproximado
10	6	$n + 10$	$6 \cdot 10^6$
100	3	$n + 100$	$3 \cdot 10^6$
1000	2	$n + 1000$	$2 \cdot 10^6$
10^6	1	$n + 10^6$	$2 \cdot 10^6$ (é o counting sort direto)

Aumentar a base reduz o número de passadas enquanto b é pequeno perto de n; quando b se aproxima de n, o custo dos contadores passa a pesar, e não há mais ganho.

Chaves de tamanhos variados. Se os números têm quantidades diferentes de dígitos, $d_1, \ldots, d_n$, completar todos com zeros até o maior tamanho desperdiça trabalho. É possível ordenar em tempo $O(\sum_i d_i)$, linear no tamanho da entrada, agrupando os números pelo número de dígitos (um número com menos dígitos é menor) e fazendo cada número participar apenas das passadas correspondentes aos seus próprios dígitos. Por exemplo, em 7, 42, 3, 105, 18, os grupos são $\{7, 3\}$, $\{42, 18\}$ e $\{105\}$; ordenado cada grupo, a concatenação $3, 7, 18, 42, 105$ já é a resposta, e o 105 foi o único a passar por três passadas. Para **strings**, ordena-se da direita para a esquerda completando as menores com um símbolo "menor que todos", ou usa-se a versão que começa pelo caractere mais significativo, descrita adiante.

Exemplo 7.6 (Strings de tamanhos diferentes)**.** Ordenemos as palavras `ba`, `a` e `ab`, completando-as até 2 letras com um símbolo $\perp$ menor que todas as letras: `ba`, `a`$\perp$, `ab`. Pela segunda letra ($\perp <$ `a` $<$ `b`): `a`$\perp$, `ba`, `ab`. Pela primeira letra, de forma estável: `a`$\perp$, `ab`, `ba`. Removendo o $\perp$, obtemos a ordem alfabética `a`, `ab`, `ba`: o símbolo $\perp$ garante que um prefixo vem antes das palavras que o estendem.

7.3 Bucket sort

O bucket sort supõe que as chaves são reais **uniformemente distribuídos** em $[0, 1)$. Divide-se o intervalo em r baldes de mesmo tamanho, coloca-se $A[i]$ no balde $\lfloor r\, A[i] \rfloor$, ordena-se cada balde (por exemplo, por inserção) e concatenam-se os baldes. Como os baldes correspondem a intervalos disjuntos e crescentes, todo elemento de um balde é menor que todo elemento do balde seguinte, e a concatenação fica ordenada.

Exemplo 7.7 (Bucket sort com 10 baldes)**.** Com $r = 10$ baldes, ordenemos os números 0,42; 0,07; 0,93; 0,15; 0,48; 0,61; 0,11; 0,88; 0,44 e 0,35. O balde de cada número é o seu primeiro dígito depois da vírgula: $\lfloor 10 \cdot 0{,}42 \rfloor = 4$, $\lfloor 10 \cdot 0{,}07 \rfloor = 0$, e assim por diante. O balde 4 recebe três números, o balde 1 recebe dois, e os baldes 2, 5 e 7 ficam vazios (Figura 7.3). Ordenando cada balde e concatenando, obtemos 0,07; 0,11; 0,15; 0,35; 0,42; 0,44; 0,48; 0,61; 0,88; 0,93.

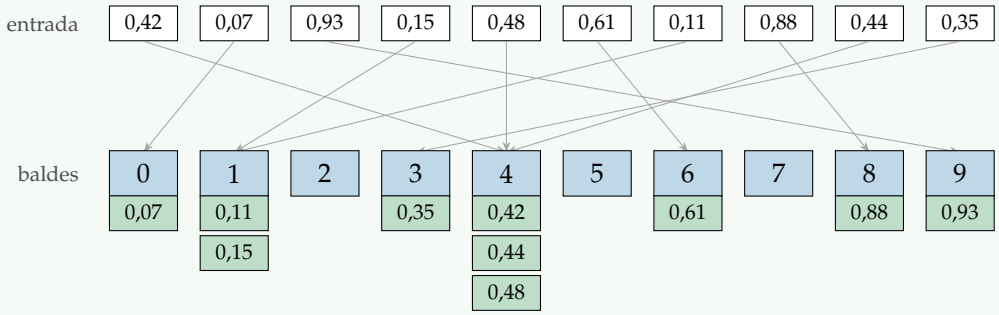


Figura 7.3. Bucket sort com 10 baldes. Cada número da entrada (em cima) vai para o balde $\lfloor 10x \rfloor$; cada balde é ordenado separadamente (aqui, já ordenados), e a saída é a concatenação dos baldes da esquerda para a direita.

Com entrada uniforme, cada balde recebe em média n/r elementos; com $r = \Theta(n)$, o tempo **esperado** é $\Theta(n)$. Sendo n_i o número de elementos no balde i, a ordenação por inserção desse balde custa $O(n_i^2)$, e o que importa é a média de n_i^2, e não a de n_i. Com $r = n$ baldes, cada elemento cai no balde i com probabilidade $1/n$, de modo que n_i tem distribuição binomial de parâmetros n e $1/n$, com média 1 e variância $1 - 1/n$. Portanto

$$E[n_i^2] = \mathrm{Var}(n_i) + E[n_i]^2 = \left(1 - \frac{1}{n}\right) + 1 = 2 - \frac{1}{n}, \qquad \text{e} \qquad \sum_i E[O(n_i^2)] = O(n).$$

O pior caso, quando todos os elementos caem no mesmo balde, é o do algoritmo usado dentro dos baldes.

Intuição

Jogue n bolinhas, ao acaso, em n baldes. Alguns baldes ficam vazios, a maioria recebe uma ou duas bolinhas, e é raríssimo um balde receber muitas. Ordenar um balde com n_i bolinhas por inserção custa cerca de n_i^2, mas, como os baldes cheios são raros, a soma desses custos fica em torno de $2n$. O algoritmo aposta na distribuição dos dados: se ela não for uniforme — por exemplo, se quase todos os números estiverem perto de 0 —, a aposta falha.

7.4 American Flag sort

O American Flag sort é um radix sort que começa pelo dígito **mais** significativo e trabalha **in-place**. Ele conta quantos elementos há com cada valor do dígito (um vetor C de tamanho b), calcula onde começa o bloco de cada valor e *permuta no próprio vetor* cada elemento para o seu bloco, por meio de trocas; depois, chama a si mesmo recursivamente em cada bloco, com o dígito seguinte. O nome vem da imagem do vetor, depois da primeira passada, dividido em faixas, como uma bandeira listrada.

A permutação é feita bloco a bloco. Para cada valor de dígito δ, mantém-se um ponteiro $prox[\delta]$ para a primeira posição do bloco de δ que ainda não recebeu o seu elemento definitivo. Enquanto o bloco de δ não estiver completo, olha-se o elemento x na posição $prox[\delta]$: se o dígito de x é δ, ele já está no bloco certo e o ponteiro avança; se é outro valor ε, troca-se x com o elemento da posição $prox[\varepsilon]$, e $prox[\varepsilon]$ avança. Cada troca põe pelo menos um elemento no seu bloco definitivo, e por isso uma passada faz menos de n trocas.

Exemplo 7.8 (Uma passada do American Flag sort)**.** Vamos ordenar pelo dígito das dezenas o vetor $A = (52, 17, 34, 19, 58, 31, 12, 36)$ (Figura 7.4). As contagens são três números com dezena 1, três com dezena 3 e dois com dezena 5; os blocos começam, portanto, nas posições 1, 4 e 7. Começando pelo bloco do 1: a posição 1 tem 52, que pertence ao bloco do 5, e é trocado com o elemento da posição 7, o 12; o 12 já está no bloco certo, assim como o 17; na posição 3 está 34, trocado com o 19 da posição 4. O bloco do 1 está completo. No bloco do 3, o 58 da posição 5 é trocado com o 36 da posição 8, e todos os elementos já ocupam os seus blocos. Foram três trocas. Depois, cada bloco é ordenado recursivamente pelo dígito das unidades.

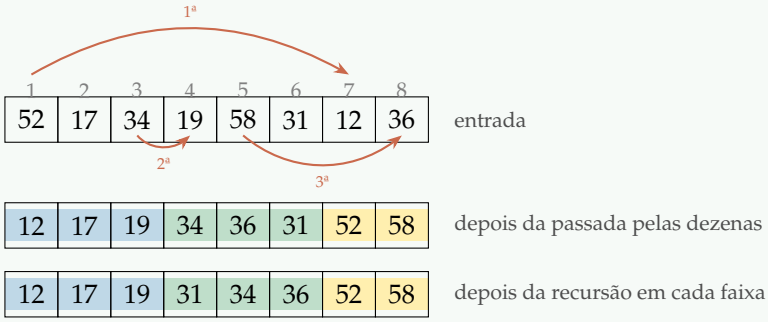


Figura 7.4. American Flag sort sobre $(52, 17, 34, 19, 58, 31, 12, 36)$. Depois da passada pelas dezenas, o vetor fica dividido em faixas (dezena 1, 3 e 5); cada faixa é então ordenada recursivamente pelas unidades. As setas indicam as três trocas da primeira passada.

Note que a faixa da dezena 3 ficou $(34, 36, 31)$, embora na entrada o 31 viesse antes do 36: as trocas não preservam a ordem relativa. Isso não atrapalha, porque a recursão reordena a faixa pelas unidades.

A memória auxiliar se resume aos contadores: $O(b)$ por nível de recursão, $O(b \cdot d)$ no total, independente de n. As trocas mudam a ordem relativa de elementos iguais, de modo que o algoritmo **não é estável** — e não precisa ser, porque cada bloco é ordenado pelos dígitos seguintes de forma independente.

algoritmo	hipótese sobre as chaves	tempo	memória extra	estável?
counting sort	inteiros em $[0, k]$	$\Theta(n + k)$	$\Theta(n + k)$	sim
radix sort	d dígitos na base b	$\Theta(d(n + b))$	$\Theta(n + b)$	sim
bucket sort	reais uniformes em $[0, 1)$	$\Theta(n)$ esperado	$\Theta(n)$	sim, com inserção
American Flag	d dígitos na base b	$\Theta(d(n + b))$	$O(b \cdot d)$	não

7.5 Qual algoritmo de ordenação usar?

Situação	Escolha
Inteiros num intervalo pequeno ($k = O(n)$)	**Counting sort**, $\Theta(n + k)$
Inteiros ou identificadores com poucos dígitos (d fixo) e k enorme	**Radix sort**, $\Theta(d(n + b))$
Reais uniformemente distribuídos num intervalo	**Bucket sort**, $\Theta(n)$ esperado
Chaves arbitrárias, com garantia de tempo e/ou estabilidade	**Mergesort**, $\Theta(n \log n)$, memória $\Theta(n)$
Chaves arbitrárias, pouca memória, com garantia de tempo	**Heapsort**, $\Theta(n \log n)$, in-place
Chaves arbitrárias, desempenho médio na prática	**Quicksort** com pivô aleatório ou mediana de três
Vetor quase ordenado, ou n pequeno	**Inserção**
Dados em lista encadeada ou em arquivos externos	**Mergesort**

Três perguntas antes de ordenar

1. **As chaves são inteiros (ou podem ser vistas como inteiros)?** Se sim, qual o tamanho do intervalo? Intervalo comparável a n: counting sort. Intervalo enorme, mas com poucos dígitos: radix sort.
2. **São reais com distribuição conhecida e uniforme?** Bucket sort. Se a distribuição é desconhecida, não aposte nela.
3. **Nada disso?** Só resta comparar: escolha entre mergesort, heapsort e quicksort pela memória disponível, pela necessidade de estabilidade e pelo tipo de garantia exigida (pior caso ou média).

Exemplo 7.9 (Escolhendo o algoritmo)**.** (A) Os horários de embarque de 10^7 passageiros de um dia, em minutos desde a meia-noite (inteiros de 0 a 1439): **counting sort**, com 1440 contadores e tempo $\Theta(n)$; uma ordenação por comparação faria cerca de $n \log_2 n \approx 2{,}3 \cdot 10^8$ comparações. (B) 10^6 números de telefone de 11 dígitos: o intervalo tem 10^{11} valores, e o counting sort direto é inviável; o **radix sort** com 11 passadas na base 10 (ou 4 passadas na base 10^3) é linear. (C) Os tempos de chegada de 10^5 clientes, sorteados uniformemente em $[0, 60)$ minutos: dividindo por 60, são reais uniformes em $[0, 1)$, e o **bucket sort** ordena em tempo esperado linear. (D) 10^5 nomes de clientes, que precisam ficar em ordem alfabética e, nos empates, na ordem de cadastro: são chaves arbitrárias e a estabilidade importa, e o **mergesort** dá as duas coisas.

Armadilhas da ordenação linear

- Dizer que o counting sort é "sempre linear": ele é $\Theta(n + k)$, e só é linear quando $k = O(n)$.
- Esquecer a memória: counting e radix sort usam vetores auxiliares de tamanho $\Theta(n + k)$ ou $\Theta(n + b)$; "linear" se refere ao tempo, e não dispensa essa memória.
- Começar o radix sort (com passadas estáveis) pelo dígito *mais* significativo: a última passada desfaria o trabalho das anteriores. Pelo dígito mais significativo, só com a recursão em blocos do American Flag sort.
- Usar o bucket sort sem garantia de distribuição uniforme: o tempo linear é apenas *esperado*, e só sob essa hipótese.

Pontos-chave

Counting sort: $\Theta(n + k)$, estável (último laço de trás para frente), linear se $k = O(n)$. Radix sort (do menos para o mais significativo): $\Theta(d(n + b))$, exige passadas estáveis. Bucket sort: $\Theta(n)$ esperado para entrada uniforme. American Flag sort: radix pelo dígito mais significativo, in-place e não estável. Para chaves arbitrárias, o limite $\Omega(n \log n)$ continua valendo.

7.6 Exercícios

Exercícios do curso

7.1 [Avaliação] resposta: p. 179
Uma empresa precisa ordenar grandes volumes de dados com características diferentes. Para cada cenário, escolha o algoritmo mais adequado entre counting sort, radix sort, bucket sort,

mergesort e quicksort, justificando por tempo, espaço e características dos dados: (A) 10^7 notas entre 0 e 100; (B) 10^6 identificadores numéricos de 12 dígitos; (C) 10^5 números reais uniformemente distribuídos entre 0 e 1; (D) 10^6 inteiros com valores entre -10^9 e 10^9; (E) 10^5 registros com chaves arbitrárias e sem restrição de domínio.

7.2 [Slides] resposta: p. 179
Modifique o radix sort para ordenar inteiros com quantidades diferentes de dígitos em tempo $O\left(\sum_{i=1}^{n} d_i\right)$, onde d_i é o número de dígitos do i-ésimo inteiro. Faça o mesmo para o American Flag sort.

7.3 [Slides] resposta: p. 179
Que modificações são necessárias no radix sort e no American Flag sort para ordenar *strings* em vez de inteiros?

Exercícios complementares

7.4 [Complementar] resposta: p. 179
Execute o counting sort em $A = [2, 5, 3, 0, 2, 3, 0, 3]$ com $k = 5$: mostre o vetor C após a contagem e após as somas acumuladas, e a saída B.

7.5 [Complementar] resposta: p. 179
Execute o radix sort na base 10 sobre $329, 457, 657, 839, 436, 720, 355$, mostrando a lista após cada passada.

7.6 [Complementar] resposta: p. 179
Mostre como ordenar n inteiros do intervalo $[0, n^3 - 1]$ em tempo $O(n)$.

7.7 [Complementar] resposta: p. 179
No último laço do counting sort, o que muda se j for de 1 até n (em vez de n até 1)? O resultado continua ordenado? E o radix sort que usa esse counting sort em cada passada?

7.8 [Complementar] resposta: p. 180
Qual o pior caso do bucket sort com ordenação por inserção nos baldes? Como garantir $O(n \log n)$ no pior caso sem perder o tempo esperado $\Theta(n)$?

7.9 [Complementar] resposta: p. 180
Por que o radix sort que começa pelo dígito menos significativo precisa de passadas estáveis, enquanto o American Flag sort, que começa pelo mais significativo, funciona mesmo não sendo estável?

7.10 [Complementar] resposta: p. 180
Como usar o counting sort para ordenar inteiros no intervalo $[-m, m]$? Qual o custo?

7.11 [Complementar] resposta: p. 180
Para ordenar n inteiros no intervalo $[0, n^2 - 1]$, compare o counting sort direto com o radix sort na base n.

7.12 [Complementar] resposta: p. 180
Ordene pelo radix sort as palavras COW, DOG, SEA, RUG, ROW, MOB, BOX, TAB, BAR, EAR, TAR, DIG, BIG, TEA, NOW, FOX, mostrando a lista após cada uma das três passadas.

7.13 [Complementar] resposta: p. 180
Execute o bucket sort com 10 baldes sobre 0,78; 0,17; 0,39; 0,26; 0,72; 0,94; 0,21; 0,12; 0,23; 0,68. Mostre o conteúdo de cada balde não vazio.

7.14 [Complementar] resposta: p. 180
Para ordenar 10^6 inteiros de 32 bits com o radix sort, compare as bases 2^4, 2^8 e 2^{16}: quantas passadas cada uma faz e qual o custo aproximado de cada passada? Qual parece a melhor escolha?

7.15 [Complementar] resposta: p. 180
Prove por indução no número de passadas que o radix sort (do dígito menos significativo para o mais significativo, com passadas estáveis) ordena corretamente.

7.16 [Complementar] resposta: p. 180
Uma lista de alunos já está em ordem alfabética e deve ser ordenada pela nota (um inteiro de 0 a 10), mantendo em ordem alfabética os alunos com a mesma nota. Que algoritmo usar e por quê?

7.17 [Complementar] resposta: p. 180
Datas no formato (dia, mês, ano) devem ser ordenadas cronologicamente com o radix sort. Em que ordem devem ser feitas as passadas e qual a base de cada uma?

7.18 [Complementar] resposta: p. 180
Dados n inteiros no intervalo $[0, n^2]$, decida em tempo $O(n)$ se há elementos repetidos.

7.19 [Complementar] resposta: p. 180
Dados n inteiros no intervalo $[0, k]$, faça um pré-processamento em tempo $O(n + k)$ que permita responder em $O(1)$ a perguntas do tipo "quantos elementos estão no intervalo $[a, b]$?".

7.20 [Complementar] resposta: p. 180
Dê uma entrada de n números em $[0, 1)$ para a qual o bucket sort com n baldes e inserção nos baldes gasta $\Theta(n^2)$.

7.21 [Complementar] resposta: p. 180
Quanta memória auxiliar o American Flag sort usa para ordenar n inteiros de d dígitos na base b? Por que se diz que ele é in-place, se usa vetores de contadores?

8

Seleção em tempo linear: a mediana das medianas

Neste capítulo

- O problema da seleção do k-ésimo menor elemento e as soluções ingênuas.
- O quickselect e a sua análise de tempo esperado linear.
- O algoritmo da mediana das medianas, que seleciona em tempo linear no pior caso.
- Por que grupos de 5 funcionam e grupos de 3 não.

Para achar a mediana de um conjunto — ou, em geral, o k-ésimo menor elemento — não é preciso ordenar. O quickselect resolve o problema em tempo linear *esperado*, recursando em apenas um dos lados da partição. O algoritmo da mediana das medianas, de Blum, Floyd, Pratt, Rivest e Tarjan, garante tempo linear *no pior caso*, escolhendo como pivô a mediana das medianas de grupos de cinco elementos. Sua análise é um belo exemplo de recorrência com subproblemas de tamanhos diferentes.

8.1 O problema e o quickselect

No **problema da seleção**, recebemos um conjunto A de n números e um inteiro k, e queremos o k-ésimo menor elemento de A, isto é, o elemento que ocuparia a posição k se A fosse ordenado. Casos particulares são o mínimo ($k = 1$), o máximo ($k = n$) e as medianas inferior, $k = \lfloor (n+1)/2 \rfloor$, e superior, $k = \lceil (n+1)/2 \rceil$; quando n é ímpar, as duas medianas coincidem.

Exemplo 8.1 (Medianas de um conjunto pequeno)**.** Em $A = \{9, 4, 7, 1, 8, 2\}$, com $n = 6$, a ordem crescente é $1, 2, 4, 7, 8, 9$. A mediana inferior é o elemento de posição $\lfloor 7/2 \rfloor = 3$, o 4; a superior, o de posição $\lceil 7/2 \rceil = 4$, o 7. O terceiro maior é o quarto menor, também o 7: em geral, o k-ésimo maior é o $(n-k+1)$-ésimo menor. Note que, para responder, *não* precisamos saber a ordem dos outros elementos entre si: basta saber quantos são menores e quantos são maiores que a resposta.

Há várias maneiras de resolver o problema: (1) ordenar e tomar a posição k, em $O(n \log n)$; (2) retirar o mínimo k vezes, em $O(kn)$, o que dá $O(n^2)$ para a mediana; (3) o **quickselect**, que particiona como o quicksort mas recursa em *um* lado só, em $O(n)$ esperado e $O(n^2)$ no pior caso; (4) o algoritmo da **mediana das medianas**, em $O(n)$ no **pior caso**.

Intuição

O quicksort particiona e depois ordena *os dois* lados. Para a seleção, depois de particionar já sabemos em que lado está a resposta: se há pelo menos k elementos menores que o pivô, ela está entre eles; senão, está entre os maiores (ou é o próprio pivô). O outro lado pode ser descartado inteiro. É como procurar uma palavra no dicionário: abrir numa página

qualquer diz se a palavra está antes ou depois, e metade do livro deixa de importar.

Quickselect: tempo esperado $O(n)$

Função QuickSelect(S, k)
  escolha um pivô m **aleatório** em S
  $S_1 := \{s < m\}$; $S_2 := \{s = m\}$; $S_3 := \{s > m\}$
  se $|S_1| \geq k$ **então**
    retorna QuickSelect(S_1, k)
  senão se $|S_1| + |S_2| \geq k$ **então**
    retorna m
  senão
    retorna QuickSelect(S_3, $k - |S_1| - |S_2|$)

Exemplo 8.2 (Quickselect passo a passo)**.** Procuramos o 5º menor elemento do conjunto $S = \{15, 3, 22, 8, 11, 6, 19, 2, 13\}$ e supomos que os pivôs sorteados sejam, em sequência, 13, 3, 8 e 11 (Figura 8.1).

- Pivô 13: $S_1 = \{3, 8, 11, 6, 2\}$, $S_2 = \{13\}$, $S_3 = \{15, 22, 19\}$. Como $|S_1| = 5 \geq 5$, a resposta está em S_1, e k continua 5.
- Pivô 3: $S_1 = \{2\}$, $S_2 = \{3\}$, $S_3 = \{8, 11, 6\}$. Como $|S_1| + |S_2| = 2 < 5$, a resposta está em S_3; os 2 elementos de $S_1 \cup S_2$ ficaram para trás, e k passa a $5 - 2 = 3$.
- Pivô 8: $S_1 = \{6\}$, $S_2 = \{8\}$, $S_3 = \{11\}$. Como $1 + 1 = 2 < 3$, segue para S_3 com $k = 3 - 2 = 1$.
- Pivô 11: $|S_1| = 0 < 1 \leq |S_1| + |S_2| = 1$, e a resposta é **11**.

De fato, a ordem crescente de S é 2, 3, 6, 8, **11**, 13, 15, 19, 22. As quatro chamadas examinaram $9 + 5 + 3 + 1 = 18$ elementos, contra cerca de $n \log_2 n \approx 29$ comparações de uma ordenação.

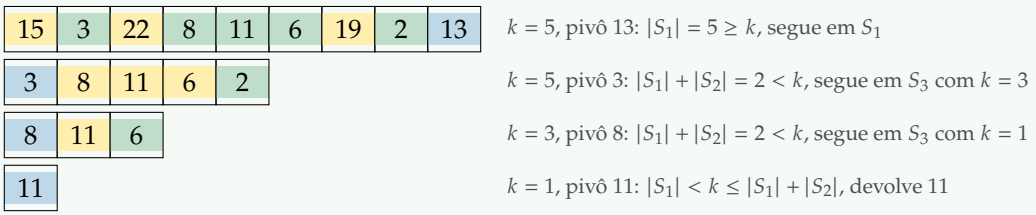


Figura 8.1. O quickselect procurando o 5º menor. Em cada linha, o pivô está em azul, os menores que ele em verde e os maiores em amarelo; só a parte que contém a resposta passa para a linha seguinte.

Análise. Com probabilidade pelo menos 1/2, o pivô cai entre o primeiro e o terceiro quartis, e então o lado mantido tem no máximo $3n/4$ elementos (Figura 8.2). Agrupe as chamadas em *fases*: uma fase termina quando o tamanho cai para no máximo $3n/4$. O número de chamadas numa fase é dominado por uma variável geométrica de parâmetro 1/2, cuja esperança é 2, e cada chamada custa no máximo cn. Assim, $E[T(n)] \leq E[T(3n/4)] + 2cn$, e, somando a série geométrica, $E[T(n)] \leq 2cn \sum_{t \geq 0} (3/4)^t = 8cn = O(n)$.

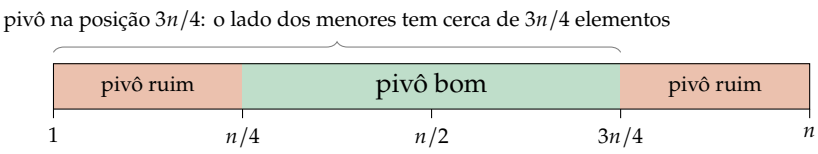


Figura 8.2. Os elementos em ordem crescente. Se o pivô cai na metade central (entre o primeiro e o terceiro quartis), cada lado da partição tem no máximo $3n/4$ elementos, qualquer que seja o lado mantido. Um pivô aleatório cai ali com probabilidade 1/2.

Exemplo 8.3 (Quantos pivôs são bons?)**.** Com $n = 100$ elementos distintos, os pivôs das posições 26 a 75 na ordem crescente, a metade central, deixam no máximo 74 elementos de cada lado, menos que $3n/4 = 75$: são 50 dos 100 possíveis. Em média, portanto, são necessários dois sorteios para obter um pivô bom, e cada pivô bom reduz o problema a no máximo 3/4 do tamanho. Os custos das fases formam a série $2cn + 2c\frac{3}{4}n + 2c\frac{9}{16}n + \cdots$, que soma no máximo $8cn$.

8.2 O algoritmo da mediana das medianas

O quickselect só é linear *em média*: um azar persistente na escolha do pivô o leva a $\Theta(n^2)$. A ideia da mediana das medianas é garantir que o pivô seja *sempre* bom, sem sorteio: ele é a mediana das medianas de grupos de 5 elementos.

Seleção(S, k) pela mediana das medianas

```
Função Seleção(S, k)
    se |S| ≤ 49 então
        ordene S e retorna o k-ésimo menor
    divida S em ⌈|S|/5⌉ grupos de até 5 elementos
    determine a mediana de cada grupo                  % ordenar 5 elementos custa O(1)
    M := conjunto das medianas;   m := Seleção(M, ⌈|M|/2⌉)   % recursão sobre n/5 elementos
    S1 := {s ∈ S : s < m}; S2 := {s ∈ S : s = m}; S3 := {s ∈ S : s > m}
    se |S1| ≥ k então
        retorna Seleção(S1, k)                         % recursão sobre no máximo 3n/4
    senão se |S1| + |S2| ≥ k então
        retorna m
    senão
        retorna Seleção(S3, k − |S1| − |S2|)
```

Repare que o algoritmo faz *duas* chamadas recursivas: uma para encontrar o pivô, sobre as $n/5$ medianas, e outra para continuar a busca, sobre um dos lados da partição. É essa segunda chamada que precisa ser pequena, e é para isso que o pivô é escolhido com tanto cuidado.

Exemplo 8.4 (Escolhendo o pivô)**.** Tome os 35 números

$$24, 16, 85, 58, 43, \quad 31, 26, 63, 81, 64, \quad 99, 62, 38, 59, 34, \quad 93, 33, 98, 42, 67,$$
$$23, 94, 22, 78, 84, \quad 47, 65, 75, 90, 28, \quad 39, 97, 1, 44, 52,$$

já divididos em 7 grupos de 5. (Como $35 \leq 49$, o algoritmo ordenaria tudo diretamente; o exemplo serve para ver como o pivô é escolhido.) As medianas dos grupos são 43, 63, 59, 67, 78, 65, 44; em ordem, 43, 44, 59, 63, 65, 67, 78, e a mediana delas é $m = 63$. A partição dá $|S_1| = 20$ elementos menores que 63 e $|S_3| = 14$ maiores. A mediana verdadeira dos 35 números é 58, e não 63: o pivô não é a mediana, mas é *bom o bastante*. A Figura 8.3 mostra por quê: pelo menos 12 elementos são certamente ≥ 63, e pelo menos 12 são certamente ≤ 63; logo, cada lado da partição tem no máximo $35 - 12 = 23$ elementos, menos que $3 \cdot 35/4 \approx 26$.

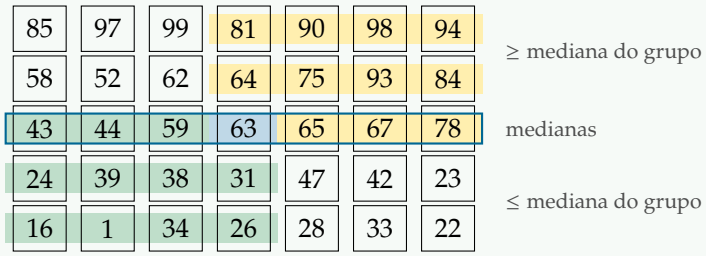


Figura 8.3. Os grupos do Exemplo 8.4, cada um ordenado de cima (maior) para baixo (menor), com as colunas em ordem crescente de mediana. A linha do meio são as medianas, e o pivô $m = 63$ está em azul. Em amarelo, 12 elementos que são certamente $\geq m$ (maiores ou iguais à mediana do seu grupo, que é $\geq m$); em verde, 12 que são certamente $\leq m$. Os elementos brancos podem estar de qualquer lado.

Armadilhas comuns

O teste que decide ir para S_1 é $|S_1| \geq k$, e não $|S_1| = k$: o k-ésimo menor está em S_1 sempre que há pelo menos k elementos menores que o pivô. E, ao ir para S_3, o índice muda para $k - |S_1| - |S_2|$, pois os elementos de S_1 e S_2 ficaram para trás.

8.3 Análise

A Figura 8.3 contém toda a ideia da análise: metade das colunas tem mediana $\geq m$, e cada uma delas tem três elementos $\geq$ a sua mediana. O lema abaixo faz a conta para um n qualquer.

Lema 8.5. $|S_1| \leq n - 3\lfloor n/10 \rfloor$ *e* $|S_3| \leq n - 3\lfloor n/10 \rfloor$. *Para* $n \geq 50$, *ambos são no máximo* $3n/4$.

Demonstração. Das $\lfloor n/5 \rfloor$ medianas de grupos completos, pelo menos metade, isto é, $\lfloor \lfloor n/5 \rfloor / 2 \rfloor = \lfloor n/10 \rfloor$, são maiores ou iguais a m. Cada uma delas vem de um grupo que tem mais dois elementos maiores ou iguais a ela, e portanto a m. Assim, pelo menos $3\lfloor n/10 \rfloor$ elementos são maiores ou iguais a m, ou seja, $|S_2| + |S_3| \geq 3\lfloor n/10 \rfloor$ e $|S_1| \leq n - 3\lfloor n/10 \rfloor$. O argumento para S_3 é simétrico. Para que $n - 3\lfloor n/10 \rfloor \leq 3n/4$ basta $12\lfloor n/10 \rfloor \geq n$, isto é, $2\lfloor n/10 \rfloor \geq n - 10\lfloor n/10 \rfloor$. Como o lado direito é no máximo 9, basta $\lfloor n/10 \rfloor \geq 5$, ou seja, $n \geq 50$. Por isso o caso base é $n \leq 49$. □

Por exemplo, com $n = 1000$, há pelo menos $3 \cdot 100 = 300$ elementos de cada lado do pivô, e a chamada seguinte recebe no máximo 700 elementos, menos que $3n/4 = 750$.

Teorema 8.6. $T(n) \leq 20cn$; *portanto, a seleção pela mediana das medianas custa* $O(n)$ *no pior caso.*

Demonstração. $T(n) = O(1)$ para $n \leq 49$, e $T(n) \leq T(n/5) + T(3n/4) + cn$ para $n > 49$, pois o cálculo das medianas dos grupos e a partição são lineares. Por indução forte, escolhendo c grande o suficiente para que a base valha:

$$T(n) \leq 20c\,\frac{n}{5} + 20c\,\frac{3n}{4} + cn = 4cn + 15cn + cn = 20cn. \qquad \square$$

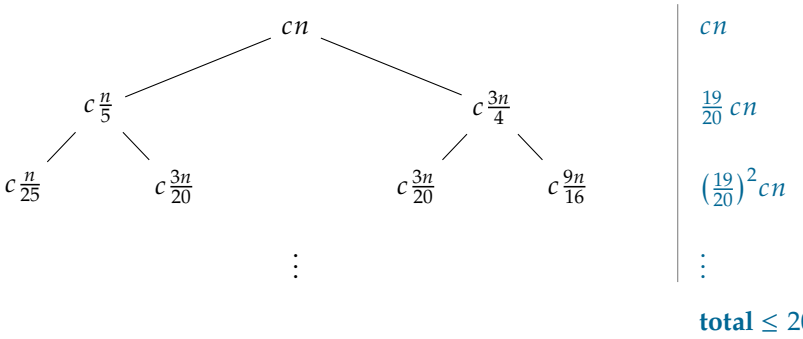


Figura 8.4. Árvore de recursão de $T(n) = T(n/5) + T(3n/4) + cn$. Os dois subproblemas somam $\frac{19}{20}$ do tamanho do pai, e o custo de cada nível é $\frac{19}{20}$ do custo do nível acima: a soma é uma série geométrica, limitada por $20cn$.

Por que grupos de 5? Para que o tempo seja linear, a soma dos tamanhos dos dois subproblemas precisa ser uma fração menor que 1 de n: com grupos de 5, $\frac{n}{5} + \frac{3n}{4} = \frac{19n}{20} < n$. Com grupos de 3, o pivô só garante cerca de $n/3$ elementos de cada lado, e a recorrência $T(n) = T(n/3) + T(2n/3) + cn$, com $\frac{1}{3} + \frac{2}{3} = 1$, tem solução $\Theta(n \log n)$. Grupos de 7, 9 ou mais também funcionam, com outras constantes.

> **Intuição**
>
> Na árvore de recursão, o que decide o custo é a soma das frações dos subproblemas. Se ela é menor que 1, cada nível custa uma fração fixa do anterior, e o total é dominado pela raiz: $O(n)$, como na Figura 8.4. Se ela é exatamente 1, todos os níveis custam cn, e são $\Theta(\log n)$ níveis: $\Theta(n \log n)$, como no mergesort. Grupos maiores dão um pivô mais garantido, mas cada mediana de grupo fica mais cara; grupos de 5 são o menor tamanho ímpar em que a soma das frações fica abaixo de 1.

Aplicação. Usando como pivô do quicksort a mediana exata, calculada pela mediana das medianas, obtém-se $T(n) = 2T(n/2) + O(n) = O(n \log n)$ no **pior** caso. Na prática, porém, esse quicksort é mais lento que o quicksort com pivô aleatório, por causa das constantes.

método	pior caso	caso esperado	observação
ordenar e indexar	$\Theta(n \log n)$	$\Theta(n \log n)$	devolve também a ordem completa
retirar o mínimo k vezes	$\Theta(kn)$	$\Theta(kn)$	bom só para k pequeno
quickselect (pivô aleatório)	$\Theta(n^2)$	$\Theta(n)$	simples e rápido na prática
mediana das medianas	$\Theta(n)$	$\Theta(n)$	garantia de pior caso, constantes altas

Pontos-chave

Seleção em $O(n)$ no pior caso: grupos de 5, mediana das medianas como pivô, pelo menos $3\lfloor n/10 \rfloor$ elementos de cada lado, $T(n) \leq T(n/5) + T(3n/4) + cn \leq 20cn$. O quickselect com pivô aleatório é mais simples e linear em média. Não é preciso ordenar para achar o k-ésimo menor.

8.4 Exercícios

Exercícios do curso

8.1 [Slides] resposta: p. 180

Escreva um algoritmo de tempo *esperado* $O(n)$ para determinar a mediana de uma lista com n

elementos, mais simples que o método da mediana das medianas.

8.2 [Slides] resposta: p. 181
Argumente como o quicksort pode ser implementado com complexidade $O(n \log n)$ no pior caso. Qual o problema dessa abordagem na prática?

Exercícios complementares

8.3 [Complementar] resposta: p. 181
Para $S = 12, 3, 25, 7, 18,\ 1, 22, 9, 15, 6,\ 20, 2, 11, 24, 5,\ 17, 8, 23, 4, 14,\ 10, 21, 13, 19, 16$ (grupos de 5 na ordem dada), calcule as medianas dos grupos, o pivô m e os tamanhos de S_1 e S_3. Confira que ambos são pelo menos $3\lfloor n/10 \rfloor$.

8.4 [Complementar] resposta: p. 181
Refaça a análise do algoritmo com grupos de 7 e com grupos de 3. Em qual dos casos ele continua linear?

8.5 [Complementar] resposta: p. 181
Dê um algoritmo $O(n)$ que devolve os k menores elementos de um conjunto, em qualquer ordem. E se eles forem pedidos em ordem crescente?

8.6 [Complementar] resposta: p. 181
Um oleoduto horizontal (leste-oeste) deve ligar n poços, cada um por um duto vertical. Dadas as coordenadas $y_1, \ldots, y_n$ dos poços, em que altura colocar o oleoduto para minimizar o comprimento total dos dutos verticais? Mostre que a resposta pode ser encontrada em $O(n)$.

8.7 [Complementar] resposta: p. 181
Mostre que o quickselect pode custar $\Theta(n^2)$ no pior caso, mesmo para encontrar o mínimo.

8.8 [Complementar] resposta: p. 181
Execute o quickselect sobre $[7, 2, 9, 4, 3, 8, 6, 1]$ com $k = 4$, usando sempre o *último* elemento do conjunto (na ordem dada) como pivô. Mostre S_1, S_2, S_3 e o novo k em cada chamada.

8.9 [Complementar] resposta: p. 181
Calcule $n - 3\lfloor n/10 \rfloor$ e $3n/4$ para $n = 49$ e para $n = 50$. O que isso mostra sobre o caso base do algoritmo?

8.10 [Complementar] resposta: p. 181
Suponha que, em vez de calcular a mediana das medianas recursivamente, o algoritmo ordene as $n/5$ medianas com o mergesort para achar a do meio. Qual passa a ser a complexidade?

8.11 [Complementar] resposta: p. 181
O algoritmo da mediana das medianas continua linear se os elementos estiverem numa lista encadeada, em vez de um vetor? Justifique.

8.12 [Complementar] resposta: p. 181
Mostre como calcular os três quartis de um conjunto de n números em $O(n)$.

8.13 [Complementar] resposta: p. 181
Dados n números distintos e um inteiro k, encontre em $O(n)$ os k números mais próximos da mediana (em valor absoluto da diferença).

8.14 [Complementar ★] resposta: p. 181
Cada elemento x_i tem um peso $w_i > 0$, com $\sum w_i = 1$. A *mediana ponderada* é o elemento x_k tal que a soma dos pesos dos elementos menores que x_k é menor que $1/2$ e a soma dos pesos dos maiores é no máximo $1/2$. Encontre-a em $O(n)$.

8.15 [Complementar ★] resposta: p. 181
Dados dois vetores ordenados X e Y, cada um com n elementos, encontre a mediana dos $2n$ elementos em $O(\log n)$.

8.16 [Complementar ★] resposta: p. 182
Mostre como encontrar o segundo menor de n elementos com $n + \lceil \log_2 n \rceil - 2$ comparações, organizando as comparações como um torneio.

8.17 [Complementar ★] resposta: p. 182
Mostre como encontrar a mediana de 5 elementos com apenas 6 comparações.

8.18 [Complementar] resposta: p. 182
Mostre que todo algoritmo baseado em comparações que encontra a mediana de n elementos distintos faz pelo menos $n - 1$ comparações. (Pense no grafo cujas arestas são as comparações feitas.)

8.19 [Complementar ★] resposta: p. 182
Mostre que o número esperado de comparações do quickselect satisfaz

$$E[T(n)] \leq n + \frac{2}{n} \sum_{i=\lfloor n/2 \rfloor}^{n-1} E[T(i)]$$

e conclua, por substituição, que $E[T(n)] \leq 4n$.

8.20 [Complementar] resposta: p. 182
Como usar um algoritmo que seleciona o k-ésimo *menor* para encontrar o k-ésimo *maior*? E para encontrar o percentil 90 de um conjunto de n notas?

9

Divisão e conquista

Neste capítulo

- O paradigma de divisão e conquista e um roteiro para projetar e analisar algoritmos com ele.
- A contagem de inversões e o subvetor contíguo de soma máxima.
- A multiplicação de matrizes de Strassen.
- Outros exemplos clássicos: busca binária, exponenciação rápida, máximo e mínimo e elemento majoritário.

O mergesort, o quicksort e a seleção pela mediana das medianas já usaram a mesma ideia: quebrar o problema em pedaços menores do mesmo tipo, resolvê-los recursivamente e combinar as respostas. Este capítulo trata essa ideia como um método de projeto, com um roteiro para escrever o pseudocódigo e analisar a recorrência, e a aplica a vários problemas.

9.1 O paradigma

Um algoritmo de **divisão e conquista** tem três passos: (1) *divisão*: quebrar o problema Π em subproblemas menores **do mesmo tipo**, $\Pi_1, \ldots, \Pi_k$; (2) *conquista*: resolvê-los recursivamente, resolvendo diretamente os casos pequenos; (3) *combinação*: juntar as soluções dos subproblemas, por um procedimento relativamente simples, numa solução de Π. Busca-se um **bom balanceamento**, com partes de tamanhos parecidos, que leva a poucas camadas de recursão. A técnica funciona bem quando os subproblemas são **disjuntos**; quando eles se repetem muito, a técnica adequada é a programação dinâmica do Capítulo 11.

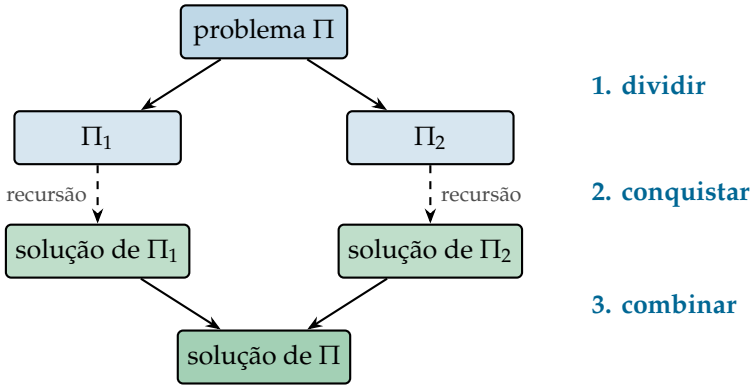


Figura 9.1. Os três passos da divisão e conquista. A divisão e a combinação são feitas pelo próprio algoritmo; a conquista é delegada às chamadas recursivas, que repetem o esquema até chegar a casos pequenos, resolvidos diretamente.

Como escrever um algoritmo de divisão e conquista

1. **Assinatura**: uma função Resolve(V, ini, fim) que trabalha num intervalo do vetor, para não copiar vetores. Diga o que a função **devolve**; às vezes é mais de um valor, como a soma e os índices, ou a contagem e o vetor ordenado.
2. **Caso base**: $ini = fim$ (ou n menor que uma constante), com resposta direta.
3. **Divisão**: $meio := \lfloor (ini + fim)/2 \rfloor$.
4. **Conquista**: duas chamadas recursivas, sobre $[ini, meio]$ e $[meio + 1, fim]$.
5. **Combinação**: trata o que "atravessa" o meio; é a parte criativa e costuma custar $O(n)$.
6. **Complexidade**: escreva a recorrência e resolva-a (§6.2); por exemplo, $2T(n/2) + \Theta(n) = \Theta(n \log n)$.
7. **Correção**, em uma frase: toda solução está inteira à esquerda, inteira à direita ou cruza o meio, e cada caso é tratado.

9.2 Contagem de inversões

Dada uma sequência $a_1, \ldots, a_n$, um par $i < j$ é uma **inversão** se $a_i > a_j$. O número de inversões mede quão distante uma ordem está da ordem crescente, e serve como medida de semelhança entre dois rankings: se uma pessoa ordena n itens como $1, \ldots, n$ e outra pessoa os ordena de outra forma, as inversões da segunda ordem contam os pares em que as duas discordam. A força bruta testa todos os pares, em $\Theta(n^2)$.

Exemplo 9.1 (Inversões e rankings)**.** A sequência $(4, 1, 3, 2)$ tem 4 inversões: os pares $(4, 1)$, $(4, 3)$, $(4, 2)$ e $(3, 2)$, em que um número maior aparece antes de um menor (Figura 9.2). Uma sequência crescente não tem nenhuma inversão, e uma decrescente tem todas as possíveis. Agora, duas amigas ordenam cinco filmes por preferência. Numerando os filmes pela ordem da primeira, a segunda escolheu $(2, 1, 4, 5, 3)$. As inversões são $(2, 1)$, $(4, 3)$ e $(5, 3)$: as duas discordam sobre 3 dos $\binom{5}{2} = 10$ pares de filmes. Quanto menos inversões, mais parecidos os gostos.

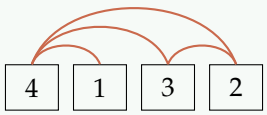


Figura 9.2. As 4 inversões de $(4, 1, 3, 2)$: cada arco liga um par em que o elemento da esquerda é maior.

Divisão e conquista. Divida a sequência em metades A e B; conte recursivamente as inversões dentro de cada uma; e **combine** contando os pares (a, b) com $a \in A$, $b \in B$ e $a > b$. Esse último passo é fácil se A e B estiverem *ordenadas*: durante a intercalação, ao comparar a_i com b_j, se $a_i \leq b_j$, então a_i não forma inversão com nenhum elemento restante de B; se $a_i > b_j$, então b_j forma inversão com **todos os elementos restantes de** A. Por isso a função também devolve o vetor ordenado: ela é um mergesort "instrumentado".

Intuição

Ordenar cada metade não muda o número de inversões *entre* as metades: um par (a, b), com a na esquerda e b na direita, é inversão se e só se $a > b$, e isso não depende de onde a e b estão dentro das suas metades. Então podemos ordenar as metades à vontade — e, com elas ordenadas, as inversões cruzadas são contadas em blocos, e não uma a uma.

Contagem de inversões

```
Função CalculaInversões(V, ini, fim)                  % conta e deixa V[ini..fim] ordenado
    se ini = fim então
        retorna 0
    meio := ⌊(ini + fim)/2⌋
    a := CalculaInversões(V, ini, meio)
    b := CalculaInversões(V, meio + 1, fim)
    c := IntercalaContando(V, ini, meio, fim)
    retorna a + b + c
Função IntercalaContando(V, ini, meio, fim)
    i := ini; j := meio + 1; k := 1; cont := 0
    enquanto i ≤ meio e j ≤ fim faça
        se V[i] ≤ V[j] então
            W[k] := V[i]; i := i + 1
        senão
            W[k] := V[j]; j := j + 1
            cont := cont + (meio − i + 1)          % V[j] é menor que todos os restantes da esquerda
        k := k + 1
    copie os restantes de V[i..meio] e de V[j..fim] para W; copie W para V[ini..fim]
    retorna cont
```

Exemplo 9.2 (Contando inversões cruzadas na intercalação)**.** Intercalemos $A = (2, 5, 8)$ com $B = (1, 3, 9)$, contando as inversões entre as duas metades.

- $2 > 1$: sai o 1, que é menor que *todos* os 3 elementos restantes de A (2, 5 e 8); soma 3.
- $2 \leq 3$: sai o 2, que não forma inversão com nenhum elemento restante de B.
- $5 > 3$: sai o 3, menor que os 2 restantes de A (5 e 8); soma 2.
- $5 \leq 9$ e $8 \leq 9$: saem o 5 e o 8; por fim, o 9.

O total é $3 + 2 = 5$, e de fato as inversões cruzadas são $(2, 1)$, $(5, 1)$, $(8, 1)$, $(5, 3)$ e $(8, 3)$. Cada elemento de B "cobrou" de uma vez as inversões com o que restava de A, sem examiná-las uma a uma.

O custo é $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$. Dois detalhes importam: o teste é "$\leq$", porque elementos iguais não formam inversão, e soma-se $meio - i + 1$, a quantidade de elementos que restam em A, e não 1. Por exemplo, a sequência $1, 5, 4, 8, 10, 2, 6, 9, 12, 11, 3, 7$ tem **22** inversões.

Exemplo 9.3 (A recursão completa)**.** A Figura 9.3 mostra a contagem para $(6, 2, 8, 4, 1, 7, 3, 5)$. No nível de baixo, os pares $(6, 2)$ e $(8, 4)$ têm uma inversão cada; $(1, 7)$ e $(3, 5)$, nenhuma. No nível do meio, a intercalação de $(2, 6)$ com $(4, 8)$ encontra 1 inversão cruzada (o par $(6, 4)$), e a de $(1, 7)$ com $(3, 5)$ encontra 2 ($(7, 3)$ e $(7, 5)$). Na última intercalação, de $(2, 4, 6, 8)$ com $(1, 3, 5, 7)$, o 1 sai antes dos quatro elementos da esquerda, o 3 antes de três, o 5 antes de dois e o 7 antes de um: $4 + 3 + 2 + 1 = 10$. O total é $1 + 1 + 0 + 0 + 1 + 2 + 10 = 15$.

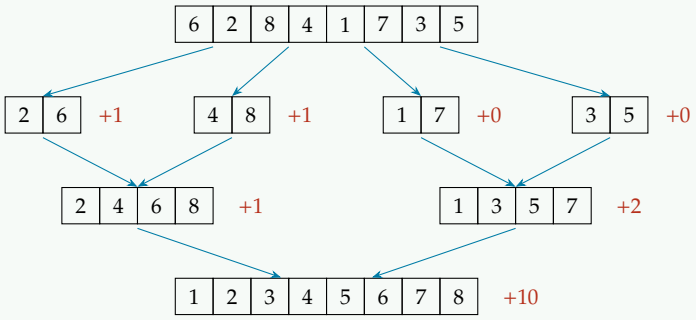


Figura 9.3. Contagem de inversões de $(6, 2, 8, 4, 1, 7, 3, 5)$ de baixo para cima. Ao lado de cada vetor intercalado está o número de inversões cruzadas encontradas na intercalação que o produziu.

9.3 Subvetor contíguo de soma máxima

Dado um vetor $A[1..n]$ de inteiros, que podem ser negativos, queremos $i \le j$ que maximizem $A[i] + \cdots + A[j]$, isto é, o subvetor contíguo **não vazio** de maior soma. Pense, por exemplo, nos resultados diários de uma empresa, positivos ou negativos, e na busca do período de dias consecutivos de maior resultado acumulado. A força bruta testa os $\binom{n}{2} + n$ intervalos; com somas acumuladas, cada um custa $O(1)$, e o total é $\Theta(n^2)$.

Ideia. O melhor subvetor de $A[ini..fim]$ está (i) inteiro na metade esquerda, (ii) inteiro na direita, ou (iii) **cruza o meio**, sendo da forma $A[i..meio] \cup A[meio+1..j]$ (Figura 9.4). O caso (iii) se resolve em tempo linear: é o melhor sufixo da esquerda que termina em $meio$ somado ao melhor prefixo da direita que começa em $meio + 1$. As duas escolhas são independentes, porque o valor do sufixo não depende de qual prefixo vem depois, e vice-versa.

Subvetor de soma máxima por divisão e conquista (devolve soma, início e fim)

```
Função MaxSub(A, ini, fim)
    se ini = fim então
        retorna (A[ini], ini, ini)                    % não vazio: um elemento
    meio := ⌊(ini + fim)/2⌋
    (s_E, i_E, j_E) := MaxSub(A, ini, meio)
    (s_D, i_D, j_D) := MaxSub(A, meio + 1, fim)
    (s_C, i_C, j_C) := MaxCruzado(A, ini, meio, fim)
    retorna a tripla de maior soma entre (s_E, i_E, j_E), (s_D, i_D, j_D) e (s_C, i_C, j_C)
Função MaxCruzado(A, ini, meio, fim)
    soma := 0; melhorE := −∞
    para i := meio decrescendo até ini faça
        soma := soma + A[i]
        se soma > melhorE então
            melhorE := soma; iMax := i
    soma := 0; melhorD := −∞
    para j := meio + 1 até fim faça
        soma := soma + A[j]
        se soma > melhorD então
            melhorD := soma; jMax := j
    retorna (melhorE + melhorD, iMax, jMax)
```

Exemplo 9.4 (O subvetor cruzado)**.** Seja $A = (1, -4, 3, -2, 5, -1, -6, 2)$, com $meio = 4$. Na metade esquerda, o melhor subvetor é (3), de soma 3; na direita, (5), de soma 5. Para o cruzado, MaxCruzado acumula os sufixos da esquerda, de trás para frente, e os prefixos da direita:

sufixo $A[i..4]$, $i=$	4	3	2	1	prefixo $A[5..j]$, $j=$	5	6	7	8
soma	−2	**1**	−3	−2	soma	**5**	4	−2	0

O melhor sufixo é $A[3..4] = (3, -2)$, de soma 1, e o melhor prefixo é $A[5..5] = (5)$, de soma 5: o cruzado é $A[3..5] = (3, -2, 5)$, de soma 6. Como $6 > \max\{3, 5\}$, a resposta é o subvetor cruzado. Repare que o melhor sufixo inclui o −2: sozinho ele é ruim, mas é o preço para ligar o 3 ao 5.

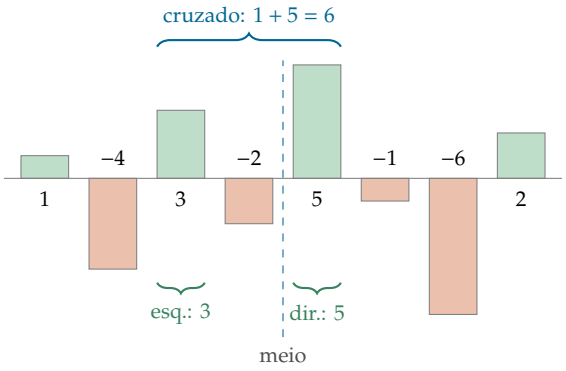


Figura 9.4. Os três casos para o subvetor máximo de $A = (1, -4, 3, -2, 5, -1, -6, 2)$: inteiro à esquerda do meio (melhor soma 3), inteiro à direita (5) ou cruzando o meio (6, a resposta).

- **Complexidade**: $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.
- **Todos os elementos negativos**: o algoritmo funciona, porque o caso base devolve um elemento e o subvetor cruzado usa pelo menos $A[meio]$ e $A[meio + 1]$; o máximo das três opções acaba sendo o maior elemento. Por exemplo, $[-8, -3, -6, -2, -5, -4]$ dá −2.
- Em $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$, a resposta é a soma 6, com o subvetor $[4, -1, 2, 1]$, das posições 4 a 7 (com índices a partir de 1).
- Existem soluções lineares: a programação dinâmica de Kadane (Capítulo 11) e uma divisão e conquista em que cada metade devolve quatro valores (Exercício 9.13).

9.4 Multiplicação de matrizes: o algoritmo de Strassen

Multiplicando matrizes $n \times n$ por blocos de tamanho $n/2$, o produto ingênuo faz 8 multiplicações de blocos, e $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$, sem ganho sobre o método escolar. Strassen mostrou como fazer apenas **7** multiplicações de blocos, à custa de mais somas e subtrações: $T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2,807})$, pelo caso 1 do Teorema Mestre, pois $n^2 = O(n^{\log_2 7 - \varepsilon})$.

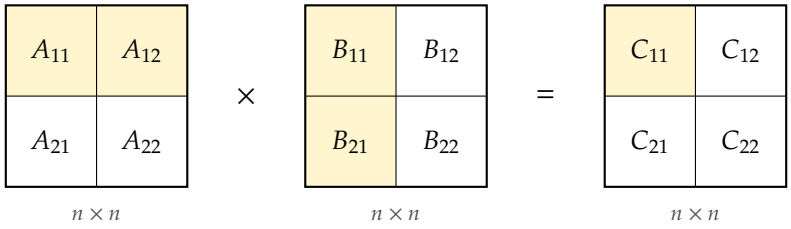


Figura 9.5. Multiplicação por blocos: cada matriz $n \times n$ é vista como uma matriz 2×2 de blocos $\frac{n}{2} \times \frac{n}{2}$, e o bloco C_{11} é $A_{11}B_{11} + A_{12}B_{21}$, como no produto de matrizes 2×2 de números. As fórmulas de Strassen valem para blocos porque só usam somas, subtrações e produtos, sem nunca trocar a ordem dos fatores.

Strassen(A, B, n), com n potência de 2 e soma e subtração de matrizes dadas

Função Strassen(A, B, n)
  se $n = 1$ **então**
    retorna $[\,A[1,1] \cdot B[1,1]\,]$
  particione $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$ e $B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$ em blocos $\frac{n}{2} \times \frac{n}{2}$
  $M_1 :=$ Strassen$(A_{11} + A_{22},\ B_{11} + B_{22},\ n/2)$
  $M_2 :=$ Strassen$(A_{21} + A_{22},\ B_{11},\ n/2)$
  $M_3 :=$ Strassen$(A_{11},\ B_{12} - B_{22},\ n/2)$
  $M_4 :=$ Strassen$(A_{22},\ B_{21} - B_{11},\ n/2)$
  $M_5 :=$ Strassen$(A_{11} + A_{12},\ B_{22},\ n/2)$
  $M_6 :=$ Strassen$(A_{21} - A_{11},\ B_{11} + B_{12},\ n/2)$
  $M_7 :=$ Strassen$(A_{12} - A_{22},\ B_{21} + B_{22},\ n/2)$
  $C_{11} := M_1 + M_4 - M_5 + M_7$; $C_{12} := M_3 + M_5$
  $C_{21} := M_2 + M_4$; $C_{22} := M_1 - M_2 + M_3 + M_6$
  retorna $\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$

Cada chamada faz 18 somas ou subtrações de matrizes $\frac{n}{2} \times \frac{n}{2}$, de custo $\Theta(n^2)$. Na prática, usa-se o método clássico abaixo de um certo tamanho, pois as constantes de Strassen são maiores e o algoritmo é numericamente menos estável.

Exemplo 9.5 (As fórmulas de Strassen com números)**.** Com $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ e $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$, os "blocos" são números, e as sete multiplicações dão

$$M_1 = (1+4)(5+8) = 65, \quad M_2 = (3+4)\cdot 5 = 35, \quad M_3 = 1\cdot(6-8) = -2,$$
$$M_4 = 4\cdot(7-5) = 8, \quad M_5 = (1+2)\cdot 8 = 24, \quad M_6 = (3-1)(5+6) = 22,$$
$$M_7 = (2-4)(7+8) = -30.$$

Combinando: $C_{11} = 65 + 8 - 24 - 30 = 19$, $C_{12} = -2 + 24 = 22$, $C_{21} = 35 + 8 = 43$ e $C_{22} = 65 - 35 - 2 + 22 = 50$. É o mesmo que o produto direto, $\begin{bmatrix} 1\cdot 5 + 2\cdot 7 & 1\cdot 6 + 2\cdot 8 \\ 3\cdot 5 + 4\cdot 7 & 3\cdot 6 + 4\cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$, que usa 8 multiplicações. Para números, economizar uma multiplicação não compensa as 18 somas; para *blocos* grandes, compensa, porque uma multiplicação de blocos custa muito mais que uma soma.

Exemplo 9.6 (O ganho de uma multiplicação a menos)**.** Com $n = 2^{10} = 1024$, a recursão tem 10 níveis. O produto por blocos com 8 multiplicações chega a $8^{10} \approx 1{,}07 \cdot 10^9$ multiplicações de números, que é n^3; o de Strassen, a $7^{10} \approx 2{,}8 \cdot 10^8$, quase 4 vezes menos. A cada nível, a diferença entre 8 e 7 se multiplica, e por isso uma única multiplicação a menos muda o expoente.

9.5 Outros exemplos clássicos

Busca binária, potência rápida e máximo e mínimo simultâneos

Função BuscaBin(V, ini, fim, x) % V ordenado; $T(n) = T(n/2) + O(1) = O(\log n)$
  se $ini > fim$ **então**
    retorna -1

```
    meio := ⌊(ini + fim)/2⌋
    se V[meio] = x então
        retorna meio
    senão se x < V[meio] então
        retorna BuscaBin(V, ini, meio − 1, x)
    senão
        retorna BuscaBin(V, meio + 1, fim, x)
Função Pot(a, n)                                   % a^n com O(log n) multiplicações
    se n = 0 então
        retorna 1
    t := Pot(a, ⌊n/2⌋)
    se n par então
        retorna t · t
    senão
        retorna t · t · a
Função MaxMin(V, ini, fim)      % cerca de 3n/2 comparações, contra 2n − 2 do método ingênuo
    se ini = fim então
        retorna (V[ini], V[ini])
    se fim = ini + 1 então
        retorna (max, min) de V[ini] e V[fim], com 1 comparação
    meio := ⌊(ini + fim)/2⌋
    (M1, m1) := MaxMin(V, ini, meio); (M2, m2) := MaxMin(V, meio + 1, fim)
    retorna (max{M1, M2}, min{m1, m2})
```

Nos três, a divisão produz subproblemas do mesmo tipo, mas a combinação é trivial; o que muda é quantos subproblemas são resolvidos. A busca binária e a potência resolvem *um* subproblema de metade do tamanho, e por isso são logarítmicas; o MaxMin resolve dois.

Exemplo 9.7 (Os três algoritmos em ação)**. Busca binária**: procuramos $x = 34$ no vetor ordenado $V = (3, 8, 12, 17, 21, 30, 34, 41, 50)$ (Figura 9.6). Com $ini = 1$ e $fim = 9$, o meio é a posição 5, com $V[5] = 21 < 34$, e a busca segue em $[6, 9]$; o meio passa a ser 7, com $V[7] = 34$: encontrado, com 2 comparações, contra até 9 de uma busca sequencial.
Potência rápida 2^{10}: Pot(2, 10) chama Pot(2, 5), que chama Pot(2, 2), Pot(2, 1) e Pot(2, 0) = 1. Na volta: $2^1 = 1 \cdot 1 \cdot 2$; $2^2 = 2 \cdot 2$; $2^5 = 4 \cdot 4 \cdot 2 = 32$; $2^{10} = 32 \cdot 32 = 1024$. São 6 multiplicações (duas nos níveis ímpares, uma nos pares), contra 9 do produto $2 \cdot 2 \cdots 2$.
Máximo e mínimo de 6 elementos: o vetor se divide em duas metades de 3, e cada uma em partes de 2 e de 1. Uma parte de 2 custa 1 comparação, e uma de 1 não custa nada; combinar custa 2 (uma para o máximo, outra para o mínimo). Cada metade de 3 custa $1 + 0 + 2 = 3$, e o total é $3 + 3 + 2 = 8$ comparações, contra $2 \cdot 6 - 2 = 10$ do método que procura o máximo e depois o mínimo.

1	2	3	4	5	6	7	8	9	
3	8	12	17	21	30	34	41	50	$V[5] = 21 < 34$: descarta $[1, 5]$
3	8	12	17	21	30	34	41	50	$V[7] = 34$: encontrado

Figura 9.6. Busca binária por 34. Em cada passo, o intervalo $[ini, fim]$ ainda possível está em branco, o meio em destaque e as posições descartadas em cinza.

Elemento majoritário (aparece mais de $n/2$ vezes): $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$

```
Função Maj(V, ini, fim)          % devolve o majoritário de V[ini..fim] ou NENHUM
    se ini = fim então
        retorna V[ini]
    meio := ⌊(ini + fim)/2⌋
```

```
    a := Maj(V, ini, meio); b := Maj(V, meio + 1, fim)
    para c ∈ {a, b}, c ≠ NENHUM faça
        se c ocorre mais de (fim − ini + 1)/2 vezes em V[ini..fim] então
            retorna c
                                                    % contagem em O(n)
    retorna NENHUM
```

A correção do último algoritmo vem de uma observação simples: se x é majoritário no vetor todo, ele é majoritário em pelo menos uma das metades, pois do contrário apareceria no máximo $n/2$ vezes. Assim, os candidatos devolvidos pelas metades são os únicos que precisam ser testados.

Exemplo 9.8 (Elemento majoritário)**.** Em $V = (3, 1, 3, 3, 2, 3, 1, 3)$, o 3 aparece 5 vezes, mais que $8/2$. A Figura 9.7 mostra a recursão. A metade direita, $(2, 3, 1, 3)$, não tem majoritário: o 3 aparece só 2 vezes em 4. Mesmo assim, o 3 é majoritário na metade esquerda, $(3, 1, 3, 3)$, e chega à raiz como candidato; a contagem no vetor todo confirma as 5 ocorrências. Note também que $(3, 1)$ não tem majoritário: aparecer *metade* das vezes não basta.

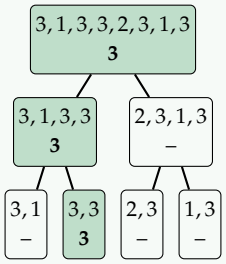


Figura 9.7. A recursão do elemento majoritário em $(3, 1, 3, 3, 2, 3, 1, 3)$. Cada nó mostra o trecho e o majoritário encontrado (ou "–", se não há).

Mais ideias. O algoritmo de *Karatsuba* multiplica inteiros de n dígitos com 3 produtos de inteiros de $n/2$ dígitos, em $\Theta(n^{\log_2 3}) \approx \Theta(n^{1{,}585})$. O *par de pontos mais próximos* no plano se resolve em $\Theta(n \log n)$, dividindo os pontos pela mediana das abscissas e combinando com uma varredura da faixa central, em que cada ponto precisa ser comparado com no máximo 7 vizinhos. O *pico de um vetor unimodal* e a *raiz de uma função monótona* se encontram por busca binária, em $O(\log n)$; e o número de ocorrências de x num vetor ordenado sai de duas buscas binárias.

Erros frequentes em pseudocódigo de divisão e conquista

- Esquecer o caso base, ou escrevê-lo errado (por exemplo, permitir subvetor vazio quando o problema pede um não vazio).
- Dividir em $[ini, meio - 1]$ e $[meio, fim]$ com $meio = \lfloor (ini + fim)/2 \rfloor$: com dois elementos, a recursão não termina. Use $[ini, meio]$ e $[meio + 1, fim]$.
- Fazer uma combinação que olha um só elemento de cada lado; o subvetor cruzado precisa do *melhor sufixo* e do *melhor prefixo*.
- Não devolver a informação extra necessária, como os índices no subvetor máximo ou o vetor ordenado na contagem de inversões.
- Não analisar a complexidade.

Pontos-chave

Divisão e conquista = dividir ao meio, conquistar com duas chamadas e combinar o que cruza o meio. $2T(n/2) + \Theta(n) = \Theta(n \log n)$; $T(n/2) + O(1) = \Theta(\log n)$; $7T(n/2) + \Theta(n^2) = \Theta(n^{2{,}807})$;

$T(n/5) + T(3n/4) + \Theta(n) = \Theta(n)$.

9.6 Exercícios

Exercícios do curso

9.1 [Avaliação] resposta: p. 182
Dado um vetor de inteiros, determine por divisão e conquista a maior soma de um subvetor contíguo **não vazio** e o subvetor responsável, com suas posições inicial e final. Determine a complexidade. Exemplos: $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$ (soma 6, subvetor $[4, -1, 2, 1]$) e $[-8, -3, -6, -2, -5, -4]$ (soma -2). Compare sua solução com a da §9.

9.2 [Avaliação] resposta: p. 182
Escreva um algoritmo recursivo que multiplica duas matrizes $n \times n$, com n potência de 2, pela estratégia de Strassen, com as sete multiplicações $M_1, \ldots, M_7$, supondo dados procedimentos de soma e subtração de matrizes.

9.3 [Slides] resposta: p. 182
Prove: um algoritmo de divisão e conquista c-balanceado ($c > 1$; cada uma das k partes tem tamanho no máximo n/c, e os tamanhos diferem por uma constante), que gasta tempo linear para dividir e combinar, roda em $O(n \log n)$.

9.4 [Slides] resposta: p. 182
Escreva o pseudocódigo completo do algoritmo de contagem de inversões por divisão e conquista.

Exercícios complementares

9.5 [Complementar] resposta: p. 182
Conte as inversões de (a) $2, 4, 1, 3, 5$; (b) $5, 4, 3, 2, 1$; (c) $3, 1, 4, 1, 5, 9, 2, 6$ (pares iguais não são inversão). Qual o máximo possível para n elementos?

9.6 [Complementar] resposta: p. 182
Um vetor $V[1..n]$ é *unimodal*: estritamente crescente até uma posição p e estritamente decrescente depois dela. Encontre p em $O(\log n)$.

9.7 [Complementar] resposta: p. 182
Conte quantas vezes x aparece num vetor ordenado em $O(\log n)$.

9.8 [Complementar] resposta: p. 182
Multiplique dois inteiros de n dígitos com três produtos de inteiros de $n/2$ dígitos (Karatsuba): escreva $x = x_1 B^{n/2} + x_0$ e $y = y_1 B^{n/2} + y_0$ e use $(x_1 + x_0)(y_1 + y_0)$. Qual a recorrência e a sua solução?

9.9 [Complementar] resposta: p. 182
Mostre que o MaxMin faz $\frac{3n}{2} - 2$ comparações quando n é potência de 2.

9.10 [Complementar ★] resposta: p. 182
Dê um algoritmo $O(n)$, com memória $O(1)$, para encontrar o elemento majoritário, se existir.

9.11 [Complementar] resposta: p. 183
Escreva a busca binária de forma iterativa e prove que ela faz no máximo $\lfloor \log_2 n \rfloor + 1$ comparações com elementos do vetor.

9.12 [Complementar] resposta: p. 183
Mostre que Pot(a, n) faz no máximo $2\lfloor \log_2 n \rfloor + 2$ multiplicações. Quantas faz para $n = 13$?

9.13 [Complementar] resposta: p. 183
Mostre como resolver o subvetor de soma máxima por divisão e conquista em $O(n)$, fazendo cada chamada devolver quatro valores: a soma total do intervalo, o melhor prefixo, o melhor sufixo e o melhor subvetor.

9.14 [Complementar] resposta: p. 183
Escreva a multiplicação de matrizes por blocos com 8 multiplicações recursivas e mostre que ela custa $\Theta(n^3)$. Quantas somas de blocos ela faz por chamada?

9.15 [Complementar] resposta: p. 183
Um vetor ordenado contém os inteiros de 1 a $n + 1$, exceto um. Encontre o que falta em $O(\log n)$.

9.16 [Complementar] resposta: p. 183
Dado um vetor ordenado $A[1..n]$ de inteiros *distintos*, decida em $O(\log n)$ se existe i com $A[i] = i$.

9.17 [Complementar] resposta: p. 183
Um vetor ordenado contém apenas zeros seguidos de uns. Conte os uns em $O(\log n)$. E se o tamanho do vetor for desconhecido (pode-se consultar qualquer posição, e posições além do fim devolvem um valor especial)?

9.18 [Complementar ★] resposta: p. 183
Um par $i < j$ é uma *inversão significativa* se $a_i > 2a_j$. Conte as inversões significativas em $O(n \log n)$.

9.19 [Complementar ★] resposta: p. 183
Descreva o algoritmo de divisão e conquista para o par de pontos mais próximos no plano e justifique por que cada ponto da faixa central precisa ser comparado com um número constante de vizinhos.

9.20 [Complementar ★] resposta: p. 183
Prédios retangulares são dados por triplas (e_i, d_i, h_i): o prédio ocupa o intervalo $[e_i, d_i]$ e tem altura h_i. O *contorno* (*skyline*) é a curva formada pelo ponto mais alto em cada abscissa. Calcule o contorno de n prédios em $O(n \log n)$.

9.21 [Complementar] resposta: p. 183
Escreva e resolva as recorrências da busca binária, da potência rápida, do MaxMin e do elemento majoritário por divisão e conquista.

10

Escolhendo estruturas de dados e algoritmos

Neste capítulo

- Um método para escolher a estrutura de dados e o algoritmo adequados a uma situação.
- Um quadro comparativo das estruturas e algoritmos da Parte I.
- Como reconhecer, num trecho de código, oportunidades de melhorar a complexidade.

Conhecer cada estrutura e cada algoritmo isoladamente não basta: na prática, o problema chega descrito em palavras, ou na forma de um programa lento, e é preciso decidir qual ferramenta usar. Este capítulo reúne os critérios espalhados pelos capítulos anteriores e os organiza num método.

10.1 Um método de escolha

Como escolher

1. **Liste as operações** que o problema (ou o código) faz repetidamente: buscar por chave, achar o máximo, inserir, remover, unir grupos, ordenar, selecionar o k-ésimo.
2. **Estime quantas vezes** cada operação acontece. A operação dominante é a que deve ficar barata.
3. **Escolha a estrutura** que torna barata a operação dominante sem tornar as outras proibitivas.
4. **Olhe o tipo das chaves** para escolher a ordenação: inteiros pequenos, inteiros com poucos dígitos, reais uniformes ou chaves arbitrárias.
5. **Verifique as restrições**: memória disponível, necessidade de tempo previsível por operação (pior caso) ou apenas de custo total (amortizado), estabilidade, necessidade de manter ordem.
6. **Justifique** com as complexidades das operações e com uma estimativa do custo total.

O passo 6 pede uma estimativa numérica, e ela é mais fácil do que parece. Um computador comum executa algo em torno de 10^8 operações simples por segundo; com isso, basta substituir n na complexidade para saber se o programa termina em milissegundos, em horas ou em séculos.

n	$\log_2 n$	$n \log_2 n$	n^2	tempo de $n \log_2 n$ / de n^2 (a 10^8 op/s)
10^3	≈ 10	$\approx 10^4$	10^6	instantâneo / 0,01 s
10^6	≈ 20	$\approx 2 \cdot 10^7$	10^{12}	0,2 s / quase 3 horas
10^9	≈ 30	$\approx 3 \cdot 10^{10}$	10^{18}	5 minutos / mais de 300 anos

A tabela mostra por que a complexidade importa mais que qualquer detalhe de implementação: com um milhão de elementos, trocar um algoritmo quadrático por um de $O(n \log n)$ transforma horas em frações de segundo, e nenhum computador mais rápido compensa a diferença.

Exemplo 10.1 (O roteiro aplicado: um painel de servidores)**.** Um painel acompanha 10^4 servidores. A cada segundo, cada servidor informa a sua carga atual, um inteiro de 0 a 100 (por cento); o painel mostra, a cada segundo, os 10 servidores mais carregados, e, uma vez por hora, gera um relatório com todos os servidores em ordem de carga.

1. *Operações*: atualizar a carga de um servidor; consultar os 10 mais carregados; ordenar todos por carga.
2. *Frequências*: 10^4 atualizações por segundo, uma consulta por segundo, uma ordenação por hora. A operação dominante é a atualização.
3. *Estrutura*: um heap de máximo com vetor de posições faz cada atualização em $O(\log n)$, uma alteração de prioridade (Capítulo 2), e os 10 maiores saem com algumas remoções e reinserções; uma AVL indexada por (carga, servidor) também faria tudo em $O(\log n)$.
4. *Tipo das chaves*: as cargas são inteiros de 0 a 100. Isso permite algo melhor: 101 listas duplamente encadeadas, uma por valor de carga, e um vetor que diz em que nó de que lista está cada servidor. Atualizar é tirar o servidor de uma lista e pô-lo em outra, em $O(1)$; os 10 mais carregados são lidos percorrendo as listas da carga 100 para baixo, em $O(101 + 10)$; e o relatório ordenado é a concatenação das listas, um counting sort, em $O(n + 101)$.
5. *Restrições*: memória $O(n + 101)$, nenhuma exigência de estabilidade.
6. *Justificativa*: 10^4 atualizações de custo $O(1)$ por segundo, contra cerca de $10^4 \cdot \log_2 10^4 \approx 1{,}3 \cdot 10^5$ passos com o heap; as duas cabem folgadamente num segundo, mas a solução com listas é mais simples de manter e não depende de n.

O ponto do exemplo é o passo 4: o tipo das chaves (inteiros num intervalo pequeno) abriu uma solução que nenhuma estrutura "genérica" daria.

10.2 Quadro comparativo

O problema faz repetidamente. . .	Estrutura ou algoritmo
pega o maior (ou o menor), insere e às vezes altera prioridades	**heap binário** (Capítulo 2)
. . . e une filas com frequência, exigindo tempo previsível	**heap binomial** (Capítulo 3)
. . . com um número enorme de aumentos de prioridade, importando o custo total	**heap de Fibonacci** (Capítulo 3)
"x e y estão no mesmo grupo?" e une grupos	**Union-Find** com rank e compressão (Capítulo 4)
. . . e precisa listar os membros de um grupo	**Union-Find por listas** (Capítulo 4)
busca por chave, inserção, remoção, percurso em ordem, sucessor, mínimo e máximo, intervalos	**árvore AVL** ou rubro-negra (Capítulo 5)
só busca, inserção e remoção por chave exata, sem ordem	**tabela hash**, $O(1)$ esperado (Apêndice A)
muitas buscas e quase nenhuma atualização	**vetor ordenado** e busca binária
o k-ésimo menor ou a mediana, uma vez	**seleção linear** (Capítulo 8), sem ordenar
os k maiores de um fluxo de dados	**heap de mínimo de tamanho** k, $O(n \log k)$
ordenar inteiros num intervalo pequeno ou com poucos dígitos	**counting** ou **radix sort** (Capítulo 7)
ordenar chaves arbitrárias com garantia e estabilidade	**mergesort** (Capítulo 6)

Para a ordenação, as perguntas podem ser feitas numa ordem fixa, como no fluxograma da Figura 10.1: as primeiras perguntas procuram uma estrutura especial nos dados que permita fugir do limite $\Omega(n \log n)$; só quando nenhuma aparece é que se escolhe entre os algoritmos por comparação.

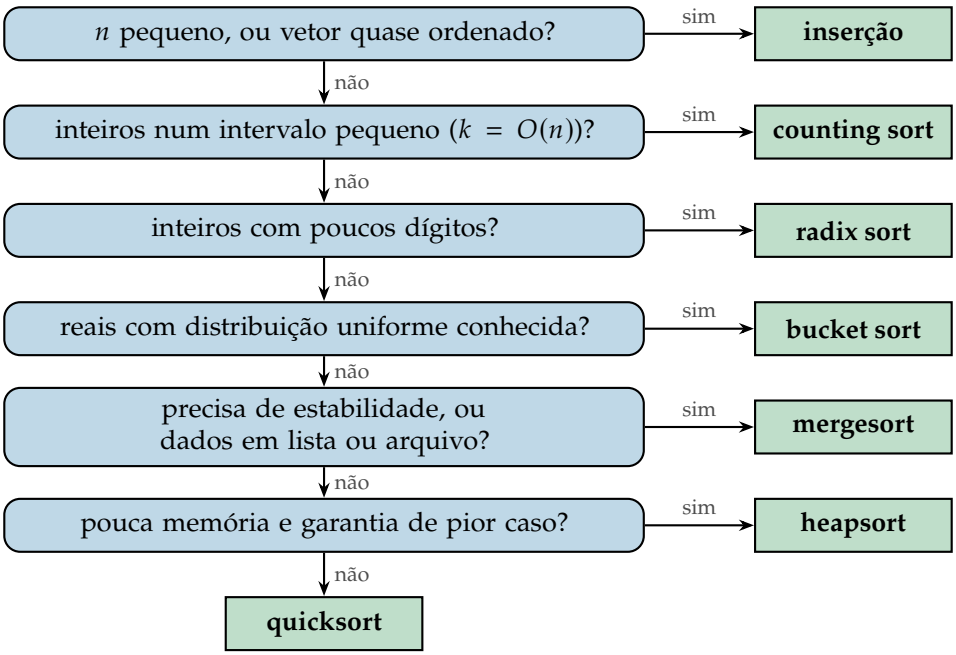


Figura 10.1. Um fluxograma para escolher o algoritmo de ordenação. Cada pergunta é feita só se as anteriores tiveram resposta "não".

Padrões a reconhecer em código. Alguns padrões indicam, quase sempre, uma estrutura inadequada:

- um laço que procura o máximo de uma lista e o remove, a cada iteração ($\Theta(n)$ por iteração): use um **heap**;
- ordenar a lista inteira depois de cada inserção: use um **heap** ou uma **AVL**, conforme seja preciso só o extremo ou a ordem toda;
- percorrer todos os elementos para saber se dois estão no mesmo grupo: use **Union-Find**;
- ordenar só para depois pegar o k-ésimo elemento: use **seleção** em $O(n)$;
- ordenar inteiros pequenos com um `sort` genérico: use **counting** ou **radix sort**;
- procurar um elemento por varredura a cada consulta: use **hash** (sem ordem) ou **AVL** (com ordem).

Exemplo 10.2 (Do código lento ao código rápido: o par mais próximo numa reta)**.** Um programa recebe as posições $x_1, \ldots, x_n$ de n postes ao longo de uma estrada e precisa da menor distância entre dois postes. O código atual compara todos os pares:

Versão quadrática

```
melhor := ∞
para i := 1 até n faça
    para j := i + 1 até n faça
        melhor := min{melhor, |x_i − x_j|}
```

São $\binom{n}{2}$ comparações: com $n = 10^6$, cerca de $5 \cdot 10^{11}$ operações, mais de uma hora. A observação que salva é que, *depois de ordenar*, o par mais próximo é necessariamente formado por dois vizinhos: se $x_{(1)} \le x_{(2)} \le \cdots \le x_{(n)}$ e $i < j - 1$, então $x_{(j)} - x_{(i)} \ge x_{(i+1)} - x_{(i)}$. Basta ordenar, em $O(n \log n)$, e comparar os $n - 1$ pares vizinhos, em $O(n)$: cerca de $2 \cdot 10^7$ operações, uma fração de segundo. Por exemplo, para as posições 17, 3, 42, 25, 9, 40, a ordem é $3, 9, 17, 25, 40, 42$, as diferenças entre vizinhos são $6, 8, 8, 15, 2$, e a resposta é 2 (os postes em 40 e 42). Se as posições forem inteiros num intervalo pequeno, a ordenação pode até ser linear, com counting ou radix sort.

10.3 Resumo das complexidades

Listas de prioridades (n elementos; * indica custo amortizado)

Operação	Lista não ordenada	Heap binário	Heap binomial	Heap de Fibonacci
Seleção do máximo	$\Theta(n)$	$O(1)$	$O(\log n)$ ou $O(1)$	$O(1)$
Inserção	$\Theta(1)$	$O(\log n)$	$O(\log n)$; $O(1)^*$	$O(1)^*$
Remoção do máximo	$\Theta(n)$	$O(\log n)$	$O(\log n)$	$O(\log n)^*$
Aumentar prioridade	$\Theta(n)$	$O(\log n)$	$O(\log n)$	$O(1)^*$
Fusão	$O(1)$	$O(n)$	$O(\log n)$	$O(1)^*$

Ordenação

Algoritmo	Pior caso	Caso médio	Memória extra	Estável?
Inserção	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	sim
Mergesort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	sim
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	não
Quicksort	$\Theta(n^2)$	$\Theta(n \log n)$	$O(\log n)$	não
Counting sort	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	sim
Radix sort	$\Theta(d(n + b))$	$\Theta(d(n + b))$	$\Theta(n + b)$	sim
Bucket sort	$\Theta(n^2)$	$\Theta(n)$ (uniforme)	$\Theta(n)$	depende

Outras estruturas. Union-Find com rank e compressão: $O(\log^* n)$ amortizado por operação (ou $O(\alpha)$). AVL: busca, inserção e remoção em $O(\log n)$ no pior caso. Seleção: $O(n)$ no pior caso pela mediana das medianas. Divisão e conquista: contagem de inversões e subvetor máximo em $O(n \log n)$; Strassen em $O(n^{2,807})$.

Armadilhas na escolha

- Escolher pela operação mais *lembrada*, e não pela mais *frequente*: uma estrutura ótima para a consulta rara e péssima para a atualização constante é uma má escolha.
- Confundir custo amortizado com custo de pior caso: o heap de Fibonacci e o vetor que dobra de tamanho são baratos *no total*, mas uma operação isolada pode ser cara; se cada operação precisa de tempo previsível, isso importa.
- Esquecer as operações "escondidas": "inserir, se ainda não existir" exige uma *busca por chave* antes de cada inserção, e um heap não busca em menos de $\Theta(n)$.
- Ignorar a memória: counting sort com k enorme, ou uma matriz de adjacência para um grafo esparso, podem ser inviáveis mesmo quando o tempo é bom.

10.4 Exemplos

Exemplo 10.3 (Uma agenda de compromissos)**.** Um aplicativo guarda compromissos com data e hora, insere e remove compromissos com frequência, mostra o próximo compromisso e lista todos os compromissos de uma semana. As operações dominantes são inserção, remoção de um compromisso específico, "próximo compromisso" e consulta por intervalo. Um heap daria o próximo compromisso, mas não remove um elemento qualquer pela chave nem lista um intervalo; uma tabela hash não mantém ordem. Uma **AVL** indexada por data e hora faz tudo: inserção e remoção em $O(\log n)$, mínimo em $O(\log n)$ e a semana em $O(\log n + k)$, com k compromissos listados.

Exemplo 10.4 (Ordenando frações com pouca memória)**.** Um sistema embarcado precisa ordenar 10^5 frações p/q, com numeradores e denominadores inteiros de até 18 dígitos, guardadas de forma exata, sem arredondar para ponto flutuante. Comparar duas frações é fácil — com denominadores positivos, $p/q < r/s$ se e somente se $ps < rq$ —, mas uma fração não serve diretamente de índice: counting, radix e bucket sort ficam de fora, e só resta comparar. Não há memória para um vetor auxiliar do tamanho da entrada, o que descarta o mergesort, e o tempo precisa ser garantido, o que descarta o quicksort. O **heapsort** atende a tudo: $\Theta(n \log n)$ comparações no pior caso, cerca de $1{,}7 \cdot 10^6$, sem memória extra. A falta de estabilidade não atrapalha, porque frações iguais podem sair em qualquer ordem.

Exemplo 10.5 (Juntando dois catálogos)**.** Duas lojas que se fundiram têm catálogos com 10^6 códigos de produto cada um, ambos já em ordem crescente, e querem um catálogo único, ordenado e sem códigos repetidos. Juntar as listas e ordenar tudo de novo custaria $\Theta(n \log n)$ e jogaria fora uma informação valiosa: as duas listas já estão ordenadas. A **intercalação** do mergesort (Capítulo 6) resolve em $\Theta(n)$: percorre as duas listas com um índice em cada, copia sempre o menor código e, quando os dois códigos apontados são iguais, copia um só e avança os dois índices. Por exemplo, $(3, 8, 15, 21)$ e $(5, 8, 9, 21, 30)$ dão $(3, 5, 8, 9, 15, 21, 30)$, numa única passada pelas duas listas.

Resumo da Parte I

Identifique a operação dominante e o tipo das chaves. Heap para extremos; binomial para fusões previsíveis; Fibonacci para muitos aumentos de prioridade. Union-Find para grupos. AVL para ordem e busca; hash para busca sem ordem. Seleção sem ordenar. Counting e radix para inteiros; mergesort para chaves arbitrárias com estabilidade; heapsort com pouca memória; quicksort para desempenho médio. Justifique sempre com complexidade, memória e o tipo de garantia (pior caso ou amortizada).

10.5 Exercícios

Para cada situação, diga qual estrutura de dados ou algoritmo você usaria e justifique com as complexidades envolvidas.

Exercícios complementares

10.1 [Complementar] resposta: p. 183
Enquanto houver tarefas na lista L **faça**: percorra L para achar a de maior prioridade, execute-a e remova-a de L; novas tarefas podem ser acrescentadas a L durante a execução.

10.2 [Complementar] resposta: p. 183
Para cada uma de q consultas: copie o vetor V (que não muda) de tamanho n, ordene a cópia com um `sort` genérico e devolva o elemento da posição k_i.

10.3 [Complementar] resposta: p. 184
Uma rede social recebe 10^6 pedidos de amizade, que unem comunidades, e 10^8 perguntas "u e v estão na mesma comunidade?". O código atual responde cada pergunta com uma busca em largura no grafo de amizades.

10.4 [Complementar] resposta: p. 184
Um sistema guarda produtos por código, com inserções e remoções frequentes, e precisa responder "quais códigos estão entre a e b?". Hoje os códigos ficam num vetor ordenado e cada inserção desloca os elementos.

10.5 [Complementar] resposta: p. 184
Ordenar as idades (inteiros entre 0 e 120) de 10^8 pessoas com o `sort` da biblioteca.

10.6 [Complementar] resposta: p. 184
Um sistema de atendimento tem uma fila de prioridade por guichê; quando um guichê fecha, sua fila é unida à de outro guichê. Fechamentos são frequentes e o tempo de cada operação precisa ser previsível. Hoje cada fila é um heap binário.

10.7 [Complementar] resposta: p. 184
Um pronto-socorro mantém os pacientes em espera ordenados por gravidade. Chegam e são atendidos milhares de pacientes por dia; a gravidade de um paciente raramente muda; o sistema roda num dispositivo com pouca memória.

10.8 [Complementar] resposta: p. 184
Um simulador mantém eventos futuros ordenados por horário; a todo momento processa o próximo evento, cria novos eventos e às vezes cancela um evento já agendado, identificado por um número.

10.9 [Complementar] resposta: p. 184
Um corretor ortográfico precisa verificar, para cada palavra de um texto, se ela está num dicionário fixo de 10^6 palavras. Não há atualizações nem consultas por ordem alfabética.

10.10 [Complementar ★] resposta: p. 184
Um placar online recebe pontuações e precisa responder "qual a posição de x no ranking?" e "quem está na posição r?", com inserções e remoções frequentes.

10.11 [Complementar] resposta: p. 184
Um sensor envia medições continuamente, e após cada medição é preciso exibir a mediana de todas as medições recebidas até então.

10.12 [Complementar] resposta: p. 184
Ordenar um arquivo de 500 GB de registros num computador com 16 GB de memória.

10.13 [Complementar] resposta: p. 184
Um vetor de 10^6 elementos está ordenado, exceto por uns poucos elementos fora do lugar, cada um a no máximo 10 posições da posição correta.

10.14 [Complementar] resposta: p. 184
Decidir se há strings repetidas numa lista de 10^7 strings.

10.15 [Complementar] resposta: p. 184
Uma malha de estradas recebe uma lista de interdições, uma por vez, e após cada interdição é preciso dizer quantas regiões desconectadas existem. Toda a lista de interdições é conhecida de antemão.

10.16 [Complementar] resposta: p. 184
Um servidor processa um fluxo de 10^9 requisições e, ao final, deve informar as 100 mais demoradas, usando pouca memória.

10.17 [Complementar] resposta: p. 184
Ordenar 10^6 números reais em $[0, 1)$ cuja distribuição é desconhecida e possivelmente muito concentrada perto de 0.

10.18 [Complementar] resposta: p. 184
Um programa responde a q perguntas "qual a soma de $V[i..j]$?" sobre um vetor fixo, somando os elementos de i a j a cada pergunta.

10.19 [Complementar] resposta: p. 184
A cada consulta, um programa percorre uma lista de n registros para descobrir se existe um registro com uma dada chave; há muitas consultas, inserções e remoções, e também é preciso, de vez em quando, listar os registros em ordem de chave.

10.20 [Complementar] resposta: p. 185
Um editor de redes permite ligar computadores, consultar se dois computadores se comunicam e desfazer as últimas ligações feitas.

Parte II

Paradigmas de projeto e algoritmos em grafos

11

Programação dinâmica

Neste capítulo

- A ideia da programação dinâmica: subestrutura ótima, sobreposição de subproblemas, memorização e cálculo de baixo para cima.
- Um roteiro para projetar algoritmos de programação dinâmica.
- Três problemas clássicos: subsequência comum máxima, ordem de multiplicação de matrizes e mochila 0-1.
- Por que o algoritmo da mochila é pseudopolinomial.

Na divisão e conquista, os subproblemas são disjuntos. Em muitos problemas, porém, a decomposição natural gera subproblemas que *se repetem*, e a recursão ingênua refaz o mesmo trabalho um número exponencial de vezes. A programação dinâmica resolve cada subproblema distinto uma única vez e guarda a resposta numa tabela. Este capítulo apresenta a técnica e três aplicações; uma quarta, o algoritmo de Floyd-Warshall, é o assunto do próximo capítulo.

11.1 A técnica

No método geral de decomposição, subproblemas idênticos podem aparecer muitas vezes ao longo do processo. Se cada aparição for resolvida de novo, o algoritmo pode ser exponencial mesmo que o número de subproblemas *distintos* seja polinomial. A **programação dinâmica** mantém uma tabela com uma entrada para cada subproblema distinto: na primeira vez em que um subproblema é considerado, ele é resolvido normalmente; nas vezes seguintes, sua solução é simplesmente lida da tabela.

Exemplo 11.1 (Fibonacci: o mesmo cálculo, muitas vezes)**.** Os números de Fibonacci são $F_0 = 0$, $F_1 = 1$ e $F_n = F_{n-1} + F_{n-2}$. A recursão direta para F_5 chama F_4 e F_3; F_4 chama F_3 de novo; e assim por diante (Figura 11.1). São 15 chamadas para calcular F_5, embora só existam 6 subproblemas distintos, $F_0, \ldots, F_5$: F_3 é calculado 2 vezes, F_2 3 vezes e F_1 5 vezes. O número de chamadas $c(n)$ satisfaz $c(n) = c(n-1) + c(n-2) + 1$ e cresce como os próprios números de Fibonacci, isto é, exponencialmente: $c(30) = 2\,692\,537$ e $c(40) = 331\,160\,281$. Guardando cada F_k na primeira vez em que é calculado, cada um dos $n+1$ subproblemas custa $O(1)$, e o total cai para $O(n)$.

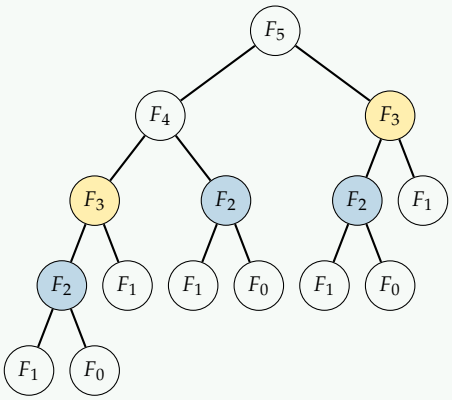


Figura 11.1. A árvore de chamadas da recursão ingênua para F_5. Subárvores repetidas têm a mesma cor: a de F_3 (amarelo) aparece duas vezes, e a de F_2 (azul), três.

- **Divisão e conquista ou programação dinâmica?** A divisão e conquista funciona bem quando os subproblemas são **disjuntos**, como no mergesort. Quando eles **compartilham** subproblemas, é melhor trocar a abordagem de cima para baixo por uma de baixo para cima: resolver dos subproblemas menores para os maiores, de modo que, ao chegar a um subproblema, todos aqueles de que ele depende já estejam resolvidos.
- **Subestrutura ótima**: uma solução ótima é composta de soluções ótimas de subproblemas. A prova típica é o argumento de "recortar e colar": se uma parte não fosse ótima, trocá-la por uma melhor daria uma solução global melhor, o que é uma contradição.
- **Sobreposição de subproblemas**: a recursão ingênua revisita os mesmos subproblemas; é isso que faz a tabela valer a pena.
- **Memorização** (de cima para baixo): a recursão original, acrescida de "se já calculei este subproblema, devolvo o valor guardado". **Cálculo de baixo para cima**: laços que preenchem a tabela na ordem certa. As duas têm a mesma complexidade; a segunda evita a pilha de recursão e, às vezes, permite economizar memória.

Exemplo 11.2 (Recortar e colar)**.** Se o caminho mais curto de Belo Horizonte a Curitiba passa por São Paulo, então o seu trecho de Belo Horizonte a São Paulo é um caminho mais curto entre essas duas cidades. De fato, se houvesse um caminho mais curto de Belo Horizonte a São Paulo, poderíamos "recortá-lo" e "colá-lo" no lugar do trecho original, obtendo um caminho até Curitiba mais curto que o ótimo, uma contradição. É essa propriedade que permite escrever a distância até Curitiba em termos de distâncias até cidades intermediárias, e ela é a base do algoritmo de Floyd-Warshall (Capítulo 12) e do de Dijkstra (Capítulo 16).

Fibonacci: recursão ingênua $\Theta(\varphi^n)$; memorização $O(n)$; de baixo para cima, $O(n)$ com memória $O(1)$

```
Função FibMemo(n)                                   % F[ ] começa preenchido com −1
    se F[n] = −1 então
        se n ≤ 1 então
            F[n] := n
        senão
            F[n] := FibMemo(n − 1) + FibMemo(n − 2)
    retorna F[n]
Função FibIterativo(n)
    a := 0; b := 1                                  % a = F_i e b = F_{i+1}
    para i := 1 até n faça
        (a, b) := (b, a + b)
```

retorna a

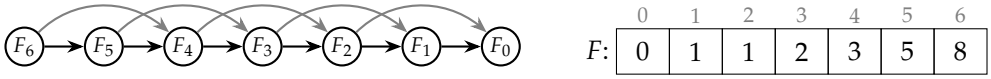


Figura 11.2. À esquerda, o grafo de dependências dos subproblemas de Fibonacci: cada F_k depende de F_{k-1} e de F_{k-2}. Há só $n+1$ nós, e qualquer ordem que respeite as setas (por exemplo, $F_0, F_1, \ldots, F_6$) serve para preencher a tabela, à direita.

Como projetar um algoritmo de programação dinâmica

1. **Defina o subproblema em palavras**, por exemplo "$C(i,j)$ = tamanho da subsequência comum máxima de X_i e Y_j". Sem essa frase, a recorrência não tem sentido.
2. **Decomponha**: o que uma solução ótima faz com o *último* elemento, ou qual é a *primeira* decisão? Cada possibilidade vira um termo da recorrência.
3. **Escreva a recorrência e os casos base**.
4. **Calcule a complexidade** como o número de subproblemas vezes o custo de cada um.
5. **Escolha a ordem de preenchimento**: cada célula só pode depender de células já calculadas (linha a linha, por diagonais, por tamanho crescente...).
6. **Indique onde está a resposta**, como $C[n,m]$ ou $m[1,n]$.
7. **Reconstrua a solução**: guarde a escolha feita em cada célula, ou recalcule-a percorrendo a tabela de volta.

11.2 Subsequência comum máxima

Uma sequência Z é **subsequência** de $X = x_1 \ldots x_n$ se os seus símbolos aparecem em X na mesma ordem, **não necessariamente contíguos**. Por exemplo, $6, 10, 12$ é subsequência de $3, 6, 9, 10, 12, 14, 6$, mas não de $3, 9, 10, 12, 14, 6$. Dadas X, com $|X| = n$, e Y, com $|Y| = m$, o problema da **subsequência comum máxima** (LCS, do inglês *longest common subsequence*) pede uma subsequência comum a X e a Y de tamanho máximo. A força bruta testaria as 2^n subsequências de X. O problema aparece na comparação de arquivos (o comando `diff` mostra o que não está numa subsequência comum máxima das linhas) e na comparação de sequências de DNA.

Decomposição. Seja $X_i = x_1 \ldots x_i$ e olhe para os últimos símbolos. Se $x_n = y_m$, esse símbolo termina alguma subsequência comum máxima, e o restante dela é uma subsequência comum máxima de X_{n-1} e Y_{m-1}. Se $x_n \neq y_m$, a resposta não usa x_n ou não usa y_m. Com $C(i,j)$ = tamanho da subsequência comum máxima de X_i e Y_j:

$$C(i,j) = \begin{cases} 0 & i = 0 \text{ ou } j = 0 \\ 1 + C(i-1, j-1) & x_i = y_j \\ \max\{C(i-1,j),\ C(i,j-1)\} & x_i \neq y_j \end{cases} \qquad \text{resposta: } C(n,m).$$

A recursão direta é exponencial, mas há apenas $(n+1)(m+1)$ subproblemas distintos.

Intuição

Compare duas palavras de trás para frente. Se as últimas letras são iguais, case-as: nada se perde, porque qualquer subsequência comum que não as use pode trocar a sua última letra por elas. Se são diferentes, pelo menos uma das duas sobra; como não sabemos qual, experimentamos descartar cada uma e ficamos com o melhor. A tabela evita repetir as

mesmas comparações de prefixos.

LCS de baixo para cima, com reconstrução: $\Theta(nm)$ de tempo e memória

```
Função LCS(X, Y)
    para j := 0 até m faça
        C[0, j] := 0
    para i := 1 até n faça
        C[i, 0] := 0
    para i := 1 até n faça
        para j := 1 até m faça
            se x_i = y_j então
                C[i, j] := 1 + C[i − 1, j − 1]
            senão
                C[i, j] := max{C[i − 1, j], C[i, j − 1]}
    i := n; j := m; Z := sequência vazia          % reconstrução em O(n + m)
    enquanto i > 0 e j > 0 faça
        se x_i = y_j então
            coloque x_i no início de Z; i := i − 1; j := j − 1
        senão se C[i − 1, j] ≥ C[i, j − 1] então
            i := i − 1
        senão
            j := j − 1
    retorna Z                                     % |Z| = C[n, m]
```

Exemplo 11.3 (X = CATGACT e Y = ACTGTA)**.** Pense em dois trechos de DNA. A tabela é preenchida linha a linha, da esquerda para a direita: cada célula depende da vizinha de cima, da da esquerda e da diagonal superior esquerda, que já foram calculadas (Figura 11.3). Por exemplo, na linha $i = 3$ (x_3 = T) e coluna $j = 3$ (y_3 = T), os símbolos casam, e $C[3,3] = 1 + C[2,2] = 2$; na mesma linha e coluna $j = 1$ (y_1 = A), não casam, e $C[3,1] = \max\{C[2,1], C[3,0]\} = \max\{1,0\} = 1$.
$C[7,6] = 4$. A reconstrução, subindo nos empates, passa pelas células destacadas: de $(7,6)$ sobe para $(6,6)$ e para $(5,6)$, onde casa **A**; vai para $(4,5)$ e anda para a esquerda até $(4,4)$, onde casa **G**; vai para $(3,3)$, onde casa **T**; vai para $(2,2)$ e sobe para $(1,2)$, onde casa **C**. Como cada símbolo é colocado no início, Z = CTGA. As subsequências comuns de tamanho 4 são exatamente ATGA, ATGT, CTGA e CTGT.

				C	T	G	T	A
		0	0	0	0	0	0	0
i=1	**C**	0	0	1	1	1	1	1
i=2	**A**	0	1	1	1	1	1	2
i=3	**T**	0	1	1	2	2	2	2
i=4	**G**	0	1	1	2	3	3	3
i=5	**A**	0	1	1	2	3	3	4
i=6	**C**	0	1	2	2	3	3	4
i=7	**T**	0	1	2	3	3	4	4

Figura 11.3. A tabela C da subsequência comum máxima de CATGACT e ACTGTA. O caminho da reconstrução está em amarelo; nas células verdes, os símbolos casam e entram na resposta, CTGA.

Para $X = 5, 2, 1, 3, 9, 10, 14$ e $Y = 2, 3, 1, 10$, o tamanho é 3, com as soluções $2, 3, 10$ e $2, 1, 10$.

- Se só o **tamanho** interessa, basta memória $O(\min\{n, m\})$, pois cada linha só usa a anterior. Mas então não é possível reconstruir a subsequência percorrendo a tabela.
- Para obter **todas** as subsequências comuns máximas, siga, na reconstrução, os dois caminhos quando $x_i \neq y_j$ e $C[i-1, j] = C[i, j-1]$, eliminando repetições; pode haver exponencialmente muitas.

11.3 Multiplicação de cadeia de matrizes

Queremos calcular $M = M_1 \times M_2 \times \cdots \times M_n$, onde M_i tem dimensões $r_{i-1} \times r_i$. Multiplicar uma matriz $p \times q$ por uma $q \times s$ custa $p\,q\,s$ multiplicações escalares. O produto de matrizes é **associativo**: a ordem em que as multiplicações são feitas não muda o resultado, mas pode mudar muito o custo. O problema é encontrar a parentização de custo mínimo.

Exemplo 11.4. Sejam $M_1 : 20 \times 50$, $M_2 : 50 \times 10$, $M_3 : 10 \times 1$ e $M_4 : 1 \times 100$, isto é, $r = (20, 50, 10, 1, 100)$. A ordem $M_1(M_2(M_3M_4))$ custa $1000 + 50\,000 + 100\,000 =$ **151 000** multiplicações; a ordem $((M_1M_2)M_3)M_4$ custa $10\,000 + 200 + 2000 =$ **12 200**. Nenhuma das duas é a melhor: a programação dinâmica abaixo encontra $(M_1(M_2M_3))M_4$, que custa $500 + 1000 + 2000 =$ **3500**. As cinco parentizações possíveis custam:

$((M_1M_2)M_3)M_4$	12 200	$M_1((M_2M_3)M_4)$	105 500
$(M_1(M_2M_3))M_4$	**3500**	$M_1(M_2(M_3M_4))$	151 000
$(M_1M_2)(M_3M_4)$	31 000		

A Figura 11.4 desenha a melhor e a pior como árvores: cada nó interno é uma multiplicação, e o seu custo é o produto das três dimensões envolvidas. A melhor ordem faz cedo o produto com a matriz 10×1, que "estreita" tudo; a pior multiplica por último matrizes de 100 colunas.

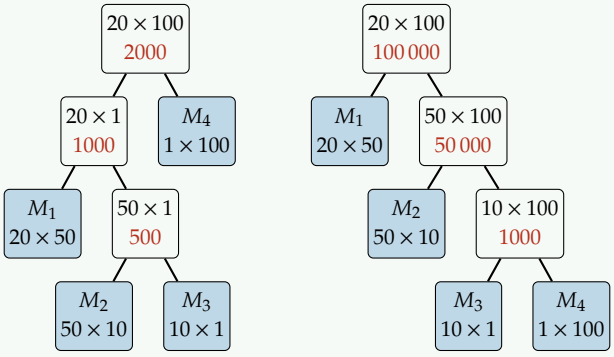


Figura 11.4. A melhor (à esquerda) e a pior (à direita) parentização de $M_1M_2M_3M_4$, com $r = (20, 50, 10, 1, 100)$. Cada nó interno mostra as dimensões do resultado e, em vermelho, o custo daquela multiplicação; o custo total é a soma dos números vermelhos.

A força bruta é exponencial: o número de parentizações de n matrizes é o número de Catalan C_{n-1} (que vale $1, 1, 2, 5, 14, 42, 132, \ldots$), e cresce como $\Omega(4^n/n^{3/2})$.

Decomposição. A *última* multiplicação de $M_i \cdots M_j$ é $(M_i \cdots M_k) \times (M_{k+1} \cdots M_j)$, para algum $i \le k < j$. Numa solução ótima, as duas partes também estão parentizadas de forma ótima, pelo argumento de recortar e colar. Com $m[i, j]$ = número mínimo de multiplicações escalares para calcular $M_i \cdots M_j$:

$$m[i,j] = \begin{cases} 0 & i = j \\ \min\limits_{i \le k < j} \{\, m[i,k] + m[k+1,j] + r_{i-1}\, r_k\, r_j \,\} & i < j \end{cases} \qquad \text{resposta: } m[1,n].$$

O termo $r_{i-1}r_k r_j$ é o custo da última multiplicação: o produto $M_i \cdots M_k$ tem dimensões $r_{i-1} \times r_k$, e o produto $M_{k+1} \cdots M_j$, dimensões $r_k \times r_j$. Há $O(n^2)$ subproblemas e cada um testa $O(n)$ valores de k: o algoritmo custa $\Theta(n^3)$ **de tempo e** $\Theta(n^2)$ **de memória**. Como $m[i,j]$ depende de intervalos *mais curtos*, a tabela é preenchida **por diagonais**, com $d = j - i = 1, 2, \ldots, n-1$; a diagonal principal é toda zero (Figura 11.5).

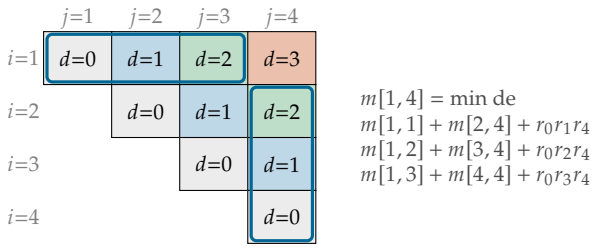


Figura 11.5. A ordem de preenchimento da tabela m para $n = 4$, por diagonais $d = j - i$. A célula $m[1,4]$ (em vermelho) usa a linha 1 à sua esquerda e a coluna 4 abaixo dela, que pertencem a diagonais anteriores.

Ordem ótima de multiplicação, com a tabela s das escolhas

```
Função OrdemÓtima(r_0, ..., r_n)
    para i := 1 até n faça
        m[i, i] := 0
    para d := 1 até n − 1 faça                          % para cada diagonal
        para i := 1 até n − d faça                      % para cada linha da diagonal
            j := i + d; m[i, j] := ∞
            para k := i até j − 1 faça
                q := m[i, k] + m[k + 1, j] + r_{i−1} r_k r_j
                se q < m[i, j] então
                    m[i, j] := q; s[i, j] := k
    retorna m[1, n] e s
Procedimento Imprime(s, i, j)
    se i = j então
        imprima "M_i"
    senão
        imprima "("; Imprime(s, i, s[i, j]); Imprime(s, s[i, j] + 1, j); imprima ")"
Função Multiplica(A, s, i, j)                           % calcula A_i ··· A_j na ordem ótima
    se i = j então
        retorna A_i
    retorna Multiplica(A, s, i, s[i, j]) × Multiplica(A, s, s[i, j] + 1, j)
```

$m[i,j]$ ($s[i,j]$)	$j=1$	2	3	4
$i=1$	0	10 000 (1)	1500 (1)	**3500 (3)**
2		0	500 (2)	5500 (3)
3			0	1000 (3)
4				0

Na tabela do Exemplo 11.4, $s[1,4] = 3$ indica que a última multiplicação é $(M_1M_2M_3) \cdot M_4$, e $s[1,3] = 1$ indica que a primeira parte é $M_1(M_2M_3)$. Logo, a ordem ótima é $((M_1(M_2M_3))M_4)$, com custo 3500.

Exemplo 11.5 (Calculando uma célula). Vejamos como sai $m[1,3]$, da diagonal $d = 2$, a partir das células da diagonal $d = 1$, já calculadas: $m[1,2] = 20 \cdot 50 \cdot 10 = 10\,000$ e

$m[2,3] = 50 \cdot 10 \cdot 1 = 500$. Há duas escolhas para a última multiplicação:

$$\begin{aligned} k = 1: &\quad m[1,1] + m[2,3] + r_0 r_1 r_3 = 0 + 500 + 20 \cdot 50 \cdot 1 = 1500; \\ k = 2: &\quad m[1,2] + m[3,3] + r_0 r_2 r_3 = 10\,000 + 0 + 20 \cdot 10 \cdot 1 = 10\,200. \end{aligned}$$

O mínimo é 1500, com $s[1,3] = 1$: multiplicar primeiro $M_2 M_3$ (que dá uma matriz 50×1) e depois M_1 por ela.

Outros exemplos:

Dimensões r	Custo ótimo e ordem
$(5, 20, 2, 30)$	**500**: $((M_1 M_2) M_3)$, com $m[1,2] = 200$ e $m[2,3] = 1200$
$(8, 2, 9, 3, 7, 4, 10)$	**392**: $(M_1((((M_2 M_3) M_4) M_5) M_6))$
$(30, 35, 15, 5, 10, 20, 25)$	15 125: $((M_1 (M_2 M_3))((M_4 M_5) M_6))$

11.4 Mochila 0-1

Há n itens; o item i tem peso p_i e valor v_i, inteiros; a mochila tem capacidade W. Queremos $S \subseteq \{1, \ldots, n\}$ com $\sum_{i \in S} p_i \le W$ que maximize $\sum_{i \in S} v_i$, usando cada item no máximo uma vez. Seja $M(w, i)$ o maior valor obtido com os itens $1, \ldots, i$ e capacidade w. O item i fica de fora, ou entra, se couber:

$$M(w,i) = \begin{cases} 0 & i = 0 \\ M(w, i-1) & p_i > w \\ \max\{M(w, i-1),\ v_i + M(w - p_i,\ i-1)\} & p_i \le w \end{cases} \qquad \text{resposta: } M(W, n).$$

A decisão sobre o item i é a "última decisão" do roteiro: se ele fica de fora, o que resta é o mesmo problema com os itens $1, \ldots, i-1$ e a mesma capacidade; se ele entra, é o mesmo problema com os itens $1, \ldots, i-1$ e a capacidade que sobra, $w - p_i$.

Mochila 0-1: $\Theta(nW)$ (pseudopolinomial)

```
Função Mochila(p, v, n, W)
    para w := 0 até W faça
        M[w, 0] := 0
    para i := 1 até n faça
        para w := 0 até W faça
            M[w, i] := M[w, i − 1]
            se p_i ≤ w e v_i + M[w − p_i, i − 1] > M[w, i] então
                M[w, i] := v_i + M[w − p_i, i − 1]
    w := W                                              % reconstrução
    para i := n decrescendo até 1 faça
        se M[w, i] ≠ M[w, i − 1] então
            o item i está na solução; w := w − p_i
    retorna M[W, n]
```

Exemplo 11.6 ($W = 5$ e itens $(p, v) = (2,3), (3,5), (1,1)$)**.** A tabela tem uma coluna por item e uma linha por capacidade (Figura 11.6). A solução ótima leva os itens 1 e 2, com peso 5 e valor **8**. Repare que $M(5,2) = \max\{M(5,1),\ 5 + M(2,1)\} = \max\{3,\ 8\} = 8$: é fácil errar essa célula esquecendo que, depois de colocar o item 2, ainda sobra capacidade 2 para o item 1. Na reconstrução, $M(5,3) = M(5,2)$, e o item 3 fica de fora; $M(5,2) \neq M(5,1)$, e o item 2 entra, sobrando $w = 5 - 3 = 2$; $M(2,1) \neq M(2,0)$, e o item 1 entra.

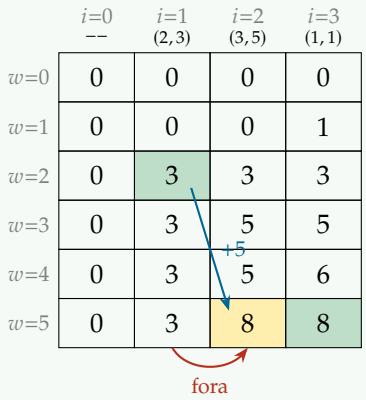


Figura 11.6. A tabela da mochila do Exemplo 11.6. A célula $M(5,2)$ escolhe o maior entre "deixar o item 2 de fora" (seta da esquerda, valor 3) e "levar o item 2" (seta diagonal, $5 + M(2,1) = 8$). Em verde, o caminho da reconstrução.

- Com $W = 6$ e itens $(p, v) = (1,6), (3,15), (4,18)$, o ótimo é **24**, com os itens 1 e 3. O guloso que escolhe pela razão v/p (que vale 6; 5; 4,5) pega os itens 1 e 2, e o item 3 já não cabe: obtém só 21. Assim, o algoritmo guloso **não** resolve a mochila 0-1, embora resolva a versão fracionária, em que frações de itens são permitidas (Capítulo 13).
- **Por que pseudopolinomial?** O tempo $\Theta(nW)$ depende do *valor* de W, mas a entrada guarda W em apenas $\log_2 W$ bits. Com $W = 10^9$ (30 bits), o algoritmo faz 10^9 iterações por item: é exponencial no tamanho da entrada. O problema da mochila é NP-difícil, e não se conhece algoritmo polinomial para ele.
- **Memória $O(W)$.** Um único vetor $K[0..W]$ basta: para cada item, percorra w **decrescendo** de W até p_i e faça $K[w] := \max\{K[w], K[w - p_i] + v_i\}$. Com w *crescendo*, o mesmo item poderia ser reaproveitado, e o algoritmo passaria a resolver a mochila *ilimitada*.

Exemplo 11.7 (Por que w decresce no vetor único)**.** Um único item, de peso 2 e valor 3, e capacidade 4; o vetor começa com $K = (0,0,0,0,0)$, para $w = 0, \ldots, 4$. Com w **decrescendo**: $K[4] := \max\{0, K[2] + 3\} = 3$, $K[3] := 3$, $K[2] := 3$; resultado $(0,0,3,3,3)$, correto: o item cabe uma vez e vale 3. Com w **crescendo**: $K[2] := 3$, $K[3] := 3$ e $K[4] := \max\{0, K[2] + 3\} = 6$, porque $K[2]$ *já foi atualizado com este mesmo item*; resultado $(0,0,3,3,6)$, como se o item tivesse sido levado duas vezes. Decrescendo, $K[w - p_i]$ ainda guarda o valor "sem o item i", que é o $M(w - p_i, i - 1)$ da recorrência.

Armadilhas da programação dinâmica

- **Subsequência não é subvetor**: a subsequência comum máxima não exige contiguidade; o subvetor de soma máxima (Capítulo 9) exige.
- Escrever a recorrência sem casos base ou sem dizer o que $C(i, j)$ significa.
- Preencher a tabela na ordem errada: na cadeia de matrizes, preencher por *linhas* $i = 1, 2, \ldots$ usaria $m[i + 1, j]$ ainda não calculado. Use diagonais, ou i decrescente.
- Na cadeia de matrizes, o termo de custo é $r_{i-1}\, r_k\, r_j$ (e não $r_i r_k r_j$), e k vai de i a $j - 1$.
- Afirmar que $O(nW)$ é polinomial: é pseudopolinomial.

Pontos-chave

Programação dinâmica = recorrência com subproblemas que se repetem + tabela. Subsequência comum máxima em $\Theta(nm)$; cadeia de matrizes em $\Theta(n^3)$, por diagonais; mochila 0-1 em $\Theta(nW)$, pseudopolinomial; Floyd-Warshall em $\Theta(n^3)$ (Capítulo 12). Sempre: significado do subproblema, recorrência, base, ordem de preenchimento, complexidade e reconstrução.

11.5 Exercícios

Exercícios do curso

11.1 [Lista] resposta: p. 185
Na variante da multiplicação de cadeia de matrizes em que se quer *maximizar* o número de multiplicações escalares, vale a subestrutura ótima?

11.2 [Lista] resposta: p. 185
Dê um algoritmo $O(n^2)$ para a maior subsequência estritamente crescente de n números. Se puder, dê um $O(n \log n)$.

11.3 [Lista] resposta: p. 185
Dê um algoritmo eficiente para o maior palíndromo que é *subsequência* de uma string; para "character", a resposta é "carac". Qual o tempo?

11.4 [Lista] resposta: p. 185
Num grafo acíclico dirigido com pesos reais nas arestas e dois vértices s e t, descreva uma programação dinâmica para o caminho simples de maior peso de s a t.

11.5 [Lista] resposta: p. 185
Resolva por programação dinâmica a mochila *inteira* (ilimitada): cada item é um tipo e pode ser usado quantas vezes se quiser.

11.6 [Lista] resposta: p. 185
Cortar uma string de n caracteres em duas partes custa n. Dados os pontos de corte $L[1..m]$, determine a ordem de cortes de menor custo total e o custo. Para $n = 20$ e cortes após os caracteres 2, 8 e 10, cortar da esquerda para a direita custa 50, e da direita para a esquerda, 38.

11.7 [Slides] resposta: p. 185
Modifique o algoritmo da subsequência comum máxima para devolver uma subsequência, e não só o tamanho. Aplique a X = ABCBDAB e Y = BDCABA.

11.8 [Monitoria] resposta: p. 185
(a) Monte a tabela $m[i, j]$ para $r = (10, 100, 5, 50)$ e para $r = (20, 50, 10, 1, 100)$, com a ordem ótima. (b) Monte a tabela da mochila com $W = 4$ e itens $(p, v) = (1, 2), (3, 7), (2, 1)$. (c) Explique por que $\Theta(nW)$ é pseudopolinomial.

11.9 [Material de apoio] resposta: p. 185
(a) Encontre a maior subsequência comum de X = dacdba e Y = cadcda. (b) Resolva a mochila 0-1 com capacidade 100 e itens (valor, peso) = $(50, 20), (70, 30), (90, 40), (120, 50), (140, 60)$. (c) Dê um algoritmo linear para o subvetor contíguo de soma máxima e aplique-o a $5, 15, -30, 10, -5, 40, 10$.

11.10 [Material de apoio] resposta: p. 185
(a) Encontre a parentização ótima para as dimensões $\langle 5, 10, 3, 12, 5, 50, 6\rangle$. (b) Adapte o algoritmo para calcular a tabela s e escreva um algoritmo recursivo que multiplica $A_i \cdots A_j$ na ordem ótima. (c) Adapte o algoritmo da subsequência comum máxima para imprimir *todas* as subsequências comuns mais longas.

11.11 [Material de apoio] resposta: p. 185
(a) Quantas árvores binárias de busca guardam 6 chaves diferentes? (b) Construa uma árvore binária de busca ótima para as palavras com números de acessos *case* (15), *else* (5), *end* (5), *if* (35), *of* (5) e *then* (35), em que o custo é a soma, sobre as palavras, de (acessos)×(profundidade +1).

11.12 [Material de apoio] resposta: p. 186
(a) Subsequência crescente de *soma* máxima em $O(n^2)$. (b) Festa da empresa: numa árvore hierárquica com notas de sociabilidade, escolha convidados sem convidar ninguém junto com o seu chefe imediato, maximizando a soma das notas. (c) Torres gêmeas: dadas duas sequências de raios de lajotas, encontre a maior torre comum obtida removendo lajotas e mantendo a ordem. (d) Bandeiras: quantas sequências de n faixas brancas, azuis e vermelhas existem sem duas faixas de mesma cor vizinhas e com toda faixa azul entre uma branca e uma vermelha?

11.13 [Material de apoio] resposta: p. 186
(a) Mochila exata (*subset sum*) com pesos $2, 7, 5, 2$: complete a tabela ("O" = há solução sem o item; "I" = só usando o item; "–" = não há solução) para os itens 3 e 4 e capacidades 0..14. (b) Como reduzir a memória da mochila para $O(K)$? (c) Encontre a submatriz consecutiva de soma máxima numa matriz $m \times n$. (d) Corte uma laje de mármore $n \times m$, com cortes horizontais ou verticais de ponta a ponta, para maximizar o lucro, dados os preços p_{ab} de cada retângulo $a \times b$, em $O(n^2 m^2)$.

Exercícios complementares

11.14 [Complementar] resposta: p. 186
A *distância de edição* entre duas strings é o menor número de inserções, remoções e substituições de caracteres que transforma uma na outra. Dê a recorrência e a complexidade, e calcule a distância entre "kitten" e "sitting".

11.15 [Complementar] resposta: p. 186
Com moedas de valores $\{1, 3, 4\}$, qual o menor número de moedas para dar troco de 6? O que faz o algoritmo guloso? Escreva a programação dinâmica e preencha $T(0..10)$.

11.16 [Complementar] resposta: p. 186
Uma haste de comprimento n pode ser cortada em pedaços de comprimentos inteiros; um pedaço de comprimento i vale p_i. Com $p = (1, 5, 8, 9, 10, 17, 17, 20)$ para $i = 1, \ldots, 8$, qual o maior valor obtido de uma haste de comprimento 8? Dê a recorrência.

11.17 [Complementar] resposta: p. 186
Num tabuleiro de 4×4 casas, quantos caminhos levam uma peça da casa do canto superior esquerdo à do canto inferior direito, andando uma casa por vez para a direita ou para baixo? E se a casa da linha 2, coluna 2 estiver bloqueada?

11.18 [Complementar] resposta: p. 186
Decida por programação dinâmica se um multiconjunto de inteiros positivos pode ser dividido em duas partes de mesma soma. Teste com $\{3, 1, 1, 2, 2, 1\}$ e $\{2, 3, 7\}$. Qual a complexidade?

11.19 [Complementar] resposta: p. 186
Seja $P(n)$ o número de parentizações de um produto de n matrizes. Mostre que $P(1) = 1$ e $P(n) = \sum_{k=1}^{n-1} P(k)P(n-k)$, e calcule $P(n)$ para $n \leq 6$.

11.20 [Complementar] resposta: p. 186
Escreva uma versão do algoritmo da subsequência comum máxima que calcula apenas o tamanho usando memória $O(\min\{n, m\})$.

11.21 [Complementar] resposta: p. 186
De quantas maneiras é possível dar troco de 5 com moedas de valores $\{1, 2, 5\}$, sem considerar a ordem das moedas? E de 10? Escreva a programação dinâmica que conta as maneiras.

11.22 [Complementar] resposta: p. 186
O maior *substring* comum de duas strings exige que os símbolos sejam contíguos nas duas. Dê uma programação dinâmica em $O(nm)$ e aplique-a a "ABABC" e "BABCA".

12

Caminhos mínimos entre todos os pares: Floyd-Warshall

Neste capítulo

- Notação e representações de grafos usadas no restante do livro.
- O problema dos caminhos mínimos entre todos os pares de vértices.
- O algoritmo de Floyd-Warshall como programação dinâmica, com reconstrução dos caminhos.
- Pesos negativos, ciclos negativos e a comparação com o algoritmo de Dijkstra.

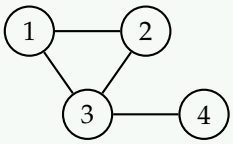
O algoritmo de Floyd-Warshall é mais uma aplicação da programação dinâmica, agora sobre um grafo: queremos o custo do caminho mais barato entre *todos* os pares de vértices. Por ser o primeiro algoritmo em grafos do livro, o capítulo começa fixando a notação e as representações de grafos usadas daqui em diante.

12.1 Grafos: notação e representação

Um grafo é um par $G = (V, E)$, em que V é o conjunto de vértices e E o de arestas; escrevemos $n = |V|$ e $m = |E|$. A **vizinhança** de um vértice v, $N(v)$, é o conjunto dos vértices ligados a v por uma aresta, e o seu **grau** é $|N(v)|$. Num **digrafo** (grafo direcionado), cada aresta $v \to w$ tem uma direção, e $N_{out}(v)$ é o conjunto dos vértices w com uma aresta $v \to w$. Há duas formas usuais de representar um grafo na memória:

Representação	Memória	"$vw \in E$?"	Percorrer $N(v)$
Matriz de adjacência (ou de pesos W)	$\Theta(n^2)$	$O(1)$	$\Theta(n)$
Listas de adjacência	$\Theta(n+m)$	$O(\text{grau}(v))$	$\Theta(\text{grau}(v))$

Exemplo 12.1 (As duas representações)**.** O grafo da Figura 12.1 tem $n = 4$ vértices e $m = 4$ arestas: 12, 13, 23 e 34. Na matriz de adjacência, a posição (v, w) vale 1 se vw é aresta; como o grafo não é direcionado, a matriz é simétrica, e cada aresta aparece duas vezes. Nas listas de adjacência, cada vértice guarda só os seus vizinhos: $N(3) = \{1, 2, 4\}$, e o grau de 3 é 3. Para saber se 24 é aresta, a matriz responde com uma consulta, à posição $(2, 4)$; as listas precisam percorrer $N(2) = \{1, 3\}$. Em compensação, as listas ocupam espaço proporcional a $n + 2m = 12$ entradas, contra $n^2 = 16$ da matriz — uma diferença que, num grafo esparso com um milhão de vértices, separa alguns megabytes de terabytes.



	1	2	3	4
1	0	1	1	0
2	1	0	1	0
3	1	1	0	1
4	0	0	1	0

1: 2, 3
2: 1, 3
3: 1, 2, 4
4: 3

Figura 12.1. Um grafo com 4 vértices, a sua matriz de adjacência e as suas listas de adjacência.

A matriz é adequada a grafos densos, com m próximo de n^2, e a testes frequentes de adjacência; as listas, a grafos esparsos e a algoritmos que percorrem vizinhanças. Uma **busca**, que visita todos os vértices e arestas do grafo examinando cada vizinhança uma vez, custa $\Theta(n+m)$ com listas e $\Theta(n^2)$ com matriz; as buscas em largura e em profundidade são estudadas nos Capítulos 15 e 17.

12.2 Matriz de pesos

Em problemas de caminhos mínimos, cada aresta tem um peso (um custo, uma distância, um tempo), e o grafo é dado por uma **matriz de pesos** W: $W(x,x) = 0$; $W(x,y)$ é o peso da aresta xy, se ela existe; e $W(x,y) = \infty$ se não existe. O custo de um caminho é a soma dos pesos das suas arestas, e a **distância** $\text{dist}(x,y)$ é o menor custo de um caminho de x a y. Um grafo não direcionado é tratado como um digrafo em que cada aresta xy de peso p dá origem às arestas $x \to y$ e $y \to x$, ambas de peso p.

Usaremos como exemplo, neste capítulo e no Capítulo 16, o digrafo de cinco vértices dado pela matriz abaixo e desenhado na Figura 12.2; os vértices são chamados $1, \ldots, 5$ aqui e $a, \ldots, e$ no capítulo do algoritmo de Dijkstra.

W	1 (a)	2 (b)	3 (c)	4 (d)	5 (e)
1 (a)	0	1	3	∞	6
2 (b)	∞	0	1	3	∞
3 (c)	1	2	0	1	∞
4 (d)	3	∞	∞	0	2
5 (e)	∞	∞	∞	1	0

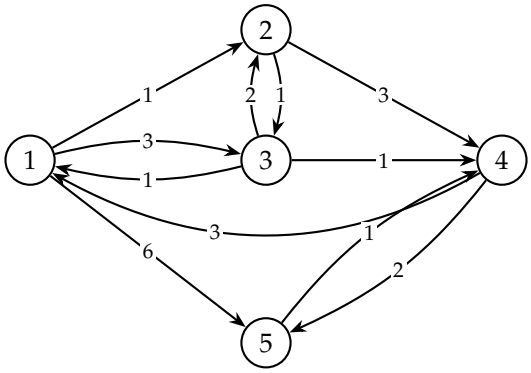


Figura 12.2. O digrafo de exemplo deste capítulo e do Capítulo 16. Os números nas arestas são os pesos; entre 1 e 3, entre 2 e 3 e entre 4 e 5 há arestas nos dois sentidos, com pesos diferentes.

12.3 O algoritmo de Floyd-Warshall

Problema: dado um digrafo com vértices $1, \ldots, n$ e matriz de pesos W, calcular $\text{dist}(i,j)$ para todos os pares ordenados (i,j). A ideia é limitar, passo a passo, quais vértices podem aparecer

no meio dos caminhos:

$D^k(i,j)$ = custo do caminho mínimo de i a j cujos vértices *intermediários* estão em $\{1,\dots,k\}$.

Então $D^0 = W$, pois sem intermediários só valem as arestas diretas, e D^n é a matriz das distâncias.

> **Intuição**
> Imagine que as cidades $1,\dots,n$ abrem, uma por vez, para servir de conexão. No começo, só há voos diretos ($D^0 = W$). Quando a cidade 1 abre, alguns trajetos ficam mais baratos fazendo escala nela; quando a 2 abre, trajetos com escala em 2 — e possivelmente também em 1 — entram em jogo; e assim por diante. Cada abertura é testada para todos os pares de uma vez, e depois que todas as cidades abriram, todos os trajetos possíveis foram considerados.

Decomposição. Um caminho mínimo de i a j com intermediários em $\{1,\dots,k\}$ ou **não passa** por k, e custa $D^{k-1}(i,j)$, ou passa **exatamente uma vez** por k — voltar a k formaria um ciclo, que não ajuda — e se divide num caminho mínimo de i a k e num de k a j, ambos com intermediários em $\{1,\dots,k-1\}$ (Figura 12.3):

$$D^k(i,j) = \begin{cases} W(i,j) & k = 0 \\ \min\{\, D^{k-1}(i,j),\ D^{k-1}(i,k) + D^{k-1}(k,j) \,\} & k \geq 1. \end{cases}$$

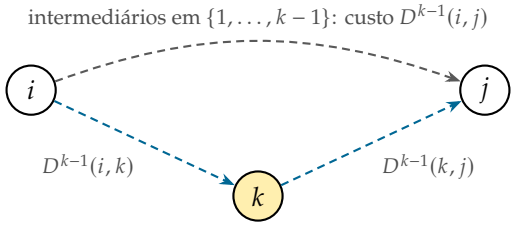


Figura 12.3. As duas possibilidades para um caminho mínimo de i a j com intermediários em $\{1,\dots,k\}$: ou ele não usa k (em cima), ou passa por k uma única vez, e os dois trechos só usam intermediários em $\{1,\dots,k-1\}$ (embaixo).

Por que só "prefixos" $\{1,\dots,k\}$? Uma formulação mais livre usaria, para cada conjunto $S \subseteq V$ de intermediários permitidos, $D(i,j,S) = \min_{v \in S}\{D(i,v,S\setminus\{v\}) + D(v,j,S\setminus\{v\})\}$. Ela também é correta, mas tem $n^2\,2^n$ subproblemas. Fixando uma ordem dos vértices e usando só os $n+1$ prefixos, sobram $O(n^2)$ subproblemas por valor de k, e $O(n^3)$ no total. Com $n = 20$, por exemplo, são $20^2 \cdot 2^{20} \approx 4{,}2 \cdot 10^8$ subproblemas na formulação por subconjuntos, contra $20^3 = 8000$ passos do Floyd-Warshall.

Floyd-Warshall com matriz de predecessores P: $\Theta(n^3)$ de tempo e $\Theta(n^2)$ de memória

```
Função FloydWarshall(W, n)
    para i, j := 1 até n faça
        D[i, j] := W[i, j]
        se i ≠ j e W[i, j] < ∞ então
            P[i, j] := i
        senão
            P[i, j] := λ
    para k := 1 até n faça                          % k é o laço externo
        para i := 1 até n faça
            para j := 1 até n faça
                se D[i, k] + D[k, j] < D[i, j] então
                    D[i, j] := D[i, k] + D[k, j]; P[i, j] := P[k, j]   % penúltimo vértice do trecho k → j
    retorna D, P
```

```
Procedimento Caminho(P, i, j)
    se i = j então
        imprima i
    senão se P[i, j] = λ então
        imprima "não há caminho"
    senão
        Caminho(P, i, P[i, j]); imprima j
```

A matriz P guarda, em $P^k(i,j)$, o **penúltimo** vértice do caminho mínimo de i a j com intermediários em $\{1,\ldots,k\}$. Se o caminho não melhora ao permitir k, $P^k(i,j) = P^{k-1}(i,j)$; se melhora, o novo caminho termina com o trecho de k a j, e $P^k(i,j) = P^{k-1}(k,j)$. Uma única matriz D basta, com atualizações "no lugar": na iteração k, a linha e a coluna k não mudam, pois $D(k,k) = 0$.

Exemplo 12.2 (A primeira iteração, $k = 1$)**.** Na iteração $k = 1$, cada entrada $D(i,j)$ é comparada com $D(i,1) + D(1,j)$: o caminho de i a j com escala em 1. Só interessam os i que chegam a 1 (os finitos da coluna 1: $i = 3$, com custo 1, e $i = 4$, com custo 3) e os j alcançados a partir de 1 (os finitos da linha 1: $j = 2, 3$ e 5). As candidatas são, portanto, $D(3,2)$, $D(3,5)$, $D(4,2)$, $D(4,3)$ e $D(4,5)$ (Figura 12.4):

- $D(3,5) = \min\{\infty,\ D(3,1) + D(1,5)\} = \min\{\infty,\ 1+6\} = 7$: o caminho $3 \to 1 \to 5$;
- $D(4,2) = \min\{\infty,\ 3+1\} = 4$ e $D(4,3) = \min\{\infty,\ 3+3\} = 6$;
- $D(3,2) = \min\{2,\ 1+1\} = 2$ e $D(4,5) = \min\{2,\ 3+6\} = 2$ não mudam: o caminho direto já é tão bom quanto o que passa por 1.

	1	2	3	4	5
1	0	1	3	∞	6
2	∞	0	1	3	∞
3	1	2	0	1	7
4	3	4	6	0	2
5	∞	∞	∞	1	0

D^1

Figura 12.4. A iteração $k = 1$ sobre $D^0 = W$. A linha e a coluna do vértice 1 (em azul) não mudam; cada entrada (i,j) é comparada com a soma da entrada da coluna 1 na sua linha com a da linha 1 na sua coluna. As três entradas que diminuem estão em amarelo.

Exemplo 12.3 (O digrafo da §12.2)**.**

k	entradas que diminuem ao permitir k como intermediário
1	$D(3,5)$: $\infty \to 7$; $D(4,2)$: $\infty \to 4$; $D(4,3)$: $\infty \to 6$
2	$D(1,3)$: $3 \to 2$; $D(1,4)$: $\infty \to 4$; $D(4,3)$: $6 \to 5$
3	$D(1,4)$: $4 \to 3$; $D(2,1)$: $\infty \to 2$; $D(2,4)$: $3 \to 2$; $D(2,5)$: $\infty \to 8$
4	$D(1,5)$: $6 \to 5$; $D(2,5)$: $8 \to 4$; $D(3,5)$: $7 \to 3$; $D(5,1)$: $\infty \to 4$; $D(5,2)$: $\infty \to 5$; $D(5,3)$: $\infty \to 6$
5	nenhuma

D^5	1	2	3	4	5
1	0	1	2	3	5
2	2	0	1	2	4
3	1	2	0	1	3
4	3	4	5	0	2
5	4	5	6	1	0

P^5	1	2	3	4	5
1	–	1	2	3	4
2	3	–	2	3	4
3	3	3	–	3	4
4	4	1	2	–	4
5	4	1	2	5	–

O caminho de 5 a 3 sai da matriz P de trás para frente: $P(5,3) = 2$, $P(5,2) = 1$, $P(5,1) = 4$ e $P(5,4) = 5$. Logo, o caminho é $5 \to 4 \to 1 \to 2 \to 3$, com custo $1 + 3 + 1 + 1 = 6 = D^5(5,3)$.

- **Pesos negativos.** O algoritmo funciona com arestas de peso negativo desde que **não haja ciclo negativo**, pois então o caminho mínimo nunca repete vértices. Há um ciclo negativo se e somente se, ao final, algum $D(i,i)$ é negativo.
- **Floyd-Warshall ou n execuções do Dijkstra?** O Floyd-Warshall custa $\Theta(n^3)$ e é muito simples. Executar o algoritmo de Dijkstra (Capítulo 16) a partir de cada vértice custa $O(n\,(n+m)\log n)$ com heap binário, ou $O(n^3)$ com vetor. Em grafos esparsos e sem pesos negativos, as n execuções do Dijkstra são mais rápidas; em grafos densos, ou com pesos negativos, o Floyd-Warshall é preferível.

Exemplo 12.4 (Pesos negativos, com e sem ciclo negativo)**.** No digrafo com as arestas $1 \to 2$ (4), $2 \to 3$ (-2) e $1 \to 3$ (3), a aresta direta de 1 a 3 custa 3, mas o caminho $1 \to 2 \to 3$ custa $4 - 2 = 2$. O algoritmo encontra isso na iteração $k = 2$: $D(1,3) = \min\{3,\ D(1,2) + D(2,3)\} = \min\{3, 2\} = 2$. Não há ciclos, e a diagonal termina toda nula.
Já no digrafo com as arestas $1 \to 2$ (1), $2 \to 3$ (-4) e $3 \to 1$ (2), o ciclo $1 \to 2 \to 3 \to 1$ pesa $1 - 4 + 2 = -1$: dando voltas nele, o custo diminui sem limite, e "caminho mínimo" deixa de fazer sentido. Ao final do algoritmo, os três elementos da diagonal são negativos (por exemplo, $D(1,1) = -1$, o peso do ciclo), e é por esse sinal que o ciclo negativo é detectado.

Armadilhas do Floyd-Warshall

- Colocar o laço de k **por dentro** (i e j por fora) dá resposta errada: k precisa ser o laço externo, pois D^k só pode ser calculada depois de toda a D^{k-1}.
- $D^k(i,j)$ restringe os vértices *intermediários* a $\{1,\ldots,k\}$; os extremos i e j podem ser quaisquer.
- $P(i,j)$ é o **penúltimo** vértice do caminho, e não o segundo; a reconstrução vai de trás para frente.
- Com ciclo negativo, "caminho mínimo" deixa de fazer sentido; o algoritmo detecta a situação por $D(i,i) < 0$.

Pontos-chave

$D^k(i,j) = \min\{D^{k-1}(i,j),\ D^{k-1}(i,k) + D^{k-1}(k,j)\}$, com $D^0 = W$; k é o laço externo; $\Theta(n^3)$ de tempo e $\Theta(n^2)$ de memória, com uma única matriz. Quando o caminho por k melhora, $P(i,j)$ recebe $P(k,j)$. Usar só os prefixos $\{1,\ldots,k\}$, e não subconjuntos quaisquer, é o que torna o algoritmo polinomial.

12.4 Exercícios

Exercícios do curso

12.1 [Slides] resposta: p. 186
Uma primeira formulação usa $D(i,j,S)$ = custo do caminho mínimo de i a j com intermediários em $S \subseteq V$, com $D(i,j,\emptyset) = W(i,j)$. Escreva a recorrência. Quantos subproblemas ela tem? O que o algoritmo de Floyd-Warshall faz para evitar isso?

12.2 [Slides] resposta: p. 187
Para a matriz de pesos W da §12.2, calcule $D^1,\ldots,D^5$ e P^5 e reconstrua o caminho mínimo de 5

a 3 e o de 2 a 1. Compare com o Exemplo da §12.

Exercícios complementares

12.3 [Complementar] resposta: p. 187
Execute o Floyd-Warshall no digrafo com arestas $1 \to 2$ (3), $1 \to 3$ (8), $1 \to 5$ (-4), $2 \to 4$ (1), $2 \to 5$ (7), $3 \to 2$ (4), $4 \to 1$ (2), $4 \to 3$ (-5) e $5 \to 4$ (6). Dê a matriz final. Há ciclo negativo?

12.4 [Complementar] resposta: p. 187
O *fecho transitivo* de um digrafo diz, para cada par (i, j), se existe caminho de i a j. Adapte o Floyd-Warshall, usando "ou" e "e" no lugar de min e +. Calcule o fecho para os vértices $1, \ldots, 5$ com as arestas $1 \to 2$, $2 \to 3$, $3 \to 1$ e $3 \to 4$.

12.5 [Complementar] resposta: p. 187
Como encontrar o peso do *ciclo* direcionado de menor peso de um digrafo com pesos positivos usando o Floyd-Warshall? Aplique ao digrafo com as arestas $1 \to 2$ (2), $2 \to 3$ (3), $3 \to 1$ (1), $3 \to 4$ (5) e $4 \to 3$ (2).

12.6 [Complementar] resposta: p. 187
Prove que, se o digrafo tem um ciclo de peso negativo, ao final do Floyd-Warshall vale $D(i, i) < 0$ para algum i do ciclo; e que, se não tem, $D(i, i) = 0$ para todo i.

12.7 [Complementar] resposta: p. 187
Um grafo tem $n = 10^4$ vértices, $m = 5 \cdot 10^4$ arestas e pesos não negativos. Para obter as distâncias entre todos os pares, compare o custo do Floyd-Warshall com o de n execuções do Dijkstra com heap binário. E se $m \approx n^2/2$?

12.8 [Complementar] resposta: p. 187
Execute o Floyd-Warshall no digrafo com as arestas $1 \to 2$ (4), $1 \to 4$ (10), $2 \to 3$ (3), $3 \to 4$ (1), $4 \to 1$ (2) e $3 \to 1$ (7), mostrando $D^1, \ldots, D^4$.

12.9 [Complementar] resposta: p. 187
No exercício anterior, calcule a matriz P^4 e reconstrua os caminhos mínimos de 2 a 1 e de 4 a 3.

12.10 [Complementar] resposta: p. 187
Dê um digrafo com dois vértices e um ciclo negativo e mostre o que o Floyd-Warshall calcula. Por que os valores de D deixam de ser custos de caminhos?

12.11 [Complementar] resposta: p. 187
Como aplicar o Floyd-Warshall a um grafo não direcionado? O que se pode dizer da matriz D final? Aresta de peso negativo num grafo não direcionado é um problema? Por quê?

12.12 [Complementar] resposta: p. 187
Quantas vezes, exatamente, o teste "$D[i, k] + D[k, j] < D[i, j]$" é executado? Quanta memória o algoritmo usa com e sem a matriz P?

12.13 [Complementar] resposta: p. 187
Prove que usar uma única matriz D, atualizada no lugar, dá o mesmo resultado que calcular $D^0, D^1, \ldots, D^n$ separadamente (suponha que não há ciclos negativos).

12.14 [Complementar] resposta: p. 187
Cada aresta tem uma *capacidade*, e a capacidade de um caminho é a menor capacidade das suas arestas. Adapte o Floyd-Warshall para calcular, para todo par (i, j), a maior capacidade de um caminho de i a j.

12.15 [Complementar] resposta: p. 187
A *excentricidade* de um vértice v é a maior distância de v a outro vértice, e o *diâmetro* do digrafo é a maior excentricidade. Use a matriz D^5 do Exemplo da §12 para calcular as excentricidades, o diâmetro e o *centro* (o vértice de menor excentricidade).

12.16 [Complementar] resposta: p. 187
Dada a matriz D final, como decidir se a aresta uv pertence a algum caminho mínimo de i a j? E a algum caminho mínimo qualquer do grafo?

12.17 [Complementar] resposta: p. 188
Depois de calcular D, o peso da aresta uv diminui para w'. Mostre como atualizar todas as distâncias em $O(n^2)$.

12.18 [Complementar] resposta: p. 188
Depois de calcular D para os vértices $1, \ldots, n$, acrescenta-se um vértice $n+1$ com algumas arestas de e para ele. Mostre como atualizar a matriz em $O(n^2)$.

12.19 [Complementar] resposta: p. 188
Com pesos positivos, adapte o Floyd-Warshall para calcular, além da distância, o *número* de caminhos mínimos distintos de i a j, para todo par.

12.20 [Complementar ★] resposta: p. 188
Mostre que o fecho transitivo pode ser calculado em $O(n^3/w)$ operações, representando cada linha da matriz como um vetor de bits e operando sobre palavras de w bits.

13

Algoritmos gulosos e matroides

Neste capítulo

- O método guloso e como provar que ele dá a solução ótima, pelo argumento de troca.
- Matroides: definição, exemplos e formas equivalentes do axioma de aumento.
- O teorema que caracteriza as matroides como as estruturas em que o guloso é sempre ótimo.
- Limites do método guloso e uma visão da interseção de matroides.

Um algoritmo guloso constrói a solução escolhendo, a cada passo, o que parece melhor naquele momento, sem nunca voltar atrás. É rápido e simples, mas nem sempre está correto. A teoria das matroides explica exatamente quando o guloso acerta: as matroides são as estruturas em que ele é ótimo para quaisquer pesos. O exemplo mais importante, a matroide gráfica, leva diretamente ao algoritmo de Kruskal do próximo capítulo.

13.1 O método guloso

No esquema geral, queremos um subconjunto $S' \subseteq S$ que satisfaça uma propriedade P e seja máximo (ou mínimo) segundo algum critério. O algoritmo guloso começa com $S'_0 = \emptyset$ e, a cada passo, acrescenta o elemento s que mantém a propriedade P e é o **melhor segundo o critério naquele momento**, sem jamais desfazer uma escolha. Ele para quando nenhum elemento pode mais ser acrescentado.

- **Quando o guloso é ótimo?** Quando vale a **propriedade gulosa**: existe uma solução ótima que contém o elemento escolhido, e, fixado esse elemento, o que resta é um subproblema bem definido do mesmo tipo (subestrutura ótima).
- **Como provar?** Pelo *argumento de troca*: tome uma solução ótima O que concorde com a solução gulosa o máximo possível; no primeiro ponto em que elas discordam, troque um elemento de O pelo escolhido pelo guloso, sem piorar a solução. Isso contradiz a escolha de O, ou mostra que a solução gulosa também é ótima.
- **O que os problemas resolvidos por gulosos têm em comum?** Em grande parte deles, a família dos conjuntos viáveis é uma **matroide** (§13.2).
- **Programação dinâmica ou guloso?** A programação dinâmica examina todas as escolhas possíveis de cada subproblema, guardando os resultados numa tabela; o guloso faz uma única escolha e nunca volta atrás. O guloso é mais rápido, mas só funciona quando vale a propriedade gulosa.

O argumento de troca

Para mostrar que a escolha gulosa não estraga nada, imagine alguém que conhece uma solução ótima e discorda do guloso no primeiro passo. Se for possível "trocar" um elemento da solução ótima pelo elemento guloso, sem perder qualidade e sem violar as restrições,

então existe uma solução ótima que começa como o guloso. Repetindo o argumento passo a passo (ou por indução), a solução gulosa inteira é ótima. Quando a troca falha — quando o elemento guloso "ocupa espaço" que a solução ótima usaria melhor —, o guloso costuma estar errado, e é aí que se procura um contraexemplo.

Troco guloso

```
Função TrocoGuloso(S, v_1 > v_2 > ··· > v_n)        % moedas em ordem decrescente
    total := 0
    para i := 1 até n faça
        q_i := ⌊S/v_i⌋; S := S − q_i v_i; total := total + q_i   % o máximo possível da maior moeda
    retorna total                                    % se S ≠ 0 ao final, não há troco exato
```

Exemplo 13.1 (Troco de 38 centavos)**.** Com as moedas $\{20, 10, 5, 1\}$, o guloso dá $\lfloor 38/20 \rfloor = 1$ moeda de 20 (sobram 18), $\lfloor 18/10 \rfloor = 1$ moeda de 10 (sobram 8), $\lfloor 8/5 \rfloor = 1$ moeda de 5 (sobram 3) e três moedas de 1: seis moedas no total (Figura 13.1). Cada passo é uma decisão local — "use a maior moeda que cabe" —, tomada sem olhar o que vem depois.

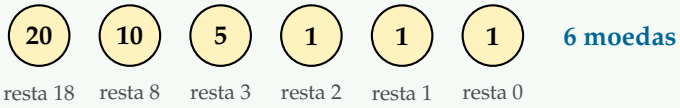


Figura 13.1. O troco guloso de 38 com as moedas $\{20, 10, 5, 1\}$: em cada passo, a maior moeda que ainda cabe no valor restante.

Exemplo 13.2 (O troco guloso é ótimo para as moedas $\{20, 10, 5, 1\}$)**.** Seja (a, b, c, d) o número de moedas de $20, 10, 5, 1$ numa solução ótima. Então $d \leq 4$, pois cinco moedas de 1 podem ser trocadas por uma de 5; $c \leq 1$, pois duas de 5 viram uma de 10; e $b \leq 1$, pois duas de 10 viram uma de 20. Logo as moedas menores que 20 somam $10b + 5c + d \leq 10 + 5 + 4 = 19 < 20$, e então $a = \lfloor S/20 \rfloor$, como no guloso. Do mesmo modo, $5c + d \leq 9 < 10$ força o número de moedas de 10 a ser o do guloso, e $d \leq 4 < 5$ força o de moedas de 5. A solução ótima *é* a gulosa.

O guloso, porém, **não funciona para qualquer sistema de moedas**. Com $\{6, 4, 1\}$ e $S = 8$, ele dá $6 + 1 + 1$ (três moedas), mas $4 + 4$ usa duas. Com $\{25, 10, 1\}$ e $S = 30$, ele usa 6 moedas, contra $10 + 10 + 10$. O caso geral se resolve por programação dinâmica, como a mochila ilimitada: $T(s) = 1 + \min_{v_i \leq s} T(s - v_i)$, em $\Theta(nS)$.

13.2 Matroides

As matroides foram introduzidas por Whitney em 1935, com o objetivo de capturar as propriedades comuns à independência linear de vetores e à ausência de ciclos em grafos.

Definição 13.3 (Matroide)**.** Uma **matroide** é um par $M = (E, \mathcal{I})$, em que E é um conjunto finito e $\mathcal{I} \subseteq 2^E$ é uma família de subconjuntos, chamados **independentes**, tal que: (I1) $\emptyset \in \mathcal{I}$; (I2) se $I \in \mathcal{I}$ e $I' \subseteq I$, então $I' \in \mathcal{I}$ (a família é fechada para subconjuntos); (I3) se $I_1, I_2 \in \mathcal{I}$ e $|I_1| < |I_2|$, existe $e \in I_2 \setminus I_1$ com $I_1 \cup \{e\} \in \mathcal{I}$ (**axioma de aumento**).

Em palavras: o vazio é independente; tirar elementos de um independente não o torna dependente; e um independente pequeno sempre pode "pegar emprestado" um elemento de qualquer independente maior.

Exemplo 13.4 (Uma matroide pequena, de dois jeitos)**.** Considere o grafo da Figura 13.2: um triângulo com as arestas $a = 12$, $b = 23$ e $c = 13$, e uma aresta pendente $d = 34$. Na **matroide gráfica**, os independentes são os conjuntos de arestas sem ciclo. O único ciclo é $\{a, b, c\}$, então são independentes todos os $2^4 = 16$ subconjuntos, exceto $\{a, b, c\}$ e $\{a, b, c, d\}$: 14 independentes. Os independentes maximais, as *bases*, são $\{a, b, d\}$, $\{a, c, d\}$ e $\{b, c, d\}$ — exatamente as árvores geradoras do grafo, todas com 3 arestas.
Agora tome os vetores de $\mathbb{R}^3$ $a = (1, 0, 0)$, $b = (0, 1, 0)$, $c = (1, 1, 0)$ e $d = (0, 0, 1)$. Na **matroide linear**, os independentes são os conjuntos linearmente independentes. Como $c = a + b$, o conjunto $\{a, b, c\}$ é dependente, e qualquer conjunto que o contenha também; todos os outros são independentes. É *a mesma* família de independentes: duas situações muito diferentes, ciclos num grafo e dependência linear, têm a mesma estrutura combinatória. É isso que a definição de matroide captura.

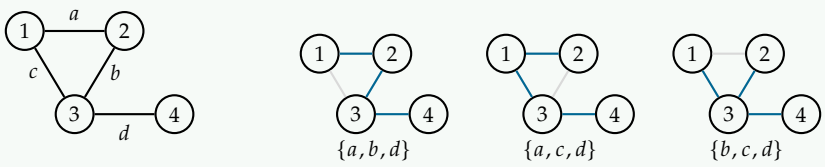
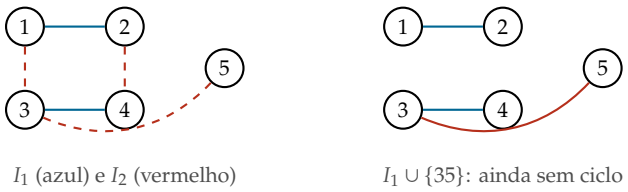


Figura 13.2. À esquerda, o grafo do Exemplo 13.4; as arestas são os elementos da matroide. À direita, as suas três bases, que são as árvores geradoras.

Matroide	Independentes
Linear	conjuntos de colunas (vetores) de uma matriz que são linearmente independentes.
Gráfica (de um grafo G)	conjuntos de arestas **acíclicos** (florestas); se G é conexo, as bases são as árvores geradoras.
Uniforme $U_{n,k}$	todos os subconjuntos $I \subseteq E$ com $\lvert I\rvert \leq k$ (*livre*, se $k = n$).
De partição	$E = E_1 \cup \cdots \cup E_t$ (uma partição) e os I com $\lvert I \cap E_i\rvert \leq k_i$ para todo i.
Transversal	num grafo bipartido com lados X e Y: os $I \subseteq X$ *emparelháveis*, isto é, cobertos por algum emparelhamento.
De emparelhamento	num grafo qualquer: os conjuntos de vértices cobertos por algum emparelhamento.
Dual M^*	as bases são os complementos $E \setminus B$ das bases B de M; o dual da gráfica é a *cográfica*.

Exemplo 13.5 (Uma matroide de partição)**.** Um restaurante monta pratos executivos com no máximo 1 entrada, 2 acompanhamentos e 1 sobremesa. O conjunto E de opções é particionado em entradas, acompanhamentos e sobremesas, e um pedido é "independente" se respeita os três limites. O pedido {salada, arroz, feijão, pudim} é independente; {arroz, feijão, purê} não é, pois tem 3 acompanhamentos. Os axiomas valem: o pedido vazio respeita os limites; tirar itens de um pedido válido mantém-no válido; e, se um pedido válido tem menos itens que outro, há alguma categoria em que o maior tem mais itens que o menor, e um desses itens pode ser acrescentado ao menor sem estourar o limite dela.

O axioma (I3) para a matroide gráfica. Uma floresta com k arestas num grafo de n vértices tem $n - k$ componentes. Como $\lvert I_1\rvert < \lvert I_2\rvert$, a floresta I_2 tem menos componentes que I_1, e por isso alguma aresta $e \in I_2$ liga duas componentes diferentes de I_1: se todas as arestas de I_2 ficassem dentro de componentes de I_1, cada componente de I_2 estaria contida numa de I_1, e I_2 teria pelo menos tantas componentes quanto I_1. Acrescentar essa aresta e a I_1 não cria ciclo (Figura 13.3).
O axioma (I3) para a matroide linear. O espaço gerado por I_1 tem dimensão $\lvert I_1\rvert < \lvert I_2\rvert$, então algum vetor de I_2 está fora dele, e acrescentá-lo a I_1 mantém a independência.



I_1 (azul) e I_2 (vermelho) $I_1 \cup \{35\}$: ainda sem ciclo

Figura 13.3. O axioma de aumento na matroide gráfica. A floresta $I_1 = \{12, 34\}$ (azul) tem 3 componentes; a floresta $I_2 = \{13, 35, 24\}$ (vermelho tracejado), com uma aresta a mais, tem só 2. A aresta 35 de I_2 liga duas componentes de I_1 e pode ser acrescentada a ela sem formar ciclo (à direita).

13.2.1 Formas equivalentes do axioma de aumento e bases

- **(I3′)** Para todo $X \subseteq E$, todos os independentes *maximais* contidos em X têm o mesmo tamanho, chamado **posto** de X, $r(X)$. Prova: se I_1 e I_2 fossem maximais em X com $|I_1| < |I_2|$, (I3) aumentaria I_1 dentro de X. Reciprocamente, dados $|I_1| < |I_2|$, estenda I_1 a um independente maximal B_1 dentro de $X = I_1 \cup I_2$; como $|B_1| \geq |I_2| > |I_1|$, existe $e \in B_1 \setminus I_1 \subseteq I_2 \setminus I_1$, e $I_1 \cup \{e\} \subseteq B_1$ é independente por (I2).
- **(I3″)** Todos os independentes maximais de $\mathcal{I}$ têm o mesmo tamanho r, e, se $|I| = r$ e $|I'| = r - 1$, existe $x \in I \setminus I'$ com $I' \cup \{x\} \in \mathcal{I}$.
- Uma **base** é um independente maximal; todas as bases têm o mesmo tamanho. A coleção $\mathcal{B}$ das bases satisfaz (B1) $\mathcal{B} \neq \emptyset$ e (B2) (**troca de bases**): se $B_1, B_2 \in \mathcal{B}$ e $x \in B_1 \setminus B_2$, existe $y \in B_2 \setminus B_1$ com $(B_1 \setminus \{x\}) \cup \{y\} \in \mathcal{B}$. Reciprocamente, toda coleção $\mathcal{B}$ que satisfaz (B1) e (B2) é a coleção das bases da matroide cujos independentes são os subconjuntos de elementos de $\mathcal{B}$.

Exemplo 13.6 (Posto e troca de bases)**.** Na matroide do Exemplo 13.4, o posto do triângulo $X = \{a, b, c\}$ é $r(X) = 2$: qualquer par de arestas do triângulo é independente maximal dentro de X, e não há um trio. O posto da matroide toda é 3. Para ver a troca de bases, tome $B_1 = \{a, b, d\}$, $B_2 = \{b, c, d\}$ e $x = a \in B_1 \setminus B_2$: retirando a de B_1 sobra $\{b, d\}$, e o elemento $y = c \in B_2 \setminus B_1$ completa uma nova base, $\{b, c, d\}$. No grafo, isso é trocar uma aresta de uma árvore geradora por outra que reconecta as duas partes — a mesma troca que aparece na prova da propriedade do corte do Capítulo 14.

13.2.2 O algoritmo guloso para matroides

Problema da base de peso máximo: dada uma matroide M e uma função de peso $c : E \to \mathbb{R}$, encontrar uma base B com $c(B) = \sum_{e \in B} c(e)$ máximo. Em geral, os independentes são dados de forma *implícita*, por um **oráculo** que responde "X é independente?", pois listá-los todos poderia exigir tempo exponencial em $|E|$. Na matroide gráfica, por exemplo, o oráculo é "as arestas de X formam ciclo?", respondido com Union-Find; na linear, é um escalonamento de Gauss.

Algoritmo guloso para matroides: $O(n \log n + n\, h)$**, sendo** h **o custo do oráculo**

```
Função Guloso(M, c)
    B := ∅
    ordene E de modo que c(e_1) ≥ c(e_2) ≥ ··· ≥ c(e_n)        % ordem não crescente
    para i := 1 até n faça
        se B ∪ {e_i} é independente em M então
            B := B ∪ {e_i}
    retorna B
```

Exemplo 13.7 (O guloso na matroide gráfica)**.** No grafo da Figura 13.4, com as arestas AB (7), AC (5), BC (8), BD (6), CD (9), CE (4) e DE (3), queremos uma árvore geradora de peso *máximo*. Em ordem decrescente de peso, o guloso aceita CD (9), BC (8) e AB (7); rejeita BD (6), que fecharia o ciclo B, C, D, e AC (5), que fecharia A, B, C; aceita CE (4); e rejeita DE (3). A árvore tem peso $9 + 8 + 7 + 4 = 28$, e, pelo teorema a seguir, nenhuma árvore geradora pesa mais (o que se confirma testando as poucas árvores possíveis).

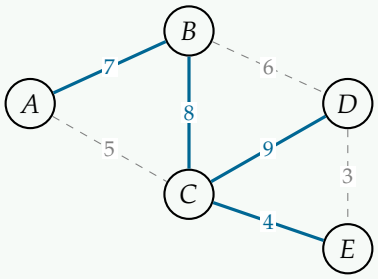


Figura 13.4. O guloso para a árvore geradora de peso máximo. As arestas aceitas estão em azul; as rejeitadas, que fechariam ciclos, em cinza tracejado.

Teorema 13.8. *Se M é uma matroide, o algoritmo* Guloso *devolve uma base de peso máximo, qualquer que seja a função de peso c.*

Demonstração. Seja B a saída, que é uma base, pois nada mais pode ser acrescentado a ela, e seja B^* uma base de peso máximo com $|B \cap B^*|$ o maior possível. Suponha $B \neq B^*$ e seja f o primeiro elemento, na ordem do algoritmo, de $B \setminus B^*$; os elementos escolhidos antes de f formam um conjunto $B' \subseteq B^*$. Pela troca de bases, existe $e \in B^* \setminus B$ tal que $(B^* \setminus \{e\}) \cup \{f\}$ é base. Pela otimalidade de B^*, $c(f) \leq c(e)$. Se $c(f) = c(e)$, essa nova base também seria ótima e teria mais elementos em comum com B, uma contradição. Logo $c(f) < c(e)$, e e vem antes de f na ordem. Mas, quando e foi examinado, o conjunto escolhido estava contido em B', e $B' \cup \{e\} \subseteq B^*$ é independente: o algoritmo teria escolhido e. Contradição. □

A recíproca também vale. Se $\mathcal{I}$ satisfaz (I1) e (I2) e o guloso encontra um independente maximal de peso máximo para *toda* função de peso, então vale (I3). De fato, suponha $|I_1| = k < |I_2|$ e que nenhum $e \in I_2 \setminus I_1$ aumente I_1. Dê peso $k + 2$ aos elementos de I_1, $k + 1$ aos de $I_2 \setminus I_1$ e 0 aos demais. O guloso escolhe I_1 primeiro, não consegue acrescentar nada de $I_2 \setminus I_1$ e termina com peso $k(k + 2)$; mas I_2 pesa pelo menos $(k + 1)(k + 1) > k(k + 2)$, uma contradição. Ou seja, **as matroides são exatamente os sistemas em que o guloso é sempre ótimo**.

- **Base de peso mínimo**: use a ordem não decrescente, ou os pesos $-c$. O algoritmo de Kruskal é exatamente isso na matroide gráfica (Capítulo 14).
- **Testar se k colunas de uma matriz $m \times k$ são linearmente independentes**: escalonamento de Gauss, em $O(m\,k \min\{m, k\})$; se $k > m$, elas já são dependentes.
- **Seleção de intervalos** com pesos unitários: o guloso que escolhe "o que termina primeiro" é ótimo, mas a família dos conjuntos de intervalos disjuntos **não** é uma matroide. Com $I_1 = \{[1, 10]\}$ e $I_2 = \{[1, 2], [3, 4]\}$, nenhum intervalo de I_2 é compatível com $[1, 10]$, e (I3) falha (Figura 13.5). A otimalidade vem de um argumento de troca específico; com pesos arbitrários, o guloso falha e é preciso programação dinâmica.
- **Emparelhamentos** (conjuntos de arestas sem extremos em comum) num grafo qualquer **não** formam uma matroide: no caminho $a-b-c-d$, o emparelhamento $\{bc\}$ não pode ser aumentado por ab nem por cd, que formam o emparelhamento $\{ab, cd\}$. Um guloso que escolha bc primeiro fica com uma aresta, e o máximo é 2.

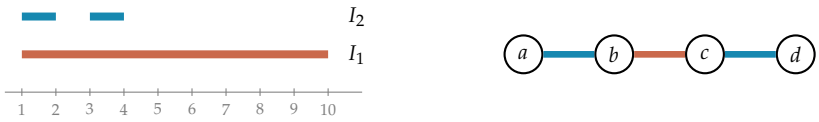


Figura 13.5. Duas famílias que não são matroides. À esquerda, intervalos: $I_1 = \{[1,10]\}$ tem menos elementos que $I_2 = \{[1,2],[3,4]\}$, mas nenhum intervalo de I_2 pode ser acrescentado a I_1. À direita, emparelhamentos no caminho $a-b-c-d$: $\{bc\}$ (vermelho) é maximal, mas menor que $\{ab, cd\}$ (azul).

Para saber mais: interseção de matroides

Dadas duas matroides $M_1 = (E, \mathcal{I}_1)$ e $M_2 = (E, \mathcal{I}_2)$, o problema da **interseção de matroides** pede um conjunto $I \in \mathcal{I}_1 \cap \mathcal{I}_2$ de tamanho (ou peso) máximo. Vários problemas são desse tipo: o *emparelhamento máximo em grafos bipartidos* é a interseção de duas matroides de partição (no máximo uma aresta por vértice de cada lado); a *floresta colorida* — o maior conjunto de arestas que é uma floresta e tem no máximo uma aresta de cada cor — é a interseção da matroide gráfica com uma de partição pelas cores.

A interseção de duas matroides, em geral, não é uma matroide, e o guloso pode falhar, como no caminho $a-b-c-d$ acima, que é bipartido. Mesmo assim, o problema é **polinomial**, por um algoritmo de *caminhos aumentantes*. Dado um conjunto comum I, constrói-se um *grafo de troca*: para $y \in I$ e $x \notin I$, há um arco $y \to x$ se $I - y + x \in \mathcal{I}_1$ e um arco $x \to y$ se $I - y + x \in \mathcal{I}_2$; as fontes são os x com $I + x \in \mathcal{I}_1$ e os destinos, os x com $I + x \in \mathcal{I}_2$. Um caminho *mais curto* de uma fonte a um destino dá, por diferença simétrica com I, um conjunto comum com um elemento a mais; se não há caminho, I é máximo. São no máximo r aumentos, cada um com $O(nr)$ consultas ao oráculo. Vale ainda um teorema de dualidade: $\max |I| = \min_{U \subseteq E}\{r_1(U) + r_2(E \setminus U)\}$.

Com **três** matroides, o problema já é NP-difícil: um *caminho hamiltoniano* num digrafo é um conjunto comum de tamanho $n-1$ a três matroides — a gráfica do grafo subjacente (sem ciclos), a de partição "grau de entrada no máximo 1" e a de partição "grau de saída no máximo 1". Também é polinomial o **particionamento** de matroides, que cobre E com o menor número de independentes (por exemplo, cobrir as arestas de um grafo com o menor número de florestas). Reconhecer que um problema é uma interseção ou um particionamento de matroides prova, portanto, que ele é polinomial.

Pontos-chave

Guloso: escolha local, sem retrocesso; correto quando vale a propriedade gulosa, provada por troca. Matroide: (I1) $\emptyset$ independente, (I2) fechada para subconjuntos, (I3) aumento. Nas matroides, o guloso (ordem decrescente de peso e oráculo de independência) encontra a base de peso máximo, e apenas nelas ele acerta para todo peso. Exemplos: linear, gráfica, uniforme, de partição, transversal. Não exemplos: emparelhamentos de arestas e intervalos disjuntos. A interseção de duas matroides é polinomial; a de três, NP-difícil.

13.3 Exercícios

Exercícios do curso

13.1 [Lista] resposta: p. 188

Escreva um algoritmo guloso para o problema do troco: dar S centavos com o menor número de moedas, de valores $v_1, \ldots, v_n$. Mostre que ele é ótimo para as moedas $\{10, 5, 2, 1\}$ e que não é ótimo para qualquer sistema monetário.

13.2 [Slides] resposta: p. 188
Prove o axioma de aumento (I3) para a matroide linear, para a gráfica, para a transversal e para a de emparelhamento.

13.3 [Slides] resposta: p. 188
Mostre que, se $\mathcal{I}$ satisfaz (I1) e (I2) e o algoritmo guloso devolve um independente maximal de peso máximo para *toda* função de peso, então vale (I3).

13.4 [Slides] resposta: p. 188
(a) É possível usar matroides no problema de seleção de intervalos com pesos unitários? (b) Faça um algoritmo para verificar se um conjunto S de vetores-coluna de uma matriz é linearmente independente. Qual a complexidade?

13.5 [Pesquisa] resposta: p. 188
Investigue: os axiomas das matroides e sua interpretação; a relação entre a propriedade de troca e a correção do guloso; o algoritmo de Kruskal como guloso numa matroide; quando a maximização de pesos pelo guloso é ótima; o que muda quando há duas famílias de restrições de independência; como funciona a interseção de duas matroides por caminhos aumentantes; e por que duas matroides levam a problemas polinomiais enquanto três já podem levar a problemas NP-difíceis.

Exercícios complementares

13.6 [Complementar] resposta: p. 188
Seleção de atividades: 11 atividades têm inícios $s = (1, 3, 0, 5, 3, 5, 6, 8, 8, 2, 12)$ e términos $f = (4, 5, 6, 7, 9, 9, 10, 11, 12, 14, 16)$. Aplique o guloso "a que termina primeiro" e prove que ele é ótimo.

13.7 [Complementar] resposta: p. 188
Mostre com contraexemplos que, para a seleção de atividades, os gulosos "a de menor duração primeiro" e "a que começa primeiro" não são ótimos.

13.8 [Complementar] resposta: p. 189
Na mochila *fracionária*, em que se podem levar frações dos itens, prove que o guloso pela razão valor/peso é ótimo. Aplique a $W = 50$ e itens $(p, v) = (10, 60), (20, 100), (30, 120)$ e compare com a mochila 0-1.

13.9 [Complementar] resposta: p. 189
Prove que a matroide uniforme $U_{n,k}$ e a matroide de partição satisfazem (I1), (I2) e (I3).

13.10 [Complementar ★] resposta: p. 189
Tarefas unitárias com prazos e multas: cada tarefa leva uma unidade de tempo, tem prazo d_i e multa w_i se terminar depois do prazo. Um conjunto de tarefas é *independente* se pode ser executado todo dentro dos prazos, o que vale se e somente se, para todo t, no máximo t tarefas do conjunto têm prazo $\leq t$. Esses conjuntos formam uma matroide; use o guloso para minimizar a multa total com $d = (4, 2, 4, 3, 1, 4, 6)$ e $w = (70, 60, 50, 40, 30, 20, 10)$.

13.11 [Complementar ★] resposta: p. 189
Os *códigos de Huffman* são construídos por um guloso que junta, repetidamente, as duas árvores de menor frequência. Construa o código para as frequências a:45, b:13, c:12, d:16, e:9 e f:5 e calcule o custo $\sum$(frequência) $\times$ (tamanho do código).

13.12 [Complementar] resposta: p. 189
Tarefas com durações $t_1, \ldots, t_n$ serão executadas uma após a outra numa única máquina. Mostre que executá-las em ordem crescente de duração minimiza a soma dos instantes de término.

Aplique a durações $3, 1, 2$.

13.13 [Complementar] resposta: p. 189
Seja $E = \{1, 2, 3\}$ e $\mathcal{I} = \{\emptyset, \{1\}, \{2\}, \{3\}, \{1, 2\}\}$. Verifique se $(E, \mathcal{I})$ é uma matroide. Em caso negativo, mostre uma função de peso para a qual o guloso falha.

13.14 [Complementar] resposta: p. 189
Seja $E = \{1, 2, 3, 4\}$ e $\mathcal{I}$ a família dos conjuntos com no máximo 2 elementos, exceto $\{1, 2\}$. Verifique se é uma matroide e dê uma matriz cujas colunas realizam essa matroide.

13.15 [Complementar] resposta: p. 189
Mostre que, na matroide gráfica de um grafo com n vértices e c componentes conexas, todas as bases têm $n - c$ arestas. Qual o posto da matroide gráfica de um grafo com 6 vértices formado por um triângulo e um caminho com 3 vértices?

13.16 [Complementar] resposta: p. 189
Mostre que o dual da matroide uniforme $U_{n,k}$ é $U_{n,n-k}$.

13.17 [Complementar] resposta: p. 189
O que faz o algoritmo guloso na matroide uniforme $U_{n,k}$? E numa matroide de partição? Aplique ao problema de escolher, entre itens das categorias $A = \{9, 7, 4\}$, $B = \{8, 3\}$ e $C = \{6, 5, 2\}$ (os números são os pesos), no máximo 2 itens por categoria com peso total máximo.

13.18 [Complementar] resposta: p. 189
Mostre que o troco guloso não é ótimo para as moedas $\{1, 7, 10\}$, dando um valor S em que ele falha. O sistema $\{1, 5, 10, 25\}$ tem esse problema?

13.19 [Complementar] resposta: p. 189
Dê um exemplo de seleção de intervalos *com pesos* em que o guloso "o que termina primeiro" não encontra o conjunto compatível de peso máximo. Como resolver esse problema?

13.20 [Complementar ★] resposta: p. 189
Mostre que a mochila fracionária pode ser resolvida em $O(n)$, sem ordenar os itens, usando seleção em tempo linear.

14

Árvores geradoras mínimas: Kruskal e Prim

Neste capítulo

- O problema da árvore geradora mínima e a propriedade do corte.
- O algoritmo de Kruskal, com Union-Find, e a sua correção.
- O algoritmo de Prim e as suas implementações com vetor, heap binário e heap de Fibonacci.
- Como escolher a implementação de acordo com a densidade do grafo.

Ligar n cidades por uma rede de custo mínimo, usando apenas as ligações possíveis, é encontrar uma **árvore geradora mínima** (AGM). Os dois algoritmos clássicos são gulosos: o de Kruskal percorre as arestas em ordem crescente de peso e aceita as que não fecham ciclo, usando a estrutura Union-Find do Capítulo 4; o de Prim faz crescer uma única árvore a partir de uma raiz, usando uma fila de prioridades (Capítulos 2 e 3). A correção dos dois decorre de um único lema, a propriedade do corte.

14.1 O problema e a propriedade do corte

Dado um grafo conexo G com pesos c nas arestas, queremos uma **árvore geradora** — um subgrafo conexo e acíclico que contém todos os n vértices e, portanto, tem $n - 1$ arestas — de peso total mínimo. Um **corte** $(S, V \setminus S)$ é uma partição dos vértices em dois lados, e uma aresta **atravessa** o corte se tem um extremo em cada lado.

Exemplo 14.1 (Árvores geradoras de um grafo pequeno)**.** O grafo da Figura 14.1 tem 4 vértices e 5 arestas: o quadrado PQ (4), QR (2), RS (5), SP (3) e a diagonal PR (1). Uma árvore geradora escolhe 3 das 5 arestas sem formar ciclo, e há 8 maneiras de fazê-lo, com pesos entre 6 e 12. Por exemplo, $\{PQ, QR, RS\}$ pesa 11 e $\{PQ, QR, SP\}$ pesa 9; a mínima é $\{PR, QR, SP\}$, com peso $1 + 2 + 3 = 6$. Com n vértices o número de árvores geradoras pode chegar a n^{n-2} (no grafo completo), e testar todas é inviável: precisamos de uma regra que diga, aresta a aresta, quais são seguras.

Lema 14.2 (Propriedade do corte)**.** *Seja A um conjunto de arestas contido em alguma AGM e $(S, V \setminus S)$ um corte que nenhuma aresta de A atravessa. Se e é uma aresta de peso mínimo entre as que atravessam o corte, então $A \cup \{e\}$ está contido em alguma AGM.*

Demonstração. Seja T uma AGM com $A \subseteq T$. Se $e \in T$, não há o que provar. Senão, $T + e$ tem um ciclo que passa por e; como e atravessa o corte, o ciclo precisa atravessá-lo de novo, por outra aresta $f \in T$, e $f \notin A$, pois nenhuma aresta de A atravessa o corte. Então $T' = T + e - f$ é uma árvore geradora e $c(T') = c(T) + c(e) - c(f) \leq c(T)$, pois $c(e) \leq c(f)$. Logo T' é uma AGM e contém $A \cup \{e\}$. □

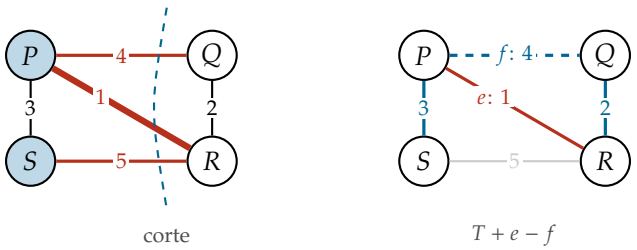


Figura 14.1. À esquerda, o corte $(\{P, S\}, \{Q, R\})$ do grafo do Exemplo 14.1: as arestas que o atravessam são PQ (4), PR (1) e RS (5), e a mais leve, PR, pertence à AGM. À direita, a troca da demonstração: a árvore $T = \{PQ, QR, SP\}$ (azul), de peso 9, mais a aresta $e = PR$, forma um ciclo que atravessa o corte também por $f = PQ$; trocando f por e, obtém-se uma árvore de peso $9 + 1 - 4 = 6$.

> **Intuição**
>
> Um corte divide as cidades em dois grupos, e a rede final precisa de pelo menos uma ligação entre eles. Qualquer que seja a rede ótima, se ela usasse uma ligação cara para atravessar o corte, poderíamos trocá-la pela mais barata que o atravessa, sem desconectar nada. Por isso a aresta mais leve de um corte é sempre "segura". Os algoritmos de Kruskal e de Prim são duas maneiras de escolher, a cada passo, um corte conveniente.

14.2 O algoritmo de Kruskal

No algoritmo de Kruskal, a solução parcial é uma **floresta**. As arestas são percorridas em ordem crescente de peso, e cada uma é aceita se **não forma ciclo** com as já aceitas, isto é, se liga duas árvores diferentes da floresta. O teste é feito com conjuntos disjuntos: cada árvore da floresta é um conjunto, e uma aresta uv fecha ciclo se e somente se u e v estão no mesmo conjunto.

Kruskal com Union-Find

```
Função Kruskal(G, c)
    E(T) := ∅
    para cada v ∈ V faça
        Make-Set(v)                                  % n árvores com um vértice
    ordene as arestas de modo que c(e_1) ≤ c(e_2) ≤ ··· ≤ c(e_m)   % O(m log m) = O(m log n)
    para j := 1 até m faça
        seja e_j = uv
        se Find(u) ≠ Find(v) então                   % une duas árvores distintas: não fecha ciclo
            E(T) := E(T) ∪ {e_j}; Union(u, v)
    retorna T                                        % pode parar quando |E(T)| = n − 1
```

Complexidade. A ordenação domina: $O(m \log m) = O(m \log n)$, pois $m \le n^2$ implica $\log m \le 2 \log n$. O laço custa $O(m + n \log n)$ com listas encadeadas e união "menor na maior", já que cada vértice muda de conjunto no máximo $\log n$ vezes, ou $O(m\,\alpha(n))$ com floresta, rank e compressão de caminhos. O total é $O(m \log n)$.

Correção. Ao aceitar $e = uv$, tome como S a árvore atual de u. Nenhuma aresta aceita atravessa esse corte, e e é a mais leve que o atravessa: uma aresta mais leve que o atravessasse teria sido examinada antes, quando os seus extremos já estavam em árvores diferentes (as árvores só crescem), e teria sido aceita. Pelo Lema 14.2, por indução, o conjunto de arestas aceitas está sempre contido numa AGM. O algoritmo de Kruskal também é o Teorema 13.8 aplicado à matroide gráfica.

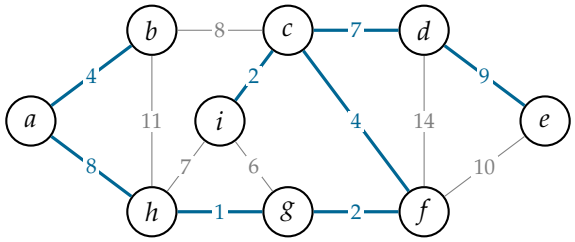


Figura 14.2. O grafo dos exemplos deste capítulo. As arestas em azul formam uma AGM, de custo 37.

Exemplo 14.3 (Kruskal e Prim no grafo da Figura 14.2)**.** O algoritmo de Kruskal aceita hg (1), ic (2), gf (2), ab (4), cf (4), cd (7), ah (8) e de (9), e rejeita ig (6), hi (7), bc (8), ef (10), bh (11) e df (14), que fechariam ciclos. A AGM tem **custo 37** (arestas em azul). O algoritmo de Prim, com raiz a, acrescenta ab (4), ah (8), hg (1), gf (2), fc (4), ci (2), cd (7) e de (9), nessa ordem, e chega à mesma árvore. A AGM não é única: ah e bc pesam 8, e trocar uma pela outra dá outra AGM de custo 37. Quando todos os pesos são distintos, a AGM é única.

Exemplo 14.4 (O Kruskal passo a passo, com os conjuntos)**.** A tabela mostra, para cada aresta na ordem examinada (com os empates na ordem em que aparecem), o teste $\textsc{Find}(u) \neq \textsc{Find}(v)$ e os conjuntos com mais de um vértice depois dela. Note como a floresta é desconexa no meio do caminho — $\{a, b\}$ e $\{c, f, g, h, i\}$ crescem separadamente — e só se junta com a aresta ah.

aresta	peso	teste	decisão	conjuntos com mais de um vértice
hg	1	h e g separados	aceita	$\{g, h\}$
ic	2	i e c separados	aceita	$\{c, i\}, \{g, h\}$
gf	2	g e f separados	aceita	$\{c, i\}, \{f, g, h\}$
ab	4	a e b separados	aceita	$\{a, b\}, \{c, i\}, \{f, g, h\}$
cf	4	c e f separados	aceita	$\{a, b\}, \{c, f, g, h, i\}$
ig	6	mesmo conjunto	**rejeita**	sem mudança
cd	7	c e d separados	aceita	$\{a, b\}, \{c, d, f, g, h, i\}$
hi	7	mesmo conjunto	**rejeita**	sem mudança
ah	8	a e h separados	aceita	$\{a, b, c, d, f, g, h, i\}$
bc	8	mesmo conjunto	**rejeita**	sem mudança
de	9	d e e separados	aceita	todos os 9 vértices

Depois de de já há $n - 1 = 8$ arestas, e o algoritmo pode parar; as arestas ef, bh e df seriam todas rejeitadas.

14.3 O algoritmo de Prim

No algoritmo de Prim, a solução parcial é uma **árvore** que cresce a partir de uma raiz: a cada passo, entra a aresta mais leve com um extremo na árvore e o outro fora dela. A correção vem do Lema 14.2, com $S = V(T)$.

Prim em $O(n^2)$ com vetores

```
Função Prim(G, W, r)
    para todo v: chave(v) := ∞; pai(v) := λ; dentro(v) := falso
    chave(r) := 0
    para it := 1 até n faça
        u := vértice com dentro(u) = falso e chave(u) mínima        % varredura em O(n)
```

```
        dentro(u) := verdadeiro                          % se u ≠ r, a aresta pai(u) u entra na árvore
        para cada v ∈ N(u) com dentro(v) = falso faça    % matriz: linha u, O(n); listas: O(grau(u))
            se W(u,v) < chave(v) então
                chave(v) := W(u,v); pai(v) := u
    retorna { pai(v) v : v ≠ r }
```

O valor $chave(v)$ é o peso da aresta mais leve que liga v à árvore. O algoritmo tem o mesmo esqueleto do algoritmo de Dijkstra (Capítulo 16), trocando "$L(u) + W(u,v)$" por "$W(u,v)$".

Exemplo 14.5 (As chaves do Prim)**.** A tabela acompanha o Prim com raiz a no grafo da Figura 14.2. Cada linha mostra o vértice que entra na árvore e as chaves dos vértices depois de atualizadas; "$*$" marca os vértices que já estão na árvore. Nos empates, escolhemos o vértice cuja chave foi definida primeiro: depois que b entra, c e h têm chave 8, mas a chave de h foi definida antes (quando a entrou), e por isso h entra antes de c.

entra	aresta	a	b	c	d	e	f	g	h	i
a	—	$*$	4	∞	∞	∞	∞	∞	8	∞
b	ab	$*$	$*$	8	∞	∞	∞	∞	8	∞
h	ah	$*$	$*$	8	∞	∞	∞	1	$*$	7
g	hg	$*$	$*$	8	∞	∞	2	$*$	$*$	6
f	gf	$*$	$*$	4	14	10	$*$	$*$	$*$	6
c	fc	$*$	$*$	$*$	7	10	$*$	$*$	$*$	2
i	ci	$*$	$*$	$*$	7	10	$*$	$*$	$*$	$*$
d	cd	$*$	$*$	$*$	$*$	9	$*$	$*$	$*$	$*$
e	de	$*$	$*$	$*$	$*$	$*$	$*$	$*$	$*$	$*$

A chave de um vértice só diminui: a de c passa de 8 (pela aresta bc) a 4 (pela fc) quando f entra, e a de i, de 7 a 6 e depois a 2. A soma das arestas que entram é $4+8+1+2+4+2+7+9 = 37$.

Implementação	Custo	Comentário
Matriz de adjacência e vetor $chave$	$O(n^2)$	n varreduras de $O(n)$; ótimo para matriz
Listas de adjacência e vetor $chave$	$O(n^2 + m) = O(n^2)$	a varredura domina
Listas e heap binário (com redução de chave)	$O(m \log n)$	n remoções e até m reduções, cada uma $O(\log n)$
Listas e heap de Fibonacci	$O(m + n \log n)$	redução de chave em $O(1)^*$

A versão com heap **não é necessariamente melhor** que a de $O(n^2)$: num grafo denso, $m = \Theta(n^2)$ e $m \log n = \Theta(n^2 \log n)$. O heap compensa quando $m = o(n^2/\log n)$.

Exemplo 14.6 (Vetor ou heap?)**.** Com $n = 1000$ vértices, a versão com vetor faz cerca de $n^2 = 10^6$ passos, qualquer que seja m. Num grafo denso, com $m \approx 5 \cdot 10^5$ arestas, o heap binário custa cerca de $(n+m)\log_2 n \approx 5 \cdot 10^6$ passos: cinco vezes mais que o vetor. Num grafo esparso, com $m = 3000$ arestas, o heap custa cerca de $4000 \cdot 10 = 4 \cdot 10^4$ passos: 25 vezes menos que o vetor. A densidade do grafo decide.

Armadilhas de árvores geradoras mínimas

- "O guloso funciona porque escolhe o melhor a cada passo" não é uma prova. A prova é a propriedade do corte, ou a estrutura de matroide.

- O algoritmo de Kruskal aceita uma aresta se ela *não fecha ciclo*, e não "se o vértice ainda não foi visitado" — isso seria o algoritmo de Prim. No Kruskal, a solução parcial pode ser desconexa.
- A AGM minimiza a soma dos pesos, e não as distâncias até uma raiz; isso é o que faz o algoritmo de Dijkstra.
- A complexidade do Kruskal é $O(m \log n)$ por causa da *ordenação*; as operações de Union-Find, sozinhas, são quase lineares.
- O Prim com heap não é sempre melhor; em grafos densos, perde para a versão com vetor.

Pontos-chave

Propriedade do corte: a aresta mais leve que atravessa um corte que A não atravessa pode ser acrescentada a A. Kruskal: arestas em ordem crescente e Union-Find, $O(m \log n)$. Prim: uma árvore que cresce pela aresta mais leve do corte, $O(n^2)$ com vetor ou matriz, $O(m \log n)$ com heap binário ou $O(m + n \log n)$ com heap de Fibonacci. Com pesos distintos, a AGM é única.

14.4 Exercícios

Exercícios do curso

14.1 [Lista] resposta: p. 190
Prove a correção do algoritmo de Kruskal.

14.2 [Lista] resposta: p. 190
Prove a correção do algoritmo de Prim.

14.3 [Lista] resposta: p. 190
Mostre como implementar o algoritmo de Prim em tempo $O(m \log n)$ com o grafo representado por listas de adjacência.

14.4 [Lista] resposta: p. 190
Mostre como implementar o algoritmo de Prim em tempo $O(n^2)$ com o grafo representado (a) por listas de adjacência; (b) por matriz de adjacência.

14.5 [Lista] resposta: p. 190
É possível implementar o algoritmo de Prim em tempo $O(n^2)$ com matriz de adjacência? É possível fazer melhor que isso nessa representação? Justifique.

14.6 [Lista] resposta: p. 190
Quais seriam as formas de resolver o problema da árvore geradora de peso **máximo**? Justifique.

Exercícios complementares

14.7 [Complementar] resposta: p. 190
No grafo com as arestas AB (6), AC (1), AD (5), BC (5), BE (3), CD (5), CE (6), CF (4), DF (2) e EF (6), execute o Kruskal (indicando as arestas rejeitadas) e o Prim a partir de A (indicando a ordem de entrada das arestas). Qual o custo da AGM? Ela é única?

14.8 [Complementar] resposta: p. 190
Prove que, se todos os pesos são distintos, a AGM é única.

14.9 [Complementar] resposta: p. 190
Prove a *propriedade do ciclo*: se e é a aresta de peso estritamente maior de algum ciclo de G, então e não pertence a nenhuma AGM.

14.10 [Complementar] resposta: p. 190
Mostre que, numa AGM T, o caminho entre dois vértices u e v minimiza o peso da *maior* aresta, entre todos os caminhos de u a v em G.

14.11 [Complementar] resposta: p. 190
Dada uma AGM T de G, acrescenta-se a G uma nova aresta uv de peso c. Como obter uma AGM do novo grafo em $O(n)$?

14.12 [Complementar] resposta: p. 190
No grafo com as arestas AB (7), AD (5), BC (8), BD (9), BE (7), CE (5), DE (15), DF (6), EF (8), EG (9) e FG (11), execute o Prim a partir de A, mostrando a ordem em que as arestas entram, e o Kruskal, mostrando as arestas rejeitadas. Qual o custo da AGM?

14.13 [Complementar ★] resposta: p. 191
No algoritmo de Borůvka, cada componente da floresta atual escolhe, em paralelo, a aresta mais leve que sai dela, e todas essas arestas são acrescentadas de uma vez. Mostre que o algoritmo está correto (supondo pesos distintos) e que faz $O(\log n)$ rodadas, cada uma em $O(m)$.

14.14 [Complementar] resposta: p. 191
Se todas as arestas têm o mesmo peso, como encontrar uma AGM em $O(n + m)$?

14.15 [Complementar] resposta: p. 191
Se todos os pesos são 1 ou 2, mostre como encontrar uma AGM em $O(m\,\alpha(n))$, sem ordenar por comparações.

14.16 [Complementar] resposta: p. 191
Prove que, se a aresta mais leve do grafo é única, ela pertence a toda AGM. Vale o mesmo para a segunda mais leve? E para a terceira?

14.17 [Complementar] resposta: p. 191
Dê um grafo conexo com pesos positivos em que a árvore de caminhos mínimos a partir de algum vértice (Capítulo 16) não é uma AGM.

14.18 [Complementar] resposta: p. 191
No algoritmo de Kruskal, quantas arestas são examinadas no melhor e no pior caso, se o laço para ao aceitar $n - 1$ arestas? Isso muda a complexidade assintótica?

14.19 [Complementar] resposta: p. 191
Explique por que, no algoritmo de Kruskal com Union-Find por listas encadeadas e união "menor na maior", o laço principal custa $O(m + n \log n)$.

14.20 [Complementar] resposta: p. 191
Se o grafo é desconexo, o que o algoritmo de Kruskal devolve? E o de Prim? Como obter uma *floresta geradora mínima*?

14.21 [Complementar ★] resposta: p. 191
A *segunda melhor* árvore geradora é a de menor peso entre as diferentes de uma AGM fixada T. Mostre que ela pode ser obtida de T trocando uma única aresta, e dê um algoritmo $O(n^2)$ para encontrá-la.

15

Busca em largura

Neste capítulo

- O que é uma busca em grafos e como a busca em largura percorre o grafo em camadas.
- A classificação das arestas em relação à floresta de largura.
- A propriedade fundamental dos níveis e suas consequências: distâncias e bipartição.
- A relação entre a estrutura de dados usada e o tipo de busca.

Uma **busca** é um procedimento sistemático para visitar todos os vértices e todas as arestas de um grafo, com o objetivo de resolver um problema ou extrair informação da sua estrutura. Na busca em largura, os vértices são visitados em camadas: primeiro a raiz, depois os vizinhos dela, depois os vizinhos dos vizinhos, e assim por diante. Por isso ela calcula distâncias em número de arestas e reconhece grafos bipartidos. A notação de grafos está na §12.1.

Intuição

Jogue uma pedra num lago: a perturbação se espalha em ondas circulares, e cada ponto é atingido pela primeira onda que chega até ele. A busca em largura faz o mesmo num grafo: a raiz é a pedra, a primeira onda atinge os vizinhos da raiz, a segunda atinge os vizinhos deles que ainda não tinham sido atingidos, e assim por diante. O número da onda que atinge um vértice é a sua distância à raiz.

15.1 O algoritmo

A regra da busca em largura é "*esgotar o pai antes de processar os filhos*": a próxima aresta examinada parte sempre do vértice **mais antigo** na busca. A estrutura que realiza essa regra é uma **fila** F: os vértices são processados na ordem em que foram alcançados. Cada vértice recebe um rótulo $L(v)$, com a ordem em que foi alcançado, e um nível $nivel(v)$, a sua profundidade na floresta de largura; os ponteiros $pai(v)$ definem a floresta.

Busca em largura com classificação das arestas

Procedimento BuscaLargura(G)
  $t := 0$; $F := \emptyset$; para todo v: $L(v) := 0$; $pai(v) := \lambda$
  enquanto existe v com $L(v) = 0$ **faça** % nova raiz, se o grafo for desconexo
    $nivel(v) := 0$; $t := t + 1$; $L(v) := t$; insira v no fim de F
    enquanto $F \neq \emptyset$ **faça**
      $v :=$ primeiro elemento de F; retire v de F
      para cada $w \in N(v)$ **faça**
        se $L(w) = 0$ **então** % aresta-**pai**
          $pai(w) := v$; $nivel(w) := nivel(v) + 1$; $t := t + 1$; $L(w) := t$; insira w no fim de F
        senão se $nivel(w) = nivel(v)$ **e** $w \in F$ **então** % $w \in F$: para não classificar duas vezes
          se $pai(w) = pai(v)$, vw é uma aresta-**irmão**; senão, uma aresta-**primo**
        senão se $nivel(w) = nivel(v) + 1$ **então**

vw é uma aresta-**tio**

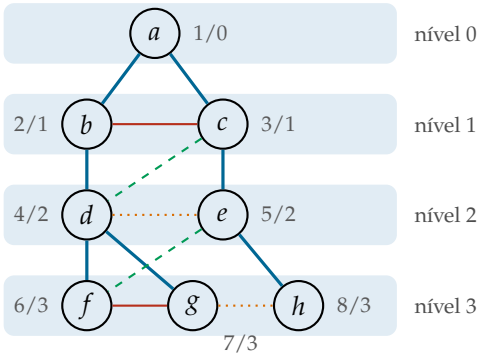


Figura 15.1. Busca em largura a partir de a, com os vizinhos em ordem alfabética; os rótulos são $L/nivel$, e cada faixa é um nível. Em azul grosso, as arestas-pai; em vermelho, as arestas-irmão (bc e fg); em laranja pontilhado, as arestas-primo (de e gh); em verde tracejado, as arestas-tio (cd e ef).

Exemplo 15.1 (A fila durante a busca)**.** Na busca da Figura 15.1, a fila evolui assim (cada linha é um vértice retirado da fila e o exame dos seus vizinhos):

sai	vizinhos examinados	fila depois
a	b e c: novos (pai a)	b, c
b	c: mesmo nível, mesmo pai (irmão); d: novo	c, d
c	d: um nível abaixo, já alcançado (tio); e: novo	d, e
d	e: mesmo nível, pais diferentes (primo); f e g: novos	e, f, g
e	f: tio; h: novo	f, g, h
f	g: irmão	g, h
g	h: primo	h
h	—	vazia

(Arestas para vértices que já saíram da fila, como ba vista a partir de b, não são classificadas de novo.) Observe que, em todo momento, a fila contém vértices de no máximo dois níveis consecutivos, e os de nível menor vêm na frente: depois que d sai, a fila é e, f, g, com e no nível 2 e f, g no nível 3. As distâncias a partir de a são 1 para b, c; 2 para d, e; e 3 para f, g, h. O grafo não é bipartido: o triângulo abc é um ciclo ímpar, e isso aparece nas arestas-irmão e primo.

15.2 Propriedades e aplicações

A floresta de largura alcança todos os vértices, e somente as arestas-pai pertencem a ela. As demais arestas são caracterizadas pelos níveis:

- vw é aresta-**pai** se e somente se $nivel(w) = nivel(v) + 1$ e $L(w) = 0$ no momento da visita;
- vw é aresta-**tio** se e somente se $nivel(w) = nivel(v) + 1$ e $L(w) \neq 0$ no momento da visita;
- vw é aresta-**irmão** se e somente se $nivel(w) = nivel(v)$ e $pai(w) = pai(v)$;
- vw é aresta-**primo** se e somente se $nivel(w) = nivel(v)$ e $pai(w) \neq pai(v)$.

Os nomes vêm da árvore genealógica: na Figura 15.1, c é irmão de b (mesmo pai, a), e d, filho de b, é "sobrinho" de c; a aresta cd liga, portanto, um tio ao sobrinho. Já d e e são primos: estão no mesmo nível, mas têm pais diferentes.

Teorema 15.2 (Propriedade fundamental)**.** *Para toda aresta vw de um grafo e toda floresta de largura,* $|nivel(v) - nivel(w)| \leq 1$.

A demonstração se reduz ao caso das arestas-tio e usa uma observação sobre a fila: em qualquer momento da busca, os vértices na fila pertencem a no máximo dois níveis consecutivos, em ordem não decrescente de nível. Desse teorema decorrem as principais aplicações.

- **Distâncias.** Numa busca com raiz x, $\text{dist}(x, v) = nivel(v)$ para todo v, e o caminho de x a v na árvore é um caminho **mínimo** em número de arestas. Por exemplo, o menor caminho da entrada à saída de um labirinto, dado por uma matriz de casas livres e paredes, é obtido por uma busca em largura no grafo das casas livres.
- **Bipartição.** Um grafo é bipartido se e somente se a sua floresta de largura não tem arestas-irmão nem arestas-primo, que fecham ciclos ímpares. Uma 2-coloração é dada pela paridade do nível.
- **Complexidade.** $O(n + m)$ com listas de adjacência, pois cada vértice entra na fila uma vez e cada lista é percorrida uma vez; $O(n^2)$ com matriz de adjacência.

Exemplo 15.3 (Um caminho mínimo, de trás para frente)**.** Na busca da Figura 15.1, para obter um caminho mínimo de a até h, sobe-se pelos ponteiros: $pai(h) = e$, $pai(e) = c$ e $pai(c) = a$. Lido ao contrário, o caminho é a, c, e, h, com $3 = nivel(h)$ arestas. Pela propriedade fundamental, não pode haver caminho mais curto: cada aresta muda o nível em no máximo 1, e é preciso ir do nível 0 ao nível 3.

Exemplo 15.4 (Reconhecendo um grafo bipartido)**.** O grafo da Figura 15.2 é um quadrado $1, 2, 3, 4$ com duas arestas pendentes, 45 e 26. A busca a partir de 1 põe 2 e 4 no nível 1 e 3, 5 e 6 no nível 2; a aresta 43 é uma aresta-tio, e não há arestas-irmão nem primo. O grafo é bipartido, e as cores pela paridade do nível, $\{1, 3, 5, 6\}$ e $\{2, 4\}$, formam uma 2-coloração: toda aresta liga um vértice de nível par a um de nível ímpar.

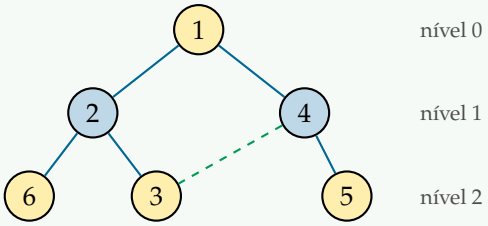


Figura 15.2. Um grafo bipartido e a 2-coloração dada pela paridade dos níveis da busca em largura a partir de 1. A aresta 43 (tracejada) é uma aresta-tio.

Pilha ou fila. Considere o seguinte esquema genérico: marque a raiz e insira-a numa estrutura D; enquanto D não estiver vazia, seja v o primeiro elemento de D; se v tem um vizinho desmarcado w, marque w, faça $pai(w) := v$ e insira w em D; senão, remova v de D. Se D é uma **pilha**, esse esquema é a busca em profundidade (Capítulo 17); se D é uma **fila**, é a busca em largura; e, com uma **fila de prioridades**, chega-se ao algoritmo de Dijkstra (Capítulo 16).

Exemplo 15.5 (O mesmo esquema, duas estruturas)**.** No grafo com as arestas 12, 13, 24, 34, 36 e 45, executamos o esquema a partir de 1, com os vizinhos em ordem crescente (Figura 15.3). Com **fila**, o vértice consultado é sempre o mais antigo ainda com vizinhos desmarcados: 1 marca 2 e 3; depois 2 marca 4; 3 marca 6; e 4 marca 5. A ordem de marcação é $1, 2, 3, 4, 6, 5$. Com **pilha**, o consultado é sempre o mais recente: 1 marca 2; 2 marca 4; 4 marca 3; 3 marca 6; 6 não tem vizinhos desmarcados e sai da pilha, e o mesmo acontece com 3; de volta a 4, ele marca 5. A ordem é $1, 2, 4, 3, 6, 5$, e a árvore é bem diferente: "funda", em vez de "larga".

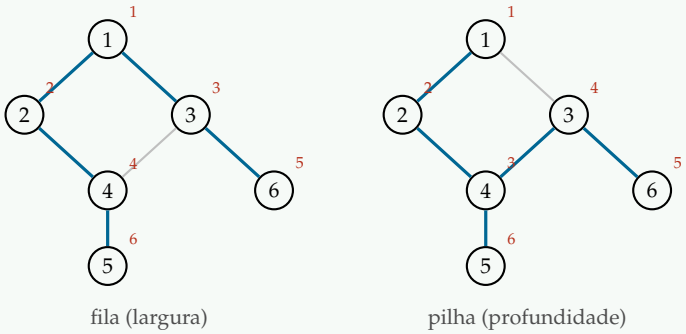


Figura 15.3. As árvores produzidas pelo esquema genérico com fila (à esquerda) e com pilha (à direita) no mesmo grafo. Em azul, as arestas da árvore; em cinza, as demais. Os números pequenos dão a ordem de marcação.

Armadilhas da busca em largura

- Esquecer o laço externo: num grafo desconexo, a busca a partir de um vértice só alcança a sua componente.
- O caminho da busca em largura é mínimo apenas em *número de arestas*; com pesos, é preciso o algoritmo de Dijkstra.
- Uma aresta entre vértices do mesmo nível deve ser classificada uma única vez; daí o teste "$w \in F$" no pseudocódigo.

Pontos-chave

Fila; "esgotar o pai antes de processar os filhos". Arestas-pai (árvore), irmão, primo e tio. Para toda aresta, $|nivel(v) - nivel(w)| \leq 1$. $nivel(v) = \text{dist}(raiz, v)$ em arestas, e o caminho na árvore é mínimo. Bipartido $\iff$ sem arestas-irmão e primo. $O(n + m)$ com listas.

15.3 Exercícios

Exercícios do curso

15.1 [Slides] resposta: p. 191
Dado um grafo conexo G e uma árvore geradora T de G, decida se T é uma árvore de largura de G, isto é, se existe uma busca em largura em G que produz T.

15.2 [Slides] resposta: p. 191
Mostre que o esquema genérico da §15 funciona como uma busca em profundidade se D for uma pilha e como uma busca em largura se D for uma fila.

15.3 [Slides] resposta: p. 192
Descreva, usando a busca em largura: (a) o cálculo das distâncias de um vértice x a todos os demais; (b) o menor caminho da entrada à saída de um labirinto dado por uma matriz; (c) o teste de bipartição e a 2-coloração.

Exercícios complementares

15.4 [Complementar] resposta: p. 192
Execute a busca em largura a partir de s no grafo com as arestas rs, rv, sw, wt, wx, tx, tu, xu, xy e uy (vizinhos em ordem alfabética). Dê a ordem de visita, os níveis, os pais e a classificação

das arestas que não são de árvore. O grafo é bipartido?

15.5 [Complementar] resposta: p. 192
O *diâmetro* de uma árvore é o maior número de arestas de um caminho entre dois vértices. Mostre que duas buscas em largura bastam: a partir de qualquer vértice, encontre o vértice a mais distante; a partir de a, o mais distante, b; o diâmetro é dist(a, b). Aplique à árvore com as arestas 12, 13, 24, 25, 36 e 57.

15.6 [Complementar] resposta: p. 192
Adapte a busca em largura para contar, para cada vértice v, *quantos* caminhos mínimos (em número de arestas) existem da raiz até v.

15.7 [Complementar] resposta: p. 192
Prove a propriedade fundamental: para toda aresta vw, $|nivel(v) - nivel(w)| \leq 1$. Conclua que $nivel(v) = \text{dist}(raiz, v)$.

15.8 [Complementar] resposta: p. 192
Num labirinto 4×4 com as linhas `S..#`, `##.#`, `....` e `#.#E` (`#` é parede; anda-se na horizontal ou na vertical), qual o menor número de passos de `S` a `E`? Dê o caminho e a distância de `S` a cada casa livre.

15.9 [Complementar] resposta: p. 192
Como fica a busca em largura num digrafo, percorrendo $N_{out}(v)$? Ela ainda calcula distâncias? Que tipos de arestas podem aparecer entre níveis?

15.10 [Complementar] resposta: p. 192
Escreva um procedimento que, depois de uma busca em largura com raiz x, imprime um caminho mínimo de x até um vértice y, ou informa que não há caminho.

15.11 [Complementar] resposta: p. 192
Justifique os custos $O(n + m)$ da busca em largura com listas de adjacência e $O(n^2)$ com matriz de adjacência. Em que tipo de grafo as duas representações dão o mesmo custo assintótico?

15.12 [Complementar ★] resposta: p. 192
Num grafo cujas arestas têm peso 0 ou 1, mostre como calcular as distâncias a partir de uma raiz em $O(n + m)$, usando uma fila de duas pontas (inserindo no início os vértices alcançados por arestas de peso 0 e no fim os alcançados por arestas de peso 1).

15.13 [Complementar] resposta: p. 192
Num mapa, há várias saídas de emergência. Calcule, para cada local, a distância (em número de passos) até a saída mais próxima, com uma única busca em largura.

15.14 [Complementar] resposta: p. 192
Use a busca em largura para decidir se o ciclo com 5 vértices e o ciclo com 6 vértices são bipartidos, mostrando as arestas-irmão ou primo encontradas.

15.15 [Complementar] resposta: p. 192
Mostre como contar as componentes conexas de um grafo e rotular cada vértice com o número da sua componente usando a busca em largura, em $O(n + m)$.

15.16 [Complementar] resposta: p. 192
Num tabuleiro de xadrez 8×8, qual o menor número de movimentos de um cavalo para ir da casa a1 à casa h8? Modele como um grafo e use a busca em largura.

15.17 [Complementar] resposta: p. 193
Com um jarro de 3 litros e outro de 5 litros, sem marcações, pode-se encher um jarro, esvaziá-lo ou despejar o conteúdo de um no outro até esvaziar o primeiro ou encher o segundo. Qual o

menor número de operações para obter exatamente 4 litros num dos jarros? Modele os estados como vértices de um grafo.

15.18 [Complementar] resposta: p. 193
Mostre que toda aresta-irmão ou aresta-primo fecha um ciclo de comprimento ímpar com os caminhos da floresta de largura. Por que uma aresta-tio não fecha, necessariamente, um ciclo ímpar?

15.19 [Complementar ★] resposta: p. 193
A *cintura* de um grafo não direcionado é o comprimento do menor ciclo. Mostre como calculá-la em $O(n(n+m))$ com buscas em largura.

15.20 [Complementar] resposta: p. 193
Execute a busca em largura no grafo bipartido completo $K_{3,3}$ (lados $\{a, b, c\}$ e $\{x, y, z\}$) a partir de a. Quais são os níveis? Que tipos de arestas aparecem?

16

Caminhos mínimos de uma origem: Dijkstra

Neste capítulo

- O problema dos caminhos mínimos a partir de uma origem, com pesos não negativos.
- O algoritmo de Dijkstra como uma busca guiada por uma fila de prioridades.
- A prova de correção e o papel dos pesos não negativos.
- As implementações com vetor, heap binário e heap de Fibonacci, e quando usar cada uma.

Quando as arestas têm pesos, a busca em largura deixa de fornecer distâncias. O algoritmo de Dijkstra generaliza a busca: em vez de uma fila, usa uma **fila de prioridades** e processa sempre o vértice mais próximo da origem que ainda não teve a distância fixada. É o primeiro algoritmo deste livro em que a escolha da implementação da fila de prioridades — vetor, heap binário ou heap de Fibonacci — muda a complexidade de forma decisiva. O exemplo do capítulo usa o digrafo da §12.2.

Intuição

Imagine que cada aresta é um pavio, que queima em tempo igual ao seu peso, e que acendemos a origem no instante 0. O fogo chega a cada vértice exatamente no instante $\text{dist}(s, v)$, pelo caminho mais rápido, e acende os pavios que saem dele. O algoritmo de Dijkstra simula esse incêndio: ele mantém, para cada vértice, a melhor estimativa de quando o fogo chegará, e processa os vértices na ordem em que o fogo os alcança. Quando um vértice é processado, nenhum caminho ainda não explorado pode chegar antes, porque esses caminhos passam por vértices que o fogo ainda não alcançou.

16.1 O algoritmo

Problema: dado um digrafo com pesos **não negativos** e uma raiz s, calcular os caminhos mínimos de s a todos os vértices. O algoritmo mantém, para cada vértice w, um rótulo $L(w)$, que é o custo do melhor caminho de s a w encontrado até o momento, e um ponteiro $pai(w)$. A cada passo, retira da fila de prioridades o vértice de menor rótulo, fixa esse rótulo como definitivo e tenta melhorar os rótulos dos seus vizinhos.

Algoritmo de Dijkstra

Procedimento DIJKSTRA(G, W, s)
  para todo w: $L(w) := \infty$; $pai(w) := \lambda$; w desmarcado
  $L(s) := 0$; insira s em D
  enquanto $D \neq \emptyset$ **faça**
    $v :=$ elemento de D com menor $L(v)$; remova v de D; marque v % $L(v)$ agora é definitivo
    para cada $w \in N_{out}(v)$ desmarcado **faça**

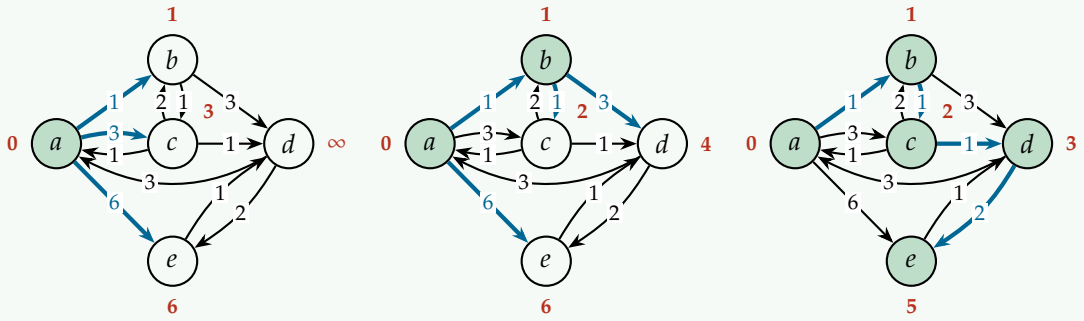
```
    se w ∉ D então
        insira w em D com prioridade L(w)
    se L(v) + W(v,w) < L(w) então                    % "relaxamento" da aresta vw
        L(w) := L(v) + W(v,w); reposicione w em D; pai(w) := v
```

O passo central é o **relaxamento** de uma aresta vw: se chegar a v e seguir a aresta vw é melhor que o melhor caminho conhecido até w, esse caminho passa a ser o novo melhor. Por exemplo, se $L(v) = 4$, $W(v,w) = 3$ e $L(w) = 9$, o relaxamento faz $L(w) := 7$ e $pai(w) := v$; se fosse $L(w) = 6$, nada mudaria.

Exemplo 16.1 (Raiz a no digrafo da §12.2)**.**

Removido	$L(a)$	$L(b)$	$L(c)$	$L(d)$	$L(e)$	mudanças em pai
a (0)	**0**	1	3	∞	6	$b, c, e \leftarrow a$
b (1)		**1**	2	4	6	$c, d \leftarrow b$
c (2)			**2**	3	6	$d \leftarrow c$
d (3)				**3**	5	$e \leftarrow d$
e (5)					**5**	—

Os ponteiros pai formam a **árvore de caminhos mínimos**: $a \to b \to c \to d \to e$. Por exemplo, o caminho mínimo de a a e é a, b, c, d, e, com custo $L(e) = 5$, embora a aresta direta custe 6. A primeira linha da matriz de distâncias calculada pelo Floyd-Warshall (Capítulo 12) confirma os valores $0, 1, 2, 3, 5$. A Figura 16.1 mostra o estado do algoritmo em três momentos: o rótulo de c começa em 3, pela aresta direta, e cai para 2 quando b é processado; o de e começa em 6 e só cai para 5 no fim, quando d é processado.



(a) depois de remover a (b) depois de remover b (c) no final

Figura 16.1. O algoritmo de Dijkstra a partir de a. Em verde, os vértices já removidos (com rótulo definitivo); em vermelho, os rótulos L; em azul, as arestas $pai(w) \to w$ atuais. À direita, a árvore de caminhos mínimos final.

Para um grafo não direcionado, basta trocar cada aresta xy de peso p pelas arestas direcionadas $x \to y$ e $y \to x$, ambas de peso p. E, se todos os pesos são iguais a 1, o algoritmo se comporta como a busca em largura: a fila de prioridades passa a liberar os vértices em ordem de nível, e $L(w)$ é a distância em número de arestas.

16.2 Correção

Teorema 16.2. *Com pesos não negativos, quando um vértice v é removido de D, vale $L(v) = \text{dist}(s, v)$.*

Demonstração. Sempre vale $L(v) \geq \text{dist}(s, v)$, pois $L(v)$ é o custo de algum caminho. Suponha que u seja o primeiro vértice removido com $L(u) > \text{dist}(s, u)$. Num caminho mínimo de s a u, seja y o primeiro vértice ainda não marcado e x o vértice anterior a ele, que já foi marcado

e, portanto, tem $L(x) = \text{dist}(s, x)$. Quando x foi marcado, a aresta xy foi relaxada, e então $L(y) \leq \text{dist}(s, x) + W(x, y) = \text{dist}(s, y) \leq \text{dist}(s, u) < L(u)$. A segunda desigualdade usa que os **pesos não são negativos**: o trecho do caminho de y até u custa pelo menos 0. Mas então u não tinha o menor rótulo em D, uma contradição. □

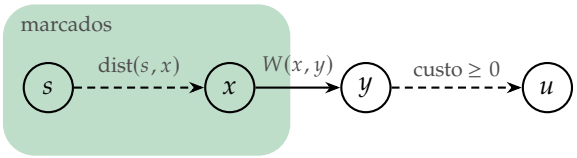


Figura 16.2. A situação da prova. A região verde contém os vértices já marcados. Um caminho mínimo de s a u sai da região pela aresta xy; como o trecho de y a u não tem custo negativo, $L(y) \leq \text{dist}(s, u) < L(u)$, e y teria saído da fila antes de u.

A hipótese sobre os pesos é essencial: com uma aresta negativa, um vértice já marcado pode ganhar, depois, um caminho mais barato, que o algoritmo nunca mais examina. Para pesos negativos sem ciclos negativos, existem outros algoritmos, como o de Bellman-Ford, em $O(nm)$; para todos os pares, o Floyd-Warshall.

Exemplo 16.3 (Uma aresta negativa engana o algoritmo)**.** No digrafo da Figura 16.3, com as arestas $s \to a$ (1), $s \to b$ (3), $b \to a$ (−3) e $a \to c$ (1), o algoritmo remove s (rótulo 0), depois a (rótulo 1), que é marcado como definitivo, e então c (rótulo 2). Só depois remove b (rótulo 3) e relaxa a aresta $b \to a$; mas a já está marcado e não é revisto. O resultado diz $L(a) = 1$ e $L(c) = 2$, enquanto o caminho $s \to b \to a$ custa $3 - 3 = 0$ e, com ele, $s \to b \to a \to c$ custa 1. O erro num vértice se propaga para os que dependem dele.

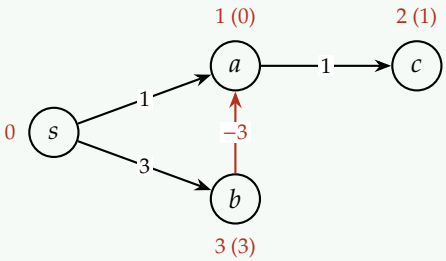


Figura 16.3. Um digrafo com uma aresta negativa em que o algoritmo de Dijkstra erra. Em vermelho, os rótulos calculados; entre parênteses, as distâncias corretas.

16.3 Implementações e complexidade

Estrutura D	Custo	Quando usar
Vetor L com varredura (matriz de adjacência)	$O(n^2)$	grafo **denso**, com $m = \Theta(n^2)$
Heap binário de mínimo com a posição de cada vértice	$O((n + m) \log n)$	grafo esparso
Heap de Fibonacci	$O(m + n \log n)$	teórico: redução de chave em $O(1)^*$

Os custos vêm de n inserções, n remoções do mínimo e até m reduções de prioridade (Capítulos 2 e 3). No heap binário, "reposicionar" é uma operação *subir*, e exige um vetor $pos[w]$ com a posição de cada vértice no heap.

Exemplo 16.4 (O heap com vetor de posições)**.** Na execução do Exemplo 16.1 com um heap binário de mínimo, depois que b é removido o heap guarda (c:2, e:6, d:4), nessa ordem no

vetor, e o vetor de posições diz $pos[c] = 1$, $pos[e] = 2$ e $pos[d] = 3$. Quando c é removido, o último elemento, d:4, vai para a raiz e desce (aqui, não precisa descer), e $pos[d]$ passa a 1. Em seguida, o relaxamento da aresta cd reduz $L(d)$ de 4 para 3: é $pos[d] = 1$ que diz onde d está no heap, e a operação subir a partir da posição 1 confirma que nada precisa mudar. Sem o vetor pos, seria preciso procurar d no heap inteiro, em $O(n)$.

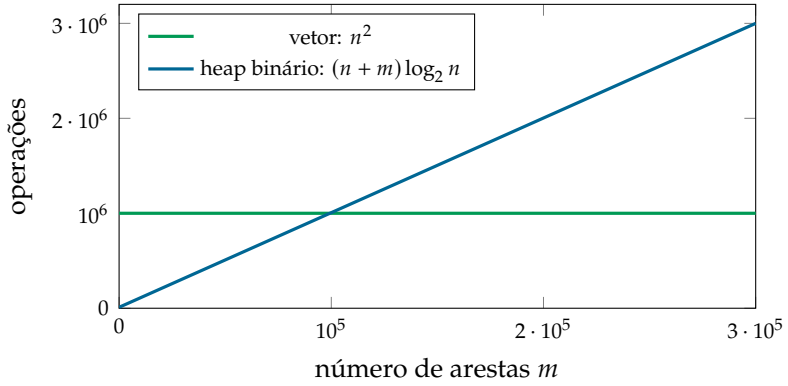


Figura 16.4. O custo das duas implementações principais para $n = 1000$ vértices, em função do número de arestas m. A versão com vetor custa cerca de n^2, qualquer que seja m; a com heap binário, cerca de $(n + m) \log_2 n$. As curvas se cruzam perto de $m \approx n^2/\log_2 n \approx 10^5$: abaixo disso, o heap é melhor; acima, o vetor.

Armadilhas do algoritmo de Dijkstra

- Usar o algoritmo com uma aresta negativa: o rótulo de um vértice já fixado pode estar errado.
- Esquecer o vetor de posições no heap binário: sem ele, encontrar o vértice a reposicionar custaria $O(n)$.
- Confundir com a árvore geradora mínima: a árvore de caminhos mínimos minimiza a distância de cada vértice à raiz; a AGM (Capítulo 14) minimiza a *soma* dos pesos da árvore. Em geral, são árvores diferentes.
- Num grafo não direcionado, cada aresta deve ser considerada nos dois sentidos.

Pontos-chave

Fila de prioridades na chave L; remove-se o menor L ainda não fixado; relaxa-se $L(w) := \min\{L(w), L(v) + W(v, w)\}$. Exige pesos não negativos. Custos: $O(n^2)$ com vetor, $O((n + m) \log n)$ com heap binário, $O(m + n \log n)$ com heap de Fibonacci. Os ponteiros pai formam a árvore de caminhos mínimos. Com todos os pesos iguais a 1, é a busca em largura.

16.4 Exercícios

Exercícios do curso

16.1 [Slides] resposta: p. 193
Mostre que o algoritmo de Dijkstra se comporta como uma busca em largura se todos os pesos são iguais a 1: a árvore de caminhos mínimos é uma árvore de largura e cada $L(w)$ é a distância em arestas.

16.2 [Slides] resposta: p. 193
Como encontrar caminhos mínimos de uma raiz a todos os vértices num grafo *não direcionado*

com pesos positivos?

16.3 [Slides] resposta: p. 193
Execute o algoritmo de Dijkstra a partir de a no digrafo da §12.2, mostrando os rótulos L e os pais a cada vértice removido.

Exercícios complementares

16.4 [Complementar] resposta: p. 193
Execute o algoritmo de Dijkstra a partir de s no digrafo com as arestas $s \to t$ (10), $s \to y$ (5), $t \to x$ (1), $t \to y$ (2), $x \to z$ (4), $y \to t$ (3), $y \to x$ (9), $y \to z$ (2), $z \to s$ (7) e $z \to x$ (6). Dê a ordem de remoção, as distâncias e a árvore de caminhos mínimos.

16.5 [Complementar] resposta: p. 193
Dê um digrafo com três vértices e uma aresta negativa, sem ciclo negativo, em que o algoritmo de Dijkstra erra. Somar uma constante a todos os pesos, para torná-los positivos, resolve? Por quê?

16.6 [Complementar] resposta: p. 193
Cada aresta uv de uma rede funciona com probabilidade $p(u, v) \in (0, 1]$, independentemente das outras. Como encontrar o caminho *mais confiável* (de maior probabilidade de funcionar) de s a t com o algoritmo de Dijkstra?

16.7 [Complementar] resposta: p. 193
Há várias fontes $s_1, \ldots, s_k$ (por exemplo, quartéis de bombeiros). Como calcular, para cada vértice, a distância até a fonte mais próxima com uma única execução do algoritmo de Dijkstra?

16.8 [Complementar] resposta: p. 193
Para um grafo com $n = 10^5$ vértices e $m = 3 \cdot 10^5$ arestas, e para outro com $n = 10^4$ e $m = 5 \cdot 10^7$, qual implementação do algoritmo de Dijkstra você usaria, vetor ou heap binário? Estime o número de operações.

16.9 [Complementar] resposta: p. 193
Execute o algoritmo de Dijkstra a partir do vértice 1 no grafo não direcionado com as arestas 12 (7), 13 (9), 16 (14), 23 (10), 24 (15), 34 (11), 36 (2), 45 (6) e 56 (9). Dê a ordem de remoção, as distâncias e os pais.

16.10 [Complementar] resposta: p. 193
O algoritmo de Dijkstra continua correto se algumas arestas têm peso zero? Justifique com a prova de correção.

16.11 [Complementar] resposta: p. 193
Se só interessa a distância de s a um único vértice t, em que momento o algoritmo pode parar? Por que isso está correto?

16.12 [Complementar] resposta: p. 194
Cada *vértice* v tem um custo $c(v) \geq 0$ de passagem, além dos pesos das arestas. Como transformar o problema para aplicar o algoritmo de Dijkstra?

16.13 [Complementar] resposta: p. 194
Queremos o caminho mínimo de s a t que usa no máximo k arestas. Por que o algoritmo de Dijkstra não resolve isso diretamente? Mostre como resolver com um grafo de estados (v, j), em que j é o número de arestas usadas.

16.14 [Complementar] resposta: p. 194
A capacidade de um caminho é a menor capacidade das suas arestas. Adapte o algoritmo de Dijkstra para encontrar, a partir de s, o caminho de maior capacidade até cada vértice.

16.15 [Complementar] resposta: p. 194
Com pesos positivos, adapte o algoritmo de Dijkstra para contar quantos caminhos mínimos distintos existem de s até cada vértice.

16.16 [Complementar] resposta: p. 194
Num digrafo *acíclico* com pesos quaisquer, inclusive negativos, mostre como calcular os caminhos mínimos a partir de s em $O(n + m)$, sem fila de prioridades.

16.17 [Complementar] resposta: p. 194
Por que não basta trocar "menor" por "maior" no algoritmo de Dijkstra para encontrar caminhos simples *mais longos*?

16.18 [Complementar ★] resposta: p. 194
Se os pesos são inteiros entre 0 e C, mostre como implementar o algoritmo de Dijkstra em $O(m + nC)$, usando um vetor de baldes indexado pelo valor do rótulo.

16.19 [Complementar] resposta: p. 194
Prove que, durante toda a execução, $L(v) \geq \text{dist}(s, v)$ para todo vértice v, e que $L(v)$ nunca aumenta.

16.20 [Complementar] resposta: p. 194
Quantas operações de cada tipo (inserção, remoção do mínimo, redução de chave) o algoritmo de Dijkstra faz no pior caso? Use isso para justificar o custo $O(m + n \log n)$ com heap de Fibonacci.

17

Busca em profundidade

Neste capítulo

- A busca em profundidade, com as profundidades de entrada e de saída de cada vértice.
- A árvore de profundidade, as arestas de retorno e o teorema dos intervalos aninhados.
- Aplicações: conexidade, caminhos, ciclos e bipartição.
- Articulações, pontes e blocos, com o valor $back(v)$.

Na busca em profundidade, a próxima aresta examinada parte sempre do vértice visitado mais recentemente: a busca avança o quanto puder e só volta quando não há mais para onde ir. Os instantes de entrada e de saída de cada vértice formam intervalos aninhados que revelam a estrutura do grafo: componentes, ciclos e bipartição e, com um valor auxiliar chamado $back(v)$, as articulações e os blocos — tudo em tempo linear. Este capítulo trata de grafos não direcionados; o próximo, de digrafos.

Intuição

É a estratégia de quem explora um labirinto desenrolando um novelo de lã: em cada encruzilhada, segue por um corredor ainda não explorado; quando chega a um beco sem saída, ou a um lugar onde já esteve, volta pelo fio até a última encruzilhada com corredores novos. O fio é a pilha de recursão: ele contém exatamente o caminho da entrada até a posição atual. A busca em largura, ao contrário, avança em ondas, como um grupo que se divide em todas as direções ao mesmo tempo.

17.1 O algoritmo

A regra da busca em profundidade é "*esgotar o filho antes de esgotar o pai*": a busca é naturalmente recursiva, ou, de forma equivalente, usa uma pilha. Um relógio global t marca, para cada vértice v, a sua profundidade de entrada $PE(v)$, quando ele é alcançado, e a sua profundidade de saída $PS(v)$, quando todos os seus vizinhos já foram examinados.

Busca em profundidade, com o cálculo de $back(v)$ e dos blocos

```
Procedimento BuscaProfundidade(G)
    t := 0; para todo v: PE(v) := 0; PS(v) := 0; pai(v) := λ
    enquanto existe v com PE(v) = 0 faça P(v)        % nova raiz; uma por componente
Procedimento P(v)
    t := t + 1; PE(v) := t; back(v) := PE(v)
    para cada w ∈ N(v) faça
        se PE(w) = 0 então                           % aresta de árvore
            pai(w) := v; empilhe vw; P(w)
            se back(w) ≥ PE(v) então
                desempilhe e imprima as arestas até vw    % um bloco
```

```
        back(v) := min{back(v), back(w)}
    senão se PS(w) = 0 e w ≠ pai(v) então          % aresta de retorno
        empilhe vw; back(v) := min{back(v), PE(w)}
t := t + 1; PS(v) := t
```

As linhas com $back$ e com a pilha de arestas só são necessárias para encontrar articulações e blocos (§17.4); sem elas, o procedimento é a busca básica.

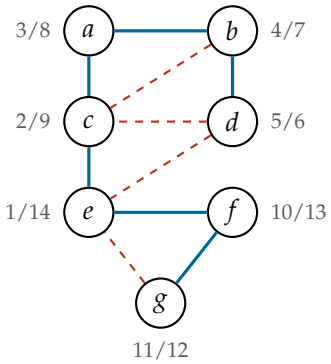


Figura 17.1. Busca em profundidade a partir de e, com os vizinhos em ordem alfabética; os rótulos são PE/PS. Em azul grosso, a árvore de profundidade; em vermelho tracejado, as arestas de retorno.

Exemplo 17.1 (A busca passo a passo)**.** Na Figura 17.1, a busca começa em e (instante 1) e examina os vizinhos em ordem alfabética. O primeiro é c, que entra (2); de c, o primeiro vizinho não visitado é a (3), e de a, o b (4). Em b, o vizinho c já está ativo — a sua chamada ainda não terminou —, e bc é uma aresta de retorno; o vizinho d é novo e entra (5). Em d, os vizinhos b (o pai), c e e já estão ativos: dc e de são arestas de retorno. Sem vizinhos novos, d sai (6); depois saem b (7), a (8) e c (9). De volta a e, o vizinho d já terminou e não é examinado de novo; f é novo (10), e dele se chega a g (11), cujo vizinho e está ativo (ge é de retorno). Saem g (12), f (13) e e (14).
No instante 5, quando d entra, a pilha de recursão é e, c, a, b, d: exatamente o caminho da raiz até d na árvore. É por isso que uma aresta para um vértice ativo sempre vai a um ancestral.

17.2 Propriedades

- Se G é conexo, as arestas de árvore formam uma **árvore geradora**, a árvore de profundidade. Se não é, o laço externo produz uma floresta, com uma árvore por componente.
- Num grafo **não direcionado**, só existem arestas de **árvore** e de **retorno**. Toda aresta de retorno liga um vértice v a um **ancestral** de v na árvore e fecha um ciclo.
- **Intervalos de vida**: seja $I(v) = [PE(v), PS(v)]$. Dois intervalos são disjuntos ou um contém o outro (o *teorema dos parênteses*), e v é descendente de w se e somente se $PE(w) < PE(v)$ e $PS(v) < PS(w)$: v "entra depois e sai antes" de w.
- **Complexidade**: $O(n + m)$ com listas de adjacência. Cada vértice é visitado uma vez, e cada aresta é examinada duas vezes, uma a partir de cada extremo.

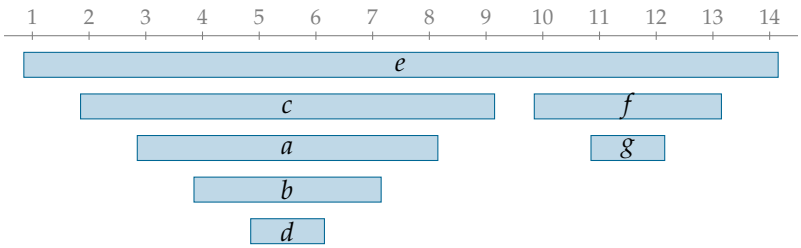


Figura 17.2. Os intervalos de vida $[PE, PS]$ da busca da Figura 17.1, desenhados sobre a linha do tempo. Escrevendo "(" na entrada e ")" na saída de cada vértice, obtém-se $(e\ (c\ (a\ (b\ (d\ d)\ b)\ a)\ c)\ (f\ (g\ g)\ f)\ e)$: os parênteses se aninham corretamente.

17.3 Aplicações

1. **Conexidade.** O número de vezes que o laço externo chama P é o número de componentes conexas; G é conexo se e somente se esse número é 1.
2. **Caminho de v a w.** Execute P(v). Se ao final $PE(w) = 0$, não há caminho; senão, suba de w até v pelos ponteiros pai, colocando cada vértice à esquerda do caminho. Esse caminho **não** é necessariamente o mais curto; para isso, use a busca em largura. Um labirinto pode ser tratado da mesma forma, modelado como um grafo.
3. **Ciclo.** Se a busca não encontra nenhuma aresta de retorno, G é acíclico, e as arestas de árvore formam uma floresta geradora. Se encontra uma aresta de retorno vw, então w é ancestral de v, e o ciclo é obtido subindo de v até w pelos ponteiros pai e fechando com a aresta vw.
4. **Bipartição.** G é bipartido (2-colorível) se e somente se não contém ciclo ímpar. Durante a busca, dê à raiz a cor 0 e, a cada aresta de árvore, faça $cor(w) := 1 - cor(v)$. Uma aresta de retorno com $cor(w) = cor(v)$ fecha um ciclo ímpar, e então o grafo não é bipartido.

Exemplo 17.2 (As aplicações no grafo da Figura 17.1)**. Conexidade**: a chamada P(e) alcança os sete vértices, e o laço externo não faz mais nenhuma chamada: o grafo é conexo. Num grafo com as componentes $\{1,2\}$ e $\{3,4,5\}$, o laço faria duas chamadas, uma para cada componente. **Caminho**: para ir de e até a, sobe-se de a pelos pais, $pai(a) = c$ e $pai(c) = e$, e o caminho é e, c, a. **Ciclo**: a primeira aresta de retorno encontrada é bc; subindo de b até c pelos pais ($b \to a \to c$) e fechando com bc, obtém-se o ciclo c, a, b, c. **Bipartição**: com $cor(e) = 0$, a árvore dá $cor(c) = 1$, $cor(a) = 0$, $cor(b) = 1$ e $cor(d) = 0$. A aresta de retorno bc liga dois vértices de cor 1: o ciclo c, a, b é ímpar, e o grafo não é bipartido.

17.4 Articulações, pontes e blocos

Uma **articulação** é um vértice cuja remoção aumenta o número de componentes conexas. Um **bloco** é um subgrafo maximal conexo e sem articulações; um bloco com uma única aresta é uma **ponte**, e os demais blocos são subgrafos biconexos maximais, em que quaisquer dois vértices são ligados por dois caminhos internamente disjuntos. Os blocos particionam as *arestas* do grafo; já um vértice pode estar em vários blocos, e isso acontece exatamente quando ele é uma articulação. Pensando no grafo como uma rede, articulações são nós críticos e pontes são ligações críticas.

Para encontrá-los, a busca calcula, para cada vértice v, o valor $back(v)$: a menor profundidade de entrada que se pode alcançar a partir de v descendo por arestas da árvore e usando, no máximo, uma aresta de retorno.

Teorema 17.3 (Articulações)**.** *Seja T uma árvore de profundidade de G. A raiz de T é articulação se e somente se tem pelo menos dois filhos em T. Um vértice v que não é a raiz é articulação se e somente se*

tem um filho w com $back(w) \geq PE(v)$; nesse caso, nenhum descendente de w "escapa" para cima de v, e w é chamado de demarcador *de v.*

Os blocos são obtidos com uma pilha de arestas, como no pseudocódigo: ao voltar de um filho w com $back(w) \geq PE(v)$, desempilham-se as arestas até vw, que formam um bloco. As **pontes** são as arestas de árvore vw com $back(w) > PE(v)$. Tudo isso custa $O(n + m)$.

Intuição

Se v não é a raiz e for removido, a subárvore de um filho w só continua ligada ao resto do grafo se algum vértice dela tiver uma aresta de retorno que pule *por cima* de v, para um ancestral próprio de v. O valor $back(w)$ mede exatamente quão alto a subárvore de w consegue "alcançar": se $back(w) < PE(v)$, há uma rota de fuga; se $back(w) \geq PE(v)$, não há, e v é uma articulação. Para uma aresta vw ser ponte, a exigência é mais forte: nem o próprio v pode ser alcançado sem a aresta, isto é, $back(w) > PE(v)$.

No exemplo da Figura 17.1, todos os vértices têm $back(v) = 1$. Com a raiz e, a regra da raiz identifica e como articulação, pois ele tem dois filhos, c e f; os blocos são $\{a, b, c, d, e\}$ e $\{e, f, g\}$. Se a busca começar em a, o vértice e deixa de ser a raiz, e passa a ser identificado pelo outro caso do teorema: ele tem o filho f com $back(f) = 5 \geq PE(e) = 5$.

Exemplo 17.4 (Articulações, pontes e blocos)**.** O grafo da Figura 17.3 tem um quadrado a, b, c, d, uma aresta de, um triângulo e, f, g e uma aresta pendente bh. A busca a partir de a, com os vizinhos em ordem alfabética, desce a, b, c, d, e, f, g em linha ($PE = 1, \ldots, 7$), encontra as arestas de retorno ge e da, volta até b e só então entra em h ($PE(h) = 13$). Os valores de $back$ são 1 para a, b, c, d (por causa da aresta da), 5 para e, f, g (por causa de ge) e 13 para h.

- e é articulação: o filho f tem $back(f) = 5 \geq PE(e) = 5$; o triângulo não escapa de e.
- d é articulação: o filho e tem $back(e) = 5 \geq PE(d) = 4$; e, como $5 > 4$, a aresta de é uma **ponte**.
- b é articulação: o filho h tem $back(h) = 13 \geq PE(b) = 2$, e bh também é ponte.
- c não é articulação: o filho d tem $back(d) = 1 < PE(c) = 3$, pela aresta de retorno da. E a raiz a tem um único filho, b.

Os blocos, na ordem em que a pilha de arestas os imprime, são $\{e, f, g\}$, $\{d, e\}$, $\{b, h\}$ e $\{a, b, c, d\}$.

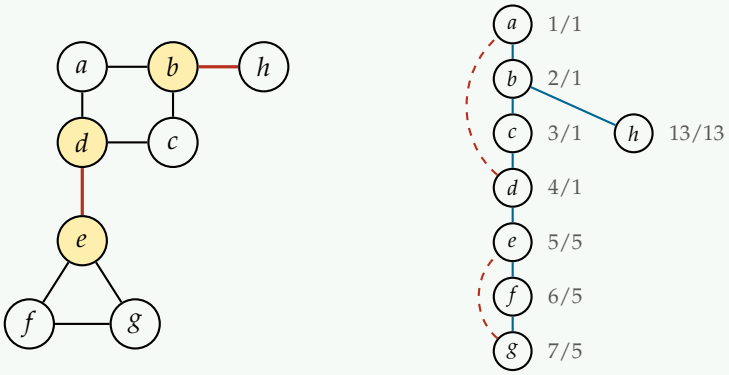


Figura 17.3. À esquerda, o grafo do Exemplo 17.4, com as articulações em amarelo e as pontes em vermelho. À direita, a árvore de profundidade a partir de a, com as arestas de retorno tracejadas e os rótulos $PE/back$.

Armadilhas da busca em profundidade

- Esquecer o laço externo: num grafo desconexo, P(v) só alcança a componente de v.
- Num grafo não direcionado, a aresta vw com $w = pai(v)$ *não* é de retorno: é a própria aresta de árvore, vista do outro extremo.
- O caminho encontrado pela busca em profundidade não é mínimo.
- Para a raiz, a regra das articulações é "pelo menos dois filhos", e não o teste com $back$: na raiz, $back(w) \geq PE(\text{raiz}) = 1$ é sempre verdadeiro.

Pontos-chave

Recursão ou pilha; PE e PS; intervalos de vida aninhados (descendente $\iff$ intervalo contido). Em grafos não direcionados, só há arestas de árvore e de retorno, e há ciclo $\iff$ há aresta de retorno. Aplicações: componentes, caminhos, ciclos, bipartição, articulações (raiz com pelo menos dois filhos, ou filho w com $back(w) \geq PE(v)$), pontes ($back(w) > PE(v)$) e blocos (pilha de arestas). Tudo em $O(n + m)$.

17.5 Exercícios

Exercícios do curso

17.1 [Slides] resposta: p. 194
Dado um grafo conexo G e uma árvore geradora T de G, decida se T é uma árvore de profundidade de G, isto é, se existe uma busca em profundidade em G que produz T.

17.2 [Slides] resposta: p. 194
Simule o cálculo dos blocos com a pilha de arestas no grafo da figura (raiz e, vizinhos em ordem alfabética), mostrando o conteúdo da pilha quando cada bloco é impresso.

17.3 [Slides] resposta: p. 194
Faça pequenos acréscimos ao procedimento com $back(v)$ para listar as articulações e as pontes do grafo.

17.4 [Slides] resposta: p. 194
Verifique se cada bloco tem o seu próprio demarcador, isto é, se o número de demarcadores é igual ao número de blocos.

17.5 [Slides] resposta: p. 194
Descreva como a busca em profundidade resolve: (a) contar as componentes conexas; (b) encontrar um caminho de v a w; (c) encontrar um ciclo ou concluir que o grafo é acíclico; (d) decidir se o grafo é 2-colorível.

Exercícios complementares

17.6 [Complementar] resposta: p. 195
Execute a busca em profundidade, com o cálculo de $back$, a partir do vértice 1 no grafo com as arestas 12, 13, 23, 34, 45, 46, 56 e 67 (vizinhos em ordem crescente). Dê PE, PS, $back$, as arestas de retorno, as articulações, as pontes e os blocos.

17.7 [Complementar] resposta: p. 195
Prove que, numa busca em profundidade num grafo *não direcionado*, não existem arestas de cruzamento: toda aresta liga um vértice a um ancestral ou a um descendente.

17.8 [Complementar] resposta: p. 195
Mostre que um grafo não direcionado com $m \geq n$ arestas tem um ciclo, e use isso para decidir se há ciclo em tempo $O(n)$, independentemente de m.

17.9 [Complementar] resposta: p. 195
Escreva a busca em profundidade sem recursão, com uma pilha explícita, mantendo os mesmos valores de PE e PS da versão recursiva.

17.10 [Complementar] resposta: p. 195
Execute a busca em profundidade a partir do vértice 1 no grafo com as arestas 12, 13, 24, 25, 36, 37, 48, 58, 67 e 78 (vizinhos em ordem crescente). Dê PE, PS, as arestas de árvore e as de retorno. O grafo tem articulações?

17.11 [Complementar] resposta: p. 195
Prove o teorema dos parênteses: para quaisquer vértices u e v, os intervalos $[PE(u), PS(u)]$ e $[PE(v), PS(v)]$ são disjuntos ou um está contido no outro.

17.12 [Complementar ★] resposta: p. 195
Prove o *teorema do caminho livre*: v é descendente de u na floresta de profundidade se e somente se, no instante $PE(u)$, existe um caminho de u a v formado apenas por vértices ainda não alcançados pela busca.

17.13 [Complementar] resposta: p. 195
Quantas componentes conexas tem o grafo com os vértices $1, \ldots, 9$ e as arestas 12, 23, 31, 45, 67, 78, 89 e 96? Quais vértices são raízes da floresta de profundidade, se a busca percorre os vértices em ordem crescente?

17.14 [Complementar] resposta: p. 195
No grafo do Exercício 17.10, use a primeira aresta de retorno encontrada para exibir um ciclo.

17.15 [Complementar] resposta: p. 195
Use a 2-coloração por busca em profundidade para decidir se é bipartido o grafo com as arestas 12, 23, 34, 41 e 13. Indique a aresta que revela o ciclo ímpar, se houver.

17.16 [Complementar] resposta: p. 195
Quais são as pontes e as articulações de uma árvore? E de um ciclo? E de um caminho com $n \geq 3$ vértices?

17.17 [Complementar] resposta: p. 195
Encontre as articulações, as pontes e os blocos do grafo com as arestas 12, 23, 31, 34, 45, 56, 64 e 47.

17.18 [Complementar] resposta: p. 195
Um grafo é *biconexo* se é conexo e não tem articulações. Como decidir isso em $O(n + m)$? Mostre que, num grafo biconexo com pelo menos 3 vértices, toda aresta está em algum ciclo.

17.19 [Complementar] resposta: p. 195
Dê um exemplo de grafo e de vértices v e w em que o caminho de v a w encontrado pela busca em profundidade é mais longo que o caminho mínimo.

17.20 [Complementar ★] resposta: p. 195
Um *circuito euleriano* percorre cada aresta exatamente uma vez e volta ao início. Sabendo que um grafo conexo tem circuito euleriano se e somente se todos os vértices têm grau par, mostre como encontrar o circuito em $O(n + m)$ com uma busca que avança por arestas ainda não usadas.

18

Busca em profundidade em digrafos

Neste capítulo

- A busca em profundidade em digrafos e os quatro tipos de aresta: árvore, retorno, avanço e cruzamento.
- A caracterização dos tipos de aresta pelos intervalos de vida e o teste de aciclicidade.
- As componentes fortemente conexas, calculadas numa única busca com o valor $old(v)$.
- A ordenação topológica de digrafos acíclicos.

Em grafos direcionados, a mesma busca em profundidade do capítulo anterior produz quatro tipos de aresta, e não apenas dois; os intervalos de vida dos vértices continuam a caracterizá-los. Com essa classificação, decide-se se um digrafo tem ciclo, obtém-se uma ordenação topológica de um digrafo acíclico e calculam-se as componentes fortemente conexas — tudo em tempo linear.

18.1 Classificação das arestas

A busca é a mesma do Capítulo 17, percorrendo a vizinhança de saída $N_{out}(v)$ e com o laço externo, de modo que a floresta de profundidade alcança todos os vértices. Quando a busca, a partir de v, examina uma aresta vw, há quatro possibilidades.

Classificação das arestas na busca em profundidade em digrafos

```
Procedimento P(v)
    t := t + 1; PE(v) := t
    para cada w ∈ N_out(v) faça
        se PE(w) = 0 então
            vw é de árvore; pai(w) := v; P(w)
        senão se PS(w) = 0 então
            vw é de retorno          % w ainda está ativo: é ancestral de v
        senão se PE(v) < PE(w) então
            vw é de avanço           % w é um descendente que já terminou
        senão
            vw é de cruzamento       % w terminou antes de v começar
    t := t + 1; PS(v) := t
```

Tipo de vw	Caracterização pelos intervalos	Observação
Árvore	$I(v) \supset I(w)$ e $PE(w) = 0$ na hora da visita	liga pai e filho
Avanço	$I(v) \supset I(w)$ e $PE(w) \neq 0$ na hora da visita	vai a um descendente que não é filho
Retorno	$I(v) \subset I(w)$	vai a um ancestral e **fecha um ciclo direcionado**
Cruzamento	$I(v)$ inteiro à **direita** de $I(w)$	vai "para trás" no tempo

Nunca há uma aresta vw com $I(v)$ inteiro à esquerda de $I(w)$: quando v examina a aresta vw, o vértice w ou já foi alcançado ou será visitado como descendente de v. Como toda aresta de retorno fecha um ciclo direcionado, e todo ciclo contém uma aresta de retorno (a que chega ao primeiro vértice do ciclo alcançado pela busca), vale: **um digrafo é acíclico se e somente se uma busca em profundidade — qualquer uma — não produz arestas de retorno**.

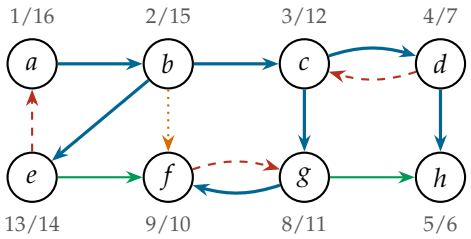


Figura 18.1. Busca a partir de a, com os vizinhos em ordem alfabética; os rótulos são PE/PS. Em azul, as arestas de árvore; em vermelho tracejado, as de retorno (dc, ea e fg); em laranja pontilhado, a de avanço (bf); em verde, as de cruzamento (ef e gh).

Exemplo 18.1 (Os quatro tipos de aresta)**.** Na busca da Figura 18.1, a sequência é: a (1), b (2), c (3), d (4); de d, a aresta dc vai a c, que está ativo: **retorno**; h entra (5) e sai (6), e d sai (7). De volta a c, entra g (8), e de g entra f (9); a aresta fg volta a g, ativo: retorno. f sai (10). De g, a aresta gh vai a h, que já terminou e começou antes de g: **cruzamento**. Saem g (11) e c (12). De volta a b, a aresta be leva a e (13), cujas arestas são ea (retorno, pois a está ativo) e ef (cruzamento, pois f terminou no instante 10, antes de e começar). e sai (14). Por fim, b examina bf: f já terminou, mas entrou *depois* de b ($PE(f) = 9 > PE(b) = 2$), logo é um descendente de b que não é filho: **avanço**. A Figura 18.2 mostra os intervalos e três dessas arestas.

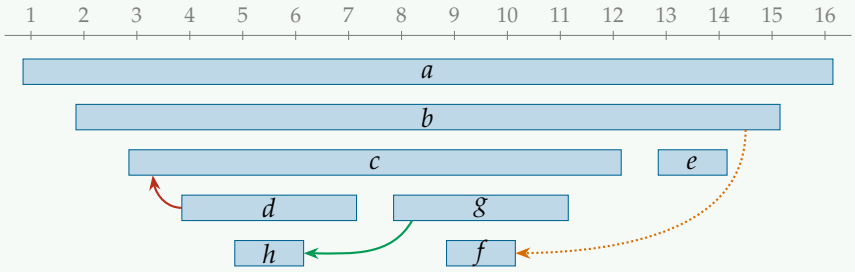


Figura 18.2. Os intervalos de vida da busca da Figura 18.1, um vértice por barra, com a profundidade na árvore crescendo para baixo. A aresta de retorno dc (vermelha) vai de um intervalo para outro que o contém; a de avanço bf (laranja), para um intervalo contido; a de cruzamento gh (verde), para um intervalo inteiramente à esquerda.

Exemplo 18.2 (Testando se há ciclo)**.** O digrafo da Figura 18.1 tem ciclos: cada aresta de retorno fecha um, por exemplo $c \to d \to c$ (com dc) e $a \to b \to e \to a$ (com ea). Já o digrafo com as arestas $1 \to 2$, $1 \to 3$, $2 \to 4$ e $3 \to 4$ é acíclico: a busca a partir de 1 encontra $1 \to 2$ e $2 \to 4$ (árvore), $1 \to 3$ (árvore) e $3 \to 4$, que vai a um vértice já terminado e não descendente de 3 (cruzamento). Nenhuma aresta de retorno: não há ciclo. Repare que, sem a direção, as quatro arestas formariam um ciclo; num digrafo, só os ciclos *direcionados* contam.

18.2 Componentes fortemente conexas

Uma **componente fortemente conexa** (CFC) de um digrafo é um subdigrafo maximal em que, para quaisquer dois vértices v e w, existe um caminho direcionado de v para w *e* um de w para v. As CFC particionam os *vértices*; algumas arestas não pertencem a nenhuma CFC, e essas são,

numa busca, arestas de árvore, de avanço ou de cruzamento. As componentes não dependem da ordem da busca, embora o tipo de cada aresta dependa.

> **Intuição**
>
> Pense numa cidade com ruas de mão única. Uma componente fortemente conexa é um bairro em que se pode ir de qualquer esquina a qualquer outra *e voltar*. Entre bairros diferentes, as ruas só permitem ir num sentido: quem sai de um bairro para outro não consegue mais voltar. Contraindo cada bairro num ponto, sobra um mapa sem ciclos — o *grafo das componentes*.

Para calculá-las numa única busca, define-se $old(v)$ como a profundidade de entrada do vértice mais antigo, **na mesma CFC** que v, que pode ser alcançado a partir de v descendo por arestas de árvore e usando, no máximo, uma aresta de retorno ou de cruzamento. Um vértice v é **forte** — é a "raiz" da sua componente na floresta — se $old(v) = PE(v)$ no momento em que v sai da busca. Os vértices são empilhados numa pilha Q ao serem alcançados; ao encontrar um vértice forte, desempilham-se todos os vértices até ele, que formam uma componente.

Componentes fortemente conexas: $O(n + m)$

```
Procedimento P(v)
    t := t + 1; PE(v) := t; old(v) := PE(v); empilhe v em Q
    para cada w ∈ N_out(v) faça
        se PE(w) = 0 então
            pai(w) := v; P(w); old(v) := min{old(v), old(w)}        % árvore
        senão se PS(w) = 0 então
            old(v) := min{old(v), PE(w)}                            % retorno
        senão se PE(v) < PE(w) então
            nada a fazer                                            % avanço
        senão se w está em Q então
            old(v) := min{old(v), PE(w)}        % cruzamento dentro de uma componente aberta
    t := t + 1; PS(v) := t
    se old(v) = PE(v) então
        desempilhe de Q todos os vértices até v, inclusive: eles formam uma CFC
```

Exemplo 18.3 (Componentes fortemente conexas)**.** Na busca da Figura 18.1, os valores de old são 1 para a, b e e; 3 para c e d; 8 para g e f; e 5 para h. Os vértices fortes, na ordem em que saem da busca, são h, g, c e a, e as componentes são $\{h\}$, $\{f, g\}$, $\{c, d\}$ e $\{a, b, e\}$. A aresta ef é de cruzamento, mas f já saiu de Q — a sua componente já tinha sido fechada —, e por isso ela não altera $old(e)$. A tabela mostra a pilha Q nos momentos em que um vértice forte sai:

instante	sai	Q antes (base à esquerda)	componente desempilhada
6	h	a, b, c, d, h	$\{h\}$
11	g	a, b, c, d, g, f	$\{f, g\}$
12	c	a, b, c, d	$\{c, d\}$
16	a	a, b, e	$\{a, b, e\}$

Note que d fica na pilha depois de sair da busca (instante 7), porque $old(d) = 3 < PE(d) = 4$: ele pertence à componente de c, que só é fechada quando c sai. A Figura 18.3 mostra o grafo das componentes, que é acíclico.

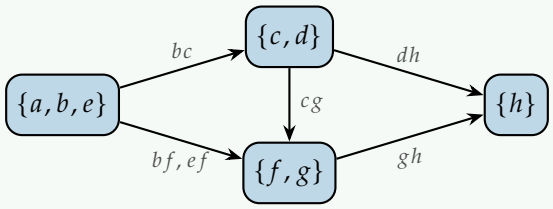


Figura 18.3. O grafo das componentes (condensação) do digrafo da Figura 18.1: cada componente fortemente conexa vira um vértice, e as arestas entre componentes diferentes viram arestas entre esses vértices. O resultado é sempre acíclico.

18.3 Ordenação topológica

Uma **ordenação topológica** de um digrafo acíclico é uma ordem $v_1, \ldots, v_n$ dos vértices tal que toda aresta $v_i v_j$ tem $i < j$. Ela aparece sempre que há tarefas com dependências: se cada aresta $a \to b$ indica que b só pode ser feita depois de a, uma ordenação topológica é uma sequência viável de execução. Desenhando os vértices numa linha, na ordem topológica, todas as arestas apontam para a direita.

Algoritmo. Execute uma busca em profundidade; se houver uma aresta de retorno, o digrafo tem ciclo e não há ordenação. Senão, ordene os vértices por PS **decrescente**. Para obter $O(n+m)$, basta colocar cada vértice na frente de uma lista no momento em que ele recebe o seu PS, o que dispensa a ordenação.

Teorema 18.4. *Num digrafo acíclico, toda aresta vw satisfaz $PS(w) < PS(v)$.*

Demonstração. Não há arestas de retorno. Se vw é de árvore ou de avanço, w é descendente de v e sai antes dele. Se é de cruzamento, $I(w)$ está inteiramente à esquerda de $I(v)$. □

Uma outra forma de ver a correção: o vértice de menor PS é um *sumidouro* (não tem arestas de saída) e pode ser o último da ordem; retirando-o, o de menor PS entre os restantes é um sumidouro do digrafo que sobrou, e assim por diante.

Exemplo 18.5. No digrafo acíclico com as arestas $a \to c$, $a \to d$, $d \to c$, $b \to c$, $e \to b$, $e \to f$, $e \to g$, $f \to g$, $h \to f$, $i \to h$, $i \to j$, $j \to h$ e $j \to g$, uma busca com as raízes em ordem alfabética dá, por PS decrescente, a ordem $i, j, h, e, f, g, b, a, d, c$ (Figura 18.4). Com as raízes na ordem i, c, b, e, d, a, obtém-se $a, d, e, b, c, i, j, h, f, g$. As duas são válidas: a ordenação topológica não é única.

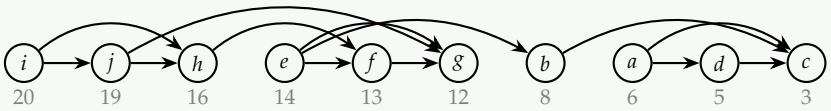


Figura 18.4. O digrafo acíclico do exemplo, com os vértices na ordem topológica $i, j, h, e, f, g, b, a, d, c$ (por PS decrescente; os números são os PS). Todas as arestas apontam para a direita.

Uma alternativa sem busca. Todo digrafo acíclico tem um sumidouro — o último vértice de um caminho de comprimento máximo — e uma fonte. Retirar repetidamente uma fonte e colocá-la no fim da ordem (ou um sumidouro, no início) também produz uma ordenação topológica. Com contadores de grau de entrada e uma fila dos vértices cujo contador chegou a zero, isso custa $O(n+m)$; é o *algoritmo de Kahn*.

Exemplo 18.6 (O algoritmo de Kahn)**.** No mesmo digrafo, os graus de entrada são zero para a, e e i, que começam na fila, nessa ordem. Sai a: os contadores de c e d diminuem, e d chega a zero e entra na fila. Sai e: b chega a zero. Sai i: j chega a zero (h ainda depende de j). Sai d;

sai b, e c chega a zero; sai j, e h chega a zero; saem c, h (liberando f), f (liberando g) e g. A ordem é $a, e, i, d, b, j, c, h, f, g$, diferente das obtidas pela busca, e também válida. Se a fila esvaziar antes de todos os vértices saírem, os que sobraram estão em ciclos ou dependem de ciclos: o digrafo não é acíclico.

Armadilhas em digrafos

- "Acíclico" refere-se a ciclos *direcionados*: arestas de cruzamento e de avanço não indicam ciclo; só as de retorno indicam.
- O tipo de cada aresta depende da ordem da busca; as componentes fortemente conexas, não.
- A ordenação topológica é por PS *decrescente*, e não por PE; e ela não é única.
- No cálculo das componentes, uma aresta de cruzamento só atualiza $old(v)$ se o outro extremo ainda está na pilha Q, isto é, numa componente que ainda não foi fechada.

Pontos-chave

Quatro tipos de aresta: árvore e avanço ($I(v) \supset I(w)$), retorno ($I(v) \subset I(w)$, fecha ciclo) e cruzamento ($I(v)$ à direita de $I(w)$). Acíclico $\iff$ sem arestas de retorno. Ordenação topológica: PS decrescente, em $O(n + m)$. Componentes fortemente conexas: quando $old(v) = PE(v)$ na saída, desempilha-se uma componente. Tudo em $O(n + m)$.

18.4 Exercícios

Exercícios do curso

18.1 [Slides] resposta: p. 195
Prove que todo digrafo acíclico tem um sumidouro. Use isso para um algoritmo alternativo de ordenação topológica: repetidamente, encontre um sumidouro, retire-o e coloque-o no início da ordenação. É possível implementá-lo em tempo $O(n + m)$?

18.2 [Slides] resposta: p. 196
Por que ordenar os vértices por PS decrescente dá uma ordenação topológica? Aplique ao digrafo acíclico do exemplo da §18.3.

18.3 [Slides] resposta: p. 196
Uma aresta vw pode ser de cruzamento numa busca e de avanço em outra? Dê um exemplo. Por que, mesmo assim, as componentes fortemente conexas encontradas são sempre as mesmas?

Exercícios complementares

18.4 [Complementar] resposta: p. 196
No digrafo com as arestas $1 \to 2$, $1 \to 3$, $2 \to 3$, $2 \to 4$, $3 \to 1$, $4 \to 5$, $5 \to 6$, $6 \to 4$, $7 \to 3$ e $7 \to 6$, execute a busca em profundidade a partir de 1, com raízes e vizinhos em ordem crescente. Dê PE, PS, o tipo de cada aresta, os valores old e as componentes fortemente conexas.

18.5 [Complementar] resposta: p. 196
Para se vestir: cueca antes de calça e de sapatos; calça antes de cinto e de sapatos; camisa antes de cinto e de gravata; cinto e gravata antes do paletó; meias antes dos sapatos; o relógio é independente. Dê uma ordenação topológica pela busca em profundidade, com raízes e vizinhos em ordem alfabética.

18.6 [Complementar] resposta: p. 196
O *grafo das componentes* (condensação) tem um vértice por componente fortemente conexa e uma aresta $C \to C'$ se há uma aresta de algum vértice de C para algum de C'. Prove que ele é acíclico.

18.7 [Complementar] resposta: p. 196
Mostre que um digrafo acíclico tem uma *única* ordenação topológica se e somente se tem um caminho hamiltoniano, isto é, um caminho que passa por todos os vértices. Como testar isso em $O(n + m)$?

18.8 [Complementar] resposta: p. 196
Um curso tem n disciplinas e m pré-requisitos. Como decidir se é possível cursar todas e, se for, qual o menor número de semestres, se cada disciplina leva um semestre e não há limite de disciplinas por semestre?

18.9 [Complementar] resposta: p. 196
No digrafo com as arestas $1 \to 2$, $1 \to 4$, $2 \to 3$, $3 \to 4$, $4 \to 2$, $5 \to 3$, $5 \to 6$ e $6 \to 5$, execute a busca em profundidade com raízes e vizinhos em ordem crescente. Classifique todas as arestas e dê PE e PS.

18.10 [Complementar] resposta: p. 196
Calcule, com o procedimento da §18, os valores old e as componentes fortemente conexas do digrafo do exercício anterior, na ordem em que são encontradas.

18.11 [Complementar] resposta: p. 196
Execute o algoritmo de Kahn no digrafo acíclico com as arestas $a \to b$, $a \to c$, $b \to d$, $c \to d$, $d \to e$ e $f \to c$, escolhendo sempre, entre as fontes disponíveis, a de menor nome. Qual a ordem obtida?

18.12 [Complementar] resposta: p. 196
Mostre como, ao encontrar uma aresta de retorno vw numa busca em profundidade num digrafo, imprimir um ciclo direcionado.

18.13 [Complementar] resposta: p. 196
Mostre que um digrafo é fortemente conexo se e somente se, a partir de um vértice qualquer s, todos os vértices são alcançáveis no digrafo e também no digrafo com as arestas invertidas. Conclua que é possível testar a conexidade forte com duas buscas.

18.14 [Complementar ★] resposta: p. 196
O algoritmo de Kosaraju faz uma busca em profundidade no digrafo, registrando a ordem de saída dos vértices; depois, faz buscas no digrafo invertido, iniciando-as em ordem decrescente de PS. Mostre que cada árvore da segunda busca é uma componente fortemente conexa.

18.15 [Complementar] resposta: p. 196
Quantas ordenações topológicas tem o digrafo acíclico com as arestas $a \to c$, $b \to c$, $c \to d$ e $c \to e$? Liste-as.

18.16 [Complementar] resposta: p. 196
No digrafo acíclico com as arestas $s \to a$ (3), $s \to b$ (2), $a \to c$ (4), $a \to t$ (2), $b \to c$ (6), $b \to t$ (3) e $c \to t$ (1), encontre o caminho de maior peso de s a t processando os vértices em ordem topológica.

18.17 [Complementar] resposta: p. 196
As disciplinas $C_1, C_2, C_3, A_1, P_1, P_2$ têm os pré-requisitos $C_1 \to C_2$, $C_2 \to C_3$, $C_1 \to A_1$, $A_1 \to C_3$ e $P_1 \to P_2$. Qual o menor número de semestres para cursar todas, e em que semestre cada uma pode ser cursada?

18.18 [Complementar ★] resposta: p. 197
Num digrafo cujo grafo das componentes tem f fontes e s sumidouros (e mais de uma componente), mostre que são necessárias pelo menos $\max\{f, s\}$ novas arestas para torná-lo fortemente conexo.

18.19 [Complementar] resposta: p. 197
Como obter, entre todas as ordenações topológicas, a lexicograficamente menor (comparando os nomes dos vértices)? Qual o custo?

18.20 [Complementar] resposta: p. 197
Por que, num grafo não direcionado, não aparecem arestas de avanço nem de cruzamento, enquanto num digrafo elas aparecem?

A

Tabelas hash

Neste capítulo

- Dicionários, endereçamento direto e a ideia de uma função hash.
- Funções hash clássicas e o problema inevitável das colisões.
- Tratamento de colisões por encadeamento e por endereçamento aberto.
- A complexidade em função do fator de carga, e quando preferir uma tabela hash a uma árvore de busca.

Um **dicionário** guarda pares (chave, valor) e oferece inserção, busca e remoção por chave. As árvores AVL do Capítulo 5 implementam dicionários com todas as operações em $O(\log n)$ e ainda mantêm as chaves em ordem. Quando a ordem não importa, é possível fazer melhor: as **tabelas hash** transformam a chave diretamente num endereço e realizam as operações em tempo $O(1)$ esperado.

A.1 Endereçamento direto e funções hash

Se as chaves são inteiros de um universo pequeno $U = \{0, \ldots, M-1\}$, basta um vetor de tamanho M indexado pela própria chave: é o **endereçamento direto**, com todas as operações em $O(1)$. Mas o universo de chaves costuma ser enorme ou infinito (todos os inteiros de 64 bits, todas as strings), e a memória é finita. Uma **função hash** $h : U \to \{0, \ldots, m-1\}$ transforma cada chave num endereço de uma tabela de m posições, com m da ordem do número de chaves efetivamente guardadas.

Exemplo A.1 (Do endereçamento direto à função hash)**.** Uma turma tem alunos com matrículas de 0 a 9, e estão presentes os de matrícula 2, 5 e 7. Com endereçamento direto, um vetor de 10 posições guarda cada aluno na posição da sua matrícula, e "o aluno 5 está presente?" é uma única consulta (Figura A.1). Se as matrículas fossem números de 9 dígitos, o vetor teria 10^9 posições para guardar 3 alunos. Uma função hash, como $h(k) = k \bmod 7$, comprime o universo: as matrículas 201 345 002, 731 000 006 e 555 777 016 vão para os endereços 5, 2 e 4 de uma tabela com 7 posições.

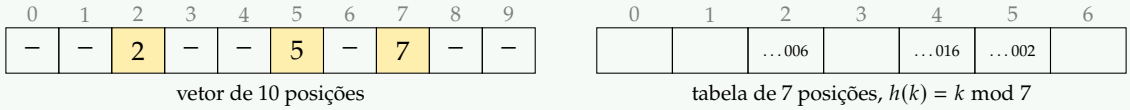


Figura A.1. À esquerda, endereçamento direto: a chave é o próprio endereço. À direita, uma função hash leva chaves de um universo enorme a uma tabela pequena.

Funções clássicas.

- *Divisão*: $h(k) = k \bmod m$, com m primo e longe de potências de 2, para que o resultado dependa de todos os bits da chave.

- *Multiplicação*: $h(k) = \lfloor m\,(kA \bmod 1)\rfloor$, com uma constante $0 < A < 1$; um valor muito usado é $A \approx (\sqrt{5}-1)/2$. Aqui o valor de m é pouco crítico, e pode ser uma potência de 2.
- *Dobra* e *meio do quadrado*: combinam partes da chave, ou extraem os dígitos centrais de k^2.
- Para *strings*, a função polinomial $h(s) = \left(\sum_i s_i\, b^i\right) \bmod m$, calculada pela regra de Horner.
- *Hashing universal*: a função é sorteada, no início, de uma família, como $h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$ com p primo. Isso garante bom desempenho esperado para *qualquer* conjunto de chaves, sem depender de a entrada ser "aleatória".

Exemplo A.2 (Calculando endereços)**. Divisão**, com $m = 13$: $h(100) = 100 \bmod 13 = 9$ e $h(2026) = 2026 \bmod 13 = 11$. **Multiplicação**, com $m = 16$ e $A = 0{,}6180339887$: $100A \approx 61{,}8034$, cuja parte fracionária é 0,8034, e $h(100) = \lfloor 16 \cdot 0{,}8034\rfloor = 12$; já $101A \approx 62{,}4214$, e $h(101) = \lfloor 16 \cdot 0{,}4214\rfloor = 6$. Chaves vizinhas caem em endereços distantes. **String**, com as letras valendo $\mathtt{a} = 1, \ldots, \mathtt{z} = 26$, $b = 31$ e $m = 97$: para "gato" (7, 1, 20, 15), a regra de Horner faz $h := 7$; $h := (7 \cdot 31 + 1) \bmod 97 = 24$; $h := (24 \cdot 31 + 20) \bmod 97 = 85$; $h := (85 \cdot 31 + 15) \bmod 97 = 31$. Em cada passo, h fica menor que m, e nunca é preciso calcular 31^3. **Universal**, com $p = 17$ e $m = 6$: com $(a, b) = (3, 4)$, as chaves 5 e 9 colidem, pois $((3 \cdot 5 + 4) \bmod 17) \bmod 6 = 2$ e $((3 \cdot 9 + 4) \bmod 17) \bmod 6 = 2$; com $(a, b) = (5, 1)$, elas vão para os endereços 3 e 0. Um adversário que não sabe qual função foi sorteada não consegue escolher chaves que colidam sempre.

A.2 Colisões

Uma **colisão** ocorre quando duas chaves diferentes recebem o mesmo endereço: $h(k_1) = h(k_2)$ com $k_1 \neq k_2$. Colisões são inevitáveis, pelo princípio da casa dos pombos, pois $|U| > m$. E elas aparecem muito antes de a tabela ficar cheia: é o "paradoxo do aniversário", pelo qual, numa turma de apenas 23 pessoas, a chance de duas fazerem aniversário no mesmo dia (365 "endereços") já passa de 50%. Há duas famílias de soluções.

Encadeamento. No **encadeamento exterior**, cada posição da tabela aponta para uma lista com as chaves que receberam aquele endereço. Inserir é colocar a chave no início da lista, em $O(1)$; buscar e remover percorrem a lista. No **encadeamento interior**, as listas são formadas dentro da própria tabela, usando uma área de colisões ou ligando posições livres por ponteiros.

Exemplo A.3 (Encadeamento exterior)**.** Inserimos 12, 44, 13, 88, 23, 94, 11, 39, 20, 16, 5, nessa ordem, numa tabela de $m = 11$ posições com $h(k) = k \bmod 11$ (Figura A.2). As chaves 44, 88 e 11 vão todas para o endereço 0, e como cada uma entra no início da lista, ela fica $11 \to 88 \to 44$. Buscar 44 percorre a lista inteira (3 comparações); buscar 33, que também iria para o endereço 0, faz 3 comparações e conclui que a chave não está lá. Buscar uma chave de endereço 3 não faz nenhuma comparação: a lista está vazia.

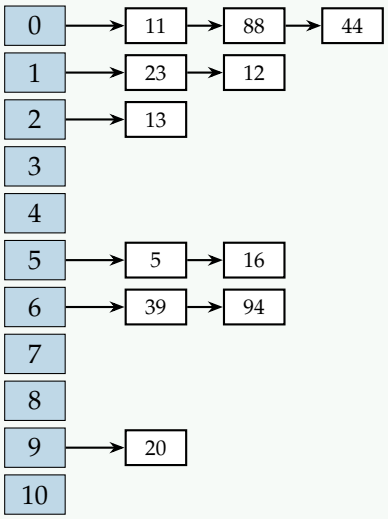


Figura A.2. Encadeamento exterior com $m = 11$. Cada posição aponta para a lista das chaves que receberam aquele endereço; as inserções são feitas no início da lista.

Endereçamento aberto. Todas as chaves ficam na própria tabela. Em caso de colisão, examina-se uma sequência de posições, chamada sequência de *sondagem*, até encontrar uma livre:

- *sondagem linear*: $h(k, i) = (h(k) + i) \bmod m$, simples, mas sujeita a **agrupamento primário**: longas sequências de posições ocupadas, que tendem a crescer;
- *sondagem quadrática*: $h(k, i) = (h(k) + c_1 i + c_2 i^2) \bmod m$, que reduz o agrupamento;
- *duplo hashing*: $h(k, i) = (h_1(k) + i\, h_2(k)) \bmod m$, com $h_2(k)$ primo com m, para que a sequência visite todas as posições.

No endereçamento aberto, uma remoção não pode simplesmente esvaziar a posição, pois isso interromperia as sequências de sondagem de outras chaves; a posição é marcada como "removida".

Exemplo A.4 (Sondagem linear e agrupamento)**.** Inserimos $18, 41, 22, 44, 59, 32, 31, 73$, nessa ordem, numa tabela de $m = 13$ posições com $h(k) = k \bmod 13$ e sondagem linear (Figura A.3). As chaves 18, 41, 22 e 59 vão direto para os endereços 5, 2, 9 e 7. O 44 tem $h = 5$, ocupado pelo 18, e vai para 6 (2 sondagens). O 32 tem $h = 6$: as posições 6 e 7 estão ocupadas, e ele vai para 8 (3 sondagens). O 31 tem $h = 5$ e só encontra lugar na posição 10, depois de 6 sondagens; o 73, de $h = 8$, vai para 11, com 4. As chaves formam um **agrupamento** contínuo das posições 5 a 11, e qualquer chave cujo endereço caia nesse trecho vai parar no seu fim, aumentando-o ainda mais.
Se outras funções de sondagem fossem usadas, a chave 31 seguiria outro caminho: com a sondagem quadrática $h(k, i) = (h(k) + i^2) \bmod 13$, a sequência seria $5, 6, 9, 1, \ldots$; com o duplo hashing de $h_2(k) = 1 + (k \bmod 11) = 10$, seria $5, 2, 12, 9, 6, 3, \ldots$ — saltos que espalham as tentativas e evitam o agrupamento.

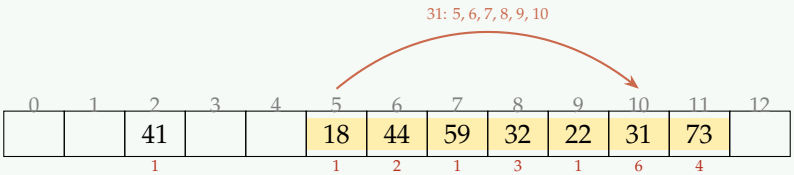


Figura A.3. Sondagem linear com $m = 13$ depois das oito inserções do Exemplo A.4. O número abaixo de cada chave é a quantidade de sondagens que a sua inserção fez; as posições 5 a 11 formam um agrupamento.

Exemplo A.5 (Por que a remoção só marca a posição)**.** Na tabela da Figura A.3, remova o 44, que está na posição 6. Se a posição simplesmente ficasse vazia, uma busca por 32 começaria em $h(32) = 6$, encontraria a posição vazia e concluiria que 32 não está na tabela — embora ele esteja na posição 8, porque na sua inserção as posições 6 e 7 estavam ocupadas. Marcando a posição 6 como "removida", a busca passa por ela e continua até encontrar o 32; uma inserção futura pode reaproveitar a posição.

A.3 Complexidade

Seja $\alpha = n/m$ o **fator de carga**. Supondo que a função hash espalha as chaves de modo uniforme:

- com encadeamento, a busca custa $\Theta(1 + \alpha)$ em média;
- com endereçamento aberto ($\alpha < 1$), uma busca sem sucesso faz no máximo $\frac{1}{1-\alpha}$ sondagens em média.

Mantendo $\alpha = O(1)$ — por exemplo, dobrando o tamanho da tabela e reinserindo as chaves sempre que α passa de um limite, o que custa $O(1)$ amortizado por inserção (Capítulo 3) —, todas as operações custam $O(1)$ **esperado**. O **pior caso**, porém, é $\Theta(n)$: todas as chaves podem cair no mesmo endereço.

Intuição

No encadeamento, as n chaves se espalham por m listas, e uma lista típica tem α chaves: a busca custa o cálculo do hash mais uma lista. No endereçamento aberto, cada sondagem encontra uma posição ocupada com probabilidade de cerca de α; o número esperado de tentativas até achar uma livre é o de uma "moeda" que dá "livre" com probabilidade $1 - \alpha$, isto é, $\frac{1}{1-\alpha}$. Enquanto a tabela tem folga, isso é quase 1; perto de cheia, explode (Figura A.4).

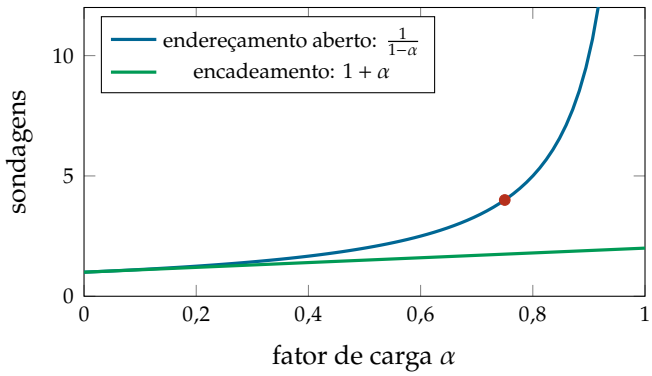


Figura A.4. O custo médio de uma busca sem sucesso em função do fator de carga α: com encadeamento, $1 + \alpha$ (cresce devagar e admite $\alpha > 1$); com endereçamento aberto, $\frac{1}{1-\alpha}$ (explode quando a tabela fica cheia). Com $\alpha = 0{,}75$, o endereçamento aberto faz em média até 4 sondagens.

Exemplo A.6 (Dimensionando a tabela)**.** Uma tabela com encadeamento e $m = 800$ posições guarda $n = 600$ chaves: $\alpha = 0{,}75$, e cada lista tem, em média, menos de uma chave; uma busca sem sucesso examina, em média, 0,75 chave depois de calcular o hash. Se as mesmas 600 chaves estivessem numa tabela com endereçamento aberto de 800 posições, uma busca sem sucesso faria em média até $\frac{1}{1-0{,}75} = 4$ sondagens; com só 650 posições ($\alpha \approx 0{,}92$), seriam até cerca de 13. Por isso, no endereçamento aberto, a tabela é redimensionada bem antes de encher.

Tabela hash ou árvore de busca?

Use uma **tabela hash** quando só forem necessárias inserção, busca e remoção por chave exata e o desempenho médio importar mais que a garantia de pior caso. Use uma **árvore balanceada** (AVL ou rubro-negra) quando for preciso manter a ordem das chaves — mínimo, máximo, sucessor, percurso em ordem, consultas por intervalo — ou quando for exigida uma garantia de $O(\log n)$ no pior caso.

Armadilhas das tabelas hash

- Esperar $O(1)$ no pior caso: o tempo constante é *esperado*, sob a hipótese de uma boa função hash; com chaves escolhidas por um adversário e uma função fixa, tudo pode cair no mesmo endereço.
- Usar uma função que ignora parte da chave (por exemplo, só os primeiros caracteres de strings longas e parecidas): as colisões se multiplicam.
- Deixar a tabela de endereçamento aberto encher: o custo cresce como $\frac{1}{1-\alpha}$, e com $\alpha = 1$ não há onde inserir.
- Remover no endereçamento aberto esvaziando a posição, em vez de marcá-la como "removida".

Pontos-chave

Função hash: chave → endereço. Colisões são inevitáveis; tratam-se por encadeamento (listas) ou por endereçamento aberto (sondagem linear, quadrática ou duplo hashing). Com fator de carga $\alpha = O(1)$, as operações custam $O(1)$ esperado; o pior caso é $\Theta(n)$. Tabelas hash não mantêm ordem.

A.4 Exercícios

Exercícios do curso

A.1 [Pesquisa] resposta: p. 197

Disserte sobre: a definição e a motivação das tabelas hash; definição e exemplos de funções de dispersão clássicas; o problema das colisões; o tratamento de colisões por encadeamento exterior e interior e por endereçamento aberto; e a complexidade de pior caso da busca por uma chave.

Exercícios complementares

A.2 [Complementar] resposta: p. 197

Insira as chaves $10, 22, 31, 4, 15, 28, 17, 88, 59$, nessa ordem, numa tabela de tamanho $m = 11$ com $h(k) = k \bmod 11$, usando (a) sondagem linear; (b) sondagem quadrática com $h(k, i) = (h(k) + i + 3i^2) \bmod 11$; (c) duplo hashing com $h_2(k) = 1 + (k \bmod 10)$.

A.3 [Complementar] resposta: p. 197

Insira $5, 28, 19, 15, 20, 33, 12, 17, 10$ numa tabela com encadeamento exterior, $m = 9$ e $h(k) = k \bmod 9$. Qual o tamanho da maior lista?

A.4 [Complementar] resposta: p. 197

Por que, no método da divisão, convém escolher m primo e longe de potências de 2? Dê um exemplo do que dá errado com $m = 2^r$.

A.5 [Complementar] resposta: p. 197
No endereçamento aberto, por que a remoção não pode simplesmente esvaziar a posição? Como contornar o problema?

A.6 [Complementar] resposta: p. 197
Uma tabela com encadeamento tem $m = 500$ posições e guarda $n = 1000$ chaves. Qual o fator de carga e o comprimento médio das listas? Quantas comparações, em média, faz uma busca sem sucesso?

A.7 [Complementar] resposta: p. 197
Com endereçamento aberto e hashing uniforme, quantas sondagens, em média, faz uma busca sem sucesso quando a tabela está com fator de carga 0,5? E com 0,9? O que isso sugere sobre quando redimensionar a tabela?

A.8 [Complementar] resposta: p. 197
Calcule $h(k) = \lfloor m\,(kA \bmod 1) \rfloor$ com $m = 1000$, $A = (\sqrt{5}-1)/2 \approx 0{,}6180339887$, para $k = 123456$ e $k = 123457$.

A.9 [Complementar] resposta: p. 197
O que é uma família universal de funções hash? Por que sortear a função de uma família universal protege contra entradas escolhidas para causar muitas colisões?

A.10 [Complementar] resposta: p. 197
Escreva o cálculo de $h(s) = \left(\sum_{i=0}^{L-1} s_i\, b^{L-1-i}\right) \bmod m$ pela regra de Horner, sem calcular potências grandes. Por que b e m não devem ter fatores em comum?

A.11 [Complementar] resposta: p. 197
Dado um vetor de n inteiros e um valor x, decida em $O(n)$ esperado se há dois elementos com soma x.

A.12 [Complementar] resposta: p. 197
Encontre, em $O(n)$ esperado, o primeiro elemento que se repete numa sequência de n valores.

A.13 [Complementar] resposta: p. 198
Mostre que encontrar o sucessor de uma chave numa tabela hash pode custar $\Theta(n)$, enquanto numa AVL custa $O(\log n)$. Que outras operações têm a mesma limitação?

A.14 [Complementar] resposta: p. 198
Uma tabela dobra de tamanho e reinsere todas as chaves sempre que o fator de carga passa de $1/2$. Mostre que cada inserção custa $O(1)$ amortizado.

A.15 [Complementar] resposta: p. 198
Explique o agrupamento primário da sondagem linear e por que, quando uma posição fica no fim de um grupo de k posições ocupadas, ela tem probabilidade maior de ser ocupada na próxima inserção.

A.16 [Complementar] resposta: p. 198
No duplo hashing, por que $h_2(k)$ precisa ser primo com m? O que acontece se $m = 10$ e $h_2(k) = 5$?

A.17 [Complementar] resposta: p. 198
Com encadeamento exterior, como remover uma chave em $O(1)$, dado um ponteiro para o seu nó? Que tipo de lista é necessário?

A.18 [Complementar ★] resposta: p. 198
Para um conjunto *fixo* de n chaves, descreva a ideia do *hashing perfeito* em dois níveis, que garante buscas em $O(1)$ no pior caso com memória $O(n)$ esperada.

A.19 [Complementar] resposta: p. 198
Conte quantos valores distintos há numa sequência de n inteiros arbitrários, em $O(n)$ esperado. Como fazer em $O(n \log n)$ no pior caso?

A.20 [Complementar] resposta: p. 198
Uma tabela com endereçamento aberto vai guardar 10^6 chaves, e o fator de carga não deve passar de 0,75. Qual o tamanho mínimo da tabela? Que valor de m você escolheria para usar o método da divisão?

B

Respostas e sugestões

Este apêndice traz respostas, sugestões e esboços de demonstração para todos os exercícios do livro, organizados por capítulo. O número de cada resposta é um link que leva de volta ao enunciado. Quando a solução completa já aparece no texto de um capítulo, a resposta indica a seção correspondente. Recomenda-se tentar cada exercício antes de consultar a resposta.

Capítulo 1: Complexidade de algoritmos

1.1 (a) Não: $(2n^2+1)/n \to \infty$. (b) Sim: $2n^2+1 \le 3n^2$ para $n \ge 1$. (c) Sim: $2n^2+1 \le 3n^3$ para $n \ge 1$. (d) Sim: $2n^2+1 \ge n$ para $n \ge 1$. (e) Não: $(2n^2+1)/n^3 \to 0$. (f) Sim, por (b) e porque $2n^2+1 \ge n^2$.

1.2 Se o tempo disponível permite T passos na máquina antiga, a nova faz $10T$. Para n: o tamanho é multiplicado por 10. Para n^2: por $\sqrt{10} \approx 3{,}16$. Para 2^n: $2^{n'} = 10 \cdot 2^n$, logo $n' = n + \log_2 10 \approx n + 3{,}3$ — só se *soma* uma constante. Algoritmos polinomiais aproveitam o avanço tecnológico multiplicando o tamanho tratável; os exponenciais quase não ganham.

1.3 O tamanho da entrada é o número de bits: n números de até $\log_2 k$ bits cada, ou seja, $\Theta(n \log k)$. Com $k = 2^n$, a entrada tem $\Theta(n^2)$ bits, mas o algoritmo gasta $\Theta(2^n)$ (vetor de contagem de tamanho k): exponencial no tamanho da entrada. Com $k = O(n)$, o custo é $\Theta(n)$. Algoritmos assim, polinomiais no *valor* dos números mas não no seu tamanho em bits, são **pseudopolinomiais**.

1.4 $\log^2 n \prec \sqrt{n} \prec 2^{\log_2 n} = n \prec n \log n \prec n^{\log_2 3} \approx n^{1{,}585} \prec n^{100} \prec 1{,}01^n \prec n! \prec 2^{2^n}$. Toda exponencial a^n com $a > 1$ supera qualquer polinômio, por menor que seja a.

1.5 (a) V: $2^{n+1} = 2 \cdot 2^n$. (b) F: $2^{2n}/2^n = 2^n \to \infty$. (c) V: $\log(n^2) = 2 \log n$. (d) V: polilogaritmos são $o(n^\varepsilon)$ para todo $\varepsilon > 0$. (e) V: $n^{\log n} = 2^{\log^2 n}$ e $\log^2 n = o(n)$. (f) F: $f = 2n$ e $g = n$ dão $2^{2n} \neq O(2^n)$.

1.6 Cota superior: $n! \le n^n$, logo $\log n! \le n \log n$. Cota inferior: os $n/2$ maiores fatores de $n!$ são $\ge n/2$, então $n! \ge (n/2)^{n/2}$ e $\log n! \ge \frac{n}{2} \log \frac{n}{2} = \frac{n}{2} \log n - \frac{n}{2} = \Omega(n \log n)$.

1.7 $f(n) = n$ se n é par e 1 se n é ímpar; $g(n) = 1$ se n é par e n se n é ímpar. Nos pares, $f/g = n \to \infty$; nos ímpares, $g/f \to \infty$. Outro exemplo: $f = n$ e $g = n^{1+\sin n}$.

1.8 (a) O laço interno roda i vezes para $i = 1, 2, 4, \ldots, \le n$: o total é $1 + 2 + 4 + \cdots + 2^{\lfloor \log n \rfloor} < 2n$, isto é, $\Theta(n)$ — e não $n \log n$. (b) Para cada i, o laço interno roda $\lfloor \log_2 n \rfloor + 1$ vezes: $\Theta(n \log n)$.

1.9 Não. A entrada é o número N, que ocupa $b = \Theta(\log N)$ bits. O algoritmo faz $\Theta(\sqrt{N}) = \Theta(2^{b/2})$ divisões: é exponencial em b (pseudopolinomial). Existe algoritmo polinomial para testar primalidade, mas não é esse.

1.10 Cota superior: $3n^3 - 100n^2 + 7 \le 3n^3 + 7 \le 4n^3$ para $n \ge 2$. Cota inferior: $3n^3 - 100n^2 + 7 \ge 2n^3$ equivale a $n^3 \ge 100n^2 - 7$, que vale para $n \ge 100$. Logo $2n^3 \le 3n^3 - 100n^2 + 7 \le 4n^3$ para $n \ge 100$: constantes $c_1 = 2$, $c_2 = 4$, $n_0 = 100$.

1.11 Se fosse, existiriam c, n_0 com $n^2 \le c\, n \log n$, isto é, $n/\log n \le c$, para todo $n \ge n_0$. Mas $n/\log n \to \infty$. Contradição.

1.12 $\log_a n = \log_b n / \log_b a$, e $1/\log_b a$ é uma constante positiva; logo cada um é constante vezes o outro. Já $a^n/b^n = (a/b)^n$ tende a 0 ou a ∞ quando $a \neq b$: a base de uma exponencial importa.

1.13 (a) $f \le c_1 g$ para $n \ge n_1$ e $g \le c_2 h$ para $n \ge n_2$ dão $f \le c_1 c_2 h$ para $n \ge \max\{n_1, n_2\}$. (b) $\max\{f, g\} \le f + g \le 2 \max\{f, g\}$. (c) Se $c_1 g \le f \le c_2 g$, então $\frac{1}{c_2} f \le g \le \frac{1}{c_1} f$.

1.14 Cota superior: cada termo é $\le n^k$, logo a soma é $\le n^{k+1}$. Cota inferior: os $\lceil n/2 \rceil$ últimos termos são $\ge (n/2)^k$, logo a soma é $\ge \frac{n}{2}(n/2)^k = n^{k+1}/2^{k+1}$.

1.15 Como $1/x$ é decrescente, $\int_i^{i+1} \frac{dx}{x} \le \frac{1}{i} \le \int_{i-1}^{i} \frac{dx}{x}$. Somando, $\ln(n+1) \le H_n \le 1 + \ln n$. Logo $H_n = \Theta(\log n)$.

1.16 Pior caso: n comparações (x na última posição, ou ausente). Melhor caso: 1. Caso médio: $\sum_{i=1}^{n} \frac{1}{n} \cdot i = \frac{n+1}{2}$.

1.17 O corpo roda $j - i + 1$ vezes para cada par $i \le j$: $\sum_{1 \le i \le j \le n} (j - i + 1) = \sum_{d=0}^{n-1} (n-d)(d+1) = \frac{n(n+1)(n+2)}{6}$ (para $n = 1$ e $n = 2$ dá 1 e 4). Logo $\Theta(n^3)$.

1.18 Após t iterações, $j = 2^{2^t}$. O laço para quando $2^{2^t} > n$, isto é, quando $2^t > \log n$, ou $t > \log\log n$: $\Theta(\log\log n)$.

1.19 $\frac{2^n}{n!} = \prod_{i=1}^{n} \frac{2}{i} \le 2 \cdot 1 \cdot \left(\frac{2}{3}\right)^{n-2} \to 0$. $\frac{n!}{n^n} = \prod_{i=1}^{n} \frac{i}{n} \le \frac{1}{n} \to 0$ (cada fator é ≤ 1 e o primeiro é $1/n$).

1.20 No pior caso, $\lfloor \log_2 n \rfloor + 1$ comparações (cada passo descarta pelo menos metade do intervalo). Para $n = 1000$: $\lfloor 9{,}97 \rfloor + 1 = 10$. Para $n = 10^6$: $\lfloor 19{,}93 \rfloor + 1 = 20$.

1.21 Argumento do adversário: o adversário responde "diferente" a toda comparação. Se o algoritmo parar depois de menos de n comparações, alguma posição p nunca foi comparada com x. Se ele responder "não está", o adversário coloca x na posição p; se responder "está", o adversário põe em p um valor diferente de x. Em ambos os casos, as respostas dadas continuam consistentes e o algoritmo erra. Logo são necessárias n comparações, e a busca sequencial, que faz n, é ótima.

1.22 (a) V: $\max\{f, g\} \le f + g \le 2\max\{f, g\}$. (b) V: de $f \le cg$ vem $\log f \le \log c + \log g \le (|\log c| + 1)\log g$, pois $\log g \ge 1$. (c) V: $f^2 \le c^2 g^2$. (d) F: para $f = 2^n$, $f(n)/f(n/2) = 2^{n/2} \to \infty$.

Capítulo 2: Listas de prioridades e heaps binários

2.1 (a) Não: a posição 6 (29) tem pai na posição 3 (28) e $29 > 28$. (b) Sim: as 8 relações pai–filho valem ($33 \ge 32, 28$; $32 \ge 31, 29$; $28 \ge 26, 25$; $31 \ge 30, 27$).

2.2 Falso. Contraexemplo: $S = [10, 9, 1, 8, 7, 0, 0, 6]$, $i = 3$ (1), $j = 4$ (8). Após a troca, $[10, 9, 8, 1, 7, 0, 0, 6]$: a posição 8 (6) tem pai na posição 4 (1). O valor pequeno foi para uma posição cujos filhos podem ser maiores que ele.

2.3 Falso. $S = [10, 9]$, $i = 1$, $j = 2$: após a troca, $[9, 10]$ tem o filho maior que o pai.

2.4 $H(0) = H(1) = 1$ e $H(n) = \binom{n-1}{L} H(L) H(n-1-L)$, onde L é o tamanho da subárvore esquerda da raiz, determinado pelo formato da árvore completa: a raiz é obrigatoriamente o extremo, escolhem-se os L valores que vão para a esquerda e cada lado é um heap. Valores: $1, 1, 2, 3, 8, 20, 80, 210, 896, 3360$ para $n = 1, \ldots, 10$. A demonstração está na §2.7.

2.5 Processam-se $i = 4, 3, 2, 1$: $34 \leftrightarrow 50$; $41 \leftrightarrow 52$; $25 \leftrightarrow 50$ e $25 \leftrightarrow 48$; $18 \leftrightarrow 52$ e $18 \leftrightarrow 41$. Resultado: $[52, 50, 41, 48, 14, 10, 18, 34, 25]$.

2.6 É o Lema 2.10: o nó i tem altura $\ge j$ se e somente se $i\,2^{j-1} \le n$; assim, o número de nós com altura exatamente j é $\lfloor x \rfloor - \lfloor x/2 \rfloor = \lceil \lfloor x \rfloor / 2 \rceil \le \lceil n/2^j \rceil$, com $x = n/2^{j-1}$.

2.7 Basta tornar *subir* e *descer* iterativos:

Subir e descer sem recursão

```
Procedimento SUBIR(i)
    enquanto i > 1 e T[i].chave > T[⌊i/2⌋].chave faça
        T[i] ⇔ T[⌊i/2⌋]; i := ⌊i/2⌋
Procedimento DESCER(i, n)
    enquanto 2i ≤ n faça
        j := 2i
        se j < n e T[j + 1].chave > T[j].chave então
            j := j + 1
```

```
se T[i].chave < T[j].chave então
    T[i] ⇔ T[j]; i := j
senão
    pare
```

Inserir: $T[n+1] := x$; $n := n+1$; subir(n). Remover: $T[1] := T[n]$; $n := n-1$; descer($1, n$). Aumentar a chave de i: subir(i); diminuir: descer(i, n). Construir: **para** $i := \lfloor n/2 \rfloor$ **decrescendo até** 1, descer(i, n).

2.8 (a) $s_i \leq s_{\lfloor i/2 \rfloor}$ para $1 < i \leq n$: cada nó é maior ou igual aos filhos. (b) Pela transitividade ao longo do caminho até a raiz (Proposição 2.14), a raiz é o máximo; basta ler $T[1]$. (c) $O(1)$, $O(\log n)$, $O(\log n)$, $O(\log n)$ e $O(n)$.

2.9 (a) O 98 sobe trocando com 78 (posição 3) e com 95 (posição 1): $[98, 60, 95, 39, 28, 78, 70, 33]$. (b) Com 33 na raiz, ele desce trocando com 78 e depois com 70: $[78, 60, 70, 39, 28, 66, 33]$. (c) Em $i = 3$ nada muda; em $i = 2$, $15 \leftrightarrow 40$; em $i = 1$, $20 \leftrightarrow 40$: $[40, 20, 30, 15, 10, 25, 5]$. (d) Em $i = 3$, $15 \leftrightarrow 20$; em $i = 2$, $2 \leftrightarrow 9$; em $i = 1$, $7 \leftrightarrow 20$ e $7 \leftrightarrow 15$: $[20, 9, 15, 2, 1, 7, 13]$.

2.10 As operações estão invertidas. Se o novo valor ficou *maior que o pai*, a violação é com o pai e o elemento precisa **subir**; caso contrário, a única violação possível é com os filhos e ele precisa **descer**. Versão correta: **se** $i > 1$ **e** $H[i] > H[\lfloor i/2 \rfloor]$ **então** Subir(i) **senão** Descer(i, n). Equivalentemente, comparando com o valor antigo: se aumentou, sobe; se diminuiu, desce.

2.11 Use um heap de mínimo de pares (t_{livre}, número do vendedor), comparados lexicograficamente, começando com $(0, 1), \ldots, (0, n)$. Para cada ligação de duração d, remova o mínimo (t, v) — o vendedor que fica livre primeiro, com desempate pelo menor número —, conte uma ligação para v e reinsira $(t + d, v)$. Cada ligação custa $O(\log n)$, e o total é $O(n + L \log n)$.

2.12 (a) Pai de i: $\lfloor (i-2)/d \rfloor + 1$; filhos: $d(i-1) + 2, \ldots, di + 1$. (b) $\Theta(\log_d n)$. (c) Subir custa $O(\log_d n)$, com uma comparação por nível; descer custa $O(d \log_d n)$, pois em cada nível acha o maior de d filhos. (d) Aumentos de prioridade usam *subir*, que fica mais barato com d maior, enquanto as remoções (descer) ficam mais caras; quando há muito mais aumentos, compensa aumentar d.

2.13 Mantenha um heap de mínimo com o primeiro elemento de cada lista (e a lista de onde veio). Repita n vezes: remova o mínimo, coloque-o na saída e insira o próximo elemento da mesma lista. O heap tem sempre no máximo k elementos, e o custo total é $O(n \log k)$, mais $O(k)$ da construção.

2.14 Use um heap de **mínimo** com até k elementos. Para cada número x: se o heap tem menos de k elementos, insira x; senão, se x é maior que o mínimo do heap, substitua o mínimo por x e faça-o descer. Ao final, o heap contém os k maiores. Tempo $O(n \log k)$ e memória $O(k)$.

2.15 **Para** $i := 2$ **até** n: **se** $s_i > s_{\lfloor i/2 \rfloor}$, **retorna** falso. Ao final, **retorna** verdadeiro. São $n - 1$ comparações, uma por aresta da árvore, e isso basta porque a propriedade de heap é exatamente essa condição local.

2.16 Entrada $[1,\ 2_a,\ 2_b]$. O *arranjar* troca 1 com 2_a (o primeiro filho, já que 2_b não é maior): $[2_a, 1, 2_b]$. O primeiro passo do heapsort troca a raiz com a última posição: $[2_b, 1, 2_a]$; descer não muda nada. O segundo passo dá $[1, 2_b, 2_a]$. Na saída, 2_b aparece antes de 2_a: a ordem relativa dos iguais foi invertida.

2.17 Ao ser inserido, i é o maior elemento até então e sobe até a raiz, com $\lfloor \log_2 i \rfloor$ trocas. O total, $\sum_{i=2}^{n} \lfloor \log_2 i \rfloor$, é $\Theta(n \log n)$, pois metade dos termos é $\geq \log_2(n/2) - 1$. O *arranjar* sobre o vetor $[1, 2, \ldots, n]$ continua $O(n)$, pela análise que soma as alturas (Lema 2.10).

2.18 $[15]$; $[40, 15]$; $[40, 15, 25]$; $[50, 40, 25, 15]$ (50 sobe duas vezes); $[50, 40, 25, 15, 10]$; $[50, 40, 45, 15, 10, 25]$ (45 troca com 25).

2.19 Primeira remoção: 20 vai para a raiz e desce trocando com 80 e depois com 50, resultando em $[80, 70, 50, 30, 60, 20, 40, 10]$. Segunda: 10 vai para a raiz e desce trocando com 70 e depois com 60, resultando em $[70, 60, 50, 30, 10, 20, 40]$.

2.20 Após o *arranjar*: $[10, 5, 3, 4, 1]$. Passos do laço, para $m = 5, 4, 3, 2$:
$[5, 4, 3, 1 \mid 10]$, $[4, 1, 3 \mid 5, 10]$, $[3, 1 \mid 4, 5, 10]$, $[1 \mid 3, 4, 5, 10]$.

2.21 Copie $T[n]$ para a posição i e faça $n := n - 1$. O valor que chegou a i pode ser maior que o pai de i (ele vinha de outro ramo da árvore) ou menor que algum filho de i; então, **se** $i > 1$ **e** $T[i] > T[\lfloor i/2 \rfloor]$, suba; **senão**, desça. Apenas uma das duas violações pode ocorrer, e o custo é $O(\log n)$.

2.22 Guarde a metade menor dos elementos num heap de **máximo** A e a metade maior num heap de **mínimo** B, com $|A| = |B|$ ou $|A| = |B| + 1$. Para inserir x: se $x \le$ topo de A, insira em A, senão em B; depois, se os tamanhos ficarem desequilibrados, mova o topo de um heap para o outro. A mediana (inferior) é o topo de A, lido em $O(1)$; cada inserção custa $O(\log n)$.

2.23 Cada passo do *descer* faz no máximo duas comparações (uma entre os dois filhos e outra entre o maior filho e o pai). Um nó de altura j faz no máximo $j-1$ passos. Pelo Lema 2.10, o total é no máximo $2\sum_{j\ge 2}(j-1)\lceil n/2^j\rceil \le 2n + O(\log^2 n)$.

Capítulo 3: Heaps binomiais, heaps de Fibonacci e análise amortizada

3.1 Faça n inserções sucessivas. Inserir é somar 1 em binário, e cada *link* corresponde a um "vai um". Contando de 0 até n, o "vai um" do bit k para o bit $k+1$ ocorre no máximo $\lfloor n/2^{k+1}\rfloor$ vezes, de modo que o número total de links é $\sum_{k\ge 1}\lfloor n/2^k\rfloor < n$, e o custo total é $O(n)$. Alternativa equivalente: agrupe os elementos em pares formando árvores B_1, depois os pares de B_1 formando B_2, e assim por diante, como numa soma binária de n uns.

3.2 A inserção é o pior caso quando o novo B_0 provoca "vai um" em todas as posições: o heap contém $B_0, B_1, \ldots, B_{k-1}$, isto é, $n = 2^k - 1$, com todos os bits iguais a 1. As alturas são $0, 1, \ldots, k-1$ em arestas (ou $1, \ldots, k$ em níveis). A inserção faz k links e produz uma única árvore B_k.

3.3 A remoção percorre as k raízes, transforma os h filhos da raiz removida no heap $H'' = \{B_0, \ldots, B_{h-1}\}$ e funde H'' com o restante H'. O número de links é máximo quando H' também contém $B_0, \ldots, B_{h-1}$ e h é o maior possível: $H = \{B_0, B_1, \ldots, B_{k-1}\}$ com o máximo na raiz de B_{k-1}, isto é, $h = k-1$ (em arestas).

3.4 Falso. H_1 com $3 = 11_2$ nós ($k_1 = 2$) e H_2 com $5 = 101_2$ nós ($k_2 = 2$). A fusão tem $8 = 1000_2$ nós, numa única árvore: $k = 1 < 2$.

3.5 Verdadeiro. A união das listas de raízes tem $k_1 + k_2$ árvores e cada link transforma duas árvores em uma; logo $k = k_1 + k_2 - \#\text{links} \le k_1 + k_2$.

3.6 Suponha árvores de posto k formadas por subordinações de árvores de mesmo posto, com cortes controlados por marcas em nós que não são raízes, e meça alturas em arestas. **Altura mínima**: $\lceil k/2\rceil$. Pelo lema $\text{grau}(y_i) \ge i-2$, a menor altura possível satisfaz $A(k) = 1 + A(k-2)$, com $A(0) = 0$ e $A(1) = 1$. **Altura máxima**: k, atingida pela própria B_k; por indução, subordinar duas árvores de posto h e altura $\le h$ dá posto $h+1$ e altura $\le h+1$, e cortes não aumentam a altura. Se as raízes puderem perder filhos livremente, só o *grau* continua sendo $O(\log n)$; a altura pode crescer.

3.7 Ver §3.1 e §3.4. Heap binomial: a seleção percorre as raízes (ou mantém um ponteiro); a inserção é a fusão com $\{B_0\}$; a fusão intercala as listas de raízes por ordem e faz links como numa soma binária; a remoção do máximo retira a árvore, transforma os filhos num heap e funde; o aumento sobe trocando com o pai; a diminuição remove o nó (aumentando-o para $+\infty$ e removendo o máximo) e o reinsere; a construção faz n inserções. Heap de Fibonacci: inserção e fusão concatenam listas circulares; a remoção do máximo promove os filhos e consolida por grau; o aumento corta o nó e faz cortes em cascata; a diminuição remove e reinsere.

3.8 (a) B_0 é um nó; B_k é obtida ligando duas B_{k-1}. Tem 2^k nós e a raiz tem grau k, que é a ordem da árvore. (b) Como uma lista de raízes de árvores com a condição de heap e **ordens distintas**; B_k aparece se e somente se o bit k de n é 1. (c) A raiz 10 fica por cima e a raiz 15 vira sua filha; o resultado é uma B_3, com 8 nós e raiz de grau 3. (d) O extremo está numa das no máximo $\lfloor \log n\rfloor + 1$ raízes: $O(\log n)$, ou $O(1)$ com ponteiro. (e) $O(\log n)$: a união percorre só as listas de raízes, que têm $O(\log n)$ árvores, fazendo links como numa soma binária; os nós internos não são visitados.

3.9 (a) O heap de Fibonacci é preguiçoso: permite várias árvores de mesmo grau na lista de raízes, consolidando só na remoção, e usa listas circulares duplas e marcas nos nós. (b) Inserção, união, seleção e redução de chave (*decrease-key*) custam $O(1)$ amortizado; *Extract-Min* custa $O(\log n)$ amortizado. (c) A nova chave pode ficar menor que a do pai. Corta-se x com sua subárvore, e ele vai para a lista de raízes, desmarcado. (d) Se o pai já tinha perdido um filho (estava marcado), ele também é cortado, e assim por diante. Isso garante que cada nó perde no máximo um filho antes de ser cortado, o que mantém o grau $O(\log n)$. Com o potencial $\Phi = t + 2m$, cada corte em cascata desmarca um nó: o potencial cai 2 e sobe 1, e o corte paga a si mesmo. Por isso o custo amortizado é $O(1)$.

3.10 São $21 = 10101_2$ e $3 = 11_2$ elementos. Os links, na ordem, são: $B_0 + B_0 \to B_1$; esse B_1 com o B_1 de H_2 forma B_2; essa B_2 com a B_2 de H_1 forma B_3. O resultado é $\{B_3, B_4\}$, isto é, $24 = 11000_2$.

3.11 (1) X é cortado e vai para a lista de raízes, desmarcado. (2) Y, que estava marcado, perdeu o segundo filho: é cortado, desmarcado e vai para as raízes. (3) O mesmo acontece com Z. (4) Se o pai W de Z é raiz, nada muda; se não é raiz e está desmarcado, é marcado e a cascata termina; se estava marcado, a cascata continua. Vão para a lista de raízes X, Y e Z (e possivelmente outros); ao final, X, Y, Z estão desmarcados e W marcado, se não for raiz.

3.12 Sem as marcas, um nó poderia perder quase todos os filhos sem que nada acontecesse: cortando todos os netos de uma raiz, obtém-se uma árvore de posto k com apenas $k+1$ nós. O tamanho deixaria de crescer exponencialmente com o posto (como F_{k+2}), o grau máximo deixaria de ser $O(\log n)$ e a consolidação, que depende do número de graus distintos, ficaria cara.

3.13 **Heap binomial**. A fusão custa $O(\log n)$ *no pior caso de cada execução* e cresce só logaritmicamente com o número de aeronaves: é previsível. No heap binário, a fusão exige reconstruir o vetor, $\Theta(n)$ por fusão; milhares de fusões com 10^5 elementos estouram as janelas de tempo. No heap de Fibonacci, a fusão é $O(1)$ apenas *amortizado*: o trabalho adiado aparece de uma vez numa remoção, cuja consolidação pode custar $\Theta(n)$, o que é imprevisível; além disso, a grande vantagem dessa estrutura, o aumento de prioridade em $O(1)$ amortizado, não seria usada, já que as alterações são raras.

3.14 **Heap binário em vetor**. Os dados ocupam $2 \cdot 10^6 \times 16 = 32\,000\,000$ bytes, cerca de 30,5 MB, abaixo do limite de 41 943 040 bytes. Um heap binomial precisaria de pelo menos 3 ponteiros por nó (pai, filho, irmão), isto é, 24 bytes, mais o grau: pelo menos 40 bytes por nó e $80 \cdot 10^6$ bytes no total, o que não cabe; o de Fibonacci, com 4 ponteiros, grau e marca, ainda menos. Como não há fusões e as alterações são poucas, inserção e remoção em $O(\log n)$ bastam; o heap binário é o único que cabe e o mais simples.

3.15 **Heap de Fibonacci**. O aumento de prioridade custa $O(1)$ amortizado, sem retirar e reinserir a tarefa. Uma estimativa do custo total é $5 \cdot 10^6 + 8 \cdot 10^7 + 5 \cdot 10^6 \log_2(5 \cdot 10^6) \approx 2 \cdot 10^8$ passos. Nos heaps binário e binomial, cada um dos $8 \cdot 10^7$ aumentos custa $O(\log n) \approx 22$ passos, cerca de $2 \cdot 10^9$ no total. Como importa o custo total da execução, e não a previsibilidade de cada operação, a garantia amortizada é adequada; as poucas uniões não pesam.

3.16 Com k elementos e capacidade c, vale $c \geq k \geq c/2$ depois da primeira duplicação, logo $\Phi = 2k - c \geq 0$. Uma inserção que não duplica tem custo 1 e $\Delta\Phi = 2$: custo amortizado 3. Uma inserção que duplica (com $k = c$) custa $c + 1$, pois copia c elementos e insere um; o potencial passa de $2c - c = c$ para $2(c+1) - 2c = 2$, ou seja, $\Delta\Phi = 2 - c$, e o custo amortizado é $(c+1) + (2-c) = 3$. Logo n inserções custam no máximo $3n$.

3.17 Comece em $2^{k-1} - 1 = 0111\ldots1$. Um incremento leva a $1000\ldots0$, trocando k bits, e um decremento volta ao estado anterior, trocando k bits. Alternando as duas operações, cada uma custa k, e o total é $\Theta(mk)$. Com apenas incrementos, o potencial "número de bits iguais a 1" pagava os "vai um" porque os uns, uma vez zerados, só voltam um a um; com decrementos, o mesmo trabalho pode ser repetido indefinidamente sem que nada o pague.

3.18 *Enfileirar*: empilhe na pilha de entrada. *Desenfileirar*: se a pilha de saída estiver vazia, transfira todos os elementos da entrada para ela, invertendo a ordem; depois desempilhe da saída. Cada elemento é empilhado e desempilhado no máximo uma vez em cada pilha, de modo que m operações custam $O(m)$. Com o potencial $\Phi = 2 \cdot |\text{entrada}|$, enfileirar custa 3 amortizado e desenfileirar custa 1.

3.19 $100 = 1100100_2$: $\{B_2, B_5, B_6\}$. $101 = 1100101_2$: $\{B_0, B_2, B_5, B_6\}$, numa inserção sem links. De 100 a $104 = 1101000_2$: $\{B_3, B_5, B_6\}$. As quatro inserções ($100 \to 101 \to 102 \to 103 \to 104$) fazem $0, 1, 0, 3$ links, respectivamente: o número de uns no final de 100, 101, 102 e 103.

3.20 Por indução. As árvores nascem com um único nó (B_0). Sem cortes, uma árvore só muda por *link*, que na consolidação liga duas raízes de mesmo grau k; se ambas são árvores binomiais B_k (não ordenadas), o resultado é B_{k+1}. Na remoção do máximo, os filhos da raiz de uma B_k são raízes de $B_0, \ldots, B_{k-1}$, também binomiais. Logo todas as árvores são binomiais; a diferença em relação ao heap binomial é que, antes da consolidação, pode haver várias árvores de mesma ordem.

3.21 Insira n elementos — cada inserção só acrescenta uma raiz, e ficam n raízes — e depois remova o máximo: a consolidação percorre as $n-1$ raízes restantes e faz $\Theta(n)$ links, com custo real $\Theta(n)$. Não há contradição, porque o custo amortizado $O(\log n)$ vale para a *sequência*: as n inserções pagaram, cada uma com um crédito (o aumento de potencial $\Delta t = +1$), pelo trabalho da consolidação.

Capítulo 4: Conjuntos disjuntos: Union-Find

4.1 (a) Manter uma partição de n elementos em conjuntos disjuntos, respondendo se dois elementos estão no mesmo conjunto e unindo conjuntos. (b) Make-Set(x) cria $\{x\}$ ($pai[x] := x$); Find(x) sobe pelos pais até a raiz, que é o representante; Union(x, y) encontra as duas raízes e faz uma apontar para a outra. (c) O **pai** de cada elemento na floresta; a raiz guarda também o rank (ou o tamanho). (d) A árvore de menor rank é ligada sob a de maior; o rank só aumenta quando os dois são iguais, e então a árvore pelo menos dobra de tamanho. Por isso uma árvore de rank h tem pelo menos 2^h nós, e a altura é no máximo $\log n$ (Teorema 4.4).

4.2 Ver §4.5. (a) Representantes finais A (de $\{A, B, C, D\}$, com rank 2) e E (de $\{E, F\}$): 2 conjuntos. (b) Devolve A e deixa B, C, D, E apontando diretamente para A. (c) A raiz é A, com rank 2: só no empate a altura cresce. (d) Só com rank, Find custa $\Theta(\log n)$; só com compressão, o total é $O(m \log n)$, com Find individual possivelmente $\Theta(n)$; juntas, $O(m \log^* n)$ (ou $O(m\,\alpha)$).

4.3 A prova é o Teorema 4.4: por indução, uma árvore de rank (igual à altura, sem compressão) h tem pelo menos 2^h nós, pois a altura só aumenta ao unir duas árvores de mesmo rank h, cada uma com pelo menos 2^h nós. Logo $2^h \le n$. Para atingir a cota com $n = 2^k$, una os elementos aos pares (altura 1), depois as árvores resultantes aos pares (altura 2), e assim por diante. Após k rodadas há uma única árvore de altura $k = \log_2 n$, que é uma árvore binomial B_k.

4.4 Faça Union$(1, 2)$, Union$(1, 3)$, ..., Union$(1, n)$, sempre pendurando a raiz da árvore grande sob o novo elemento unitário ($pai[\text{raiz}] := i$). Forma-se uma cadeia de altura $n - 1$ sem nenhum Find caro no caminho; um Find no fundo custa $\Theta(n)$ e achata tudo. Com compressão e união arbitrária, m operações custam $O(m \log n)$. A ideia da prova é classificar cada passo $x \to pai[x]$ da compressão como "leve" quando a subárvore de x tem menos da metade dos nós da subárvore do pai: cada nó só passa por $O(\log n)$ passos leves, e cada Find tem $O(\log n)$ passos pesados.

4.5 (a) Union-Find. Como as consultas dominam (10^8 contra 10^5 uniões) e precisam ser rápidas, a **lista encadeada com ponteiro direto ao representante** responde Find em $O(1)$ *no pior caso*, deixando o custo para as raras uniões: $O(\log n)$ amortizado com "menor na maior", no máximo $n \log_2 n \approx 10^7 \cdot 23$ atualizações de ponteiro no total. A floresta com rank e compressão também é defensável ($O(\alpha)$ amortizado por operação e menos memória), desde que se diga que a garantia é amortizada; floresta sem balanceamento não serve. (b) **Anexar o menor ao maior**. Na lista, cada elemento muda de representante no máximo $\log_2 n$ vezes, pois o tamanho do seu grupo pelo menos dobra a cada mudança; na árvore, a altura fica no máximo $\log_2 n$. Sem isso, listas e árvores degeneram e cada operação pode custar $\Theta(n)$.

4.6 Ver os cenários da §4.4. (a) Não há como desfazer uniões, e a compressão destrói a forma das árvores. Resolva *offline*, de trás para frente, transformando cada destruição numa união a partir do grafo final; guarde o número de componentes a cada passo e apresente as respostas na ordem original. (b) Floresta sem compressão, só com união por rank ou tamanho (Find em $O(\log n)$), empilhando cada ponteiro e rank alterado por Union; desfazer é desempilhar e restaurar, em $O(1)$. (c) A floresta só tem ponteiros para cima, e listar membros custaria $O(n)$; use listas encadeadas com ponteiro ao representante e união "menor na maior": *MesmoGrupo* em $O(1)$, *ListarMembros* em $O(k)$ e *Juntar* em $O(\log n)$ amortizado.

4.7 Cada bloco de células contíguas é um conjunto, representado por uma **lista encadeada** dos gatinhos na ordem das células, com cabeça e cauda guardadas no representante. No dia i, encontre os representantes de x_i e y_i e concatene uma lista na outra (cauda de uma $\to$ cabeça da outra), unindo também os conjuntos. Como os blocos de x_i e y_i eram vizinhos, colocá-los lado a lado é consistente com alguma ordem inicial. Ao final, percorra a única lista restante. Custo $O(n\,\alpha(n))$.

4.8 Union$(1, 2)$ torna 1 raiz, com rank 1; o mesmo ocorre com 3 (unido a 4), 5 (a 6) e 7 (a 8). Union$(1, 3)$: empate, $pai[3] = 1$ e rank$(1) = 2$. Union$(5, 7)$: $pai[7] = 5$ e rank$(5) = 2$. Union$(1, 5)$: empate, $pai[5] = 1$ e rank$(1) = 3$. Antes do Find, o vetor pai (posições 1..8) é $1, 1, 1, 3, 1, 5, 5, 7$. Find(8) percorre $8 \to 7 \to 5 \to 1$, devolve 1 e comprime: $pai = 1, 1, 1, 3, 1, 5, 1, 1$. Os ranks não mudam: rank$(1) = 3$, rank$(5) = 2$, rank$(3) =$ rank$(7) = 1$ e os demais são 0.

4.9 Faça Make-Set para todo vértice; para cada aresta uv, se Find$(u) =$ Find(v), a aresta fecha um ciclo; senão, faça Union(u, v). Custo $O(n + m\,\alpha(n))$. Uma busca em profundidade (Capítulo 17) também resolve o problema, em $O(n + m)$.

4.10 Quando x deixa de ser raiz, ele é ligado a uma raiz r com rank$(r) >$ rank(x), ou com rank$(r) =$ rank(x),

e nesse caso rank(r) é incrementado; em ambos os casos, rank(x) < rank(r). Depois disso, o rank de x nunca muda (só raízes mudam de rank), e o rank de $pai[x]$ nunca diminui. A compressão troca $pai[x]$ por um ancestral, que, pela mesma desigualdade aplicada ao longo do caminho, tem rank ainda maior. Logo a desigualdade se mantém.

4.11 Guarde na raiz o maior elemento e o tamanho do conjunto. Na Union de raízes r e s, com s indo para baixo de r, faça $max[r] := \max\{max[r], max[s]\}$ e $tam[r] := tam[r] + tam[s]$. As consultas fazem Find(x) e leem o valor na raiz: $O(\alpha(n))$ amortizado.

4.12 Indução: uma árvore de altura h tem pelo menos 2^h nós. Ao pendurar T_v (a menor) sob a raiz de T_w, com $|T_v| \le |T_w|$, a nova altura é $\max\{h_w, h_v + 1\}$. Se é h_w, vale pela hipótese. Se é $h_v + 1$, o novo tamanho é $|T_v| + |T_w| \ge 2|T_v| \ge 2 \cdot 2^{h_v} = 2^{h_v+1}$. Logo $2^h \le n$.

4.13 Faça Union$(1,2)$, Union$(2,3)$, …, Union$(n-1,n)$: a raiz do conjunto de i é sempre i (o último elemento a virar raiz é o de y), e cada união faz a raiz atual apontar para o novo elemento, formando a cadeia $1 \to 2 \to \cdots \to n$, de altura $n-1$. Sem compressão, n chamadas Find(1) custam $\Theta(n)$ cada: $\Theta(n^2)$ no total.

4.14 Na primeira rodada, 4 uniões de listas de tamanho 1: 4 atualizações. Na segunda, 2 uniões de listas de tamanho 2, atualizando 2 ponteiros cada: 4. Na terceira, 1 união de listas de tamanho 4: 4. Total: 12. Para $n = 2^k$, são k rodadas com $n/2$ atualizações cada: $\frac{n}{2}\log_2 n$.

4.15 O ponteiro de um elemento x só é atualizado quando a lista de x é a menor das duas (ou igual); a nova lista tem pelo menos o dobro do tamanho da lista anterior de x. Como nenhuma lista tem mais de n elementos, o ponteiro de x é atualizado no máximo $\log_2 n$ vezes. Somando sobre os n elementos, no máximo $n \log_2 n$ atualizações.

4.16 Una sempre o conjunto grande ao elemento seguinte, anexando a lista grande à unitária: Union$(2,1)$, Union$(3,\{1,2\})$, …, com a lista do novo elemento como base. A i-ésima união atualiza i ponteiros, e o total é $1 + 2 + \cdots + (n-1) = \Theta(n^2)$.

4.17 Com $F = 1, 2, 4, 16, 65536, 2^{65536}$: $\log^* 2 = 1$, $\log^* 16 = 3$, $\log^* 65536 = 4$ e $\log^* 2^{65536} = 5$. Como $65536 < 10^{80} \le 2^{65536}$, $\log^* 10^{80} = 5$.

4.18 Uniões efetivas: 12, 34, 23, 56, 78, 89 (seis). A aresta 65 liga vértices que já estão juntos (repete 56) e a aresta 97 fecha o ciclo $7, 8, 9$. Componentes finais: $\{1,2,3,4\}$, $\{5,6\}$, $\{7,8,9\}$ e $\{10\}$, ou seja, $10 - 6 = 4$ componentes.

4.19 Crie um elemento por casa e dois elementos virtuais, $\top$ e $\bot$. Ao abrir uma casa, una-a a cada vizinha já aberta; se ela está na primeira linha, una-a a $\top$; se está na última, a $\bot$. Há caminho se e somente se Find$(\top)$ = Find$(\bot)$. Cada abertura faz no máximo 6 operações; com rank e compressão, n^2 aberturas custam $O(n^2\,\alpha(n^2))$.

4.20 Cada declaração $x \equiv y$ é uma Union(x, y), e cada consulta compara Find(x) com Find(y). No exemplo, a, b, c, d ficam equivalentes ($a \equiv b$, $b \equiv c$, $c \equiv d$), e e, f formam outra classe.

4.21 Para cada uma das m arestas, duas operações Find (uma por extremo): $2m$. Uniões efetivas: exatamente $n-1$, uma por aresta da árvore. Com rank e compressão, essas operações custam $O(m\,\alpha(n))$, muito menos que a ordenação das arestas, $O(m \log m) = O(m \log n)$, que domina o algoritmo.

4.22 Atualizar os ranks exigiria recalcular alturas de subárvores depois de cada compressão, o que custaria caro. Com compressão, o rank deixa de ser a altura e passa a ser uma *cota superior* para ela. Ele continua servindo para decidir as uniões porque as propriedades usadas na análise — uma árvore de rank k tem pelo menos 2^k nós e o rank cresce ao subir na árvore — dependem apenas de como os ranks evoluem nas uniões, e não da altura real.

Capítulo 5: Árvores binárias de busca e árvores AVL

5.1 (a) Uma ABB é balanceada se sua altura é $O(\log n)$ e o mesmo vale para cada subárvore (com m nós, altura $O(\log m)$). (b) AVL: em todo nó, as alturas das subárvores esquerda e direita diferem de no máximo 1. (c) Toda AVL é balanceada, pois $h < 1{,}44 \log_2(n+2)$ (Teorema 5.6); nem toda árvore balanceada é AVL. (d) $\text{bal}(v) = h_D(v) - h_E(v)$.

5.2 (a) bal(p) = 1 → 0: a altura não muda e o processo para. 0 → −1: a altura aumenta e o processo continua com o pai. −1 → −2: rotação, e o processo para. (b) Com sinais iguais (−2 e −1), rotação **simples à direita** em p. Com sinais opostos (−2 e +1), rotação **dupla à direita**: à esquerda em u e depois à direita em p. No primeiro caso o excesso está "em linha" (esquerda-esquerda) e uma rotação resolve; no segundo, está em zigue-zague (esquerda-direita), e é preciso primeiro alinhar.

5.3 (a) h indica se a altura da subárvore aumentou. (b) Se a altura não mudou, nenhum ancestral muda de balanço; só quando h é verdadeiro o pai precisa se atualizar. (c) A rotação muda a raiz da subárvore; como pt é passado por referência, atribuir a nova raiz a pt religa a subárvore ao pai (ou atualiza a raiz da árvore). (d) Os testes precisam ser exclusivos: com "se" em sequência, um nó com balanço 1 vira 0 e, logo em seguida, o teste "balanço = 0" dispara e o deixa com −1, com h verdadeiro, o que está errado.

5.4 Ver §5.7. (a) Rotação simples à direita em 40 ao inserir 10, e dupla à direita em 40 ao inserir 35. Árvore final: $20^{+1}(10,\ 35(30, 40))$. (b) Rotação simples à esquerda em 10 (ao inserir 30) e simples à esquerda em **30** (ao inserir 50; o primeiro desregulado é 30, não 20). Árvore final: $20^{+1}(10,\ 40(30, 50))$. (c) Inserir 49 desregula 20, cujo filho direito 40 tem balanço +1: rotação simples à esquerda em 20. Árvore final: $40(20(10, 30),\ 50^{-1}(49, \cdot))$.

5.5 Ver o Exemplo 5.11. Inserir 5 desregula 30 (caso esquerda-esquerda, rotação simples à direita em 30); 15 não causa rotação; 35 desregula 50 (caso esquerda-direita, rotação dupla à direita: à esquerda em 20, à direita em 50, e 30 vira a raiz); 45 não causa rotação. Árvore final: $30(20(10(5, 15), 25),\ 50(40(35, 45), 70(60, 80)))$. (c) Numa ABB, inserções em ordem crescente, decrescente ou em zigue-zague geram altura $\Theta(n)$ e operações $\Theta(n)$; a AVL garante $O(\log n)$ qualquer que seja a ordem de chegada, com $O(1)$ rotações por inserção.

5.6 Use uma árvore de Fibonacci (uma AVL mínima) em que todos os nós internos têm balanço −1 e remova a folha mais à direita. Cada ancestral perde altura do lado que já era mais baixo, fica com balanço −2 e tem o filho esquerdo com balanço −1; a rotação simples diminui a altura da subárvore e propaga o problema para cima. Por exemplo, na árvore de Fibonacci de altura 7, com 33 nós (chaves $1, \ldots, 33$) e raiz 21, remover a folha 33 (cujos ancestrais são 32, 29 e 21) causa rotações em 32, 29 e 21.

5.7 Se todas as chaves de T_1 são menores que as de T_2 (junção): retire o mínimo x de T_2; supondo $h(T_1) \geq h(T_2)$, desça pela espinha direita de T_1 até um nó v de altura $h(T_2)$ ou $h(T_2) + 1$; crie um nó x com filhos $T(v)$ e T_2, coloque-o no lugar de v e suba rebalanceando como numa inserção. Custo $O(\log n)$. No caso geral, com chaves intercaladas: faça os percursos em ordem das duas árvores ($O(n + m)$), intercale as duas listas ordenadas e construa uma árvore completa tomando sempre a mediana como raiz, em $O(n + m)$ no total. Alternativa: inserir os nós da menor na maior, em $O(m \log(n + m))$.

5.8 Sim: cada conjunto é uma AVL e cada nó guarda um ponteiro para o pai. Find(x) sobe até a raiz da AVL de x, que identifica o conjunto: $O(\log n)$. Union insere os elementos da AVL menor na maior: $O(k \log n)$, sendo k o tamanho da menor; com "menor na maior", cada elemento é reinserido no máximo $\log n$ vezes, e o total é $O(n \log^2 n)$. Ganha-se ordem dentro de cada conjunto, mas a estrutura é bem pior que o Union-Find, de custo praticamente $O(1)$ amortizado.

5.9 (a) AVL: a ABB pode degenerar para altura $\Theta(n)$ com inserções em ordem, enquanto a AVL garante $O(\log n)$ para busca, inserção e remoção. (b) Heap: o máximo está na raiz ($O(1)$) e inserir ou remover custa $O(\log n)$, num vetor simples. A AVL também faria tudo em $O(\log n)$, mas é mais pesada, e só compensaria se também fosse preciso buscar ou remover tarefas específicas por identificador.

5.10 A inserção e as rotações estão na §5. Remoção: remova como numa ABB (com dois filhos, troque pelo maior da subárvore esquerda), suba a partir do nó fisicamente removido usando a tabela "a altura diminuiu?" e, ao rotacionar, trate o caso em que o filho do lado mais pesado tem balanço 0 com rotação *simples*; nesse caso a altura não muda e o processo para.

5.11 1195: insira as chaves numa ABB e imprima os três percursos. 1200 e 1201: ABB com inserção, busca, percursos e, no segundo, remoção. 1191: o primeiro símbolo da pré-ordem é a raiz; sua posição no percurso em ordem separa a subárvore esquerda (à esquerda dela) da direita; resolva recursivamente as duas partes e imprima a raiz por último. Custa $O(n^2)$ na forma simples, ou $O(n)$ com uma tabela de posições.

5.12 Ver §5.9: definição (raiz e folhas nulas pretas, nunca dois vermelhos seguidos, mesma altura negra em todos os caminhos); $h \leq 2 \log_2(n + 1)$, porque a subárvore de x tem pelo menos $2^{bh(x)} - 1$ nós internos e pelo menos metade dos nós de um caminho é preta; inserção com no máximo duas rotações (casos do

tio vermelho e do tio preto) e remoção com no máximo três.

5.13 Há rotação simples à esquerda em 1 (ao inserir 3), em 3 (ao inserir 5), em 2 (ao inserir 6; 2 é a raiz nesse momento) e em 5 (ao inserir 7). A árvore final é perfeita: $4(2(1,3),\ 6(5,7))$, com todos os balanços iguais a 0.

5.14 Inserir 30: rotação simples à esquerda em 10, dando $20(10, 30)$. Inserir 25 e 28: o nó 30 fica com -2 e o filho 25 com $+1$; rotação dupla à direita em 30, dando $20(10,\ 28(25, 30))$. Inserir 27 (à direita de 25): o nó 20 fica com $+2$ e o filho 28 com -1; rotação dupla à esquerda em 20. Árvore final: $25^{0}(20^{-1}(10, \cdot),\ 28^{0}(27, 30))$.

5.15 Remover 1: $4(2^{+1}(\cdot, 3),\ 6(5,7))$, sem rotação. Remover 3: $4^{+1}(2,\ 6(5,7))$, sem rotação. Remover 2: o nó 4 fica com $+2$ e o filho 6 com balanço 0, caso exclusivo da remoção; rotação simples à esquerda em 4, dando $6^{-1}(4^{+1}(\cdot, 5),\ 7)$, e a altura não diminuiu.

5.16 $N(1), \ldots, N(7) = 1, 2, 4, 7, 12, 20, 33$. Com 20 nós cabe uma AVL mínima de altura 6 (uma árvore de Fibonacci), e a altura 7 exigiria 33 nós: a altura máxima é 6. Com 32 nós, ainda $N(7) = 33 > 32$: a altura máxima continua 6.

5.17 Na rotação simples à direita em p, com filho esquerdo u: antes, o percurso em ordem é T_1, u, T_2, p, T_3; depois, com u na raiz, T_1 à esquerda e p à direita com T_2 e T_3, ele é T_1, u, T_2, p, T_3 — o mesmo. A rotação à esquerda é simétrica, e a dupla é a composição de duas simples. Como uma árvore binária é ABB exatamente quando o percurso em ordem é crescente, a propriedade se mantém.

5.18 $C_3 = 5$ e $C_4 = 14$ (números de Catalan). Escolhendo a raiz r, a subárvore esquerda guarda $r-1$ chaves e a direita $n-r$, logo $C_n = \sum_{r=1}^{n} C_{r-1}C_{n-r}$, com $C_0 = 1$. Os valores são $1, 1, 2, 5, 14, 42, 132$.

5.19 O maior elemento da subárvore esquerda é 25, que toma o lugar de 30 e é retirado da sua posição. O nó 20 fica sem filho direito e com a subárvore esquerda $10(5, 15)$ de altura 2: $\mathrm{bal}(20) = -2$, com o filho 10 de balanço 0. Rotação simples à direita em 20 (caso exclusivo da remoção): a subárvore passa a ser $10^{+1}(5,\ 20^{-1}(15, \cdot))$, sem diminuir de altura, e o processo para. Árvore final: $25(10(5, 20(15, \cdot)),\ 50(40(35, 45), 70(60, 80)))$.

5.20 Rotações: ao inserir 1, simples à direita em 3; ao inserir 5, simples à esquerda em 3; ao inserir 6, simples à esquerda em 2 (a raiz); ao inserir 7, simples à esquerda em 5; ao inserir 15, dupla à esquerda em 7; ao inserir 14, dupla à esquerda em 6. As inserções de 3, 2, 4 e 16 não causam rotação. Árvore final: $4^{+1}(2(1,3),\ 7(6^{-1}(5, \cdot),\ 15(14, 16)))$.

5.21 Uma árvore binária de altura h tem no máximo $1 + 2 + \cdots + 2^{h-1} = 2^h - 1$ nós; logo $n \le 2^h - 1$, isto é, $h \ge \log_2(n+1)$, e, como h é inteiro, $h \ge \lceil \log_2(n+1) \rceil$. Com o Teorema 5.6, a altura de uma AVL fica entre $\log_2(n+1)$ e $1{,}44 \log_2(n+2)$: é $\Theta(\log n)$.

5.22 Uma função recursiva que, para cada nó, devolve (é válida, altura, menor chave, maior chave) da subárvore: a subárvore de v é válida se as dos filhos são válidas, se a maior chave da esquerda é $\le v.chave \le$ a menor chave da direita e se as alturas dos filhos diferem de no máximo 1. Cada nó é visitado uma vez: $O(n)$.

5.23 Se x tem filho direito, o sucessor é o mínimo da subárvore direita (desça uma vez à direita e depois sempre à esquerda). Senão, suba pelos pais enquanto x for filho direito; o sucessor é o primeiro ancestral do qual x está na subárvore esquerda (ou não existe, se chegar à raiz por um caminho só de filhos direitos). Em ambos os casos o caminho percorrido tem comprimento no máximo h.

Capítulo 6: Ordenação por comparação e limite inferior

6.1 Ver o procedimento Intercala da §6.3: dois índices percorrem as metades, o menor elemento apontado vai para o vetor auxiliar ("$\le$" para manter a estabilidade), o que sobra de uma das metades é copiado ao final e o auxiliar volta para o vetor. Custo $\Theta(n)$.

6.2 Intercale blocos de tamanho $1, 2, 4, \ldots$:

Mergesort iterativo (de baixo para cima)

$tam := 1$
enquanto $tam < n$ **faça**

```
ini := 1
enquanto ini + tam ≤ n faça
    meio := ini + tam − 1; fim := min{ini + 2 tam − 1, n}
    Intercala(V, ini, meio, fim); ini := ini + 2 tam
tam := 2 tam
```

São $\lceil \log_2 n \rceil$ passadas de custo $O(n)$: $\Theta(n \log n)$, sem pilha de recursão.

6.3 Se as chaves são inteiros em $[0, k]$, conte as ocorrências de cada valor e use as contagens para posicionar os elementos (*counting sort*, $\Theta(n + k)$). Se têm d dígitos na base b, ordene dígito a dígito, do menos significativo para o mais significativo, com um método estável (*radix sort*, $\Theta(d(n + b))$). Se são reais uniformemente distribuídos em $[0, 1)$, distribua-os em n baldes (*bucket sort*, $\Theta(n)$ esperado). Todas usam a estrutura das chaves (Capítulo 7); para chaves arbitrárias, só resta comparar.

6.4 (a) Sim: na intercalação, em caso de empate, copie o elemento da metade esquerda (teste "$\leq$"). Por indução, as metades estão ordenadas de forma estável, e a intercalação preserva a ordem relativa dos iguais, que vêm todos da esquerda antes dos da direita. (b) A versão in-place usual não é estável: em $[2_a, 2_b, 1]$, com pivô 1, a troca de 2_a com 1 coloca 2_a depois de 2_b. É possível torná-lo estável com uma partição que copia os menores, os iguais e os maiores para vetores auxiliares na ordem original, ao custo de $\Theta(n)$ de memória extra, ou usando como chave o par (valor, posição original).

6.5 (a) $E = 2$ e $f = n = O(n^{2-\varepsilon})$: $\Theta(n^2)$. (b) $f = \Theta(n^2)$: $\Theta(n^2 \log n)$. (c) $f = \Omega(n^{2+\varepsilon})$ e $4(n/2)^3 = n^3/2$: $\Theta(n^3)$. (d) $E = 1/2$ e $f = \Theta(\sqrt{n})$: $\Theta(\sqrt{n} \log n)$. (e) $E = 2$: $\Theta(n^2)$. (f) $E = 0$, $f = n$ e $n/2 \leq cn$: $\Theta(n)$.

6.6 Cada nível custa no máximo n, pois os subproblemas de um nível somam no máximo n. O ramo mais curto ($n \to n/3 \to \cdots$) tem $\log_3 n$ níveis completos e o mais longo ($n \to 2n/3 \to \cdots$) tem $\log_{3/2} n$ níveis. Logo $n \log_3 n \leq T(n) \leq n \log_{3/2} n + O(n)$: $\Theta(n \log n)$.

6.7 $W(1) = 0$ e $W(n) = 2W(n/2) + (n - 1)$, pois a intercalação de duas listas de tamanho $n/2$ faz no máximo $n - 1$ comparações. $W(2) = 1$, $W(4) = 5$ e $W(8) = 17$. Em geral, $W(n) = n \log_2 n - n + 1$ para n potência de 2.

6.8 Todo elemento que não é o máximo precisa "perder" pelo menos uma comparação; senão, o algoritmo não teria como descartá-lo, e um adversário poderia torná-lo o maior. Cada comparação tem um único perdedor, logo são necessárias pelo menos $n - 1$ comparações. A árvore de decisão dá só $\log_2 n$, porque há apenas n respostas possíveis — um limite fraco.

6.9 Com todos os elementos iguais, o teste $V[j] < x$ nunca é verdadeiro: o pivô fica na primeira posição e as partes têm tamanhos 0 e $n - 1$. Então $T(n) = T(n - 1) + \Theta(n) = \Theta(n^2)$. Correção: particionar em três partes ($< x$, $= x$ e $> x$) e não recursar na parte dos iguais, ou usar uma partição que divide os iguais entre os dois lados.

6.10 Ao inserir $V[i]$, ele passa por cima exatamente dos elementos anteriores maiores que ele, isto é, das inversões (j, i) com $j < i$. Cada deslocamento desfaz uma inversão e nenhuma é criada. O total é o número de inversões: até $\binom{n}{2}$ no pior caso (vetor invertido) e $\binom{n}{2}/2$ em média, pois cada par está invertido com probabilidade $1/2$. Logo $\Theta(n^2)$ nos dois casos.

6.11 Considere a entrada $a_1 < b_1 < a_2 < b_2 < \cdots < a_n < b_n$. Cada par de elementos vizinhos na saída, (a_i, b_i) ou (b_i, a_{i+1}), precisa ser comparado: se um deles não fosse, seria possível trocar os dois valores sem alterar nenhuma resposta das comparações feitas, e a saída ficaria errada. São $2n - 1$ pares vizinhos, todos distintos.

6.12 As intercalações acontecem nesta ordem: $[27, 38]$, $[3, 43]$, $[3, 27, 38, 43]$, $[9, 82]$, $[9, 10, 82]$ e, por fim, $[3, 9, 10, 27, 38, 43, 82]$.

6.13 Resultado $[2, 1, 3, 4, 7, 5, 6, 8]$, e a posição devolvida é 4 (o pivô 4 ocupa sua posição definitiva).

6.14 Num vetor ordenado, o pivô (o último elemento) é sempre o máximo do intervalo: uma partição de k elementos faz $k - 1$ comparações e deixa uma parte com $k - 1$ elementos e outra vazia. O total é $\sum_{k=2}^{n}(k - 1) = \frac{n(n-1)}{2}$.

6.15 Para $n = 1$, $T(1) = 0 = 1 \cdot \log_2 1$. Supondo $T(n/2) = \frac{n}{2} \log_2 \frac{n}{2}$, temos $T(n) = 2 \cdot \frac{n}{2}(\log_2 n - 1) + n = n \log_2 n - n + n = n \log_2 n$.

6.16 A ordenação por seleção faz sempre $\sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2}$ comparações e no máximo $n-1$ trocas. Não é estável: em $[2_a, 2_b, 1]$, a primeira troca leva 2_a para o fim, depois de 2_b. A ordenação por inserção é estável, pois um elemento só passa por cima dos estritamente maiores.

6.17 A árvore de decisão precisa de pelo menos $5! = 120$ folhas, logo de altura $\geq \lceil \log_2 120 \rceil = 7$. Para 12 elementos, $\lceil \log_2(12!) \rceil = \lceil 28{,}8 \rceil = 29$.

6.18 Se cada um dos k valores aparece n/k vezes, há $\frac{n!}{((n/k)!)^k}$ saídas distintas possíveis, e a altura da árvore de decisão é pelo menos o logaritmo desse número, que é $n \log n - n \log(n/k) - O(n) = \Omega(n \log k)$ (usando $\log m! = m \log m - O(m)$). Algoritmo: insira os elementos numa árvore AVL de valores *distintos*, com um contador por valor; a árvore tem no máximo k nós, então cada inserção custa $O(\log k)$; depois, percorra-a em ordem, escrevendo cada valor tantas vezes quanto seu contador. Total $O(n \log k)$.

6.19 O nível j tem 2^j subproblemas de tamanho $n/2^j$, com custo total $n \log(n/2^j) = n(\log n - j)$. Somando de $j = 0$ a $\log n - 1$: $n \sum_j (\log n - j) = n \cdot \frac{\log n(\log n+1)}{2} = \Theta(n \log^2 n)$. O Teorema Mestre não se aplica porque $f(n) = n \log n$ é maior que $n^E = n$, mas não por um fator polinomial n^ε: cai no "vão" entre os casos 2 e 3.

6.20 (a) $T(n) = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$. (b) $T(n) = 2^n - 1$: por indução, $2(2^{n-1} - 1) + 1 = 2^n - 1$. (c) $T(n) = 1 + \sum_{i=2}^{n} \log i = 1 + \log(n!) = \Theta(n \log n)$.

Capítulo 7: Ordenação em tempo linear

7.1 (A) **Counting sort**: $k = 101$ é muito menor que n, o custo é $\Theta(n + k) = \Theta(n)$ e bastam 101 contadores; uma ordenação por comparação faria cerca de $n \log_2 n \approx 2{,}3 \cdot 10^8$ comparações. (B) **Radix sort**: o counting sort é inviável ($k = 10^{12}$); com $d = 12$ e $b = 10$ são 12 passadas lineares, $\Theta(d(n + b)) \approx 1{,}2 \cdot 10^7$ (ou 4 e 3 passadas nas bases 10^3 e 10^4). (C) **Bucket sort** com n baldes: $\Theta(n)$ esperado para entrada uniforme; counting e radix não se aplicam a reais. (D) **Radix sort** depois de somar 10^9 a todos os valores (as chaves passam a estar em $[0, 2 \cdot 10^9]$, com cerca de 31 bits: 2 passadas na base 2^{16}); o counting sort exigiria $2 \cdot 10^9$ contadores. Mergesort ou quicksort, em $\Theta(n \log n)$, seriam aceitáveis, mas o radix é linear. (E) Só resta comparar: **mergesort**, com garantia de $\Theta(n \log n)$ e estabilidade, boa para registros; ou quicksort, mais rápido em média e in-place, mas $\Theta(n^2)$ no pior caso e não estável.

7.2 Radix: agrupe os números pelo número de dígitos (um counting sort por d_i). Como um número com menos dígitos é menor, basta ordenar cada grupo separadamente com um radix de d passadas, sendo d o tamanho dos números do grupo, e concatenar os grupos em ordem crescente de d. Um grupo com n_d números de d dígitos custa $O(d\,(n_d + b))$; somando, $O\big(\sum_i d_i + b \cdot D\big)$, onde $D \leq \max d_i$ é o número de tamanhos distintos. American Flag: primeiro separe por tamanho (mesma ideia); dentro de cada grupo, a recursão por dígito processa cada dígito de cada número uma única vez, com $O(b)$ de trabalho por chamada.

7.3 Radix (do último caractere para o primeiro): complete as strings menores à direita com um símbolo "menor que todos" até o comprimento máximo L e ordene da última posição para a primeira, com counting sort estável sobre o alfabeto, em $O(L(n + |\Sigma|))$; para não pagar o custo de completar, agrupe por comprimento como no exercício anterior. American Flag (do primeiro caractere para o último): distribua pelo primeiro caractere, com um balde extra para "a string terminou", que vem antes de todos, e chame recursivamente cada balde com o caractere seguinte; strings que terminam saem da recursão.

7.4 Contagens: $C = [2, 0, 2, 3, 0, 1]$. Depois das somas acumuladas: $C = [2, 2, 4, 7, 7, 8]$. Percorrendo A de trás para frente, cada $A[j]$ vai para a posição $C[A[j]]$, que é então decrementada. Saída: $B = [0, 0, 2, 2, 3, 3, 3, 5]$.

7.5 Pelas unidades: 720, 355, 436, 457, 657, 329, 839. Pelas dezenas: 720, 329, 436, 839, 355, 457, 657. Pelas centenas: 329, 355, 436, 457, 657, 720, 839.

7.6 Escreva cada número na base n: como $x < n^3$, ele tem no máximo 3 dígitos, $x \bmod n$, $\lfloor x/n \rfloor \bmod n$ e $\lfloor x/n^2 \rfloor$. O radix sort com 3 passadas de counting sort, cada uma com $k = n$, custa $\Theta(3(n + n)) = \Theta(n)$.

7.7 A saída continua ordenada, pois cada valor ainda ocupa o seu bloco, mas os elementos iguais saem em ordem *invertida*: o primeiro a ser lido vai para a última posição do bloco. O counting sort deixa de ser estável, e o radix sort que o usa em cada passada **deixa de ordenar corretamente**, porque cada passada desfaz a ordem obtida pelos dígitos anteriores.

7.8 Se todos os elementos caírem no mesmo balde, a inserção custa $\Theta(n^2)$. Ordene cada balde com mergesort ou heapsort: o pior caso passa a $O(n \log n)$, e o caso esperado continua $\Theta(n)$, porque um balde com n_i elementos custa $O(n_i \log n_i) \leq O(n_i^2)$ e $\sum_i E[n_i^2] = O(n)$.

7.9 No radix que começa pelo dígito menos significativo, a passada pelo dígito i precisa preservar, entre elementos com o mesmo dígito i, a ordem estabelecida pelos dígitos $1, \ldots, i-1$, que é a única informação sobre eles. No que começa pelo mais significativo, depois de separar pelo primeiro dígito, cada bloco é ordenado recursivamente e de forma independente pelos dígitos seguintes; a ordem dentro do bloco antes da recursão não importa, porque a recursão a refaz.

7.10 Some m a todos os valores, que passam a estar em $[0, 2m]$, ordene com o counting sort e subtraia m na saída (ou indexe o vetor de contadores por $x + m$). O custo é $\Theta(n + m)$.

7.11 O counting sort direto usaria $k = n^2$ contadores e custaria $\Theta(n^2)$. Na base n, cada número tem 2 dígitos, e o radix sort faz 2 passadas de counting sort com $k = n$: $\Theta(n)$ de tempo e de memória.

7.12 Pela terceira letra: SEA TEA MOB TAB DOG RUG DIG BIG BAR EAR TAR COW ROW NOW BOX FOX. Pela segunda: TAB BAR EAR TAR SEA TEA DIG BIG MOB DOG COW ROW NOW BOX FOX RUG. Pela primeira: BAR BIG BOX COW DIG DOG EAR FOX MOB NOW ROW RUG SEA TAB TAR TEA.

7.13 Balde 1: 0,17; 0,12. Balde 2: 0,26; 0,21; 0,23. Balde 3: 0,39. Balde 6: 0,68. Balde 7: 0,78; 0,72. Balde 9: 0,94. Ordenando cada balde e concatenando: 0,12; 0,17; 0,21; 0,23; 0,26; 0,39; 0,68; 0,72; 0,78; 0,94.

7.14 Com $b = 2^4$: 8 passadas de custo $\approx n + 16$. Com $b = 2^8$: 4 passadas de custo $\approx n + 256$. Com $b = 2^{16}$: 2 passadas de custo $\approx n + 65536$. Como $n = 10^6$, o termo b é desprezível nas três, e a base 2^{16} faz menos passadas, com custo total $\approx 2 \cdot 10^6$. Na prática, a base 2^8 também é muito usada, porque o vetor de contadores cabe na memória cache.

7.15 Hipótese: depois da passada i, a lista está ordenada pelos i dígitos menos significativos. Para $i = 1$ vale, pois a passada ordena pelo dígito 1. Suponha válido para $i - 1$ e tome dois números x e y. Se os seus dígitos i diferem, a passada i os coloca na ordem desse dígito, que decide a comparação entre os i últimos dígitos. Se são iguais, a passada, por ser estável, mantém a ordem relativa que eles tinham, que, pela hipótese de indução, é a ordem pelos $i - 1$ últimos dígitos. Em ambos os casos, ficam na ordem certa pelos i últimos dígitos.

7.16 Counting sort pela nota ($k = 11$), que é **estável**: alunos com a mesma nota continuam na ordem em que estavam, isto é, em ordem alfabética. O custo é $\Theta(n)$. Um algoritmo não estável, como o quicksort ou o heapsort, poderia desfazer a ordem alfabética.

7.17 Do menos para o mais significativo: primeiro pelo dia (base 31 ou 32), depois pelo mês (base 12) e por fim pelo ano (base igual à amplitude dos anos, ou em duas passadas de dígitos), sempre com passadas estáveis.

7.18 Ordene com o radix sort na base $n + 1$: os números de $[0, n^2]$ têm no máximo 3 dígitos nessa base (2 bastam para valores menores que $(n + 1)^2$), e a ordenação custa $O(n)$. Depois, verifique se há dois elementos vizinhos iguais.

7.19 Calcule as contagens acumuladas $C[i] = \#\{j : A[j] \leq i\}$, como no counting sort, em $O(n + k)$. A resposta para $[a, b]$ é $C[b] - C[a - 1]$ (com $C[-1] = 0$), em $O(1)$.

7.20 Por exemplo, $x_i = \frac{i}{n^2}$ para $i = 1, \ldots, n$: todos ficam no intervalo $[0, 1/n)$ e caem no balde 0. Se estiverem em ordem decrescente na entrada, a inserção nesse balde faz $\Theta(n^2)$ deslocamentos.

7.21 Cada nível da recursão usa um vetor de b contadores (e as posições de início dos blocos); a recursão tem profundidade d, de modo que a memória auxiliar é $O(b \cdot d)$, mais a pilha de recursão. Diz-se que ele é in-place porque essa memória não depende de n: os elementos são permutados dentro do próprio vetor, sem um vetor auxiliar de tamanho n.

Capítulo 8: Seleção em tempo linear: a mediana das medianas

8.1 Quickselect com pivô aleatório (§8): particione em $S_1 = \{< m\}$, $S_2 = \{= m\}$ e $S_3 = \{> m\}$ e recursione apenas no lado que contém o k-ésimo, com $k = \lceil n/2 \rceil$. Com probabilidade pelo menos $1/2$, o pivô fica entre os quartis e o lado mantido tem no máximo $3n/4$ elementos; agrupando as chamadas em fases, $E[T(n)] \leq E[T(3n/4)] + 2cn$, e portanto $E[T(n)] \leq 8cn = O(n)$.

8.2 Use como pivô a mediana exata, obtida pela mediana das medianas em $O(n)$. As duas partes têm no máximo $n/2$ elementos, e $T(n) = 2T(n/2) + O(n) = O(n \log n)$ no **pior** caso. Na prática, as constantes da mediana das medianas (grupos, medianas, duas recursões) tornam esse algoritmo bem mais lento que o quicksort com pivô aleatório ou mediana de três, cujo pior caso é raríssimo. Por isso, implementações reais preferem trocar para o heapsort quando a recursão fica profunda demais.

8.3 Medianas dos grupos: $12, 9, 11, 14, 16$; pivô $m = 12$, a mediana das medianas. $|S_1| = 11$ (os elementos $1, \ldots, 11$) e $|S_3| = 13$; ambos são pelo menos $3\lfloor 25/10 \rfloor = 6$. (O algoritmo, de fato, ordenaria diretamente, pois $n \leq 49$; aqui o exemplo serve só para praticar a escolha do pivô.)

8.4 Grupos de 7: pelo menos metade das $n/7$ medianas é maior ou igual a m, e cada uma traz 4 elementos maiores ou iguais a m; há pelo menos $4 \cdot \frac{n}{14} = \frac{2n}{7}$ elementos de cada lado, e cada parte tem no máximo $\frac{5n}{7}$ (mais uma constante). Então $T(n) \leq T(n/7) + T(5n/7) + O(n)$, com $\frac{1}{7} + \frac{5}{7} = \frac{6}{7} < 1$: o algoritmo é **linear**. Grupos de 3: só há $2 \cdot \frac{n}{6} = \frac{n}{3}$ elementos garantidos de cada lado, e $T(n) \leq T(n/3) + T(2n/3) + O(n)$, com $\frac{1}{3} + \frac{2}{3} = 1$: $\Theta(n \log n)$, **não** linear.

8.5 Encontre o k-ésimo menor x pela mediana das medianas, em $O(n)$, e percorra o conjunto separando os elementos menores que x, completando com cópias de x até ter k elementos: $O(n)$. Para devolvê-los em ordem crescente, ordene esses k elementos: $O(n + k \log k)$.

8.6 O comprimento total é $\sum_i |y_i - y|$, que é minimizado em qualquer **mediana** dos y_i: se há mais poços acima do que abaixo de y, subir um pouco o oleoduto diminui a soma, e vice-versa; na mediana, os dois lados se equilibram. A mediana é encontrada em $O(n)$.

8.7 Se o pivô escolhido for sempre o maior elemento — por exemplo, com o pivô determinístico "último elemento" num vetor crescente, ou por um azar sistemático na escolha aleatória —, cada chamada descarta apenas o pivô, e $T(n) = T(n-1) + \Theta(n) = \Theta(n^2)$. Isso acontece mesmo procurando o mínimo ($k = 1$). Com pivô aleatório, esse caso tem probabilidade desprezível.

8.8 1ª chamada: pivô 1, $S_1 = \emptyset$, $S_2 = \{1\}$, $S_3 = \{7, 2, 9, 4, 3, 8, 6\}$; como $0 < 4$ e $1 < 4$, segue para S_3 com $k = 3$. 2ª: pivô 6, $S_1 = \{2, 4, 3\}$, $S_2 = \{6\}$, $S_3 = \{7, 9, 8\}$; como $|S_1| = 3 \geq 3$, segue para S_1 com $k = 3$. 3ª: pivô 3, $S_1 = \{2\}$, $S_2 = \{3\}$, $S_3 = \{4\}$; como $1 < 3$ e $2 < 3$, segue para S_3 com $k = 1$. 4ª: pivô 4, $S_2 = \{4\}$: devolve **4**, que é de fato o quarto menor.

8.9 Para $n = 49$: $49 - 3 \cdot 4 = 37 > 36{,}75 = 3 \cdot 49/4$. Para $n = 50$: $50 - 3 \cdot 5 = 35 \leq 37{,}5$. A garantia "cada lado tem no máximo $3n/4$" só vale a partir de $n = 50$; por isso os conjuntos com até 49 elementos são resolvidos diretamente.

8.10 Ordenar as $n/5$ medianas custa $O(\frac{n}{5} \log \frac{n}{5}) = O(n \log n)$, e a recorrência passa a ser $T(n) = T(3n/4) + O(n \log n)$, cuja solução é $O(n \log n)$ (soma geométrica). O algoritmo continua correto, mas deixa de ser linear: a recursão sobre as medianas é essencial.

8.11 Sim. Formar os grupos de 5, calcular suas medianas e particionar em S_1, S_2, S_3 são operações que percorrem a sequência uma vez, o que se faz numa lista encadeada em tempo linear; nenhum passo depende de acesso direto por posição.

8.12 Selecione o elemento de posição $\lceil n/2 \rceil$ (a mediana) e particione; depois selecione, em cada metade, o elemento do meio. São três seleções sobre conjuntos de tamanho no máximo n, em $O(n)$ no total.

8.13 Ache a mediana m em $O(n)$. Calcule $d_i = |x_i - m|$ para todo i e selecione o k-ésimo menor d, digamos d^*, em $O(n)$. Os k números mais próximos são os x_i com $d_i < d^*$, completados com elementos de $d_i = d^*$ até somar k (excluindo a própria mediana, se ela não deve contar). Total $O(n)$.

8.14 Recursão parecida com a seleção: ache a mediana m (por valor) do conjunto atual em $O(n)$ e some os pesos $W_<$ dos elementos menores que m e $W_>$ dos maiores. Se $W_< < 1/2$ e $W_> \leq 1/2$, m é a resposta. Se $W_< \geq 1/2$, a resposta está entre os menores: descarte os maiores (e m), somando o peso deles a um contador do lado direito. Caso contrário, faça o simétrico. Cada passo descarta metade dos elementos: $T(n) = T(n/2) + O(n) = O(n)$.

8.15 Compare as medianas $X[\lceil n/2 \rceil]$ e $Y[\lceil n/2 \rceil]$. Se a de X é menor, a mediana procurada não está na metade inferior de X nem na metade superior de Y; descarte essas duas metades, de mesmo tamanho, o que não muda a mediana do conjunto restante. Se é maior, descarte as outras duas metades. Repita até sobrarem poucos elementos e resolva diretamente. Cada passo divide n por 2: $O(\log n)$.

8.16 Organize um torneio eliminatório: compare os elementos aos pares e faça os vencedores (os menores) se enfrentarem, até restar o mínimo, com $n-1$ comparações. O segundo menor perdeu, necessariamente, para o mínimo em algum confronto. O mínimo disputou no máximo $\lceil \log_2 n \rceil$ confrontos; o menor dos que perderam para ele, encontrado com $\lceil \log_2 n \rceil - 1$ comparações, é o segundo menor. Total: $n + \lceil \log_2 n \rceil - 2$.

8.17 Sejam a, b, c, d, e. Compare a com b e c com d; suponha $a < b$ e $c < d$ (2 comparações). Compare os menores, a e c; suponha $a < c$ (3). Então a é menor que b, c e d, e não pode ser a mediana; a mediana dos cinco é o segundo menor de $\{b, c, d, e\}$, com $c < d$ já sabido. Compare b com e; suponha $b < e$ (4). Compare b com c (5): se $b < c$, o menor dos quatro é b e a resposta é $\min\{c, e\}$; se $c < b$, o menor é c e a resposta é $\min\{b, d\}$. A última comparação (6) decide.

8.18 Considere o grafo com os n elementos como vértices e uma aresta para cada comparação feita. Se ele fosse desconexo, haveria uma componente C que não contém a mediana devolvida x. Somando a todos os elementos de C uma constante muito grande (se algum elemento de C é menor que x) ou subtraindo (se todos são maiores), todas as comparações feitas continuariam com o mesmo resultado, mas o número de elementos menores que x mudaria, e x deixaria de ser a mediana. Logo o grafo é conexo e tem pelo menos $n-1$ arestas.

8.19 Com pivô aleatório, cada posição final i do pivô tem probabilidade $1/n$, e a partição custa no máximo n comparações. No pior caso, a recursão vai para o lado maior, de tamanho $\max\{i-1, n-i\}$, e cada tamanho entre $\lfloor n/2 \rfloor$ e $n-1$ aparece no máximo duas vezes; daí a desigualdade. Supondo $E[T(i)] \le 4i$ para $i < n$: $E[T(n)] \le n + \frac{2}{n}\sum_{i=\lfloor n/2 \rfloor}^{n-1} 4i \le n + \frac{8}{n} \cdot \frac{3n^2}{8} = 4n$, usando $\sum_{i=\lfloor n/2 \rfloor}^{n-1} i \le \frac{3n^2}{8}$.

8.20 O k-ésimo maior é o $(n-k+1)$-ésimo menor. O percentil 90 de n notas é o elemento de posição $\lceil 0{,}9n \rceil$ na ordem crescente, obtido por uma seleção em $O(n)$.

Capítulo 9: Divisão e conquista

9.1 Ver §9: o melhor subvetor está inteiro na metade esquerda, inteiro na direita ou cruza o meio; o cruzado é o melhor sufixo que termina em $meio$ mais o melhor prefixo que começa em $meio+1$, ambos calculados em $O(n)$. $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$. O caso base com um elemento garante subvetor não vazio e resolve o caso em que todos os elementos são negativos.

9.2 Ver §9: particione A e B em blocos $\frac{n}{2} \times \frac{n}{2}$, calcule recursivamente $M_1, \ldots, M_7$ e combine $C_{11} = M_1 + M_4 - M_5 + M_7$, $C_{12} = M_3 + M_5$, $C_{21} = M_2 + M_4$ e $C_{22} = M_1 - M_2 + M_3 + M_6$. $T(n) = 7T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7})$.

9.3 Na árvore de recursão, os subproblemas de um mesmo nível são partes disjuntas da entrada, então a soma dos seus tamanhos é no máximo n, e o trabalho linear de dividir e combinar, somado no nível, é $O(n)$. Como o tamanho cai por um fator de pelo menos c a cada nível, há no máximo $\log_c n + 1$ níveis. O total é $O(n \log_c n) = O(n \log n)$. Para partes iguais, é a recorrência $T(n) = c\,T(n/c) + an$, resolvida pelo caso 2 do Teorema Mestre.

9.4 Ver as funções CalculaInversões e IntercalaContando da §9: um mergesort que, ao copiar um elemento da metade direita, soma $meio - i + 1$, a quantidade de elementos que restam na esquerda, todos maiores que ele. $\Theta(n \log n)$.

9.5 (a) 3: $(2,1)$, $(4,1)$ e $(4,3)$. (b) $10 = \binom{5}{2}$. (c) 8. O máximo é $\binom{n}{2}$, atingido pelo vetor decrescente.

9.6 Compare $V[meio]$ com $V[meio+1]$: se $V[meio] < V[meio+1]$, o pico está em $[meio+1, fim]$; senão, está em $[ini, meio]$. $T(n) = T(n/2) + O(1) = O(\log n)$.

9.7 Faça duas buscas binárias: uma pela primeira posição com $V[i] \ge x$ e outra pela primeira posição com $V[i] > x$; a resposta é a diferença entre elas. $O(\log n)$.

9.8 $xy = x_1y_1B^n + \big((x_1+x_0)(y_1+y_0) - x_1y_1 - x_0y_0\big)B^{n/2} + x_0y_0$: são três produtos de números com cerca de $n/2$ dígitos, mais somas e deslocamentos em $O(n)$. $T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1{,}585})$, contra $\Theta(n^2)$ do método escolar.

9.9 $T(2) = 1$ e $T(n) = 2T(n/2) + 2$: uma comparação para o máximo e outra para o mínimo na combinação. Por indução, $T(n) = \frac{3n}{2} - 2$, pois $2\big(\frac{3n}{4} - 2\big) + 2 = \frac{3n}{2} - 2$. Os valores são $1, 4, 10, 22$ para $n = 2, 4, 8, 16$, contra $2n-2$ do método ingênuo.

9.10 Algoritmo de votação de Boyer e Moore: mantenha um candidato e um contador. Para cada elemento,

se o contador é 0, o elemento vira o candidato, com contador 1; senão, some 1 se ele é igual ao candidato e subtraia 1 se é diferente. Cada decremento "cancela" um par de elementos diferentes, e um elemento majoritário não pode ser totalmente cancelado; logo, se existir, ele é o candidato final. Uma segunda passada confere se o candidato aparece mais de $n/2$ vezes. Tempo $O(n)$ e memória $O(1)$.

9.11 **Enquanto** $ini \leq fim$: $meio := \lfloor (ini + fim)/2 \rfloor$; se $V[meio] = x$, devolva $meio$; se $x < V[meio]$, faça $fim := meio - 1$; senão, $ini := meio + 1$. Ao final, devolva -1. Cada iteração reduz o tamanho s do intervalo para no máximo $\lfloor s/2 \rfloor$, então, partindo de n, após t iterações sobram no máximo $\lfloor n/2^t \rfloor$ elementos; o intervalo fica vazio após no máximo $\lfloor \log_2 n \rfloor + 1$ iterações, cada uma com uma comparação (contando o teste de igualdade e o de ordem como uma comparação de três resultados).

9.12 Cada nível da recursão faz uma ou duas multiplicações, e há $\lfloor \log_2 n \rfloor + 1$ níveis com $n \geq 1$; logo, no máximo $2\lfloor \log_2 n \rfloor + 2$ multiplicações. Para $n = 13 = 1101_2$: os níveis com $n = 1$, 3 e 13 são ímpares (2 multiplicações cada, contando a do nível $n = 1$, que multiplica $1 \cdot 1 \cdot a$) e o nível com $n = 6$ é par (1 multiplicação): 7 ao todo, ou 5 se não contarmos as duas multiplicações por 1 do nível $n = 1$.

9.13 Para cada intervalo, devolva $(tot, pref, suf, melhor)$. No caso base, com um elemento a, todos valem a. Na combinação de E (esquerda) com D (direita): $tot = tot_E + tot_D$; $pref = \max\{pref_E,\ tot_E + pref_D\}$; $suf = \max\{suf_D,\ tot_D + suf_E\}$; $melhor = \max\{melhor_E,\ melhor_D,\ suf_E + pref_D\}$. A combinação custa $O(1)$, e $T(n) = 2T(n/2) + O(1) = \Theta(n)$.

9.14 $C_{11} = A_{11}B_{11} + A_{12}B_{21}$, $C_{12} = A_{11}B_{12} + A_{12}B_{22}$, $C_{21} = A_{21}B_{11} + A_{22}B_{21}$ e $C_{22} = A_{21}B_{12} + A_{22}B_{22}$: 8 produtos recursivos e 4 somas de blocos $\frac{n}{2} \times \frac{n}{2}$ por chamada. $T(n) = 8T(n/2) + \Theta(n^2) = \Theta(n^{\log_2 8}) = \Theta(n^3)$.

9.15 Antes do número que falta, $A[i] = i$; a partir dele, $A[i] = i + 1$. Por busca binária, encontre a primeira posição i com $A[i] \neq i$: o número que falta é i (ou $n + 1$, se não houver tal posição). $O(\log n)$.

9.16 Como os elementos são inteiros distintos e ordenados, $A[i + 1] - (i + 1) \geq A[i] - i$: a função $g(i) = A[i] - i$ é não decrescente. Faça uma busca binária por um zero de g: se $g(meio) < 0$, procure à direita; se $g(meio) > 0$, à esquerda. $O(\log n)$.

9.17 Busca binária pela primeira posição com valor 1; o número de uns é n menos essa posição mais 1. Se n é desconhecido, consulte as posições $1, 2, 4, 8, \ldots$ até encontrar um 1 (ou o fim): isso delimita, em $O(\log n)$ consultas, um intervalo onde fazer a busca binária (busca exponencial).

9.18 Como na contagem de inversões, recursione nas metades, que voltam ordenadas. Antes de intercalar, conte os pares cruzados com dois ponteiros: para cada a_i da esquerda, em ordem crescente, avance um ponteiro j na direita enquanto $a_i > 2b_j$; o número de elementos da direita já ultrapassados é a contagem de a_i, e o ponteiro nunca volta, porque os a_i crescem. Esse passo custa $O(n)$; depois intercale normalmente. Total $O(n \log n)$.

9.19 Ordene os pontos pela abscissa e divida-os pela mediana. Resolva cada lado, obtendo as distâncias mínimas δ_E e δ_D, e seja $\delta = \min\{\delta_E, \delta_D\}$. Um par melhor teria de cruzar a reta divisória e ter os dois pontos na faixa de largura 2δ em torno dela. Ordenando os pontos da faixa pela ordenada, cada ponto só precisa ser comparado com os 7 seguintes: num retângulo $\delta \times 2\delta$ acima dele cabem no máximo 8 pontos com distâncias mútuas $\geq \delta$ dentro de cada lado. Mantendo as listas ordenadas por ordenada durante a recursão (como no mergesort), a combinação custa $O(n)$ e o total é $O(n \log n)$.

9.20 Represente um contorno como uma lista de pontos (x, h) em que a altura muda. Divida os prédios em duas metades, calcule recursivamente os dois contornos e *intercale-os* como no mergesort: percorra as duas listas em ordem de x, mantendo a altura atual de cada contorno, e em cada x registre $\max\{h_1, h_2\}$ sempre que esse máximo mudar. A intercalação custa $O(n)$ e o total é $O(n \log n)$.

9.21 Busca binária: $T(n) = T(n/2) + O(1) = \Theta(\log n)$. Potência: $T(n) = T(n/2) + O(1) = \Theta(\log n)$ multiplicações. MaxMin: $T(n) = 2T(n/2) + 2$, isto é, $\frac{3n}{2} - 2$ comparações. Elemento majoritário: $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$, por causa das contagens na combinação.

Capítulo 10: Escolhendo estruturas de dados e algoritmos

10.1 Cada iteração custa $\Theta(n)$ só para achar o máximo, e o total é $\Theta(n^2)$. Com um **heap** de máximo, achar e remover custa $O(\log n)$, e inserir também: $O(n \log n)$ no total.

10.2 Cada consulta custa $\Theta(n \log n)$. Se q é pequeno, use **seleção linear** (quickselect ou mediana das

medianas) sobre a cópia: $O(n)$ por consulta. Se q é grande, **ordene uma única vez**, em $O(n \log n)$, e responda cada consulta em $O(1)$: total $O(n \log n + q)$.

10.3 Cada busca em largura custa $O(n + m)$, o que é inviável para 10^8 perguntas. Use **Union-Find**: cada amizade é uma Union e cada pergunta compara Find(u) com Find(v). Com rank e compressão, o custo é praticamente $O(1)$ por operação; com listas e ponteiro ao representante, Find é $O(1)$ no pior caso.

10.4 Inserir ou remover num vetor ordenado custa $\Theta(n)$. Use uma **AVL** (ou rubro-negra): inserção e remoção em $O(\log n)$, e a consulta por intervalo em $O(\log n + k)$, sendo k o número de códigos no intervalo (desça até a e percorra em ordem até passar de b). Uma tabela hash não serve, pois não mantém a ordem.

10.5 Uma ordenação por comparação custa $\Theta(n \log n)$, cerca de $2{,}7 \cdot 10^9$ comparações. As chaves são inteiros num intervalo pequeno ($k = 121$): **counting sort**, em $\Theta(n + k)$, com 121 contadores.

10.6 No heap binário a fusão custa $\Theta(n)$. Use o **heap binomial**: fusão em $O(\log n)$ no pior caso de *cada* operação. O heap de Fibonacci faria a fusão em $O(1)$, mas só amortizado — uma remoção pode consolidar tudo de uma vez —, o que fere a previsibilidade exigida.

10.7 **Heap binário em vetor**: inserção e remoção do mais grave em $O(\log n)$, sem fusões, com as raras alterações de gravidade em $O(\log n)$ (com um vetor de posições), e sem gastar memória com ponteiros.

10.8 **Heap** com um vetor (ou tabela) de posições indexado pelo número do evento, para encontrar o evento cancelado e removê-lo de uma posição qualquer em $O(\log n)$ (Exercício 2.21). Uma **AVL** indexada por (horário, número) também resolve, com todas as operações em $O(\log n)$.

10.9 **Tabela hash**: cada verificação custa $O(1)$ esperado (mais o custo de calcular o hash da palavra). Um vetor ordenado com busca binária também serviria, em $O(\log n)$ por consulta, já que o dicionário é fixo.

10.10 Uma **AVL aumentada**: cada nó guarda o tamanho da sua subárvore, atualizado nas inserções, remoções e rotações em $O(1)$ por nó afetado. A posição de x se calcula descendo da raiz e somando os tamanhos das subárvores à esquerda do caminho; o elemento da posição r se encontra descendo e comparando r com esses tamanhos. Tudo em $O(\log n)$.

10.11 Dois heaps: um de máximo com a metade menor das medições e um de mínimo com a metade maior, com tamanhos que diferem de no máximo 1. Cada medição é inserida em $O(\log n)$, com um rebalanceamento eventual, e a mediana é lida no topo de um dos heaps em $O(1)$.

10.12 **Mergesort externo**: leia o arquivo em blocos que cabem na memória (por exemplo, de 12 GB), ordene cada bloco e grave-o; depois intercale os cerca de 42 blocos ordenados com um heap de mínimo de tamanho 42 (intercalação de k listas), lendo e gravando sequencialmente. O custo é dominado pelas duas passadas pelo disco.

10.13 **Ordenação por inserção**: cada elemento se desloca no máximo 10 posições, então o custo é $O(10n) = O(n)$. Um heap de mínimo de tamanho 11 percorrendo o vetor também ordena em $O(n \log 11)$.

10.14 **Tabela hash**: insira as strings uma a uma e verifique se já estão presentes, em $O(1)$ esperado cada, mais o custo de calcular os hashes. Ordenar (por exemplo, com radix sort de strings ou mergesort) e comparar vizinhas também resolve.

10.15 **Union-Find offline, de trás para frente**: comece com a malha sem as estradas que serão interditadas em algum momento, una as regiões ligadas pelas estradas restantes e conte as componentes; depois percorra as interdições da última para a primeira, transformando cada uma numa Union e atualizando o contador. Guarde as respostas e apresente-as na ordem original.

10.16 Um **heap de mínimo de tamanho 100**: para cada requisição, se ela for mais demorada que o mínimo do heap, substitua o mínimo. Memória $O(100)$ e tempo $O(n \log 100)$.

10.17 O bucket sort perde a garantia sem distribuição uniforme: pode cair no pior caso, com quase tudo num balde. Use uma ordenação por comparação com garantia, como o **mergesort** ou o **heapsort**, em $\Theta(n \log n)$; ou converta os reais para inteiros de 64 bits preservando a ordem e use radix sort.

10.18 Cada pergunta custa $O(n)$. Pré-calcule as **somas de prefixo** $P[i] = V[1] + \cdots + V[i]$ em $O(n)$; cada pergunta passa a custar $O(1)$: $P[j] - P[i-1]$.

10.19 Uma **AVL** indexada pela chave: busca, inserção e remoção em $O(\log n)$ e listagem em ordem em

$O(n)$. Uma tabela hash daria buscas em $O(1)$ esperado, mas a listagem em ordem exigiria ordenar a cada vez.

10.20 **Union-Find sem compressão**, com união por rank ou tamanho e uma pilha das alterações: cada ligação empilha o ponteiro e o rank alterados; desfazer é desempilhar e restaurar em $O(1)$; a consulta custa $O(\log n)$.

Capítulo 11: Programação dinâmica

11.1 **Sim.** Se a última multiplicação de uma parentização máxima de $M_i \cdots M_j$ é feita no ponto k, o custo é $c(i..k) + c(k+1..j) + r_{i-1} r_k r_j$, e o último termo depende só de k, não de como cada parte foi parentizada. Se uma das partes não fosse máxima, trocá-la por uma máxima aumentaria o total, uma contradição. A recorrência é a mesma, com max no lugar de min, e o custo é $\Theta(n^3)$.

11.2 Seja $L[i]$ o tamanho da maior subsequência crescente que *termina em* a_i: $L[i] = 1 + \max\{L[j] : j < i,\ a_j < a_i\}$, com $\max \emptyset = 0$; a resposta é $\max_i L[i]$, e guardar o j escolhido em $ant[i]$ permite reconstruí-la. Custo $O(n^2)$. Por exemplo, $5, 2, 8, 6, 3, 6, 9, 7$ dá $L = 1, 1, 2, 2, 2, 3, 4, 4$ e resposta 4 ($2, 3, 6, 9$). Em $O(n \log n)$: mantenha $d[\ell]$ = menor final possível de uma subsequência crescente de tamanho ℓ, que é um vetor crescente; para cada a_i, encontre por busca binária o primeiro $d[\ell] \geq a_i$ e substitua-o por a_i (ou acrescente a_i no fim, se não houver). O tamanho final de d é a resposta.

11.3 Seja $P(i, j)$ o tamanho do maior palíndromo que é subsequência de $s_i..s_j$: $P(i, i) = 1$ e $P(i, i-1) = 0$; se $s_i = s_j$, $P(i, j) = 2 + P(i+1, j-1)$; senão, $P(i, j) = \max\{P(i+1, j), P(i, j-1)\}$. Preencha por tamanho crescente do trecho: $\Theta(n^2)$ de tempo e memória. Para "character", o valor é 5 ("carac"). Esse valor também é o tamanho da subsequência comum máxima entre s e s invertida.

11.4 Ordene os vértices topologicamente (Capítulo 18). Faça $D(s) = 0$ e $D(v) = -\infty$ para os demais; na ordem topológica, $D(v) = \max_{uv \in E}\{D(u) + w(u, v)\}$, guardando o predecessor que deu o máximo. A resposta é $D(t)$, em $\Theta(n + m)$. Num grafo acíclico todo caminho é simples, e por isso a subestrutura ótima vale; em grafos gerais, o caminho simples mais longo é um problema NP-difícil.

11.5 Para $w = 0..W$:

$$K(w) = \max\Big\{0,\ \max_{i:\, p_i \leq w}\{K(w - p_i) + v_i\}\Big\}.$$

$\Theta(nW)$. Por exemplo, com itens $(p, v) = (3, 5), (4, 7), (2, 3)$ e $W = 10$, o máximo é 17 (por exemplo, $4 + 4 + 2$). É o mesmo que a mochila 0-1 em vetor único, com w *crescente*.

11.6 Ordene os pontos e acrescente as sentinelas $L_0 = 0$ e $L_{m+1} = n$. Seja $C(i, j)$ o custo mínimo de fazer todos os cortes estritamente entre L_i e L_j: $C(i, i+1) = 0$ e $C(i, j) = (L_j - L_i) + \min_{i<k<j}\{C(i, k) + C(k, j)\}$, em que L_k é o primeiro corte feito no trecho. A resposta é $C(0, m+1)$; guardando o k ótimo, a sequência de cortes sai em pré-ordem. Custo $\Theta(m^3)$. No exemplo, o ótimo é **38**: corte primeiro em 10 e depois em 2 e 8, em qualquer ordem (as seis ordens custam $50, 46, 40, 40, 38, 38$).

11.7 Percorra a tabela a partir de (n, m): se $x_i = y_j$, coloque x_i no início da resposta e vá para $(i-1, j-1)$; senão, vá para a vizinha, $(i-1, j)$ ou $(i, j-1)$, de maior valor. Custo $O(n + m)$. No exemplo, subindo nos empates, obtém-se **BCBA**; as subsequências comuns de tamanho 4 são BCAB, BCBA e BDAB (§11.2).

11.8 (a) Para $r = (10, 100, 5, 50)$: $m[1, 2] = 5000$, $m[2, 3] = 25\,000$ e $m[1, 3] = \min\{75\,000, 7500\} = 7500$, com a ordem $((M_1 M_2) M_3)$. Para $r = (20, 50, 10, 1, 100)$: $m[1, 4] = 3500$, com a ordem $((M_1 (M_2 M_3)) M_4)$; a tabela completa está na §11.3. (b) A última linha é $M(4, \cdot) = 0, 2, 9, 9$; o ótimo é 9, com os itens 1 e 2. (c) O tempo depende do *valor* de W, mas W ocupa só $\log_2 W$ bits: o algoritmo é exponencial no tamanho da entrada.

11.9 (a) Tamanho 4: acda ou dcda. (b) 240, com os itens 1, 2 e 4 (pesos $20 + 30 + 50 = 100$). (c) Algoritmo de Kadane: $S(i) = \max\{a_i,\ S(i-1) + a_i\}$ é a melhor soma de um subvetor que *termina* em i, e a resposta é $\max_i S(i)$, em $O(n)$. No exemplo, 55, com o subvetor $10, -5, 40, 10$.

11.10 (a) Custo 2010, com $((A_1 A_2)((A_3 A_4)(A_5 A_6)))$. (b) Guarde em $s[i, j]$ o k que deu o mínimo e use Multiplica(A, s, i, j) da §11.3. (c) Na reconstrução, quando $x_i \neq y_j$ e $C[i-1, j] = C[i, j-1]$, siga os dois caminhos recursivamente e elimine as repetições; pode haver exponencialmente muitas respostas.

11.11 (a) O número de Catalan $C_6 = \binom{12}{6}/7 = 132$. (b) $c(i, j) = \sum_{k=i}^{j} a_k + \min_{i \leq r \leq j}\{c(i, r-1) + c(r+1, j)\}$,

com $c(i, i-1) = 0$, em $\Theta(n^3)$. Resultado: custo 185, com raiz *if*; à esquerda, *case*, com o caminho *else*→*end* à direita; à direita da raiz, *then*, com *of* como filho esquerdo.

11.12 (a) $S[i] = a_i + \max\{S[j] : j < i,\ a_j < a_i\}$ (0 se não houver tal j), e a resposta é $\max_i S[i]$; por exemplo, $1, 101, 2, 3, 100, 4, 5$ dá 106 $(1, 2, 3, 100)$.
(b) Em pós-ordem: $com(v) = s(v) + \sum_f sem(f)$ e $sem(v) = \sum_f \max\{com(f), sem(f)\}$, somando sobre os filhos f; a resposta é $\max\{com, sem\}$ na raiz, em $\Theta(n)$.
(c) É a subsequência comum máxima das duas sequências.
(d) $f(1) = f(2) = 2$ e $f(n) = f(n-1) + f(n-2)$. A última faixa é branca ou vermelha; se a penúltima não é azul, a sequência vem de uma de tamanho $n-1$, estendida de um único jeito; se é azul, vem de uma de tamanho $n-2$, estendida pela faixa azul e pela cor que falta. Valores: $2, 2, 4, 6, 10, 16, 26, \ldots$

11.13 (a) Capacidades 0..14:
item 3 (peso 5): `0 - 0 - - I - 0 - 0 - - I - I`
item 4 (peso 2): `0 - 0 - I 0 - 0 - 0 - I 0 - 0`
(b) Um único vetor $K[0..W]$: para cada item, percorra w *decrescendo* de W até p_i, fazendo $K[w] := \max\{K[w], K[w-p_i] + v_i\}$. A memória cai para $O(K)$; com w crescendo, o item poderia ser reaproveitado. (c) Fixe o par de linhas (i_1, i_2), some cada coluna entre elas e aplique Kadane ao vetor de somas: $O(m^2 n)$, usando somas de prefixo por coluna. (d) $P(a, b) = \max\{p_{ab},\ \max_{a'<a} P(a', b) + P(a - a', b),\ \max_{b'<b} P(a, b') + P(a, b - b')\}$ para todos os $a \le n$ e $b \le m$: $O(nm(n+m)) \subseteq O(n^2 m^2)$.

11.14 $D(i, 0) = i$, $D(0, j) = j$ e $D(i, j) = \min\{D(i-1, j) + 1,\ D(i, j-1) + 1,\ D(i-1, j-1) + [x_i \neq y_j]\}$, em $\Theta(nm)$. De "kitten" para "sitting" a distância é **3**: trocar k por s, trocar e por i e inserir g.

11.15 São 2 moedas (3 + 3). O guloso pega 4 + 1 + 1, com 3 moedas. Programação dinâmica: $T(0) = 0$ e $T(s) = 1 + \min_{c \le s} T(s - c)$; os valores de $T(0)$ a $T(10)$ são $0, 1, 2, 1, 1, 2, 2, 2, 2, 3, 3$. Custo $\Theta(nS)$.

11.16 $r(0) = 0$ e $r(n) = \max_{1 \le i \le n}\{p_i + r(n-i)\}$. Os valores de $r(1)$ a $r(8)$ são $1, 5, 8, 10, 13, 17, 18, 22$: o máximo para 8 é **22**, com os pedaços 2 + 6.

11.17 São 6 passos, 3 para a direita e 3 para baixo, em qualquer ordem: $\binom{6}{3} = 20$ caminhos. Programação dinâmica: $P(i, j) = P(i-1, j) + P(i, j-1)$, com $P(1, 1) = 1$ e $P = 0$ nas casas bloqueadas. Com a casa $(2, 2)$ bloqueada, sobram 8 caminhos.

11.18 Se a soma S é ímpar, não é possível. Senão, decida se algum subconjunto soma $S/2$: $R(i, s) = R(i-1, s) \vee R(i-1, s - a_i)$, em $\Theta(nS)$, pseudopolinomial. $\{3, 1, 1, 2, 2, 1\}$: sim (3 + 2 = 5 de um lado). $\{2, 3, 7\}$: a soma é 12, mas nenhum subconjunto soma 6: não.

11.19 A última multiplicação separa os k primeiros fatores dos $n-k$ últimos, e cada lado pode ser parentizado de forma independente: $P(n) = \sum_{k=1}^{n-1} P(k)P(n-k)$. Os valores para $n = 1, \ldots, 6$ são $1, 1, 2, 5, 14, 42$ — os números de Catalan C_{n-1}.

11.20 Troque X e Y se for preciso, para que Y seja a menor. Mantenha dois vetores, $ant[0..m]$ e $atual[0..m]$, com as linhas $i-1$ e i da tabela: $atual[0] := 0$ e, para $j = 1..m$, $atual[j] := 1 + ant[j-1]$ se $x_i = y_j$ e $\max\{ant[j], atual[j-1]\}$ caso contrário; ao fim de cada linha, troque os papéis dos vetores. A resposta é o último valor calculado. Memória $O(\min\{n, m\})$ e tempo $\Theta(nm)$.

11.21 Seja $F(s)$ o número de maneiras; processando as moedas uma de cada vez (para não contar ordens diferentes como maneiras diferentes): para cada moeda c e para $s = c, \ldots, S$, faça $F(s) := F(s) + F(s - c)$, com $F(0) = 1$. Para 5 há **4** maneiras: 5; $2 + 2 + 1$; $2 + 1 + 1 + 1$; $1 + 1 + 1 + 1 + 1$. Para 10 há 10 maneiras.

11.22 Seja $L(i, j)$ o tamanho do maior substring comum que *termina* em x_i e em y_j: $L(i, j) = L(i-1, j-1) + 1$ se $x_i = y_j$ e 0 caso contrário. A resposta é o maior $L(i, j)$, em $O(nm)$. Para "ABABC" e "BABCA", o valor é 4, com o substring "BABC".

Capítulo 12: Caminhos mínimos entre todos os pares: Floyd-Warshall

12.1 $D(i, j, S) = \min\left\{W(i, j),\ \min_{v \in S}\{D(i, v, S \setminus \{v\}) + D(v, j, S \setminus \{v\})\}\right\}$. Há $n^2 \cdot 2^n$ subproblemas, um por par e por subconjunto: exponencial. O Floyd-Warshall fixa uma ordem dos vértices e só usa os $n + 1$ prefixos $\{1, \ldots, k\}$; em cada passo, decide-se apenas se o vértice k entra ou não como intermediário. Isso dá $O(n^2)$ subproblemas por valor de k e $\Theta(n^3)$ no total.

12.2 Ver o Exemplo da §12: as linhas de D^5 são $(0,1,2,3,5)$, $(2,0,1,2,4)$, $(1,2,0,1,3)$, $(3,4,5,0,2)$ e $(4,5,6,1,0)$. De 5 a 3: $P(5,3)=2$, $P(5,2)=1$, $P(5,1)=4$ e $P(5,4)=5$, logo $5\to4\to1\to2\to3$, com custo 6. De 2 a 1: $P(2,1)=3$ e $P(2,3)=2$, logo $2\to3\to1$, com custo 2.

12.3 Linhas da matriz final, de 1 a 5: $(0,1,-3,2,-4)$, $(3,0,-4,1,-1)$, $(7,4,0,5,3)$, $(2,-1,-5,0,-2)$ e $(8,5,1,6,0)$. A diagonal é toda nula: não há ciclo negativo. Por exemplo, $D(1,2)=1$ pelo caminho $1\to5\to4\to3\to2$, de custo $-4+6-5+4$.

12.4 $T^0(i,j)$ é verdadeiro se $i=j$ ou se $ij\in E$, e $T^k(i,j)=T^{k-1}(i,j)\vee\big(T^{k-1}(i,k)\wedge T^{k-1}(k,j)\big)$, com $\Theta(n^3)$ operações lógicas. No exemplo, os vértices 1, 2 e 3 alcançam $\{1,2,3,4\}$; o 4 alcança apenas a si mesmo, e o 5 também.

12.5 Inicialize $D(i,i)=\infty$, em vez de 0, e execute o algoritmo: ao final, $D(i,i)$ é o peso do menor ciclo que passa por i, e a resposta é $\min_i D(i,i)$. No exemplo, $D(1,1)=D(2,2)=D(3,3)=6$ (o ciclo $1\to2\to3\to1$) e $D(4,4)=7$ (o ciclo $4\to3\to4$): a resposta é 6.

12.6 Se não há ciclo negativo, todo passeio fechado tem peso maior ou igual a zero; $D(i,i)$ começa em 0 e só poderia diminuir por meio de um passeio fechado de peso negativo, logo continua 0. Se há um ciclo negativo, seja k o vértice de maior índice do ciclo e i outro vértice dele. Os dois trechos do ciclo, de i a k e de k a i, só têm intermediários de índice menor que k; por isso, depois da iteração $k-1$, $D(i,k)$ e $D(k,i)$ são no máximo os pesos desses trechos. Na iteração k, então, $D(i,i)\le D(i,k)+D(k,i)\le$ peso do ciclo <0.

12.7 Floyd-Warshall: $n^3=10^{12}$ operações. n execuções do Dijkstra com heap: $n\cdot(n+m)\log n\approx 10^4\cdot6\cdot10^4\cdot14\approx8\cdot10^9$, bem melhor nesse grafo esparso. Com $m\approx n^2/2$, as n execuções do Dijkstra custam cerca de $n\cdot m\log n\approx n^3\log n/2$, mais que o Floyd-Warshall; com o Dijkstra em vetor ($O(n^2)$ por execução), o total também é $\Theta(n^3)$, e o Floyd-Warshall é mais simples e aceita pesos negativos.

12.8 D^1: linhas $(0,4,\infty,10)$, $(\infty,0,3,\infty)$, $(7,11,0,1)$ e $(2,6,\infty,0)$. D^2: $(0,4,7,10)$, $(\infty,0,3,\infty)$, $(7,11,0,1)$ e $(2,6,9,0)$. D^3: $(0,4,7,8)$, $(10,0,3,4)$, $(7,11,0,1)$ e $(2,6,9,0)$. D^4: $(0,4,7,8)$, $(6,0,3,4)$, $(3,7,0,1)$ e $(2,6,9,0)$.

12.9 P^4 tem as linhas $(-,1,2,3)$, $(4,-,2,3)$, $(4,1,-,3)$ e $(4,1,2,-)$. De 2 a 1: $P(2,1)=4$, $P(2,4)=3$ e $P(2,3)=2$, logo $2\to3\to4\to1$, com custo $3+1+2=6$. De 4 a 3: $P(4,3)=2$, $P(4,2)=1$ e $P(4,1)=4$, logo $4\to1\to2\to3$, com custo $2+4+3=9$.

12.10 Com as arestas $1\to2$ (1) e $2\to1$ (-3), o ciclo $1\to2\to1$ pesa -2. Na iteração $k=1$, $D(2,2)$ passa a $D(2,1)+D(1,2)=-2$; na iteração $k=2$, $D(1,1)$ passa a $D(1,2)+D(2,1)=-2$. Os valores negativos na diagonal sinalizam o ciclo. Os demais valores passam a corresponder a *passeios* que repetem vértices e não são mais custos de caminhos mínimos, que nem estão definidos, pois percorrer o ciclo mais vezes diminuiria o custo indefinidamente.

12.11 Troque cada aresta não direcionada por duas arestas direcionadas de mesmo peso; a matriz W fica simétrica, e D também. Uma aresta não direcionada de peso negativo uv é, ela mesma, um ciclo negativo $u\to v\to u$; por isso, com o Floyd-Warshall, pesos negativos só são aceitáveis em digrafos.

12.12 O teste é executado exatamente n^3 vezes (três laços de 1 a n). A matriz D ocupa n^2 posições; com a matriz P, o total é $2n^2$. Calcular $D^0,\ldots,D^n$ separadamente usaria $\Theta(n^3)$ de memória, o que a atualização no lugar evita.

12.13 Na iteração k, os valores usados são $D(i,k)$ e $D(k,j)$. Eles não mudam durante a iteração: $D^k(i,k)=\min\{D^{k-1}(i,k),\ D^{k-1}(i,k)+D^{k-1}(k,k)\}=D^{k-1}(i,k)$, pois $D(k,k)=0$ sem ciclos negativos, e o mesmo vale para $D(k,j)$. Assim, não importa se uma entrada da linha ou da coluna k já foi atualizada; e as demais entradas só podem diminuir para valores que correspondem a caminhos reais, nunca abaixo de D^k.

12.14 Troque min por max e + por min: $C^k(i,j)=\max\{C^{k-1}(i,j),\ \min\{C^{k-1}(i,k),\ C^{k-1}(k,j)\}\}$, com $C^0(i,j)$ igual à capacidade da aresta ij (0 se ela não existe e ∞ para $i=j$). O argumento de correção é o mesmo, e o custo é $\Theta(n^3)$.

12.15 Excentricidades (o maior valor de cada linha de D^5): 5, 4, 3, 5 e 6 para os vértices 1 a 5. O diâmetro é 6 (a distância de 5 a 3), e o centro é o vértice 3, de excentricidade 3.

12.16 A aresta uv está em algum caminho mínimo de i a j se e somente se $D(i,u)+W(u,v)+D(v,j)=D(i,j)$. Ela está em algum caminho mínimo do grafo se e somente se $W(u,v)=D(u,v)$, pois então ela própria é

um caminho mínimo de u a v.

12.17 Os novos caminhos mínimos ou não usam a aresta uv, e já estão em D, ou a usam uma única vez (com pesos que não criam ciclo negativo). Logo, para todo par, $D'(i,j) = \min\{D(i,j),\ D(i,u) + w' + D(v,j)\}$: $O(n^2)$.

12.18 Calcule primeiro as distâncias de e para o novo vértice $n+1$: $D(i,n+1) = \min_u\{D(i,u)+W(u,n+1)\}$ e $D(n+1,j) = \min_u\{W(n+1,u) + D(u,j)\}$, em $O(n^2)$ no total. Depois, para todo par (i,j), faça $D(i,j) := \min\{D(i,j),\ D(i,n+1) + D(n+1,j)\}$, em $O(n^2)$. É exatamente uma iteração do Floyd-Warshall com $k = n+1$.

12.19 Mantenha $C(i,j)$, com $C(i,i) = 1$ e $C(i,j) = 1$ se há aresta ij (0 se não há). Na iteração k, para $i \neq k \neq j$: se $D(i,k) + D(k,j) < D(i,j)$, faça $D(i,j) := D(i,k) + D(k,j)$ e $C(i,j) := C(i,k) \cdot C(k,j)$; se $D(i,k) + D(k,j) = D(i,j)$, faça $C(i,j) := C(i,j) + C(i,k) \cdot C(k,j)$. Com pesos positivos, cada caminho mínimo é contado exatamente uma vez: na iteração do seu vértice intermediário de maior índice.

12.20 Represente a linha i da matriz como um vetor de n bits, ocupando $\lceil n/w \rceil$ palavras. A atualização $T(i,\cdot) := T(i,\cdot) \vee T(k,\cdot)$, feita sempre que $T(i,k)$ é verdadeiro, custa $O(n/w)$ operações sobre palavras. Para cada k e cada i, isso dá $O(n^2 \cdot n/w) = O(n^3/w)$ no total.

Capítulo 13: Algoritmos gulosos e matroides

13.1 Ver a §13.1: em ordem decrescente, use o máximo possível de cada moeda. Para $\{10,5,2,1\}$, uma solução ótima tem no máximo uma moeda de 1, uma de 5 e duas de 2, e nunca $2+2+1$; então as moedas pequenas somam no máximo 9, e o número de moedas de 10 é $\lfloor S/10 \rfloor$, como no guloso; o mesmo argumento fixa as demais. Contraexemplo geral: $\{4,3,1\}$ com $S = 6$ (o guloso dá $4+1+1$; o ótimo é $3+3$).

13.2 *Linear* e *gráfica*: ver a §13.2 (dimensão do espaço gerado; número de componentes de uma floresta). *Transversal*: sejam $I_1, I_2 \subseteq X$ emparelháveis, com $|I_1| < |I_2|$, por emparelhamentos M_1 e M_2 que cobrem, do lado X, exatamente I_1 e I_2. A diferença simétrica $M_1 \triangle M_2$ é uma união de caminhos e ciclos alternantes; como $|M_2| > |M_1|$, algum caminho tem mais arestas de M_2, e começa e termina em vértices cobertos só por M_2. Esse caminho tem um número ímpar de arestas e alterna os lados, então um dos extremos está em X; ele pertence a $I_2 \setminus I_1$. Trocando as arestas do caminho em M_1, obtém-se um emparelhamento que cobre I_1 e mais esse vértice. *Emparelhamento*: o mesmo argumento, com caminhos alternantes num grafo qualquer: o caminho com uma aresta de M_2 a mais acrescenta à cobertura um vértice de I_2 sem descobrir os de I_1.

13.3 Ver a §13.2. Suponha $|I_1| = k < |I_2|$ e que nenhum $e \in I_2 \setminus I_1$ aumente I_1. Com pesos $k+2$ em I_1, $k+1$ em $I_2 \setminus I_1$ e 0 nos demais, o guloso escolhe I_1 primeiro e depois só elementos de peso 0, totalizando $k(k+2)$; mas I_2 pesa pelo menos $(k+1)^2 > k(k+2)$, uma contradição.

13.4 (a) Não diretamente: os conjuntos de intervalos dois a dois disjuntos não formam uma matroide (com $I_1 = \{[1,10]\}$ e $I_2 = \{[1,2],[3,4]\}$, (I3) falha). Mesmo assim, o guloso "o que termina primeiro" é ótimo para pesos unitários, por um argumento de troca próprio (Exercício 13.6); com pesos arbitrários, é preciso programação dinâmica (Exercício 13.19). (b) Escalonamento de Gauss na matriz $m \times k$ formada pelas colunas de S: elas são linearmente independentes se e somente se o posto é k. O custo é $O(m\,k\min\{m,k\})$; se $k > m$, elas já são dependentes.

13.5 A §13.2 e o quadro "Para saber mais" reúnem os pontos centrais: os axiomas; o teorema do guloso, provado pela troca de bases; o algoritmo de Kruskal na matroide gráfica; a interseção de duas matroides por caminhos aumentantes no grafo de troca; o emparelhamento bipartido e a floresta colorida como interseções de duas matroides; e o caminho hamiltoniano como interseção de três. Para aprofundar, consulte as obras de Oxley e de Schrijver listadas nas Referências.

13.6 Em ordem de término, o guloso escolhe as atividades $1, 4, 8, 11$, isto é, os intervalos $[1,4)$, $[5,7)$, $[8,11)$ e $[12,16)$: quatro atividades, que é o máximo possível. Prova pelo argumento de troca: seja O uma solução ótima e g a atividade que termina primeiro. Se $g \notin O$, troque a primeira atividade de O por g; como g termina antes, continua compatível com as demais, e $|O|$ não muda. Logo existe uma solução ótima que contém g, e o restante é o mesmo problema restrito às atividades que começam depois do fim de g; por indução, o guloso é ótimo.

13.7 Menor duração: com $[1,5)$, $[4,7)$ e $[6,10)$, o guloso escolhe $[4,7)$, de duração 3, e perde as outras

duas, que são compatíveis entre si (o ótimo é 2). Começa primeiro: com $[0, 10)$, $[1, 2)$ e $[3, 4)$, o guloso escolhe $[0, 10)$ e fica com uma atividade; o ótimo é 2.

13.8 Ordene os itens por v_i/p_i decrescente e leve o máximo possível de cada um. Pela troca: se uma solução ótima leva menos do item i de maior razão e algo de um item j de razão menor, trocar uma quantidade ε de peso de j por i não diminui o valor; repetindo, chega-se à solução gulosa. No exemplo: os itens 1 e 2 inteiros e $20/30$ do item 3, valendo $60 + 100 + 80 =$ **240**. Na mochila 0-1, o ótimo é 220 (itens 2 e 3), e o guloso pela razão dá só 160.

13.9 (I1) e (I2) são imediatos: as condições $|I| \leq k$ e $|I \cap E_i| \leq k_i$ valem para $\emptyset$ e passam para os subconjuntos. (I3) na uniforme: se $|I_1| < |I_2| \leq k$, qualquer $e \in I_2 \setminus I_1$ serve, pois $|I_1 \cup \{e\}| \leq k$. (I3) na de partição: como $|I_1| < |I_2|$, há uma parte E_i com $|I_2 \cap E_i| > |I_1 \cap E_i|$; tome $e \in (I_2 \setminus I_1) \cap E_i$. Então $|(I_1 \cup \{e\}) \cap E_i| \leq |I_2 \cap E_i| \leq k_i$, e as outras partes não mudam.

13.10 Minimizar a multa das tarefas atrasadas é o mesmo que maximizar o peso das que cumprem o prazo, isto é, encontrar uma base de peso máximo da matroide. Em ordem decrescente de multa, o guloso aceita as tarefas 1, 2, 3 e 4, rejeita a 5 e a 6 (cada uma tornaria o conjunto impossível de escalonar) e aceita a 7. A multa total é $w_5 + w_6 = 30 + 20 =$ **50**. Uma ordem válida de execução: as tarefas aceitas em ordem crescente de prazo $(2, 4, 1, 3, 7)$, seguidas das atrasadas.

13.11 As junções são: $f + e = 14$; $c + b = 25$; $14 + d = 30$; $25 + 30 = 55$; $a + 55 = 100$. Um código possível: $a = 0$, $c = 100$, $b = 101$, $f = 1100$, $e = 1101$, $d = 111$. O custo é $45 \cdot 1 + 13 \cdot 3 + 12 \cdot 3 + 16 \cdot 3 + 9 \cdot 4 + 5 \cdot 4 =$ **224**.

13.12 Se, numa ordem, uma tarefa i vem imediatamente antes de uma tarefa j com $t_i > t_j$, trocar as duas não altera os términos das demais, e a soma dos términos diminui de $t_i - t_j$. Logo, numa ordem ótima, não há tal par, e as durações aparecem em ordem crescente. Para as durações $3, 1, 2$, a ordem $1, 2, 3$ termina as tarefas nos instantes $1, 3, 6$, com soma **10**; a pior ordem, $3, 2, 1$, soma 14.

13.13 Não é matroide: com $I_1 = \{3\}$ e $I_2 = \{1, 2\}$, nem $\{1, 3\}$ nem $\{2, 3\}$ são independentes, e (I3) falha. Com $c(3) = 3$ e $c(1) = c(2) = 2$, o guloso escolhe 3 e não consegue acrescentar mais nada, ficando com peso 3, enquanto $\{1, 2\}$ pesa 4.

13.14 É uma matroide: é a matroide linear das colunas $1 = (1, 0)$, $2 = (1, 0)$, $3 = (0, 1)$ e $4 = (1, 1)$ de $\mathbb{R}^2$. Os pares independentes são todos, exceto $\{1, 2\}$, cujas colunas são iguais (elementos "paralelos"), e nenhum trio é independente em $\mathbb{R}^2$.

13.15 Uma base é uma floresta geradora maximal, isto é, uma árvore geradora em cada componente; uma componente com n_i vértices contribui com $n_i - 1$ arestas, e o total é $\sum_i (n_i - 1) = n - c$. No exemplo, $n = 6$ e $c = 2$: o posto é 4 (duas arestas do triângulo e as duas do caminho).

13.16 As bases de $U_{n,k}$ são todos os subconjuntos de tamanho k; os seus complementos são todos os subconjuntos de tamanho $n - k$, que são exatamente as bases de $U_{n,n-k}$.

13.17 Na uniforme $U_{n,k}$, o guloso escolhe os k elementos mais pesados. Na de partição, escolhe os k_i mais pesados de cada parte E_i. No exemplo: 9 e 7 de A, 8 e 3 de B, 6 e 5 de C, com peso total 38.

13.18 Com $S = 14$, o guloso dá $10 + 1 + 1 + 1 + 1$ (5 moedas), e o ótimo é $7 + 7$ (2 moedas). O sistema $\{1, 5, 10, 25\}$ não tem esse problema: o guloso é ótimo para todo valor, o que se prova com um argumento como o do exemplo da §13.1.

13.19 Com $A = [0, 3)$ de peso 1 e $B = [1, 5)$ de peso 10, o guloso escolhe A, que termina primeiro, e não pode escolher B, ficando com peso 1; o ótimo é 10. O problema se resolve por programação dinâmica: ordene os intervalos por término, seja $p(j)$ o último intervalo compatível com o j-ésimo e $OPT(j) = \max\{OPT(j-1),\ w_j + OPT(p(j))\}$, em $O(n \log n)$.

13.20 Encontre, por seleção em $O(n)$, a razão mediana r dos itens e some os pesos dos itens com razão maior que r. Se essa soma é no máximo a capacidade restante, leve todos esses itens inteiros e continue com os itens de razão no máximo r e a capacidade que sobra; senão, descarte os itens de razão no máximo r e continue com os de razão maior. Cada passo descarta metade dos itens: $T(n) = T(n/2) + O(n) = O(n)$. No fim, sobra no máximo um item, levado fracionado.

Capítulo 14: Árvores geradoras mínimas: Kruskal e Prim

14.1 Por indução, o conjunto A das arestas aceitas está contido em alguma AGM. Ao aceitar $e = uv$, tome como S a árvore atual de u na floresta. Nenhuma aresta de A atravessa o corte $(S, V \setminus S)$, e e é a mais leve que o atravessa: uma aresta mais leve que o atravessasse teria sido examinada antes, quando os seus extremos já estavam em árvores diferentes (as árvores só crescem), e teria sido aceita. Pelo Lema 14.2, $A \cup \{e\}$ está contido em alguma AGM. Ao final, a saída é acíclica por construção e conexa, pois uma aresta entre duas componentes finais teria sido aceita; logo é uma árvore geradora, e é mínima. (Também segue do Teorema 13.8 aplicado à matroide gráfica.)

14.2 Invariante: as arestas de T estão contidas em alguma AGM. Vale no início, com T sem arestas. A cada passo, o corte $(V(T), V \setminus V(T))$ não é atravessado por nenhuma aresta de T, e o algoritmo de Prim escolhe a mais leve que o atravessa; pelo Lema 14.2, o invariante se mantém. Depois de $n-1$ passos, T é geradora e está contida numa AGM, logo é uma AGM.

14.3 Use um heap de mínimo com os vértices fora da árvore, com chave $chave(v)$ = peso da aresta mais leve de v até a árvore, e um vetor de posições para reduzir chaves. São n remoções do mínimo e, para cada aresta uv examinada quando u entra na árvore, no máximo uma redução de chave (uma operação *subir*, em $O(\log n)$). O total é $O((n+m)\log n) = O(m \log n)$, pois um grafo conexo tem $m \geq n-1$.

14.4 Use o vetor $chave$, sem heap (pseudocódigo da §14.3): em cada um dos n passos, uma varredura de $O(n)$ encontra o vértice de menor chave fora da árvore, e depois as chaves dos vizinhos são atualizadas. (a) Com listas, as atualizações percorrem cada lista uma vez, $O(m)$ no total; o custo é $O(n^2 + m) = O(n^2)$. (b) Com matriz, percorre-se a linha u, em $O(n)$, a cada passo: $O(n^2)$.

14.5 Sim, como no exercício anterior. E não é possível fazer melhor nessa representação: qualquer algoritmo correto precisa ler $\Omega(n^2)$ posições da matriz. Se alguma posição (u, v) não fosse lida, um adversário poderia colocar ali uma aresta levíssima, que estaria em toda AGM, e o algoritmo erraria. Logo $O(n^2)$ é ótimo com matriz, e a versão com heap seria pior, $O(n^2 \log n)$.

14.6 (i) Executar o Kruskal com as arestas em ordem **não crescente** de peso, o que é o guloso de matroides aplicado diretamente aos pesos c (Teorema 13.8). (ii) Trocar os pesos por $c' = C - c$, com $C > \max c$ para mantê-los positivos, ou por $-c$, e executar qualquer algoritmo de AGM: toda árvore geradora tem exatamente $n-1$ arestas, então $c'(T) = (n-1)C - c(T)$, e minimizar c' é maximizar c. (iii) Executar o Prim escolhendo sempre a aresta *mais pesada* do corte. No grafo do Exemplo 14.3, a árvore geradora máxima pesa 71.

14.7 Kruskal: aceita AC (1), DF (2), BE (3) e CF (4); rejeita AD (5) e CD (5), que fecham ciclo com A, C, F, D; aceita BC (5); rejeita todas as arestas de peso 6. Prim a partir de A: AC (1), CF (4), FD (2), CB (5) e BE (3). O custo é **15**. A AGM é única: cada aresta escolhida é a *única* mais leve de algum corte — AC no corte $\{A\}$, DF em $\{D\}$, BE em $\{E\}$, CF em $\{A, C\}$ e BC em $\{B, E\}$ —, e uma aresta assim pertence a toda AGM, pois uma AGM sem ela poderia ser melhorada pela troca do Lema 14.2.

14.8 Suponha duas AGMs distintas, $T_1 \neq T_2$, e seja e a aresta de menor peso em $T_1 \triangle T_2$; digamos que $e \in T_1$. $T_2 + e$ tem um ciclo, que contém alguma aresta $f \in T_2 \setminus T_1$, pois do contrário o ciclo estaria todo em T_1. Como e é a menor aresta da diferença simétrica e os pesos são distintos, $c(e) < c(f)$; então $T_2 + e - f$ é uma árvore geradora mais leve que T_2, uma contradição.

14.9 Suponha que $e = uv$ pertença a uma AGM T. Retirando e, T se divide em dois lados; o ciclo de G que contém e precisa atravessar esse corte de novo, por outra aresta f, com $c(f) < c(e)$. Então $T - e + f$ é uma árvore geradora mais leve, uma contradição.

14.10 Suponha um caminho P de u a v em G cuja maior aresta pese menos que a maior aresta e do caminho de u a v em T. Retirando e de T, u e v ficam em lados diferentes; P atravessa esse corte por alguma aresta f com $c(f) < c(e)$. Então $T - e + f$ é mais leve que T, uma contradição.

14.11 Em $T + uv$ há um único ciclo, formado pelo caminho de u a v em T e pela nova aresta. Encontre esse caminho com uma busca em T, em $O(n)$, tome a aresta mais pesada do ciclo e retire-a; se for a própria uv, a AGM não muda. Pela propriedade do ciclo, a aresta retirada não está em nenhuma AGM do novo grafo, e as outras arestas de G fora de T continuam excluídas pelos mesmos ciclos de antes.

14.12 Prim a partir de A: AD (5), DF (6), AB (7), BE (7), EC (5) e EG (9). Kruskal: aceita AD (5), CE (5), DF (6), AB (7), BE (7) e EG (9), e rejeita BC (8), EF (8), BD (9), FG (11) e DE (15). O custo da AGM é **39**.

14.13 Com pesos distintos, a aresta mais leve que sai de uma componente C é a mais leve do corte $(C, V \setminus C)$, e pertence à AGM (que é única) pela propriedade do corte; logo todas as arestas escolhidas numa rodada pertencem à AGM e podem ser acrescentadas juntas, sem formar ciclo. Cada componente se junta a pelo menos uma outra em cada rodada, então o número de componentes pelo menos cai à metade, e há no máximo $\log_2 n$ rodadas. Cada rodada examina todas as arestas uma vez e une as componentes, em $O(m)$ (ou $O(m\,\alpha(n))$ com Union-Find). Total $O(m \log n)$.

14.14 Toda árvore geradora tem o mesmo peso, $(n-1)c$, e portanto é mínima. Basta encontrar uma árvore geradora qualquer, por exemplo a árvore de uma busca em largura ou em profundidade, em $O(n+m)$.

14.15 Separe as arestas em duas listas, as de peso 1 e as de peso 2, em $O(m)$; essa é a "ordenação". Execute o Kruskal processando primeiro a lista de peso 1 e depois a de peso 2. O custo é o das operações de Union-Find, $O(m\,\alpha(n))$.

14.16 A aresta mais leve $e = uv$, única, é a mais leve do corte $(\{u\}, V \setminus \{u\})$, e é a única mais leve desse corte; pela troca do Lema 14.2, uma AGM sem ela poderia ser melhorada, então ela está em toda AGM. A segunda mais leve, $f = xy$, se única, também está em toda AGM (num grafo sem arestas paralelas): um dos extremos de f, digamos y, não é extremo da mais leve, e então f é a única aresta mais leve do corte $(\{y\}, V \setminus \{y\})$. A terceira, não necessariamente: ela pode fechar um triângulo com as duas primeiras.

14.17 O triângulo com AB (2), BC (2) e AC (3). A partir de A, a árvore de caminhos mínimos usa AB e AC (pois $\text{dist}(A, C) = 3 < 4$), com peso total 5; a AGM é $\{AB, BC\}$, com peso 4.

14.18 No melhor caso, as $n-1$ primeiras arestas da ordem formam uma árvore, e só elas são examinadas. No pior caso, a última aresta da ordem é necessária (por exemplo, é a única que liga um vértice ao resto), e todas as m arestas são examinadas. A complexidade assintótica não muda, porque a ordenação, de custo $O(m \log m)$, é feita de qualquer forma.

14.19 São $2m$ operações Find, cada uma em $O(1)$ (um ponteiro), e $n-1$ uniões. Com "menor na maior", cada vértice muda de representante no máximo $\log_2 n$ vezes, pois o tamanho do seu conjunto pelo menos dobra a cada mudança; o custo total das uniões é, portanto, $O(n \log n)$. O laço custa $O(m + n \log n)$.

14.20 O Kruskal, executado até o fim, devolve uma *floresta geradora mínima*: uma AGM de cada componente. O Prim só alcança a componente da raiz; para obter a floresta, repita-o a partir de um vértice ainda não alcançado, até cobrir todos os vértices.

14.21 Seja T' a segunda melhor árvore e $e \in T' \setminus T$ a aresta de menor peso dessa diferença. Pelo argumento de troca, $T + e$ tem um ciclo com uma aresta $f \in T \setminus T'$, e $T + e - f$ não é mais pesada que T'; repetindo o argumento, conclui-se que alguma árvore da forma $T + e - f$ é uma segunda melhor. Algoritmo: para cada vértice u, faça uma busca em T a partir de u calculando, para todo v, a aresta de maior peso no caminho de u a v em T, $\max(u, v)$: $O(n^2)$ no total. Depois, para cada aresta $uv \notin T$, o custo da troca é $c(uv) - \max(u, v)$; a melhor troca dá a segunda melhor árvore, em $O(n^2 + m) = O(n^2)$.

Capítulo 15: Busca em largura

15.1 Fixe uma raiz candidata r (tente cada vértice). *Condição 1*: $\text{dist}_T(r, v) = \text{dist}_G(r, v)$ para todo v (calcule por busca em largura em G); ela é necessária, porque o nível na árvore de largura é a distância. *Condição 2 (ordem)*: numa busca em largura, os vértices saem da fila nível a nível; dentro de um nível, irmãos ficam juntos, e os blocos de irmãos seguem a ordem dos pais; e cada w é filho do *primeiro* vizinho de w do nível anterior a sair da fila. Assim, a única liberdade da busca é a ordem dos filhos de cada vértice, e a condição 2 se traduz em restrições "x antes de y": para cada w e cada outro vizinho v de w no nível anterior, $pai(w)$ deve vir antes de v. Se x e y são irmãos, é uma restrição sobre a ordem dos filhos do pai deles; se não são, ela equivale a "$pai(x)$ antes de $pai(y)$", e sobe-se até chegar a dois irmãos. T é árvore de largura com raiz r se e somente se, para cada vértice, as restrições sobre a ordem dos seus filhos não formam ciclo, o que se verifica com uma ordenação topológica (Capítulo 18).

15.2 Com pilha, o vértice consultado é sempre o último inserido que ainda não foi esgotado: a busca avança a partir do vértice mais recente e só volta quando ele não tem vizinhos desmarcados, que é a busca em profundidade; os pais formam a árvore de profundidade. Com fila, o vértice consultado é o mais antigo: todos os seus vizinhos desmarcados são inseridos antes que qualquer outro vértice seja examinado, ou seja, "esgota-se o pai antes dos filhos", que é a busca em largura.

15.3 (a) Busca em largura com raiz x: $\text{dist}(x, v) = nivel(v)$, e o caminho na árvore é mínimo. (b) Modele cada casa livre como um vértice, ligando casas vizinhas livres; faça a busca a partir da entrada; se a saída for alcançada, o caminho na árvore (subindo pelos pais) é o menor, com $nivel$(saída) passos. (c) O grafo é bipartido se e somente se a floresta de largura não tem arestas-irmão nem primo, que fecham ciclos ímpares; a 2-coloração é dada pela paridade do nível.

15.4 Ordem de visita: s, r, w, v, t, x, u, y. Níveis: s no nível 0; r, w no 1; v, t, x no 2; u, y no 3. Pais: $r, w \leftarrow s$; $v \leftarrow r$; $t, x \leftarrow w$; $u \leftarrow t$; $y \leftarrow x$. Arestas que não são de árvore: tx é irmão (mesmo nível e mesmo pai w); xu é tio (u já tinha sido alcançado por t, um nível abaixo de x); uy é primo (mesmo nível, pais t e x). O grafo não é bipartido: o triângulo w, t, x é um ciclo ímpar.

15.5 O vértice a mais distante de qualquer vértice é extremo de algum caminho de comprimento máximo; se não fosse, trocando trechos de caminhos obter-se-ia um caminho mais longo que o diâmetro. Então o vértice mais distante de a dá o diâmetro. No exemplo, a partir do vértice 1, o mais distante é o 7 (distância 3); a partir de 7, o mais distante é o 6, à distância **5** (caminho $7, 5, 2, 1, 3, 6$).

15.6 Faça $cnt(raiz) = 1$. Ao processar uma aresta vw com $nivel(w) = nivel(v)+1$ (aresta-pai ou aresta-tio), faça $cnt(w) := cnt(w) + cnt(v)$. Como todos os vértices do nível ℓ são processados antes dos do nível $\ell + 1$, $cnt(v)$ já está completo quando v é processado. Custo $O(n + m)$.

15.7 Os vértices na fila pertencem sempre a no máximo dois níveis consecutivos, em ordem não decrescente de nível; por indução, retira-se um vértice de nível ℓ e entram no fim vértices de nível $\ell + 1$. Seja vw uma aresta, com v processado antes de w. Se w ainda não havia sido alcançado, vira filho de v e $nivel(w) = nivel(v) + 1$. Se já estava na fila, seu nível é $nivel(v)$ ou $nivel(v) + 1$. Ele não pode já ter saído da fila, pois então w teria sido processado antes de v. Logo $|nivel(v) - nivel(w)| \leq 1$. Quanto à distância: o caminho na árvore tem $nivel(v)$ arestas, então dist $\leq nivel(v)$; e qualquer caminho da raiz a v com k arestas muda o nível em no máximo 1 por aresta, então $nivel(v) \leq k$.

15.8 Numerando linhas e colunas a partir de 0, o caminho mínimo é $(0,0) \to (0,1) \to (0,2) \to (1,2) \to (2,2) \to (2,3) \to (3,3)$, com **6** passos. Distâncias: $(0,1)$: 1; $(0,2)$: 2; $(1,2)$: 3; $(2,2)$: 4; $(2,1)$ e $(2,3)$: 5; $(2,0)$, $(3,1)$ e $(3,3)$: 6.

15.9 O algoritmo é o mesmo, percorrendo $N_{out}(v)$; ele calcula a distância (em arestas) da raiz a cada vértice alcançável. Uma aresta $v \to w$ sempre satisfaz $nivel(w) \leq nivel(v) + 1$, mas pode "voltar" vários níveis ($nivel(w)$ muito menor que $nivel(v)$), pois a desigualdade só vale num sentido.

15.10 **Se** $L(y) = 0$, não há caminho. **Senão**, suba de y por $pai(\cdot)$ até x, empilhando os vértices, e depois desempilhe-os, imprimindo de x para y (ou insira cada vértice à esquerda numa lista). Custo $O(nivel(y))$.

15.11 Cada vértice entra na fila e é processado uma vez. Com listas, processar v custa $O(1 + \text{grau}(v))$, e a soma dos graus é $2m$: $O(n + m)$. Com matriz, processar v exige percorrer a linha inteira, $O(n)$: $O(n^2)$. As duas coincidem assintoticamente em grafos densos, com $m = \Theta(n^2)$.

15.12 Mantenha uma fila de duas pontas com os vértices e rótulos de distância d. Ao retirar v do início, para cada aresta vw de peso p: se $d(v) + p < d(w)$, atualize $d(w)$ e insira w no *início* se $p = 0$, ou no *fim* se $p = 1$. A fila mantém a propriedade de conter no máximo dois valores de distância consecutivos, com os menores no início, como a fila de prioridades do algoritmo de Dijkstra; cada vértice é processado um número constante de vezes, e o custo é $O(n + m)$.

15.13 Coloque todas as saídas na fila no início, com nível 0, como se fossem filhas de uma raiz virtual, e execute a busca em largura normalmente. O nível final de cada local é a distância até a saída mais próxima. Custo $O(n + m)$.

15.14 No ciclo $1, 2, 3, 4, 5$, a partir de 1: os vértices 2 e 5 ficam no nível 1, e 3 e 4 no nível 2; a aresta 34 liga dois vértices do nível 2 com pais diferentes (2 e 5): é uma aresta-primo, e o ciclo não é bipartido. No ciclo de 6 vértices, a partir de 1: 2 e 6 no nível 1, 3 e 5 no nível 2 e 4 no nível 3, alcançado por 3; a aresta 54 é uma aresta-tio, e não há arestas-irmão nem primo: o ciclo é bipartido.

15.15 Use o laço externo da busca em largura com um contador c: cada vez que um vértice ainda não alcançado inicia uma nova busca, incremente c e rotule com c todos os vértices alcançados nessa busca. Ao final, c é o número de componentes. Custo $O(n + m)$.

15.16 Cada casa é um vértice, ligado às até 8 casas alcançáveis por um movimento de cavalo. Uma busca em largura a partir de a1 dá distância **6** até h8.

15.17 Os estados são pares (a, b), com $0 \le a \le 3$ e $0 \le b \le 5$, ligados pelas operações possíveis. Uma busca em largura a partir de $(0, 0)$ encontra 4 litros com **6** operações: encher o de 5, despejar no de 3 (sobram 2 no de 5), esvaziar o de 3, despejar os 2 litros no de 3, encher o de 5 e despejar no de 3 até enchê-lo, o que deixa 4 litros no de 5.

15.18 Numa aresta-irmão vw, v e w têm o mesmo pai u: o ciclo u, v, w tem 3 arestas. Numa aresta-primo vw, v e w estão no mesmo nível ℓ; os caminhos na árvore de v e de w até o seu ancestral comum mais próximo u, no nível j, têm $\ell - j$ arestas cada; com a aresta vw, o ciclo tem $2(\ell - j) + 1$ arestas, um número ímpar. Numa aresta-tio, os níveis dos extremos diferem de 1, e o ciclo formado tem comprimento par.

15.19 Faça uma busca em largura a partir de cada vértice s. Durante a busca, toda aresta vw que não é de árvore fecha, com os caminhos da árvore até s, um passeio fechado de comprimento $nivel(v) + nivel(w) + 1$, que contém um ciclo de comprimento no máximo esse valor. O menor valor dessa expressão, sobre todas as raízes e arestas, é exatamente a cintura: quando s pertence a um ciclo mínimo, a aresta desse ciclo "oposta" a s dá o seu comprimento exato. Cada busca custa $O(n + m)$, e o total é $O(n(n + m))$.

15.20 A partir de a: x, y, z ficam no nível 1 (filhos de a) e b, c no nível 2 (filhos de x, o primeiro a sair da fila). As arestas yb, yc, zb e zc são arestas-tio. Não há arestas-irmão nem primo, coerente com o fato de $K_{3,3}$ ser bipartido.

Capítulo 16: Caminhos mínimos de uma origem: Dijkstra

16.1 Por indução nos vértices removidos: com pesos 1, os rótulos dos vértices na fila de prioridades assumem, em cada momento, no máximo dois valores consecutivos, d e $d + 1$, e os de valor d saem antes, como na fila da busca em largura. Quando v sai com $L(v) = d$, cada vizinho w ainda não alcançado recebe $L(w) = d + 1$ e $pai(w) = v$, exatamente como na busca em largura; vizinhos já alcançados têm $L(w) \le d + 1$ e não mudam. Logo $L(w) = nivel(w) = \text{dist}(raiz, w)$, e a árvore é de largura (com os empates resolvidos na mesma ordem).

16.2 Transforme cada aresta xy de peso p em duas arestas direcionadas, $x \to y$ e $y \to x$, ambas de peso p (e use ∞ onde não há aresta), e execute o algoritmo no digrafo resultante.

16.3 Ver o Exemplo da §16: são removidos a (0), b (1), c (2), d (3) e e (5), e a árvore é $a \to b \to c \to d \to e$.

16.4 São removidos s (0), y (5), z (7), t (8) e x (9). Pais: $y \leftarrow s$, $z \leftarrow y$, $t \leftarrow y$ e $x \leftarrow t$. Por exemplo, o caminho mínimo até x é $s \to y \to t \to x$, com custo $5 + 3 + 1$.

16.5 Com $x \to y$ (1), $x \to z$ (2) e $z \to y$ (-2), o algoritmo remove x, depois y com $L(y) = 1$, e o fixa; mas $x \to z \to y$ custa 0. Somar uma constante c a todos os pesos **não** resolve: um caminho com k arestas ganha kc, o que favorece os caminhos com menos arestas. No exemplo, com $c = 2$: $x \to y$ passa a valer 3 e $x \to z \to y$ passa a valer $4 + 0 = 4$; o caminho ótimo muda.

16.6 A probabilidade de um caminho é o produto $\prod p(e)$; maximizá-la equivale a minimizar $\sum -\log p(e)$. Como $p(e) \in (0, 1]$, os pesos $-\log p(e)$ são não negativos, e o algoritmo de Dijkstra se aplica. (Alternativa: um Dijkstra que mantém o maior produto e remove sempre o maior rótulo, o que funciona porque os fatores são no máximo 1.)

16.7 Crie uma *superfonte* s^*, com arestas de peso 0 para cada s_i, e execute o algoritmo a partir de s^*; ou, de forma equivalente, comece com $L(s_i) = 0$ para todas as fontes, todas inseridas na fila de prioridades. A árvore resultante também indica qual fonte é a mais próxima de cada vértice.

16.8 No primeiro grafo, esparso, use heap binário: $(n + m) \log n \approx 4 \cdot 10^5 \cdot 17 \approx 7 \cdot 10^6$ operações, contra $n^2 = 10^{10}$ com vetor. No segundo, denso ($m \approx n^2/2$), use vetor: $n^2 = 10^8$, contra $(n + m) \log n \approx 5 \cdot 10^7 \cdot 14 = 7 \cdot 10^8$ com heap.

16.9 São removidos 1 (0), 2 (7), 3 (9), 6 (11), 4 (20) e 5 (20). Pais: $2 \leftarrow 1$, $3 \leftarrow 1$, $6 \leftarrow 3$, $4 \leftarrow 3$ e $5 \leftarrow 6$. Por exemplo, o caminho mínimo até 5 é 1, 3, 6, 5, com custo $9 + 2 + 9 = 20$, mais barato que $1 \to 6 \to 5$, que custa 23.

16.10 Sim. A prova de correção só usa que o trecho de y até u num caminho mínimo custa pelo menos 0, o que continua valendo com pesos nulos.

16.11 Pode parar assim que t for removido da fila de prioridades: pelo teorema de correção, nesse momento $L(t) = \text{dist}(s, t)$, e as iterações seguintes só fixam vértices mais distantes, sem alterar $L(t)$.

16.12 Divida cada vértice v em dois, v_{in} e v_{out}, ligados por uma aresta $v_{in} \to v_{out}$ de peso $c(v)$; cada aresta $u \to v$ do grafo original passa a ser $u_{out} \to v_{in}$. Os caminhos mínimos no novo grafo correspondem aos do problema original. (Alternativa: some $c(v)$ ao peso de cada aresta que entra em v.)

16.13 O algoritmo de Dijkstra só guarda o melhor custo até cada vértice, sem considerar quantas arestas foram usadas; o caminho mais barato até um vértice intermediário pode usar arestas demais. No grafo de estados, o vértice (v, j) significa "estou em v depois de usar j arestas", com arestas $(u, j) \to (v, j+1)$ de peso $W(u, v)$ para $j < k$. A resposta é $\min_{j \le k} \text{dist}((s, 0), (t, j))$; como esse grafo é acíclico (o j cresce), basta uma programação dinâmica em $O(k(n+m))$.

16.14 Troque $L(v) + W(v, w)$ por $\min\{L(v),\ cap(v, w)\}$, comece com $L(s) = \infty$ e $L(w) = 0$ para os demais, e remova sempre o vértice de *maior* rótulo. A prova de correção é análoga, porque estender um caminho nunca aumenta a sua capacidade (o papel dos pesos não negativos).

16.15 Mantenha $cnt(s) = 1$ e, ao relaxar a aresta vw ao remover v: se $L(v) + W(v, w) < L(w)$, faça $L(w) := L(v) + W(v, w)$ e $cnt(w) := cnt(v)$; se $L(v) + W(v, w) = L(w)$, faça $cnt(w) := cnt(w) + cnt(v)$. Com pesos positivos, quando v é removido, $cnt(v)$ já está completo, pois todos os predecessores de v em caminhos mínimos foram removidos antes dele.

16.16 Ordene os vértices topologicamente (Capítulo 18) e processe-os nessa ordem, relaxando as arestas que saem de cada um, a partir de $L(s) = 0$. Quando um vértice é processado, todos os seus predecessores já foram, e o seu rótulo é definitivo. O custo é $O(n + m)$, e pesos negativos não são problema, pois não há ciclos.

16.17 A correção do algoritmo depende de que estender um caminho nunca o torne "melhor"; para caminhos mais longos, estender sempre melhora, e um vértice fixado cedo pode ter um rótulo errado. Além disso, a melhor extensão pode precisar repetir vértices. O problema do caminho simples mais longo é NP-difícil em grafos gerais; só em grafos acíclicos ele é fácil.

16.18 Os rótulos finitos estão sempre entre d e $d + C$, onde d é o último rótulo removido. Use um vetor de baldes (listas) indexado pelo rótulo, de tamanho $nC + 1$ (ou circular, de tamanho $C + 1$). Para remover o mínimo, avance um ponteiro pelos baldes até encontrar um não vazio; o ponteiro só avança, no máximo nC posições no total. Reduzir a chave é mover o vértice de um balde para outro, em $O(1)$. Total $O(m + nC)$.

16.19 Por indução: no início, $L(s) = 0 = \text{dist}(s, s)$ e os demais valem ∞. Um relaxamento só muda $L(w)$ para $L(v) + W(v, w) \ge \text{dist}(s, v) + W(v, w) \ge \text{dist}(s, w)$, pela hipótese e pela desigualdade triangular; e só muda quando o novo valor é menor, de modo que $L(w)$ nunca aumenta.

16.20 No máximo n inserções, n remoções do mínimo e m reduções de chave (no máximo uma por aresta, quando a aresta é relaxada). Com heap de Fibonacci, inserção e redução de chave custam $O(1)$ amortizado e remoção do mínimo $O(\log n)$ amortizado: o total é $O(n + m + n \log n) = O(m + n \log n)$.

Capítulo 17: Busca em profundidade

17.1 Numa árvore de profundidade de um grafo não direcionado, toda aresta fora de T liga um vértice a um ancestral (Exercício 17.7). Reciprocamente, se para alguma raiz r toda aresta de G liga ancestral e descendente em T enraizada em r, a busca que, em cada vértice, explora primeiro os filhos que ele tem em T reproduz T. Algoritmo: para cada raiz candidata, calcule PE e PS de T e teste, para cada aresta uv de G, se um intervalo contém o outro: $O(n + m)$ por raiz.

17.2 Empilham-se ec, ca, ab, bc (retorno), bd, dc (retorno) e de (retorno). Ao voltar de c para e, $back(c) = 1 \ge PE(e) = 1$: desempilha-se até ec, e o bloco tem os vértices $\{a, b, c, d, e\}$. Depois empilham-se ef, fg e ge (retorno); ao voltar de f, $back(f) = 1 \ge PE(e)$, e o bloco é $\{e, f, g\}$.

17.3 Depois da chamada P(w) pela aresta de árvore vw: se v não é a raiz e $back(w) \ge PE(v)$, marque v como articulação; se $back(w) > PE(v)$, a aresta vw é ponte. Para a raiz, conte os filhos e marque-a se forem pelo menos dois.

17.4 Sim. Cada bloco é impresso exatamente quando a busca volta de um demarcador w para o seu pai, e cada demarcador imprime um único bloco; há, portanto, uma bijeção entre demarcadores e blocos (contando os filhos da raiz, que são sempre demarcadores).

17.5 (a) Conte as chamadas do laço externo. (b) Execute P(v); se $PE(w) \neq 0$, suba de w até v pelos pais.

(c) Sem aresta de retorno, é acíclico; uma aresta de retorno vw fecha o ciclo formado pelo caminho de w a v na árvore e pela aresta vw. (d) Faça $cor(w) := 1 - cor(v)$ nas arestas de árvore; uma aresta de retorno com cores iguais revela um ciclo ímpar.

17.6 A busca desce $1, 2, \ldots, 7$ em linha: $PE = 1, \ldots, 7$ e $PS(7), \ldots, PS(1) = 8, \ldots, 14$. Retornos: 31 e 64. $back$: 1 para os vértices 1, 2 e 3; 4 para 4, 5 e 6; 7 para o 7. Articulações: 3, 4 e 6. Pontes: 34 e 67. Blocos: $\{1, 2, 3\}$, $\{3, 4\}$, $\{4, 5, 6\}$ e $\{6, 7\}$.

17.7 Seja vw uma aresta, com v alcançado antes de w. Enquanto v está ativo, ele examina a aresta vw; se w ainda não tiver sido alcançado nesse momento, ele será alcançado dentro da chamada de v. Em ambos os casos, w é alcançado entre $PE(v)$ e $PS(v)$, e é descendente de v.

17.8 Uma floresta com n vértices tem no máximo $n - 1$ arestas; logo $m \geq n$ implica ciclo. Se $m \geq n$, a resposta é "sim"; senão, $m < n$ e a busca custa $O(n + m) = O(n)$.

17.9 Empilhe pares (vértice, próximo índice na lista de adjacência). Olhe o topo (v, i): se i passou do grau de v, desempilhe e faça $PS(v)$; senão, avance o índice e, se o i-ésimo vizinho w não foi alcançado, faça $pai(w) := v$, $PE(w)$ e empilhe $(w, 1)$. Os tempos coincidem com os da versão recursiva.

17.10 PE de 1 a 8: $1, 2, 8, 3, 5, 9, 7, 4$; PS: $16, 15, 11, 14, 6, 10, 12, 13$. Arestas de árvore: 12, 24, 48, 85, 87, 73 e 36. Retornos: 52, 31 e 67. Não há articulações: todo filho w de um vértice não raiz tem $back(w) < PE(v)$, e a raiz tem um único filho.

17.11 Suponha $PE(u) < PE(v)$. Se $PE(v) < PS(u)$, então v foi alcançado enquanto u estava ativo e é descendente de u; a chamada de v termina antes da de u, e $I(v) \subset I(u)$. Senão, $PS(u) < PE(v)$, e os intervalos são disjuntos.

17.12 Se v é descendente de u, o caminho de u a v na árvore só tem vértices alcançados depois de $PE(u)$. Reciprocamente, se no instante $PE(u)$ há um caminho de vértices não alcançados até v e v não fosse descendente, tome o primeiro vértice do caminho que não é descendente de u; o anterior a ele é descendente e o examina antes de terminar, tornando-o descendente também, uma contradição.

17.13 Três componentes: $\{1, 2, 3\}$, $\{4, 5\}$ e $\{6, 7, 8, 9\}$. As raízes são 1, 4 e 6.

17.14 A primeira aresta de retorno é 52: subindo de 5 por pai até 2 ($5 \to 8 \to 4 \to 2$) e fechando com 52, obtém-se o ciclo $2, 4, 8, 5, 2$.

17.15 A partir de 1: $cor(1) = 0$, $cor(2) = 1$, $cor(3) = 0$, $cor(4) = 1$. A aresta de retorno 13 liga dois vértices de cor 0: o ciclo $1, 2, 3, 1$ é ímpar, e o grafo não é bipartido.

17.16 Numa árvore, todas as arestas são pontes, e as articulações são os vértices de grau pelo menos 2. Num ciclo, não há pontes nem articulações. Num caminho com $n \geq 3$ vértices, todas as arestas são pontes, e as articulações são os vértices internos.

17.17 Articulações: 3 e 4. Pontes: 34 e 47. Blocos: $\{1, 2, 3\}$, $\{3, 4\}$, $\{4, 5, 6\}$ e $\{4, 7\}$.

17.18 Faça uma busca em profundidade: o grafo é biconexo se ela alcança todos os vértices e não encontra articulações, em $O(n + m)$. Num grafo biconexo com pelo menos 3 vértices não há pontes, e uma aresta que não é ponte está em algum ciclo.

17.19 No ciclo $1, 2, 3, 4$ com a aresta 14, a busca a partir de 1 com vizinhos em ordem crescente percorre $1 \to 2 \to 3 \to 4$, encontrando o caminho de 1 a 4 com 3 arestas, enquanto a aresta 14 é um caminho com uma aresta.

17.20 Algoritmo de Hierholzer: a partir de um vértice, siga arestas ainda não usadas, marcando-as, até não haver saída (como todos os graus são pares, isso só acontece no vértice inicial); ao voltar na recursão, insira cada vértice no início do circuito. Os subciclos encontrados ao retroceder são encaixados automaticamente. Cada aresta é usada uma vez: $O(n + m)$.

Capítulo 18: Busca em profundidade em digrafos

18.1 Tome um caminho direcionado de comprimento máximo; o seu último vértice u é um sumidouro, pois uma aresta $u \to x$ com x fora do caminho daria um caminho maior, e com x no caminho daria um ciclo. Algoritmo linear: guarde o grau de saída de cada vértice e as listas de *entrada*, e ponha numa fila todos os sumidouros. Ao retirar um sumidouro u, coloque-o no início da ordenação e decremente o grau

de saída de cada x com $x \to u$; quem chegar a zero entra na fila. Cada aresta é processada uma vez: $O(n+m)$. Se a fila esvaziar antes de todos os vértices saírem, há um ciclo.

18.2 Num digrafo acíclico não há arestas de retorno. Numa aresta vw de árvore ou de avanço, w é descendente de v e sai antes dele; numa de cruzamento, w já tinha saído quando v começou. Logo $PS(w) < PS(v)$ para toda aresta, e a ordem por PS decrescente põe v antes de w. No exemplo, com as raízes em ordem alfabética, obtém-se $i, j, h, e, f, g, b, a, d, c$.

18.3 Sim. Com as arestas $a \to b$, $a \to c$ e $b \to c$: se a busca a partir de a visita b primeiro, c vira filho de b e ac é de avanço; se visita c primeiro, c termina antes de b começar, e bc passa a ser de cruzamento (e ac, de árvore). As componentes são definidas pela existência de caminhos nos dois sentidos, uma propriedade do digrafo, e o algoritmo é correto qualquer que seja a ordem da busca.

18.4 PE dos vértices 1 a 7: $1, 2, 3, 5, 6, 7, 13$; PS: $12, 11, 4, 10, 9, 8, 14$. Árvore: $1 \to 2$, $2 \to 3$, $2 \to 4$, $4 \to 5$ e $5 \to 6$. Retorno: $3 \to 1$ e $6 \to 4$. Avanço: $1 \to 3$. Cruzamento: $7 \to 3$ e $7 \to 6$. old: 1 para os vértices 1, 2 e 3; 5 para 4, 5 e 6; 13 para o 7. Componentes, na ordem em que são encontradas: $\{4, 5, 6\}$ (quando 4 sai), $\{1, 2, 3\}$ (quando 1 sai) e $\{7\}$.

18.5 Relógio, meias, cueca, camisa, gravata, calça, sapatos, cinto, paletó. Qualquer outra ordem que respeite as dependências também é válida.

18.6 Se houvesse um ciclo $C_1 \to C_2 \to \cdots \to C_k \to C_1$ com $k \geq 2$, de qualquer vértice de C_1 se chegaria a qualquer vértice de C_2 (seguindo o ciclo de componentes e andando dentro de cada uma) e vice-versa. Então $C_1 \cup C_2$ seria fortemente conexo, contradizendo a maximalidade das componentes.

18.7 Se há um caminho hamiltoniano $v_1 \to v_2 \to \cdots \to v_n$, toda ordenação topológica precisa pôr v_i antes de v_{i+1}, e portanto é única. Se não há, em alguma ordenação topológica dois vértices consecutivos v_i e v_{i+1} não são ligados por aresta, e trocá-los dá outra ordenação válida. Teste: obtenha uma ordenação topológica e verifique se cada par de vértices consecutivos é uma aresta, em $O(n+m)$.

18.8 Monte o digrafo com uma aresta $a \to b$ se a é pré-requisito de b. Se houver ciclo (aresta de retorno numa busca), é impossível. Senão, o menor número de semestres é o número de vértices do caminho mais longo: $sem(v) = 1 + \max\{sem(u) : u \to v\}$ (1 se não há pré-requisitos), calculado em ordem topológica em $O(n+m)$ — é a programação dinâmica de caminho mais longo num digrafo acíclico do Capítulo 11.

18.9 PE dos vértices 1 a 6: $1, 2, 3, 4, 9, 10$; PS: $8, 7, 6, 5, 12, 11$. Árvore: $1 \to 2$, $2 \to 3$, $3 \to 4$ e $5 \to 6$. Retorno: $4 \to 2$ e $6 \to 5$. Avanço: $1 \to 4$. Cruzamento: $5 \to 3$.

18.10 old: 1 para o vértice 1; 2 para 2, 3 e 4; 9 para 5 e 6. Componentes, na ordem em que são encontradas: $\{2, 3, 4\}$ (quando 2 sai), $\{1\}$ e $\{5, 6\}$. A aresta $5 \to 3$ é de cruzamento, mas 3 já saiu da pilha, e não altera $old(5)$.

18.11 As fontes iniciais são a e f; sai a, e b vira fonte; sai b; sai f, e c vira fonte; sai c, e d vira fonte; saem d e e. Ordem: a, b, f, c, d, e.

18.12 Como a aresta é de retorno, w é ancestral de v. Suba de v até w pelos ponteiros pai, guardando os vértices; o ciclo é $w \to \cdots \to v \to w$, impresso na ordem inversa da subida, seguido de w.

18.13 Se o digrafo é fortemente conexo, de s se alcança todo vértice e todo vértice alcança s, o que equivale a alcançar todos a partir de s no digrafo invertido. Reciprocamente, se isso vale, para quaisquer u e v há caminhos $u \rightsquigarrow s \rightsquigarrow v$ e $v \rightsquigarrow s \rightsquigarrow u$. O teste faz uma busca no digrafo e outra no digrafo invertido, em $O(n+m)$.

18.14 Para duas componentes C e C' com uma aresta de C para C', o maior PS de C é maior que o maior PS de C' (se a busca entra primeiro em C, termina C' antes; se entra primeiro em C', termina C' inteira antes de alcançar C). No digrafo invertido, as arestas vão de C' para C. Começando pelo vértice de maior PS, a segunda busca está numa componente da qual, no digrafo invertido, só saem arestas para componentes já visitadas; por isso, cada árvore da segunda busca é exatamente uma componente.

18.15 São 4: a, b, c, d, e; a, b, c, e, d; b, a, c, d, e; b, a, c, e, d.

18.16 Em ordem topológica s, a, b, c, t: $L(s) = 0$, $L(a) = 3$, $L(b) = 2$, $L(c) = \max\{3+4,\ 2+6\} = 8$ e $L(t) = \max\{3+2,\ 2+3,\ 8+1\} = 9$. O caminho de maior peso é $s \to b \to c \to t$, com peso **9**.

18.17 São necessários **3** semestres: no 1º, C_1 e P_1; no 2º, C_2, A_1 e P_2; no 3º, C_3. O caminho mais longo, $C_1 \to C_2 \to C_3$ (ou $C_1 \to A_1 \to C_3$), tem 3 vértices.

18.18 Num digrafo fortemente conexo com mais de um vértice, todo vértice tem grau de entrada e de saída positivos. Cada fonte do grafo das componentes precisa receber pelo menos uma nova aresta entrando nela, e cada sumidouro, pelo menos uma saindo dele; uma nova aresta entra em no máximo uma fonte e sai de no máximo um sumidouro. Logo são necessárias pelo menos $\max\{f, s\}$ arestas. (Sabe-se que esse número também é suficiente.)

18.19 Use o algoritmo de Kahn com um heap de mínimo no lugar da fila: entre as fontes disponíveis, retire sempre a de menor nome. Custo $O((n + m) \log n)$.

18.20 Num grafo não direcionado, toda aresta vw é examinada a partir dos dois extremos; quando o primeiro dos dois a ser alcançado, digamos v, a examina, w ainda não foi alcançado ou está ativo como descendente, e a aresta vira de árvore ou de retorno. Num digrafo, a aresta $v \to w$ só é examinada a partir de v, e w pode já ter sido completamente explorado antes, numa outra parte da floresta, o que dá as arestas de cruzamento; ou pode ser um descendente alcançado por outro caminho, o que dá as de avanço.

Capítulo A: Tabelas hash

A.1 O Apêndice A serve de roteiro: motivação (dicionários e endereçamento direto), funções de divisão, multiplicação, dobra e polinomial, hashing universal, encadeamento exterior e interior, sondagens linear e quadrática e duplo hashing, e a complexidade em função do fator de carga, com pior caso $\Theta(n)$. O texto da pesquisa é um trabalho individual, a ser escrito com as próprias palavras.

A.2 Posições 0..10; "–" indica posição vazia.
(a) Linear: $22, 88, -, -, 4, 15, 28, 17, 59, 31, 10$.
(b) Quadrática: $22, -, 88, 17, 4, -, 28, 59, 15, 31, 10$.
(c) Duplo hashing: $22, -, 59, 17, 4, 15, 28, 88, -, 31, 10$.

A.3 Lista 1: $28, 19, 10$; lista 2: 20; lista 3: 12; lista 5: 5; lista 6: $15, 33$; lista 8: 17; as demais ficam vazias. A maior lista tem 3 elementos.

A.4 Com $m = 2^r$, $k \bmod m$ são apenas os r bits menos significativos de k: chaves que diferem só nos bits mais altos (por exemplo, múltiplos de 2^r, ou strings codificadas que diferem só no começo) caem todas no mesmo endereço. Um primo longe de potências de 2 faz o resultado depender de todos os bits da chave.

A.5 Uma busca por k para ao encontrar uma posição vazia. Se uma chave no meio da sequência de sondagem de k for apagada, a busca pararia ali e não encontraria k, que está mais adiante. A solução é marcar a posição como "removida": a busca continua passando por ela, e a inserção pode reutilizá-la. Com muitas remoções, o desempenho cai e convém reconstruir a tabela.

A.6 $\alpha = 1000/500 = 2$: as listas têm, em média, 2 elementos. Uma busca sem sucesso percorre uma lista inteira, com 2 comparações em média, mais o cálculo do hash: $\Theta(1 + \alpha)$.

A.7 No máximo $\frac{1}{1-\alpha}$: 2 sondagens com $\alpha = 0{,}5$ e 10 com $\alpha = 0{,}9$. O custo cresce rapidamente quando a tabela se aproxima de cheia; por isso é comum redimensionar quando α passa de algo entre 0,5 e 0,75.

A.8 $123456 \cdot A \approx 76300{,}0041$, cuja parte fracionária é 0,0041: $h = \lfloor 4{,}1 \rfloor = 4$. $123457 \cdot A \approx 76300{,}6221$: $h = \lfloor 622{,}1 \rfloor = 622$. Chaves vizinhas vão para endereços distantes.

A.9 Uma família $\mathcal{H}$ de funções de U em $\{0, \ldots, m-1\}$ é universal se, para quaisquer chaves distintas $k_1 \neq k_2$, no máximo $|\mathcal{H}|/m$ funções da família as fazem colidir, isto é, a probabilidade de colisão é no máximo $1/m$ quando a função é sorteada. Como o sorteio é feito depois de a entrada ser fixada, nenhuma entrada específica é ruim para a maioria das funções, e o número esperado de colisões de uma chave é no máximo α, qualquer que seja o conjunto de chaves.

A.10 $h := 0$; **para** $i := 0$ **até** $L - 1$: $h := (h \cdot b + s_i) \bmod m$. Cada passo mantém $h < m$, sem calcular potências grandes. Se b e m tivessem um fator comum d, os termos $s_i b^j$ com $j \geq 1$ seriam múltiplos de d módulo m, e só uma fração dos endereços seria usada, aumentando as colisões.

A.11 Percorra o vetor mantendo uma tabela hash dos valores já vistos; para cada a_i, verifique se $x - a_i$ está na tabela e depois insira a_i. São n buscas e n inserções, em $O(n)$ esperado.

A.12 Percorra a sequência inserindo cada valor numa tabela hash; o primeiro valor que já estiver na tabela no momento da inserção é a resposta. $O(n)$ esperado.

A.13 A função hash espalha as chaves sem relação com a ordem, então o sucessor de k pode estar em qualquer posição, e é preciso examinar todas as chaves: $\Theta(n)$. Na AVL, ele é encontrado descendo ou subindo um caminho, em $O(\log n)$. Mínimo, máximo, percurso em ordem e consultas por intervalo têm a mesma limitação nas tabelas hash.

A.14 É a análise do vetor dinâmico do Capítulo 3: uma reconstrução com k chaves custa $O(k)$, mas só acontece depois de $\Theta(k)$ inserções baratas desde a anterior. Com o potencial Φ = constante vezes (número de chaves inseridas desde a última reconstrução), cada inserção custa $O(1)$ amortizado.

A.15 Na sondagem linear, uma chave cujo endereço cai em qualquer posição de um grupo contíguo de posições ocupadas vai parar na primeira posição livre depois do grupo. Assim, a posição logo após um grupo de k ocupadas é escolhida com probabilidade $(k+1)/m$, em vez de $1/m$, e grupos grandes tendem a crescer ainda mais e a se fundir.

A.16 A sequência $h_1(k) + i\,h_2(k) \pmod m$ só visita $m/\mathrm{mdc}(h_2(k), m)$ posições distintas. Com $m = 10$ e $h_2(k) = 5$, ela alterna entre apenas 2 posições, e a inserção pode falhar mesmo com a tabela quase vazia. Por isso usa-se m primo, ou m potência de 2 com $h_2(k)$ sempre ímpar.

A.17 Use listas *duplamente* encadeadas: com ponteiros para o anterior e o próximo, o nó é retirado da lista em $O(1)$, sem percorrê-la. (Sem o ponteiro para o nó, a remoção exige primeiro uma busca, de custo $\Theta(1+\alpha)$ esperado.)

A.18 No primeiro nível, uma função universal distribui as n chaves em $m = n$ posições; a posição j recebe n_j chaves. No segundo nível, cada posição tem uma tabela própria de tamanho n_j^2 e uma função universal sorteada até não haver colisões (com tamanho n_j^2, isso acontece com probabilidade pelo menos $1/2$ a cada sorteio). A memória total esperada é $O(\sum_j n_j^2) = O(n)$, e cada busca consulta duas posições: $O(1)$ no pior caso.

A.19 Insira cada valor numa tabela hash, contando apenas quando ele ainda não estava presente: $O(n)$ esperado. Em $O(n \log n)$ no pior caso: ordene (por exemplo, com mergesort) e conte as posições em que o valor muda.

A.20 São necessárias pelo menos $10^6/0{,}75 \approx 1\,333\,334$ posições. Para o método da divisão, escolha um primo pouco acima desse valor e longe de potências de 2, como 1 333 357.

Referências bibliográficas

Nota sobre as fontes

Este livro foi elaborado a partir do material didático da disciplina de Algoritmos do IMPA Tech, ministrada pelo Prof. Uéverton Souza — slides, notas de aula, monitorias, listas de exercícios, avaliações e propostas de trabalhos de pesquisa —, e complementado com as obras listadas abaixo. Os exercícios marcados como [Material de apoio] vêm de listas de exercícios de outras disciplinas de análise de algoritmos, indicadas aos alunos como material complementar.

Livros

[1] Cormen, T. H.; Leiserson, C. E.; Rivest, R. L.; Stein, C. *Introduction to Algorithms*. 3. ed. Cambridge, MA: MIT Press, 2009.

[2] Szwarcfiter, J. L.; Markenzon, L. *Estruturas de Dados e seus Algoritmos*. 3. ed. Rio de Janeiro: LTC, 2010.

[3] Szwarcfiter, J. L. *Teoria Computacional de Grafos: os algoritmos*. Rio de Janeiro: Elsevier, 2018.

[4] Kleinberg, J.; Tardos, É. *Algorithm Design*. Boston: Pearson/Addison-Wesley, 2006.

[5] Oxley, J. G. *Matroid Theory*. 2. ed. Oxford: Oxford University Press, 2011.

[6] Schrijver, A. *Combinatorial Optimization: Polyhedra and Efficiency*. Berlin: Springer, 2003.

Material da disciplina e material de apoio

[7] Souza, U. S. *Algoritmos*: slides, notas de aula, listas de exercícios, avaliações e propostas de pesquisa da disciplina. Rio de Janeiro: IMPA Tech.

[8] Material das monitorias da disciplina de Algoritmos do IMPA Tech (heaps binomiais e de Fibonacci, Union-Find, árvores AVL, programação dinâmica e revisão).

[9] Listas de exercícios de apoio: *Análise e Projeto de Algoritmos* (Departamento de Computação, UFF/PURO); *MAC5711 – Análise de Algoritmos* (IME-USP); *MC458 – Projeto e Análise de Algoritmos I* (Unicamp).

[10] Problemas de juízes online citados nos exercícios: *Beecrowd* (problemas 1191, 1195, 1200, 1201 e 2330), *SPOJ* (problema SID) e *Codeforces* (problema 1131F).